
Business Process Modeling

BPMN, Petri nets, workflow nets, process mining

Francesco Secoli • Academic year 2025/2026

Disclaimer. Derived from the *Business Process Modeling* course materials (Academic Year 2025/2026), MSc in Data Science & Business Informatics, University of Pisa.

These notes are open educational content created by a student. They are **not an academic source** and may contain inaccuracies. You may freely share, modify, and reuse this material for educational and non-commercial purposes with appropriate attribution. The content is my personal interpretation of the professor's course materials and should not replace official teaching resources. I assume no responsibility for any errors or misinterpretations.

These notes were produced with an **AI-in-the-middle** workflow: a first human pass, then **Claude Code** to support formulation, understanding, and rewriting, followed by a final human review.

If you find errors, have suggestions, or spot unintentionally included copyrighted material (which I will promptly remove on notification), contact me at sclfnc@proton.me.

Contents

1	Introduction	7
1.1	What is a Business Process?	7
1.2	What is Business Process Management?	7
1.3	Modelling	8
1.4	A Taste of BPMN	8
1.5	A Taste of Process Mining	8
1.6	Digression: Formal Modelling and the Importance of Proofs	9
1.7	References	11
2	Business Processes	11
2.1	Terminology	11
2.2	Organizational Structures	13
2.3	Actors, Principals, and Contractors	14
2.4	Cases and Procedures	14
2.5	Business process	16
3	Visual notation	17
3.1	Objective	17
3.2	What a model must capture	18
3.3	Representing processes	19
3.4	The BPM lifecycle	19
3.5	BPMN vs .bpmn	20
3.6	Insurance claim example	20
3.7	Tasks and links	21
3.8	Control-flow patterns	21
3.9	Ambiguity	22
3.10	Disambiguation	22
3.11	Ingredients	23
3.12	A sneak peek at standards	24
3.13	Alice–Bob car selling in BPMN	24
4	Petri nets basics	24
4.1	Object	24
4.2	An important difference	25
4.3	Basic definitions	25
4.4	Token game	28
4.5	Workflow nets	29
4.6	WoPeD	32
4.7	Syntax sugar	33
4.8	Subprocesses	34
4.9	Some patterns	34
4.10	Triggers	38
5	Process Mining	41
5.1	The process-mining framework	42
5.2	Processes, cases, events, attributes	42
5.3	Event log example	42

5.4	Discovery	43
5.5	Conformance	43
5.6	Enhancement	44
5.7	A discovery example	44
5.8	A conformance example	46
5.9	Quality criteria	46
5.10	Question time	47
5.11	Process discovery: the α -algorithm	48
5.12	A second worked example	51
5.13	Exercises	52
5.14	Limitations of the α -algorithm	53
5.15	Noise	53
6	Why process mining: purpose, goals, and requirements	54
6.1	Three functional goals	54
6.2	From question to insight: the workflow	54
6.3	Prerequisites: data, tools, and knowledge	55
6.4	Knowledge and skills	55
6.5	When process mining makes sense	56
7	Orchestration and collaboration	56
7.1	Why diagrams?	56
7.2	Event-driven process chains (EPC)	57
7.3	Orchestration	59
7.4	Collaboration	59
7.5	Choreography	60
7.6	Compatibility	60
7.7	Exercises: the travel agency	62
8	EPC and BPMN	62
8.1	EPC requirements	63
8.2	EPC guidelines	63
8.3	EPC sample diagrams	64
8.4	EPC semantics	65
8.5	Decorated EPC	66
8.6	BPMN	67
8.7	Typical patterns	69
8.8	Lanes and sub-processes	70
8.9	Exercises: translating between BPMN and EPC	71
9	Other BPMN features	71
9.1	Artefacts	71
9.2	Task types and activity markers	71
9.3	Typed events: catching and throwing	72
9.4	Message passing in collaborations	73
9.5	Deferred choice	73
9.6	Data objects and data stores	74
9.7	Call activities	75

9.8	Error events	75
9.9	Choreographies	75
10	From Automata to Nets	76
10.1	Finite automata	76
10.2	Preliminaries	77
10.3	DFA, formally	78
10.4	Reshaping automata into nets	79
10.5	Multisets	80
10.6	Petri nets, formally	80
10.7	Firing sequences	82
10.8	Exercises	83
11	Occurrence graphs	84
11.1	Boundedness	88
11.2	Coverability	89
12	Liveness	91
12.1	Live and dead nodes	91
12.2	Deadlock-freedom	93
12.3	The lattice of implications	94
12.4	Exercises: the four-property checklist	95
13	Incidence matrices	96
13.1	The unroll-once pattern	96
13.2	Vectors and matrices	96
13.3	The incidence matrix	97
13.4	Parikh vectors	98
13.5	The marking equation	99
13.6	Monotonicity	99
13.7	Exercises	100
14	Analysis of Workflow Nets	100
14.1	Workflow nets, structurally	101
14.2	Behavioural pathologies	101
14.3	The language of a workflow net	102
14.4	Simulation, reachability, and their limits	102
14.5	Soundness	103
14.6	The auxiliary net N^*	103
14.7	Strong connectedness	104
14.8	Worked example: the Computer Repair Service	105
15	Sound by construction	106
15.1	Composition by refinement	106
15.2	Building blocks	106
15.3	Refinement and abstraction at work	107
15.4	Generalised substitution	108
16	Invariants	109
16.1	S-invariants	110
16.2	S-invariants and system properties	111

16.3	T-invariants	112
16.4	T-invariants and system properties	113
16.5	Strong connectedness via invariants	113
17	S-systems and T-systems	113
17.1	S-systems	114
17.2	T-systems	115
18	EPC analysis	117
18.1	The translation pipeline	117
18.2	First approach: straight translation and relaxed soundness	117
18.3	Relaxed soundness	118
18.4	Second approach: simplified EPCs and free-choice nets	119
18.5	Third approach: decorated EPCs	119
19	BPMN analysis	120
19.1	The Dijkman–Dumas–Ouyang translation	121
19.2	Step 0: preprocessing macros	122
20	Free-choice nets	123
20.1	Stable sets of markings	124
20.2	Siphons	124
20.3	Traps	125
20.4	Complexity of liveness: NP-complete	125
20.5	Liveness and boundedness: the Rank Theorem	126
21	P and NP problems	127
21.1	P vs NP and NP-completeness	128
21.2	The SAT decision problem	128
22	Workflow modules	128
22.1	Workflow systems	129
22.2	Weak soundness	131
23	Conformance checking	132
23.1	Four models, naïve fitness	132
23.2	Token replay: missing and remaining tokens	134
23.3	Diagnostics and drill-down	135
23.4	Comparing footprints (optional reading)	135
24	Quantitative analysis	136
24.1	Cycle time analysis	138
24.2	Theoretical cycle time and efficiency	140
24.3	Little’s law	141
24.4	Cost analysis	142
25	Diagnosis for workflow nets	143
25.1	S-coverability	143
25.2	Well-structuredness	145
25.3	Error sequences	146
25.4	Non-live sequences	146
25.5	Unbounded sequences	147
25.6	Practice with WoPeD	148

Appendix: BPM Cheat Sheets	149
A.1: Definitions	149
A.2: Properties	150
A.3: General Results	151
A.4: Analysis Summary Table	153
Appendix: Process Mining hands-on with Disco	154
A Worked figures: nets to draw at the exam	155
A.1 Soundness: nets to draw	155
A.2 Liveness and boundedness: nets to draw	156
A.3 Handles: nets to draw	157
A.4 S-nets, T-nets, invariants	157
A.5 Siphons and traps	158
A.6 The short-circuit N^*	159

1 Introduction

1.1 What is a Business Process?

A **business process** (BP) is any set of steps aimed at reaching some outcome. The scope is deliberately broad: a BP ranges from mundane administrative tasks (such as opening a bank account) to complex operational flows such as processing a custom order. Any coordinated sequence of activities inside an organization qualifies.

1.2 What is Business Process Management?

Business Process Management (BPM) is the systematic design, analysis, and improvement of business processes. Its operational goals include automating workflow, orchestrating processes across organizational units, reducing the risk of errors, providing real-time metrics, enforcing deadlines, validating data, and reducing training costs.

Modern markets reward companies that launch new products rapidly and adapt existing ones at low cost. Business processes organize the activities behind each product, with IT as essential support. Historically, however, information systems started from data: information modelling came first, processes second. The **workflow management systems** of the mid-nineties were the first attempt to give processes systematic support by automating structured workflows, but they stayed confined to a few domains. BPM broadens that perspective: from intra-organizational workflow management to **process-aware information systems** spanning multiple organizations.



Figure 1.1. Benefits associated with Business Process Management.

At its core, BPM aims at the *robust and correct realization of business processes in software* that increases customer satisfaction and ultimately contributes to the competitive advantage of an enterprise. Three communities approach it from different angles:

- **Formal methods people:** investigate structural and behavioural properties, detect and correct deficiencies, abstract from the real world.
- **Software development people:** provide robust and scalable software systems, integrate existing software, track new technology trends.
- **Business administration people:** increase customer satisfaction, reduce costs, establish new products and services.

implements, and analyzes the real world. Three mining operations relate logs to models:

- **Discovery:** derive a process model from an event log with no prior model.
- **Conformance:** check how well an existing model matches the observed log.
- **Enhancement:** improve or extend an existing model using log data.

Process mining answers the question posed by abstraction: abstractions can be not only *designed* top-down but also *derived* bottom-up from recorded behaviour.

1.6 Digression: Formal Modelling and the Importance of Proofs

The Königsberg bridge problem illustrates why formal models and proofs matter in BPM. In 18th-century Prussia, the city of Königsberg was built on the banks of the river Pregel. Seven bridges connected two islands and the two riverbanks. A popular puzzle asked: can one walk a complete circuit crossing each bridge exactly once?

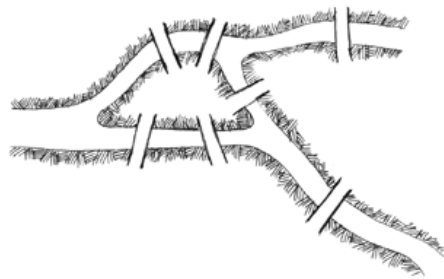


Figure 1.3. Map of Königsberg with the seven bridges highlighted.

Nobody had found a solution when Leonhard Euler heard of the puzzle in 1736. Rather than trying more routes, Euler set about *proving* that no solution was possible. He converted the map into a formal model: land areas became **nodes** (vertices), bridges became **edges**. This abstraction preserved the topology of the problem (the set of solutions) while discarding irrelevant geographic detail.

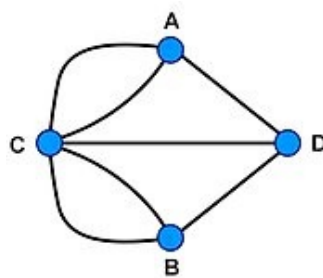


Figure 1.4. The Königsberg graph: nodes A, B, C, D represent land areas; edges represent bridges.

The original puzzle translates into a *graph problem*: redraw the graph without retracing any edge and without lifting the pencil. In general form: given a connected graph, find an **Eulerian circuit** (a circuit that visits every edge exactly once) if one exists.

1.6.1 Euler's theorem: the circuit

Definition (Graph Terminology). A **path** is a finite sequence of contiguous edges. It is a **circuit** if it ends at the same vertex where it starts.

An **Eulerian path** is a path that passes through every edge of the graph exactly once.

The **degree** of a vertex v is the number of edges attached to v . A vertex is **odd** if its degree is odd, **even** otherwise.

Definition (Euler's Theorem). A connected graph G contains an Eulerian circuit **if and only if** every vertex is even.

Informal reasoning. In a circuit, every time you enter a vertex you must also leave it, so each visit consumes one incoming and one outgoing edge: every vertex must have even degree. In the Königsberg graph all four vertices have odd degree (A, B, D have degree 3; C has degree 5), so no Eulerian circuit exists.

Example (Applying the Theorem to Königsberg). The graph has four vertices: A (degree 3), B (degree 3), C (degree 5), D (degree 3). All are odd, so Euler's theorem immediately rules out an Eulerian circuit. No amount of trial and error can find a solution because no solution exists.

1.6.2 Euler's theorem: the path

Definition (Euler's Theorem (Eulerian Path)). A connected graph G contains an Eulerian path **if and only if** G has exactly 0 or 2 vertices of odd degree.

When all vertices are even, the path is also a circuit. When exactly 2 vertices are odd, they are the start and end of the path.

Proof (necessity). Suppose G contains an Eulerian path P . At every vertex v , the path enters and leaves the same number of times, except when v is the start or end of P . If start and end are distinct, exactly two vertices are odd. If they coincide, no vertex is odd.

Proof (sufficiency). Suppose G has 0 or 2 odd vertices. If there are two odd vertices u and v , add a temporary edge (u, v) : now every vertex is even, so by Euler's theorem the modified graph contains an Eulerian circuit C , and C traverses (u, v) at some point. Remove (u, v) from C : what remains is an Eulerian path with u and v as endpoints. If there are no odd vertices, Euler's theorem applies directly and the circuit is itself an Eulerian path.

Note. The sufficiency proof is constructive: it yields an algorithm. Recursively find a cycle in the graph, remove its edges, and repeat on the remaining graph. This builds Eulerian paths and circuits directly.

Example (Historical Continuation: Königsberg Eulerised?). In the late 19th century, an eighth bridge was built across the Pregel. The new graph had degrees: A (4), B (4), C (5), D (3). With exactly two odd vertices (C and D), an Eulerian **path** exists (but no circuit), e.g. $C \rightarrow A \rightarrow C \rightarrow B \rightarrow C \rightarrow D \rightarrow A \rightarrow B \rightarrow D$. Citizens could now cross each bridge exactly once, though without returning to their starting point.

In 1944 air raids obliterated most of the bridges; five were later rebuilt. The resulting degrees were A (2), B (2), C (3), D (3). Again two odd vertices, again an Eulerian path, e.g. $C \rightarrow D \rightarrow A \rightarrow C \rightarrow B \rightarrow D$. The theorem predicted both states directly.

The algorithmic content of Euler's theorem illustrates a broader principle: when a formal proof establishes necessity and sufficiency, the constructive steps often yield executable procedures.

The modelling lesson is direct: abstraction must preserve the structure relevant to the question. Euler's graph preserves bridge connectivity and nothing else. Formal proofs then operate on the model, not on the territory; yet their conclusions transfer back to the territory precisely because the abstraction is faithful.

1.7 References

Main Textbook

M. Weske. *Business Process Management: Concepts, Languages, Architectures* (3rd ed.). Springer, 2019. <http://bpm-book.com>, chapter references in these notes follow the 2007 edition.

Suggested Reading

- J. Desel and J. Esparza. *Free Choice Petri Nets*. Cambridge Tracts in Theoretical Computer Science 40, Cambridge University Press, 1995.
- M. Dumas, M. La Rosa, J. Mendling, H. Reijers. *Fundamentals of Business Process Management* (2nd ed.). Springer, 2018. <http://fundamentals-of-bpm.org>
- W. van der Aalst. *Process Mining: Data Science in Action* (2nd ed.). Springer, 2016.
- W. van der Aalst and K. van Hee. *Workflow Management: Models, Methods, and Systems*. MIT Press, 2004.

Online Resources

Petri Nets World: <http://www.informatik.uni-hamburg.de/TGI/PetriNets>, BPMN: <http://www.bpmn.org>, Workflow Patterns: <http://www.workflowpatterns.com>, WoPeD: <http://www.woped.org>, ProM: <http://www.promtools.org>, Woflan: <http://www.win.tue.nl/woflan>¹, YAWL: <https://yawlfoundation.github.io>

2 Business Processes

2.1 Terminology

The terms introduced in this section are generic and apply to very different working situations and companies. We fix preferred terms where possible, but allow synonyms interchangeably.²³

2.1.1 Products, Work, and Specialization

We need products to live our lives (food, clothing, housing, transportation, health) and immaterial products (services) are just as frequent: credit approval, knowledge, entertainment. Every product is the outcome of some work: a specific task, duty, function, or assignment, often part of some larger activity. No one is skilled enough to produce everything they need, so products reach

¹Woflan (WOrkFLoW ANalyzer) diagnoses workflow processes for soundness. See H. M. W. Verbeek, T. Basten, W. M. P. van der Aalst. Diagnosing workflow processes using Woflan. *The Computer Journal* 44(4):246–279, 2001. <http://www.padsweb.rwth-aachen.de/wvdaalst/publications/p110.pdf>

²Ch. 1 of *Workflow Management: Models, Methods, and Systems*.

³Ch. 1 of *Business Process Management: Concepts, Languages, Architectures*.

people through markets: distribution in exchange for money. We buy what we cannot make ourselves.

Markets generate further work that would not otherwise exist (trading, banks, advertising, transportation, regulations, attorneys, insurance, eCommerce, eHealth, influencers) plus the services and products needed just to keep an organization operating, which contribute nothing directly to keeping us alive. To supply all this, people organize **specialized work units**: relatively autonomous divisions covering a limited range of products, highly efficient at producing their specific good or service.

Process orientation is based on a critical analysis of this way of organizing work units, originally introduced by Frederick Taylor (1856–1915).

Taylorism and the Evolution of Work traces a pattern across millennia. In prehistoric and ancient times, workers focused on the entire process for all products: a craftsman made a complete object from raw material to finish. The Middle Ages saw the rise of guilds, introducing intermediate specialization: workers gained deeper skill but still understood their craft holistically. The industrial era transformed this trajectory fundamentally.

Taylorism emerged in the industrial era as a systematic response to production complexity. The core insight: break a complex process into atomic work units of small granularity, then assign each unit to a specialized worker. A worker narrows focus from the entire process down to a single operation, sometimes a single gesture repeated thousands of times daily, and gains deep competence in that narrow task at the cost of losing sight of the overall product or goal.

Efficiency emerges from this radical specialization. A highly trained worker executes repetitive tasks faster and with fewer errors than a generalist. The tradeoff is rigid: the worker becomes replaceable and dependent on the organizational structure to coordinate their output with others. This model dominated manufacturing from the industrial revolution through much of the 20th century, and its legacy persists: many modern workflows still organize jobs around functional specialization rather than end-to-end processes.

Taylor’s approach rested on two mechanisms: time-and-motion studies to identify the “one best way” to perform each task, and a strict separation between mental work (planning, done by management) and manual work (execution, done by workers). Detailed plans specified what each worker should do and how to do it; workers received orders but not explanations.

This model flourished in manufacturing. Assembly lines succeeded precisely because early industrial products required few handovers between stations, and each handover incurred minimal delay. Moreover, tasks were simple and self-contained, needing no information about what happened before or after a worker’s station.

Yet this model fails in organizations that process information and immaterial goods. Knowledge work differs fundamentally: steps are interdependent, and workers must understand context from the entire case to make sound decisions. When a loan officer reviews an application, they need the customer’s history, credit report, and competing offers, not just a narrow rule to apply. Handovers become a bottleneck, less for the time lost passing work between desks than for the context that fails to travel with it.

2.1.2 Process Orientation

Radical functional specialization creates a problem that Taylorism did not face: alienation. In large organizations, workers no longer see how their narrow task contributes to a product or serves a customer. They execute orders without understanding why. This disconnect undermines motivation, self-esteem, and ultimately productivity. When workers understand they contribute to a specific customer outcome, not merely fulfill a directive, morale and performance improve measurably.

Process orientation addresses this by shifting focus from isolated tasks to connected chains of activity. Its purpose is twofold: to make visible the flow of work across an organization, and to optimize the handovers between workers. The mechanism is to combine small-granularity work units into larger-granularity units: handling a complete subprocess, a worker sees fewer handovers and keeps the context needed to decide soundly within their scope.

At the organizational level, this restructuring challenges the traditional hierarchy. A matrix structure emerges: employees report both to functional managers (who ensure technical competence) and to process owners (who coordinate end-to-end workflows). This dual accountability resists the old separation of planning from execution and enables the communication necessary for effective process management.

2.2 Organizational Structures

Organizations allocate work to resources (people, machines, or groups) according to defined capabilities and constraints. A bank teller, for example, can execute routine transactions but cannot approve loans: such restrictions reflect both competence and authority. Every process specifies three dimensions: which tasks to perform, their sequence, and which resources should execute them. The efficiency and effectiveness of workflow depends critically on how work is assigned to resources.

An organizational structure formalizes this allocation by establishing how work, authority, and responsibility are divided among staff. Resources are classified by two criteria:

- **Functions:** the competencies, skills, and specialized knowledge a resource possesses.
- **Roles:** the position held within the organization (such as a group or work unit) defining rights, duties, and place in the hierarchy.

A single resource can hold multiple roles simultaneously or in sequence; a senior accountant may serve both as a practitioner (reviewing ledgers) and as a mentor (training junior staff).

Organizations typically adopt one of three structural models:

- **Network structure:** autonomous actors collaborate to produce goods or services. Relationships are non-hierarchical and dynamic. This model permits ad-hoc clustering, outsourcing, and fluid team membership, making it flexible but harder to standardize and control.
- **Hierarchical structure:** organized as a directed tree. Formally, a *tree* is a connected acyclic graph: equivalently, a graph in which any two vertices are connected by exactly one path; in a tree with at least two vertices a *leaf* is a vertex of degree 1, and a *rooted* tree designates

one distinguished vertex (the root), implicitly orienting all edges away from it.⁴ Internal nodes represent roles and functions; leaves represent individual staff or departments; branches represent authority relationships. This creates a clear organization chart but can become rigid and slow to adapt.

- **Matrix structure:** combines hierarchical (line authority by function) and process dimensions in a grid: one column per function, one row per process. Each employee reports to a functional manager (ensuring technical competence) and to one or more process owners (also called *project leaders*) who coordinate end-to-end workflows. This model supports both specialization and process coordination but creates dual accountability and potential conflict.

2.3 Actors, Principals, and Contractors

Most work is assigned by other people, called **principals**. Principals may be individuals, departments, or external firms. They fall into two categories: a **boss** assigns work based on hierarchy and authority, while a **customer** requests work in exchange for compensation. These roles often overlap: a boss may assign work on behalf of customers.

The recipient of an assignment is called a **contractor**. Contractors need not be human; machines and software applications also serve as contractors. Between principal and contractor, a **contract** formalizes the agreement: it specifies the case to be performed, the deadline, and compensation. Contracts may also establish a communication protocol through which principal and contractor exchange information. A typical exchange runs through six messages: the principal sends a *specification* of the work; the contractor replies with a *quote* (its price and terms); the principal issues the *assignment*; the contractor sends a *confirmation*; the principal places the *order*; and the contractor reports *completion*. The quote-and-confirmation steps leave room for negotiation before the order is placed.

An actor can simultaneously hold both roles: a contractor may receive work from one principal while delegating portions of that work to other contractors, forming a chain of delegation. This generates a **contract tree**: a hierarchical decomposition of work.

2.4 Cases and Procedures

Work comes in discrete units, each with a clear beginning and end. A **case** is a unit of work to be performed or a product to be transformed. Cases vary widely: baking bread, manufacturing furniture, designing a building, or compiling survey results. Some cases yield tangible outputs (a house, a diagram); others are abstract (a lawsuit, an insurance claim, digital records). Synonyms include work, job, product, service, or item.

A **procedure** specifies how to handle a case: the tasks to be carried out and the conditions determining their order. It is a reusable template, applied to multiple cases. A **task** is a logical unit of work executed as a single indivisible whole. A **resource** (a person, machine, or group thereof) executes tasks. Not all tasks are identical in nature: some can be performed by machines alone, while others require human judgment and decision-making (a bank loan officer

⁴The degree-1 characterisation of a leaf presupposes a non-trivial tree: the single-vertex tree has its only vertex of degree 0, which is conventionally counted as the root, not as a leaf.

must evaluate risk; an algorithm cannot). Resources executing tasks must possess the relevant **knowledge**: skills, training, experience, and familiarity with organizational guidelines.

The “conditions determining their order” are a **precedence relation** $\prec$ on the tasks: $x \prec y$ means x must be completed before y starts. A single execution of the procedure lists its tasks in one order compatible with $\prec$ (a *linear extension* of the relation); such a sequence is a valid **execution trace**.

Example (Which orderings are valid?). Take the tasks $S = \{a, b, c, d, e, f\}$ with the precedence relation given by the chains

$$a \prec b \prec d \prec f, \quad a \prec c \prec e \prec f, \quad c \prec d.$$

So a starts the procedure and f ends it; b and c may run in either order after a , but d must wait for both b and c . Of the six candidate traces below, which respect every constraint?

$abcdef$	valid
$abcedf$	valid
$abdcef$	invalid: d precedes c , violating $c \prec d$
$acebdf$	valid
$acbedf$	valid
$acefbd$	invalid: f precedes b and d , violating $b \prec d \prec f$

The four valid traces among these candidates are exactly the ones that are linear extensions of $\prec$; the two invalid ones each reverse a required precedence. (These six are only a sample: $\prec$ admits further linear extensions, such as $acbdef$, that we simply did not list.) Reordering b and c is free because $\prec$ leaves them incomparable: this is precisely the room a procedure gives for concurrency.

An **activity** is the concrete performance of a task by a specific resource in a specific case. This distinction matters: cases share procedures, but each case may trigger different activities depending on case attributes. Two insurance claims follow the same procedure, yet one may require objection handling and the other may not.

Example: Making a pizza. The procedure includes checking ingredients, preparing dough and toppings, shaping, topping, and baking. These are the tasks. In case A, ingredients are fresh; in case B, fresh herbs are unavailable. Both cases follow the same procedure, but the activities differ: case A uses fresh basil, case B uses dried. The procedure is generic; the activities adapt to the case. Knowledge requirements vary too: a skilled pizzaiolo adjusts temperature and timing based on humidity and ingredient condition, while a novice follows fixed rules.

2.4.1 Economy of Scale and Procedure Design

The number of distinct procedures in a company is finite and far smaller than the number of cases. Making one hundred identical skirts with the same pattern is cheaper than making one hundred custom skirts (off-the-rack beats made-to-measure). As case volume increases, cost per case falls. Strategy follows: keep the number of procedures small while maximizing the number of cases each can handle.

This creates a design tension. Insurance companies cannot control the number of claims; they receive whatever cases arrive. They can reduce the number of procedures, but oversimplifying

risks inefficiency: a single universal procedure is theoretically possible but practically unworkable. The ideal is few well-designed procedures serving many cases.

Yet counter-examples complicate this picture. Tailors and architects seem to create one procedure per case: each custom suit fits one customer, each building is designed from scratch. Yet this appearance deceives. Tailors and architects exploit standard approaches within each case: measurement routines, pattern libraries, proven design principles. Work execution is highly case-dependent, but it reuses templates and methods.

2.5 Business process

Every product or service a company provides results from coordinated work. Business processes concern the understanding, correlation, organization, and continuous improvement of the activities that produce these outputs. Process management systems support communication between employees and make their activities more controllable. Radical redesign of business processes, not incremental improvement, is what drives organizational success.

Definition (Business Process (Hammer & Champy, 1993)). A collection of activities that take one or more kinds of input and create an output that is of value to the customer.

The focus shifts from *what* is produced to *how* it is produced: from product perspective to business logic.

Definition (Business Process (Davenport, 1993)). A specific ordering of work activities across time and space, with a beginning and an end.

Processes must have clearly defined boundaries, inputs, and outputs. Unless designers and stakeholders can agree on how work is structured, systematic improvement becomes nearly impossible. Yet following a structured process is not inherently inefficient; proper structure enables coordination and learning.

Definition (Business Process (Johansson et al., 1993)). A set of linked activities that take an input and transform it to create an output.

The transformation should add value: the output must be more useful and effective than the raw inputs. When cost decreases or customer satisfaction increases, the process itself improves.

2.5.1 Ownership and Cross-functionality

Processes require clearly defined owners responsible for design and execution. Ownership is an additional dimension of organizational structure. During periods of radical process change, ownership takes precedence over traditional hierarchies; without this authority, process owners cannot implement designs that violate functional organizational charts and norms. Processes are inherently cross-functional, spanning multiple functions and organizational boundaries.

2.5.2 Classification of Processes

Definition (Process Classification (Rummler & Brache, 1995)). Processes result in either products or services received by an organization's external customer (primary processes), or produce outputs invisible to the external customer but essential to effective business management (secondary and tertiary processes).

Primary processes produce the company's goods or services for external customers, generating revenue. Examples: raw materials purchase, service sale, design and engineering, distribution. Secondary processes support primary processes. Examples: machinery purchase and maintenance, personnel management, financial administration, marketing. Tertiary processes direct and coordinate primary and secondary processes, establishing objectives and allocating resources. Examples: maintenance of contracts with financiers and stakeholders.

2.5.3 Standardization and Notation

Processes must be represented using standard visual notation. Visual languages are intuitive, universal, and require little prior knowledge. Nodes and directed edges (graphs) are the natural choice. Shapes, colors, and symbols assign unambiguous meaning to different elements. Key concepts to represent: start, end, task, link, order, ownership, responsibility.

3 Visual notation

3.1 Objective

A visual notation is the interface between domain experts, who own the processes, and formal analysis tools, which need an unambiguous representation. The reference for the chapter is Ch. 3 of Weske.⁵

Section 2 collected three literature definitions of business process (Hammer & Champy, Davenport, Johansson). The textbook definition below distills them and is the one used throughout these notes:

Definition (Business process). A **business process** consists of a set of activities performed in coordination within an organisational and technical environment. These activities jointly realise a business goal.⁶

Each business process is enacted by a single organisation, but it may interact with processes performed by other organisations. The intra-organisational view is called *orchestration*; the inter-organisational one is *collaboration* (or *choreography*).

Definition (Business process model and instance). A **business process model** consists of a set of activity models together with the execution constraints between them. A **business process instance** represents one concrete case in the operational business of a company, consisting of activity instances.⁷

⁵Weske M., *Business Process Management: Concepts, Languages, Architectures*, Springer, 2007, Ch. 3.

⁶Weske M., *Business Process Management*, Springer, 2007.

⁷Weske M., *Business Process Management*, Springer, 2007.

An activity model is a *blueprint* for its activity instances, exactly as a process model is a blueprint for its process instances, one per *case*: each instance carrying its own case data, such as customer identifier, name, and amount of the credit request. When no confusion is possible, **activity** denotes both an activity model and an activity instance, and **process** likewise covers both sides of the model/instance distinction.

3.2 What a model must capture

Modelling starts from five questions about the process.

- **Who is the customer?** Every process starts and ends with a customer, who requests a product and ultimately receives it. The customer may be another department of the same company: the role is independent of organisational boundaries.
- **Who is the owner?** The *process owner* is responsible for the process: that instances run correctly, that business goals are met, that performance is measured and improved. The model has to support these responsibilities, not contradict them.
- **Which tasks?** The *activities* (or *tasks*) needed to realise the goals. A task is treated as an atomic action, possibly with a duration and a cost, but granularity is a modelling choice: any task may decompose into finer steps on closer inspection.
- **Which dependencies?** *Execution constraints* fix the admissible orderings: which task precedes which, which run in parallel, which exclude each other. They distribute work over *time*; a *process orchestration language* is the formalism they are written in.
- **Which roles?** Each task may require specific abilities (a customer-service agent, a back-office clerk, a software service) abstracted as *roles*. Roles distribute work over *space*: which actor is in charge of which step. Time and space are orthogonal dimensions of one orchestration.

Modelling translates informal textual requirements into a notation, ideally graphical. An explicit diagram lets stakeholders **communicate** about how work is done, **refine** the process when ambiguities surface, and **improve** it over time.

Definition (Business process management). Business process management (BPM) is the discipline that brings together concepts, methods, and techniques to support the design, administration, configuration, enactment, and analysis of business processes.⁸

BPM rests on an *explicit representation* of processes, tasks, and execution constraints. Once the representation is explicit, the process can be analysed (does it terminate? can it deadlock?), improved (where are the bottlenecks?), and enacted by software.

Definition (Business process management system). A business process management system (BPMS) is a generic software system, driven by explicit process representations, that coordinates the enactment of business processes.⁹

Process models are the main artefact for implementing a process. The implementation may rest on organisational rules and written procedures alone, or be delegated to a BPMS that reads

⁸Weske M., *Business Process Management*, Springer, 2007.

⁹Weske M., *Business Process Management*, Springer, 2007.

the model and steers the instances through it: assigning tasks to roles, enforcing execution constraints, logging what actually happened.

3.3 Representing processes

Process representations come in three flavours, one per audience:

- **Visual representations:** diagrams for *humans*, informal but intuitive: BPMN, EPC, graphical fragments of BPEL. What we see.
- **Languages:** unambiguous machine syntax, process dialects and XML schemas such as BPEL or BPMN XML. What machines see.
- **Models:** rigorous semantics for *analysis*, automata, Petri nets, workflow nets. What we analyse.

The layers complement each other, and a well-engineered BPM stack provides translations between them.

XML, the *eXtensible Markup Language*, is the serialisation format most process languages adopt. An XML document is a tree of user-defined *tags* with the values at the leaves:

```
<?xml version="1.0" encoding="UTF-8"?>
<note>
  <from>Alice</from>
  <to>Bob</to>
  <heading>Reminder</heading>
  <body>Don't forget to buy oranges!</body>
</note>
```

Verbose but unambiguous, and readable by any standard parser.

3.4 The BPM lifecycle

The BPM definition lists five capabilities: design, administration, configuration, enactment, analysis. These are not a checklist run once but a *cycle* that feeds its measurements back into the next redesign, closing back onto its own start. Each turn is a phase with a characteristic activity, a course technique that lives there, and a responsible role. Roles matter especially at *validation*: a model is only as good as the agreement between the *owner* who states the goals, the *analyst* who draws the diagram, and the *participants* who will run it: which is why the design phase closes with the three arguing over one shared diagram instead of over prose requirements.

- **Design & Analysis.** The process is modelled and checked before anyone runs it. Modelling produces a diagram in a graphical notation, **BPMN** or **EPC**; validation confronts it with stakeholders and, informally, with simulation of sample cases; verification translates it into a **workflow net** and proves properties such as *soundness*. Role: the *process analyst*, with the *owner* who states the goals and the *participants* who confirm the model matches the real work.

- **Configuration.** The validated model becomes an executable process: a **BPMS** is selected and configured, each task bound to the roles and software services that perform it, the model made executable by a compliant engine (the `.bpmn` serialisation is the exchange format). Role: the *system architect*, wiring the model into the platform.
- **Enactment.** The configured process runs: the BPMS creates one instance per case, assigns tasks, enforces the execution constraints, and *logs* every step that actually happens. Role: the *participants* who carry out the tasks, steered and recorded by the BPMS.
- **Evaluation.** The logs feed back into analysis. **Process mining** reconstructs the real process from the log; **conformance** checking compares it against the designed model; *quantitative analysis* measures durations, costs, bottlenecks. The findings drive the next redesign, returning to Design & Analysis. Role: the *process analyst*, reporting to the *owner* who decides what to improve.

Design and Evaluation are the two analytic phases, and they use different evidence: Design & Analysis reasons on the *model* before execution (a workflow net checked for soundness), Evaluation on the *log* after execution (a real trace, mined and compared). The former asks whether the process *can* misbehave; the latter, whether it *did*.

3.5 BPMN vs .bpmn

The visual/machine contrast is concrete in BPMN. A one-task process: start event, activity *Run*, end event:



Serialised as a `.bpmn` file, each visual element becomes an XML tag with a unique identifier (`StartEvent_1`, `Activity_1lzhm01`, `Event_1t0u7im`) and each sequence flow becomes a `<bpmn:sequenceFlow>` tag whose `sourceRef` and `targetRef` attributes point at those identifiers. The two representations are interchangeable: the editor renders the XML as the diagram, and saving regenerates it.

3.6 Insurance claim example

The running example for the rest of the section is a simplified insurance-claim process.¹⁰ Its textual description lists twelve steps:

1. **recording** the receipt of the claim;
2. establishing the **type** of the claim;
3. checking the covering of the client's **policy**;
4. checking the **premium** (are payments up to date?);
5. decision for **rejection**/admission;
6. if step 3 or step 4 has a negative result: producing a **rejection letter**, then jump to step 12;
7. if steps 3 and 4 have positive results: sending an **estimate** of the amount to be paid;

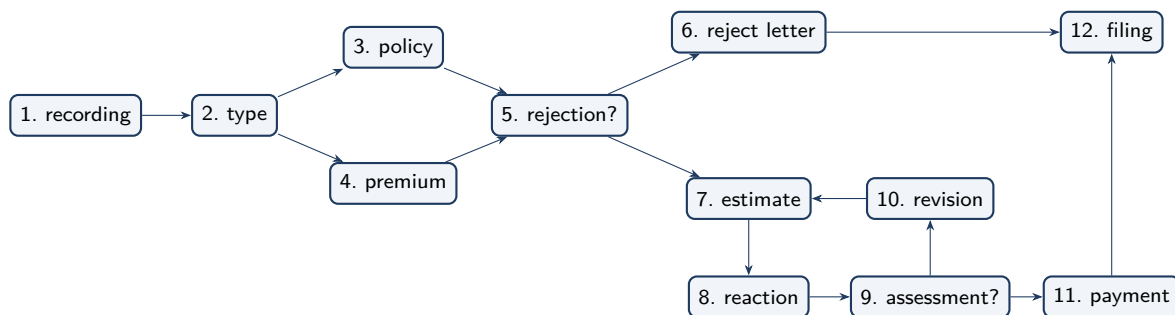
¹⁰van der Aalst W., van Hee K., *Workflow Management: Models, Methods, and Systems*, MIT Press, 2004, Sect. 1.3.

8. recording the client's **reaction**;
9. **assessment** of an objection;
10. if step 9 has a negative result: decision to **revise** step 7;
11. if step 9 has a positive result: **payment** of the claim;
12. **filing** and closure of the claim.

The description mixes activities (recording, filing, payment) with control flow (“if/then”, “then 12”) and feedback (revise step 7).

3.7 Tasks and links

Each step becomes a task node; two of them, REJECTION? and ASSESSMENT?, are decision points whose outcome is a boolean. Links then wire the tasks into a flow: a sequence, a parallel section, two choices, and a feedback loop:



3.8 Control-flow patterns

The insurance net combines four recurring control-flow patterns, visible as fragments of the diagram above.

- **Sequence:** TYPE can start only after RECORDING has finished; a single arrow makes the temporal precedence explicit.
- **Parallel:** after TYPE, both POLICY and PREMIUM must be checked before REJECTION? can be evaluated. The two branches run independently (their relative order is irrelevant) and the join waits for *both* (AND-split / AND-join).
- **Choice:** from REJECTION? the flow goes *either* to REJECT LETTER *or* to ESTIMATE, depending on the boolean outcome; exactly one branch fires per instance (XOR-split). The same construct appears at ASSESSMENT?.
- **Merge:** the two alternative paths reunite at FILING, reached through REJECT LETTER or through PAYMENT. Unlike the parallel join, the merge does not wait for both branches: whichever arrives carries the flow through (XOR-join).
- **Iteration:** a negative ASSESSMENT? routes back through REVISION to ESTIMATE; the cycle ESTIMATE → REACTION → ASSESSMENT? → REVISION repeats until the objection is finally accepted or rejected. Iteration combines an XOR-join (entering the loop body) with an XOR-split (repeat or exit).

3.9 Ambiguity

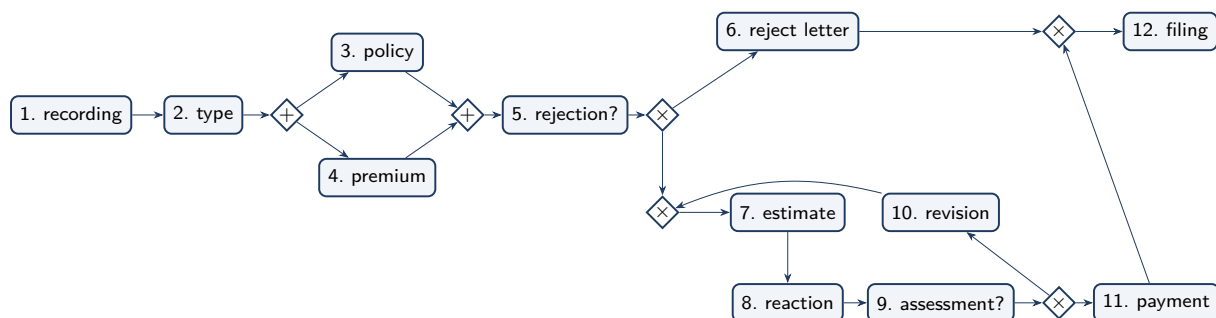
The diagram uses one kind of arrow for two different forks, and this is genuinely ambiguous. After TYPE both branches are always executed; after REJECTION? exactly one is; yet the two fan-outs look identical:



Joins suffer the same ambiguity: POLICY and PREMIUM must *both* complete before REJECTION? can fire (AND-join), whereas FILING is reached by REJECT LETTER or by PAYMENT but never by both (XOR-join), and ESTIMATE is entered from REJECTION? or from REVISION: possibly executed, but not always both. The graph as drawn does not say which fork or join is which: the semantics lives in the analyst's head, not in the diagram. A formal notation has to remove this ambiguity.

3.10 Disambiguation

The fix is to type every split and join with a *gateway*, a notation borrowed from BPMN: a diamond marked + for parallel (AND) splits and joins, a diamond marked × for choice (XOR) splits and joins. The routing logic moves from the analyst's head into explicit nodes (one gateway per fork and join) and the arrows regain a uniform meaning:



Reading the gateways: the + pair encloses the parallel checks; the first × routes to REJECT LETTER or ESTIMATE; a second × joins the loop back-edge from REVISION into ESTIMATE; a third splits after ASSESSMENT? into repeat or exit; the last one merges the two alternative paths into FILING.

Adding a start event and an end event, and enclosing the flow in a *pool* labelled with the responsible organisation, turns the diagram into valid BPMN 2.0: drawable in any editor,¹¹ exchangeable as .bpmn XML, executable by a compliant engine. A pool encloses a single participant of the

¹¹<https://camunda.com/download/modeler/>; <https://bpmn.io/>

choreography; inter-organisational interaction is modelled by adding more pools connected by message flows, a construct introduced in the next chapters. Tasks may further carry BPMN task-type markers.

The same control flow renders equally well in **EPC** (Event-driven Process Chain) syntax: tasks become rounded rectangles, events become hexagons, and the same $+/×$ connectors type the forks and joins.¹²

In **workflow-net** form the encoding is structural: tasks become square transitions, branching points become explicit places, an XOR fork is several transitions competing for one place, an AND fork is one transition with several output places.¹³ The net has a single source place (**start**) and a single sink place (**end**). This is the formal counterpart that the Petri-net chapters analyse rigorously.

3.11 Ingredients

Any graphical process language must provide:

- **nodes** for start and end: one source and one sink per process;
- **nodes** for tasks: the activities the process is made of;
- **nodes** for joins and splits, distinguishing concurrency (AND), internal decisions (XOR resolved by the system), and external decisions (XOR resolved by the environment: split only);
- **edges** for the links carrying causal and temporal dependencies.

Three further dimensions remain open: *responsibility* (who owns the process and each task), *information* (the data flowing along the edges), *platform* (bindings to concrete services and runtimes), and the BPMN, EPC, and BPEL chapters fill them in.

Concrete notations are points in a design space: events as circles, stars, or hexagons; tasks as rectangles, ellipses, or gears; gateways as diamonds, rings, or bracket icons; links as solid, dashed, or double-headed arrows. No choice is better *a priori*, as long as it is unambiguous and consistent.

Example (Exercise: design a notation). Invent your own diagrammatic notation to describe the following interaction protocol; choose symbols, shapes, and colours carefully. *Alice wants to sell her car, Bob is interested in buying it. Alice asks some quote. Bob can accept the bargain, refuse it, or make a counteroffer. Alice can accept or make a counteroffer, and so on, until either the bargain is accepted or refused.*

The winning entry of the 2024/25 edition of this exercise was a two-lane sequence diagram: one vertical lifeline per participant, the message types (MAKES AN OFFER, COUNTEROFFER, ACCEPTS, REFUSES) drawn as arrows between the lifelines and colour-coded by originator, and a shared END node closing the protocol. The bargaining loop emerges from the arrow topology alone, with no explicit gateway.

¹²Drawn in ARIS Express: <https://ariscommunity.com/aris-express>

¹³Drawn in WoPeD: <http://woped.dhbw-karlsruhe.de/>

3.12 A sneak peek at standards

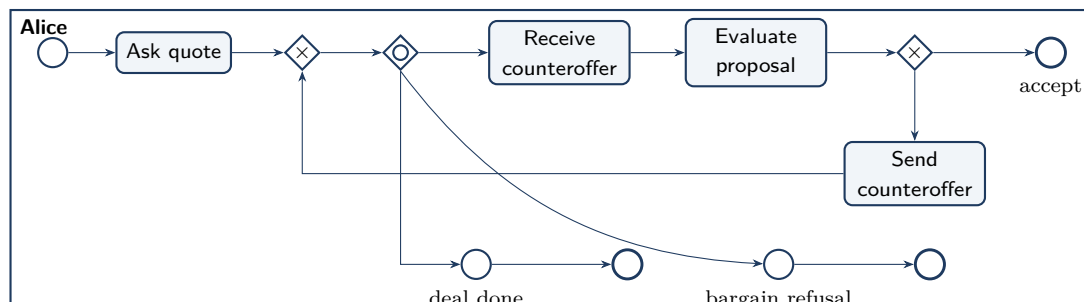
The notations studied in this course (BPMN, EPC and workflow nets) agree on the ingredient list but disagree on the symbols:

	BPMN	EPC	WfN
start / end	circle	hexagon	circle (place)
join / split	diamond	circle	square (transition)
tasks	rounded rect.	rounded rect.	square (transition)
links	solid arrow	dashed arrow	solid arrow

The course's notational baseline is BPMN; EPC and workflow nets enter the picture in the analysis chapters.

3.13 Alice–Bob car selling in BPMN

As a BPMN collaboration, each participant gets a pool, and the offer/counteroffer/accept/refuse exchange travels on message flows between the pools. Alice's pool is sketched below; Bob's mirrors it, and the message flows between the two are omitted:



After ASK QUOTE, Alice does not decide internally what happens next: she waits for whichever message arrives first: a counteroffer to evaluate, an acceptance, a refusal. The diamond with the inner circle is BPMN's *event-based gateway*: the routing decision is taken by the environment, not by the process. If Alice evaluates a counteroffer, she either accepts (closing the deal) or sends a counteroffer of her own and waits again. The event-based gateway realises the *deferred choice*: the external-decision ingredient listed above; internal and external choice return rigorously with Petri nets in the next chapter.

4 Petri nets basics

4.1 Object

This chapter is an informal introduction to *Petri nets* and to the restricted subclass of *workflow nets*. The reference text is Ch. 4.4 of Weske's *Business Process Management: Concepts, Languages,*

Architectures.¹⁴ The opening example is the credit-card order-handling workflow net studied in full at the end of the chapter: single source place, single sink place, and the transitions of an order-and-payment process: the shape of the artefact we are about to define.

4.2 An important difference

Two pictures justify the move to a new formalism. A floor plan with evacuation arrows is a static spatial map: it tells *where* things are. The BPMN diagram of the twelve-task insurance-claim process from the previous chapter is procedural: it tells *what* the process does. What BPMN cannot answer is a third kind of question: at any given moment, *where* are the running cases? No graphical artefact points at the activity each instance is sitting on, whether the model is drawn in Camunda's editor¹⁵ or at bpmn.io.¹⁶ Petri nets provide exactly that, through the explicit notion of *marking*.

4.3 Basic definitions

A Petri net is built from just three ingredients: two kinds of node, one kind of edge, and the tokens that move across them. This subsection introduces them and the bipartite structure they obey.

Petri nets are named after Carl Adam Petri, who introduced them in his 1962 PhD thesis *Kommunikation mit Automaten*.^{17,18} The field went through three phases. In the 1960s and 1970s the focus was on the theory of the new formalism. In the 1980s tools and applications took the lead, and the notation entered industry. From the 1990s on, Petri nets have been applied in many fields, from manufacturing and logistics to biochemistry and concurrent-systems verification. The success of the model rests on how little it takes: two kinds of nodes, one kind of edge, and a token-passing rule are enough to describe a wide variety of concurrent behaviours.

4.3.1 Petri nets for us

For this course Petri nets play the role of a **formal** and **abstract** business-process specification.

- **Formal**: the semantics of process instances becomes well defined and unambiguous; given a marking and a firing rule, the behaviour of the net is fixed, with no room for interpretation.
- **Abstract**: the execution environment is disregarded: we model what happens, not who or what physically performs it. The control logic is one concern (the Petri net); the resources, data, and platform are other concerns layered on top of it.

4.3.2 Places, transitions, tokens, arcs

A Petri net is built from two kinds of nodes. The first is the **transition**, the *active* component: it represents an event the system can perform. Depending on the modelling domain a transition may stand for an operation, a calculation, an evaluation, a transformation, a transportation,

¹⁴Weske M., *Business Process Management: Concepts, Languages, Architectures*, Springer, 2007, Ch. 4.4.

¹⁵<https://camunda.com/download/modeler/>

¹⁶<https://bpmn.io/>

¹⁷Petri C.A., *Kommunikation mit Automaten*, PhD thesis, 1962.

¹⁸http://www.informatik.uni-hamburg.de/TGI/mitarbeiter/profs/petri_eng.html

a task, an activity, a decision, and so on: the formal model is silent on the interpretation, all transitions are graphically identical. The symbol for a transition is a square:



The second kind of node is the **place**, the *passive* component of the net. A place represents a condition under which the system may sit, and depending on the domain it may stand for a state, a medium, a buffer, a condition, a repository of resources, a type, a memory location, and so on. The graphical symbol for a place is a circle:



Places are passive but not empty: each place may contain a number of **tokens** that record the current state of the net. A token represents an instance of whatever the place stands for: a physical object, a piece of data, a record, a resource, an activation mark, a message, a document, a process case, a value. Tokens are drawn as small filled dots inside the circle:



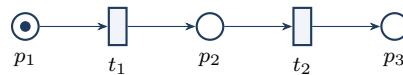
In the analogy with maps, a token is the “you are here” marker: it pinpoints a particular instance inside the abstract structure given by places and transitions.

The two kinds of nodes are connected by **arcs**, which carry directed dependencies. Two arc patterns are admitted:

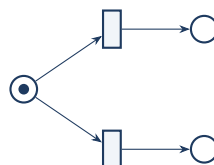
- $p \rightarrow t$: firing t *consumes* a token from p ;
- $t \rightarrow p$: firing t *produces* a token in p .

Definition (Bipartite structure). Arcs always go between a place and a transition. Place-to-place arcs and transition-to-transition arcs are **not admitted**. A Petri net is therefore a directed bipartite graph.

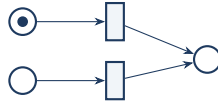
Example (Pattern: sequence). The linear chain $p_1 \rightarrow t_1 \rightarrow p_2 \rightarrow t_2 \rightarrow p_3$ with one token in p_1 models a strictly sequential two-step procedure: firing t_1 moves the token from p_1 to p_2 , then t_2 can fire and move it to p_3 .



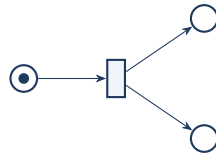
Example (Pattern: XOR-split). From p_1 (with one token) two transitions t_1 and t_2 are both enabled: whichever fires first consumes the single token, so the other becomes disabled. The result is an exclusive choice between two downstream branches. At the Petri-net level the choice is purely non-deterministic.



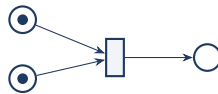
Example (Pattern: XOR-join). Two input places p_1 and p_2 each feed a distinct transition (t_1 and t_2 respectively), and both transitions produce into the same place p_3 . Whichever input place gets a token first lets the corresponding transition fire, merging the alternative paths into a common continuation.



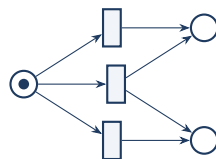
Example (Pattern: AND-split). From one input place p_1 a single transition t_1 produces into two output places p_2 and p_3 simultaneously. Firing t_1 consumes one token from p_1 and deposits one token in each of p_2 and p_3 , so the two downstream branches become concurrently active.



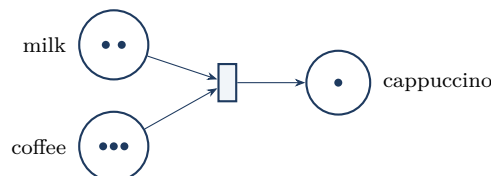
Example (Pattern: AND-join). Two input places p_1 and p_2 feed a single transition t_1 that produces into p_3 . The transition is enabled only when *both* input places hold a token; firing consumes one token from each and produces one in p_3 . This is the synchronisation construct: it waits for both branches before continuing.



Example (Pattern: OR-split (inclusive choice)). From p_1 (with one token) three transitions are simultaneously available: t_2 produces into p_2 only, t_3 produces into p_3 only, and t_1 produces into both p_2 and p_3 . The net thus expresses the inclusive disjunction “ p_2 or p_3 or both”: choosing t_1 enables both branches at once (an AND-split in disguise), while t_2 and t_3 act as the XOR alternatives that select exactly one branch.



Example (Cappuccino). Tokens also carry the intuition of *consumable resources*. Two input places, MILK (holding two tokens) and COFFEE (holding three tokens), both feed a single transition; firing the transition consumes *one* milk token and *one* coffee token and produces one CAPPUCCINO token. The recipe is encoded directly in the bipartite structure: the inputs of a transition are the resources it needs, the outputs are the resources it yields.



Definition (Petri net). A **Petri net** is a tuple (P, T, F, M_0) where:

- P is a finite set of *places*;
- T is a finite set of *transitions*, with $P \cap T = \emptyset$;
- $F \subseteq (P \times T) \cup (T \times P)$ is the *flow relation* (the arcs);
- $M_0 : P \rightarrow \mathbb{N}$ is the *initial marking*.

The constraints encode the bipartite structure: places and transitions are disjoint, and arcs only connect a place to a transition or a transition to a place. The initial marking M_0 assigns to each place the number of tokens it holds at start; its dynamic meaning is made precise below.

Definition (Pre-set of a transition). A place p is an *input place* for transition t iff $(p, t) \in F$. The set of input places of t is denoted $\bullet t$ and called the **pre-set** of t :

$$\bullet t = \{p \in P \mid (p, t) \in F\}.$$

Tokens needed by t to fire are drawn from $\bullet t$.

Definition (Post-set of a transition). A place p is an *output place* for transition t iff $(t, p) \in F$. The set of output places of t is denoted $t\bullet$ and called the **post-set** of t :

$$t\bullet = \{p \in P \mid (t, p) \in F\}.$$

Tokens produced by firing t land in $t\bullet$.

The two notations are visually symmetric: the dot sits on the input side of t in $\bullet t$ and on the output side in $t\bullet$.

The dot notation extends naturally to places. Given a place p :

- $\bullet p$ is the set of transitions that share p as an output place: the transitions tokens in p *can come from*;
- $p\bullet$ is the set of transitions that share p as an input place: the transitions tokens in p *can go to*.

The definition mirrors the one for transitions, with the roles of places and transitions swapped.

Definition (Pre-set and post-set, uniform form). For any node $x \in P \cup T$:

$$\bullet x = \{y \mid (y, x) \in F\}, \quad x\bullet = \{y \mid (x, y) \in F\}.$$

The two formulas apply uniformly to places and to transitions. The bipartite constraint then forces $\bullet p \subseteq T$ and $p\bullet \subseteq T$ when p is a place, and $\bullet t \subseteq P$ and $t\bullet \subseteq P$ when t is a transition: exactly the dual reading given above.

4.4 Token game

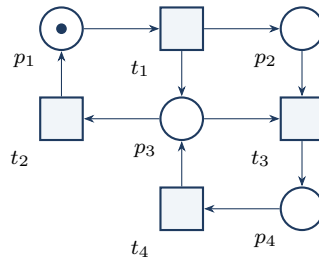
Definition (Enabling and firing). A transition t is **enabled** if each of its input places contains at least one token. When an enabled transition **fires**, it *consumes* one token from each input place and *produces* one token into each output place.

Note (Remarks on firing).

- **Atomicity:** a firing is an atomic action: the net never lingers in a half-fired state.
- **Interleaving semantics:** multiple transitions may be enabled at the same time, but only one fires at a time. Concurrency is rendered as the set of all possible interleavings of independent firings.
- **Static structure, variable population:** the net itself (P, T, F) is fixed, but the total number of tokens may grow or shrink over time. Firing a transition t adds $|t \bullet| - |\bullet t|$ tokens to the system, which can be positive, zero, or negative.

Example (Token game (start)). A running example illustrates the firing rule. The net has four places p_1, p_2, p_3, p_4 and four transitions t_1, t_2, t_3, t_4 , arranged in a grid with p_3 at the centre: t_1 consumes from p_1 and produces into *both* p_2 and p_3 ; t_3 consumes from *both* p_2 and p_3 and produces into p_4 ; t_2 moves a token from p_3 back to p_1 ; t_4 moves a token from p_4 back to p_3 . The initial marking puts a single token in p_1 , written $M_0 = p_1$. Only t_1 is enabled. We track the run as a sequence alternating markings and fired transitions:

$$p_1 / t_1 / \dots$$



Two runs from M_0 show the mechanics:

Fired	New marking	Enabled
—	p_1	t_1
t_1	$p_2 + p_3$	t_2, t_3
t_2	$p_1 + p_2$	t_1
t_1	$2 \cdot p_2 + p_3$	t_2, t_3
t_3	$p_2 + p_4$	t_4
t_4	$p_2 + p_3$	t_2, t_3

Three observations. The choice between t_2 and t_3 after t_1 is non-deterministic. The marking $2 \cdot p_2 + p_3$ puts two tokens in p_2 : a marking is in general a *multiset* over P , not a subset; and firing t_3 there consumes one token from p_2 *and* one from p_3 , leaving $p_2 + p_4$. The run is cyclic: $p_2 + p_3$ reappears and the same choice opens again. Taking t_3 at the first opportunity gives the second run, $p_2 + p_3 / t_3 / p_4 / t_4 / p_3 / t_2 / p_1$: back to the initial marking. Each such alternating sequence of markings and transitions is a *run* of the net.

4.5 Workflow nets

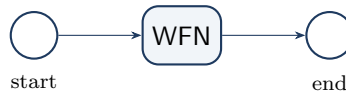
4.5.1 Features

Workflow nets are a restricted subclass of Petri nets, tailored to business-process modelling. Four features characterise them:

- **Tailored to the representation of business processes:** the structural restrictions match the lifecycle of a process instance: a single start, a single end, and a path connecting them.
- **Formal, unambiguous semantics:** workflow nets inherit the token-game semantics of Petri nets, so every modelled process has a precise meaning.
- **Structural restrictions:** the net topology must respect the workflow-net axioms (a unique source, a unique sink, every node on a path between them).
- **Decorated graphical representation:** transitions and arcs can carry additional decorations (split/join roles, triggers) that BPMN-style readers can interpret directly.

4.5.2 The intuition

The simplest way to think of a workflow net is the cloud picture: a *start* place on the left, an *end* place on the right, and an opaque “WFN” cloud in the middle that contains all the internal places, transitions, and arcs needed to do the work.



A token entering at *start* represents one new process instance; the token reaching *end* represents that instance being completed.

Definition (Workflow net). A Petri net (P, T, F) is a **workflow net** (WFN) iff:

1. there is a distinguished *initial place* $i \in P$ with $\bullet i = \emptyset$ (no incoming arcs);
2. there is a distinguished *final place* $o \in P$ with $o \bullet = \emptyset$ (no outgoing arcs);
3. every other place and every transition belongs to at least one path from i to o .

The pre/post-set notation defined above is what conditions (1) and (2) rely on: an initial place is one whose pre-set is empty; a final place is one whose post-set is empty.

Note (Basic property: uniqueness of the initial node). Lemma. In a workflow net there is a *unique* node with no incoming arc.

Proof. Let i be the initial place and o the final one. Suppose, for contradiction, that some other node v has $\bullet v = \emptyset$. By condition (3) of the workflow-net definition, v lies on some path from i to o ; since v has no incoming arc, it must be the *first* node of that path. The first node of a path from i is i itself, hence $v = i$. $\square$

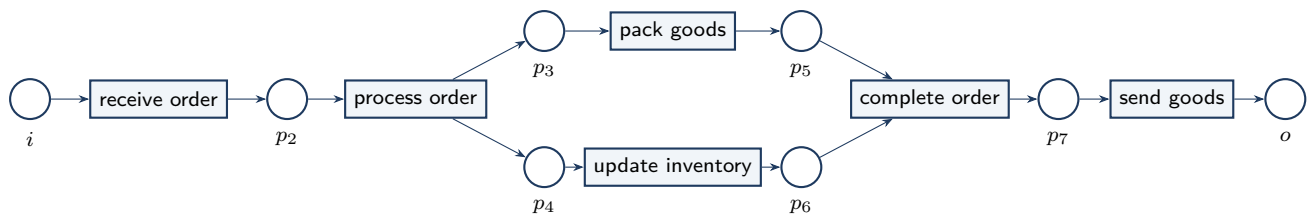
Note (Basic property: uniqueness of the final node). Lemma. In a workflow net there is a *unique* node with no outgoing arc: namely the final place o . The proof is analogous to the previous lemma, swapping pre-set for post-set and reading the path from o backwards.

Note (Rationale of the workflow-net conditions). The three structural conditions encode a precise business-process intuition:

1. a token in i represents a process instance that has not yet started;
2. a token in o represents a finished case;

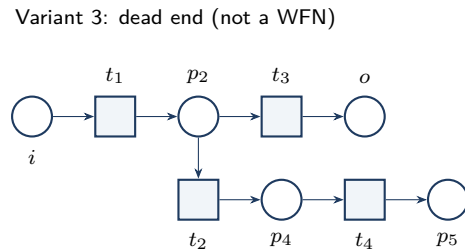
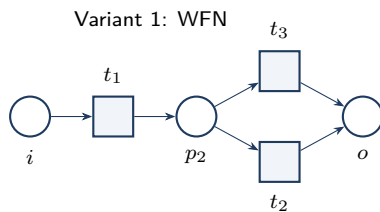
- every other place and every transition lies on a path from i to o , so each node can take part in some case: there are no orphan places or transitions disconnected from the workflow.

Example (Workflow net: example). An order-handling process modelled as a workflow net. The token enters at i ; RECEIVE_ORDER produces into p_2 ; PROCESS_ORDER is an AND-split that produces tokens into p_3 and p_4 simultaneously; the two branches run PACK_GOODS (producing into p_5) and UPDATE_INVENTORY (producing into p_6); the AND-join COMPLETE_ORDER synchronises p_5 and p_6 and produces into p_7 ; finally SEND_GOODS delivers the token to o . Eight places, six transitions, single source, single sink, every node on some path from i to o .

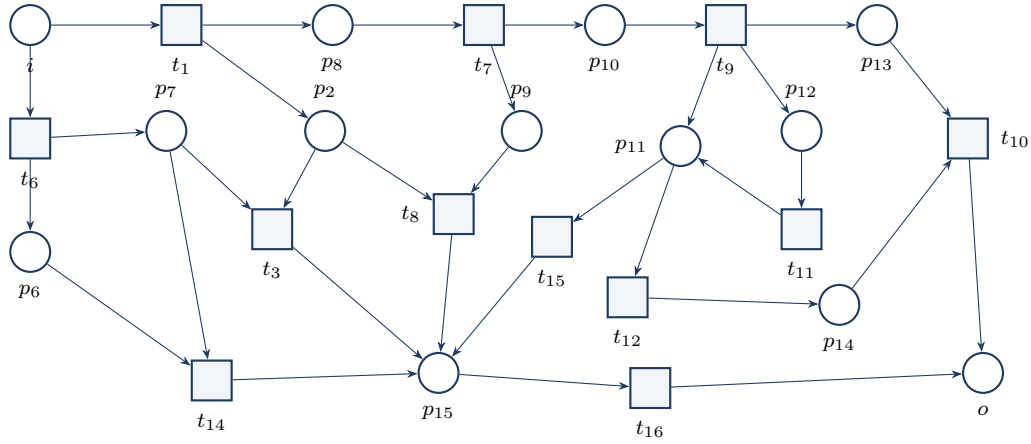


Example (Question time: four variants). Four small nets test the definition.

- Places i, p_2, o ; arcs $i \rightarrow t_1 \rightarrow p_2$, $p_2 \rightarrow t_3 \rightarrow o$, $p_2 \rightarrow t_2 \rightarrow o$. **WFN**: $\bullet i = \emptyset$, $o \bullet = \emptyset$, and every node lies on a path such as $i \rightarrow t_1 \rightarrow p_2 \rightarrow t_2 \rightarrow o$.
- Add a back-arc $t_2 \rightarrow i$. **Not a WFN**: i now has an incoming arc, so there is no proper initial place, however well-connected the net is.
- Extend the first net instead with a side branch $p_2 \rightarrow t_2 \rightarrow p_4 \rightarrow t_4 \rightarrow p_5$ ending in the dangling place p_5 . **Not a WFN**: t_2, p_4, t_4, p_5 lie on no path from i to o : a dead end violates condition (3).
- Close the dead end with a new transition t_5 from p_5 back to p_2 . **WFN again**: every loop node now reaches o , e.g. via $i \rightarrow t_1 \rightarrow p_2 \rightarrow t_2 \rightarrow p_4 \rightarrow t_4 \rightarrow p_5 \rightarrow t_5 \rightarrow p_2 \rightarrow t_3 \rightarrow o$.



Example (Question time: is this a WFN?). A larger candidate: thirteen places and twelve transitions (the node numbering, with gaps, follows the original drawing) wired in a left-to-right flow with several branches. Is it a workflow net?

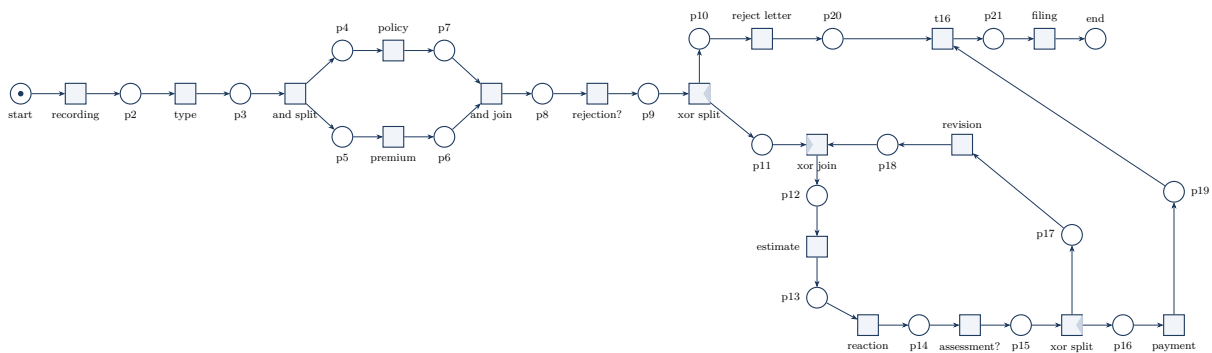


i has no incoming arc and o no outgoing one, so conditions (1) and (2) hold. Condition (3) asks that every node lie on *some* path from i to o , not that one path cross them all. Most nodes are covered at a glance, e.g. by $i \rightarrow t_1 \rightarrow p_8 \rightarrow t_7 \rightarrow p_{10} \rightarrow t_9 \rightarrow p_{13} \rightarrow t_{10} \rightarrow o$ and its branch variants. The delicate spot is t_3 : without the arc $t_3 \rightarrow p_{15}$, no path through t_3 (or reaching it via p_2 or p_7) could continue to o , and the net would *not* be a workflow net. With that arc in place (drawn in the figure), $i \rightarrow t_6 \rightarrow p_7 \rightarrow t_3 \rightarrow p_{15} \rightarrow t_{16} \rightarrow o$ covers the critical nodes, and the verdict is **yes**.

4.6 WoPeD

The course's reference tool for designing and analysing workflow nets is **WoPeD**, the Workflow Petri Net Designer¹⁹: a free desktop application offering a graphical editor, a transition-properties dialog, and automated analyses such as coverability-graph generation.

Example (Is it a Wf net?). A real-world workflow net for an insurance-claim process, drawn in WoPeD. After RECORDING and TYPE, an AND-split runs POLICY and PREMIUM in parallel; after the AND-join, REJECTION? reaches an XOR-split that either issues a REJECT LETTER or enters an assessment loop: ESTIMATE, REACTION, ASSESSMENT?, then a second XOR-split that either proceeds to PAYMENT or sends the case back through REVISION to the XOR-join at the loop entry. Both outcomes feed t_{16} and the case ends with FILING. Hatched halves mark the XOR decorations (split on the output side, join on the input side). Is it a workflow net? Verification is left as a WoPeD exercise.²⁰



¹⁹<http://woped.dhbw-karlsruhe.de/woped/>

²⁰<http://woped.dhbw-karlsruhe.de/>

4.7 Syntax sugar

Note (Transition decorations in WoPeD). WoPeD's transition-properties dialog lets a transition carry a *branching role* on top of its plain firing behaviour: NONE, AND-SPLIT, XOR-SPLIT, AND-JOIN, XOR-JOIN, and the mixed forms XOR-JOIN-SPLIT, AND-JOIN-XOR-SPLIT, AND-JOIN-SPLIT, XOR-JOIN-AND-SPLIT. The decoration is purely *syntax sugar*: each decorated transition stands for a sub-net of plain Petri-net transitions and places. The expansions are spelled out below.²¹

4.7.1 Sugar for split

A single decorated transition t_1 with two output places can stand for two different patterns depending on the split type:

- **AND-split.** t_1 behaves like a plain transition with two outgoing arcs: firing produces tokens in both p_2 and p_3 simultaneously. The sugar form is a single decorated transition; the unfolded form is the same transition with two output arcs.
- **XOR-split.** t_1 stands for two distinct transitions t_{1a} and t_{1b} that share the same input place p_1 but produce into different outputs ($t_{1a} \rightarrow p_2$, $t_{1b} \rightarrow p_3$). Either of them fires, never both at the same firing.

The decoration is a shortcut, not new semantics.

4.7.2 Sugar for join

Symmetric to the split sugar:

- **AND-join.** A decorated transition t_1 with two input places p_1 and p_2 stands for a plain transition with two incoming arcs: firing consumes one token from each and produces into p_3 .
- **XOR-join.** t_1 stands for two distinct transitions t_{1a} and t_{1b} : t_{1a} consumes from p_1 and produces into p_3 , t_{1b} consumes from p_2 and produces into p_3 . Either of the two input places, when marked, lets the corresponding transition fire.

4.7.3 Mixed sugar

The two kinds of sugar combine freely on the same transition. A four-port decorated transition t_1 with input places p_1, p_2 and output places p_3, p_4 , standing for an AND-JOIN on the input side and an XOR-SPLIT on the output side, admits two equivalent expansions:

- with an *intermediate place*: t_{1a} consumes $\{p_1, p_2\}$ and produces into a fresh place pt_1 ; from there t_{1b} produces into p_3 and t_{1c} into p_4 ;
- without: two transitions, each consuming $\{p_1, p_2\}$ and producing into one of p_3 or p_4 .

Both expansions realise the same behaviour.

Note (A personal remark on syntax sugar). The decorations carry three caveats worth spelling out before relying on them:

- the chosen glyphs for AND-join and XOR-join are visually too similar (a small vertical bar versus a hatched bar) and can be confused at first reading;

²¹<http://woped.dhbw-karlsruhe.de/woped/>

- the same glyph placed on the input side or on the output side of a transition denotes *different* patterns (join versus split), so position matters;
- the AND decoration is in fact *redundant*: a plain transition with multiple input/output arcs already behaves as an AND-join / AND-split, so the dedicated AND-bar adds nothing semantically.

Note (Why the decoration sits on the transition). The decoration is a back-port from BPMN, where the corresponding split/join roles are filled by dedicated *gateway* nodes drawn as diamonds: a + diamond for AND, a × diamond for XOR. In Petri nets the gateway is absorbed into the transition, but the conceptual role is the same.

Note (Convention adopted in the course). To avoid any source of confusion, going forward the AND-join and AND-split decorations are **abandoned**: AND-patterns are always drawn as plain transitions with multiple input/output arcs. Only the XOR-join and XOR-split decorations are retained as legitimate syntax sugar, because their unfolding is genuinely non-trivial.

4.8 Subprocesses

Example (WoPeD: subprocess). WoPeD supports hierarchical modelling: a transition of a parent workflow net (drawn as a *double square*) can be opened as a separate workflow net of its own. A transition is therefore not necessarily atomic: it can be *implemented* by another WFN, opaque from the outside.²²

Note (Hierarchical structuring). The uniqueness of the entry and exit points in a workflow net is exactly what makes hierarchical structuring possible. A transition labelled BUSINESS PROCESS A can be *realised* by an entire workflow net *A*: the single input arc of the transition becomes the entry into *A*'s initial place; the single output arc of the transition becomes the exit from *A*'s final place. From the outside the sub-process looks like a single atomic step; from the inside it is a workflow net with its own structure.

4.9 Some patterns

4.9.1 Typical control-flow aspects

The catalogue below collects the typical control-flow patterns, each implemented as a workflow-net fragment:

- **Sequencing**: one task after another;
- **Parallelism**: AND-split followed by AND-join;
- **Selection**: XOR-split followed by XOR-join;
- **Iteration**: XOR-join followed by XOR-split with a back-edge;
- **Capacity constraints**: feedback loop, mutual exclusion, alternation.

Example (Pattern: sequencing). “*B* is executed after *A*” is realised by the linear workflow net $i \rightarrow A \rightarrow p \rightarrow B \rightarrow o$. The middle place *p* holds the token between the two transitions, so *B* cannot fire until *A* has produced into *p*. Three places, two transitions, one source, one sink: the smallest non-trivial workflow net.



²²<http://woped.dhbw-karlsruhe.de/woped/>

Example (Pattern: parallelism (AND-split + AND-join)). To run A and B both, in no particular order, the workflow net puts one AND-split transition right after i and one AND-join transition right before o :

$$i \rightarrow \text{AND-split} \rightarrow \{p_A, p_B\}; \quad p_A \rightarrow A \rightarrow q_A; \quad p_B \rightarrow B \rightarrow q_B; \quad \{q_A, q_B\} \rightarrow \text{AND-join} \rightarrow o.$$

The AND-split produces tokens into both p_A and p_B , so the two branches become concurrently active. The interleaving semantics of the token game allows them to fire in any order or interleaved; the AND-join waits until both q_A and q_B are marked before producing the final token in o .

Example (Pattern: explicit choice (XOR-split + XOR-join)). To execute *either* A *or* B , never both, a single XOR-split transition after i routes the token to one of the two branches; an XOR-join before o merges the alternative outputs:

$$i \rightarrow \text{XOR-split} \rightarrow \{p_A, p_B\}; \quad p_A \rightarrow A \rightarrow q_A; \quad p_B \rightarrow B \rightarrow q_B; \quad \{q_A, q_B\} \rightarrow \text{XOR-join} \rightarrow o.$$

The choice is **explicit**: the routing transition is the place where the decision is made. Per instance, exactly one of the two branches fires.

Example (Pattern: deferred choice (implicit XOR)). Instead of inserting a routing transition, i directly feeds both A and B : the two transitions compete on the single token in i . Whichever fires first consumes the token, so the other becomes disabled. The branches reconverge at an XOR-join into o .

$$i \rightarrow \{A, B\}; \quad A \rightarrow p_A \rightarrow \text{XOR-join} \rightarrow o; \quad B \rightarrow p_B \rightarrow \text{XOR-join} \rightarrow o.$$

The choice is now **implicit**: there is no routing transition that “decides”; the decision falls out of the interleaving semantics and of which environment event triggers A or B first.

Note (Explicit choice $\neq$ implicit choice). The two choice patterns are not interchangeable: the *decision is made at different points in time*.

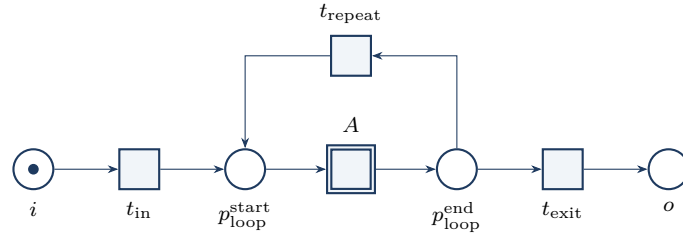
- Explicit choice (XOR-split + XOR-join). The XOR-split transition T **chooses** upfront, before A or B can start. In BPMN this maps to the **XOR gateway** ($\times$ diamond).
- Deferred choice. A and B are both pre-enabled; whichever fires first *is* the choice. “A or B chooses”. In BPMN this maps to the **event-based gateway** (pentagon-in-diamond).

The two constructs lead to identical *traces* but to different *behavioural* properties: in particular when external triggers are attached to A and B , the deferred-choice variant is the one that lets the environment decide.

Example (Pattern: one-or-more iteration (XOR-join + XOR-split)). To execute A *at least once*, and possibly several times, the loop sandwiches A between a join place and a split place. The token enters from i via t_{in} ; place $p_{\text{loop}}^{\text{start}}$ feeds A ; the output place $p_{\text{loop}}^{\text{end}}$ then offers an XOR choice: *either* fire t_{repeat} (a back-arc into $p_{\text{loop}}^{\text{start}}$, re-enabling A), *or* fire t_{exit} into o .

$$i \rightarrow t_{\text{in}} \rightarrow p_{\text{loop}}^{\text{start}} \rightarrow A \rightarrow p_{\text{loop}}^{\text{end}} \rightarrow \{t_{\text{repeat}}, t_{\text{exit}}\}; \quad t_{\text{repeat}} \rightarrow p_{\text{loop}}^{\text{start}}; \quad t_{\text{exit}} \rightarrow o.$$

The XOR-join at $p_{\text{loop}}^{\text{start}}$ accepts the token from either t_{in} or t_{repeat} . The XOR-split at $p_{\text{loop}}^{\text{end}}$ chooses whether to loop or to exit. The exit transition can fire only *after* A has fired at least once, so the iteration is guaranteed to execute at least one round.

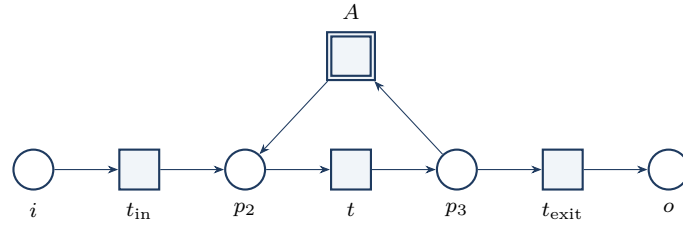


Example (One-or-more iteration: sugared). The same topology, drawn with the XOR decorations introduced in the syntax-sugar subsection: the inbound transition is a hatched XOR-join, the outbound transition is a hatched XOR-split, and the loop-back arc goes through a single intermediate place at the top of the diagram. The semantics is identical to the plain version above; the syntax is more compact.

Example (Pattern: zero-or-more iteration). To allow *zero or more* executions of A (possibly none), A sits on the *back edge* of the loop, not on the forward path:

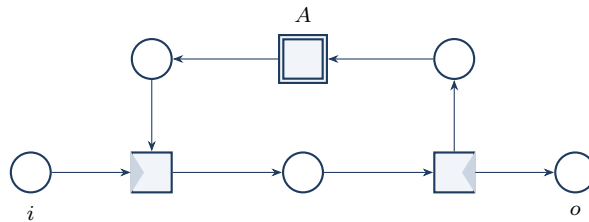
$$i \rightarrow t_{\text{in}} \rightarrow p_2; \quad p_2 \rightarrow t \rightarrow p_3; \quad p_3 \rightarrow A \rightarrow p_2; \quad p_3 \rightarrow t_{\text{exit}} \rightarrow o.$$

When p_3 holds a token, both A and t_{exit} are enabled. The token can fire t_{exit} immediately (zero executions of A), or cycle through A and t any number of times before exiting. The XOR is *deferred*: realised by the competition between A and t_{exit} on the single token in p_3 .



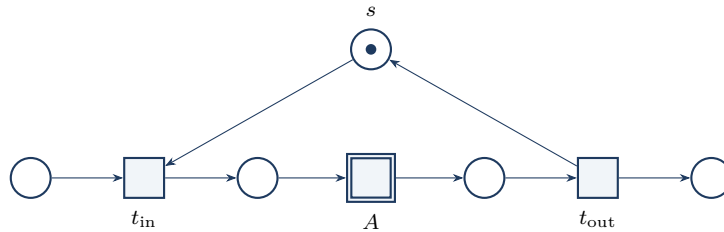
The double square marks A as a subprocess (see the Subprocesses subsection). The same glyph recurs in all the pattern figures below.

Example (Zero-or-more iteration: sugared). The decorated version makes the routing explicit. The *inbound* transition is an XOR-join: it accepts the token either from i or from the loop-back place. The *outbound* transition is an XOR-split: it routes the token either to o or back through A . The loop runs right-to-left across the top: XOR-split $\rightarrow$ place $\rightarrow A \rightarrow$ place $\rightarrow$ XOR-join. The unfolding realises the same zero-or-more behaviour as the plain net above.

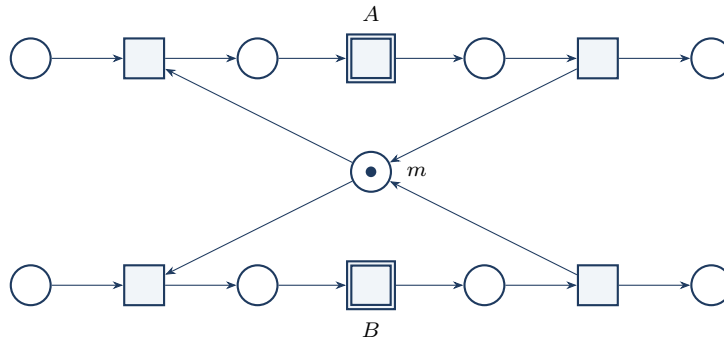


Example (Capacity constraint: one serve per time). When many activations may reach the subprocess A , but A must serve them *one at a time*, the standard pattern uses a binary semaphore: a shared place s holding exactly one token (the “capacity”). The place is wired to the transitions *bracketing*

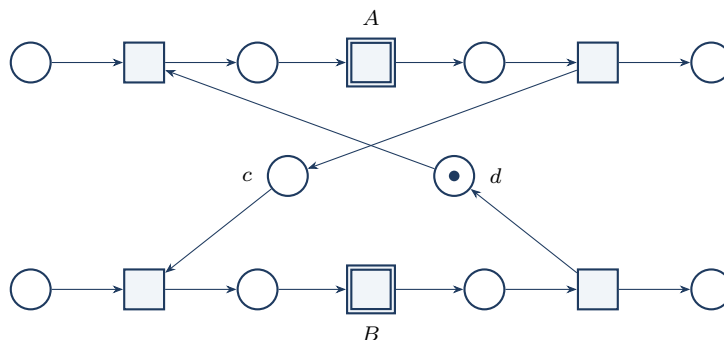
A , not to A itself: the entry transition t_{in} consumes the capacity token together with the activation, and the exit transition t_{out} returns it when A 's run completes. Between t_{in} and t_{out} the capacity is held and no second activation can enter. The bracketing is what makes the pattern meaningful: A is a subprocess, hence non-atomic, while a plain transition fires atomically and could never be observed “busy”.



Example (Capacity constraint: mutual exclusion). The same idea generalises to two subprocesses A and B that must not execute concurrently. Each runs in its own chain, and a shared *mutex* place m holding one token sits between them, wired crosswise to the bracketing transitions: the entry transition of each chain consumes the mutex token, the exit transition returns it. As long as the token is held by one chain, the other chain's entry is disabled.



Example (Capacity constraint: alternation). A and B execute one time each, A first. The wiring follows the mutual-exclusion pattern, but with *two* control places: d (initially marked) enables the entry of the A -chain; the exit of the A -chain produces into c , which enables the entry of the B -chain; the exit of the B -chain produces back into d . With the control token starting in d , the A -chain must complete before the B -chain can start.

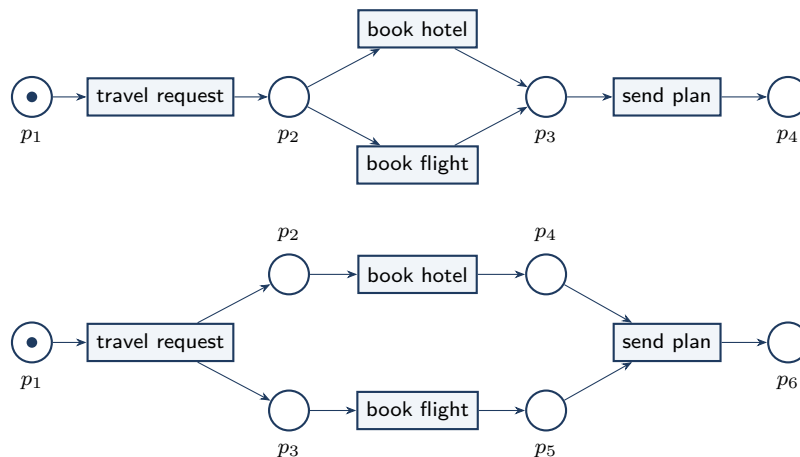


Example (Which patterns?). The insurance-claim WoPeD net of the “Is it a Wf net?” example above returns as an exercise in pattern recognition: every fork and join in it can be classified against the catalogue: sequencing along the main chain, parallelism between POLICY and PREMIUM, selection at the XOR-split after REJECTION?, iteration in the REVISION loop.²³

²³<http://woped.dhbw-karlsruhe.de/>

Example (Question time: which model is more reasonable?). Two candidate workflow nets for the same travel-booking process ($\text{TRAVEL_REQUEST} \rightarrow \text{BOOK_HOTEL}$ and $\text{BOOK_FLIGHT} \rightarrow \text{SEND_PLAN}$).

- **First model.** Place p_2 feeds both BOOK_HOTEL and BOOK_FLIGHT , and both produce into the shared place p_3 , an *implicit* (deferred) choice: the two transitions compete on the single token, so exactly *one* booking happens per case.
- **Second model.** TRAVEL_REQUEST is an AND-split producing into two parallel places; the bookings run concurrently and their output places synchronise on SEND_PLAN . *Both* bookings happen per case.



Which one is more reasonable? The second. A travel booking needs both the hotel and the flight reserved before the plan can be sent: the two tasks are independent and complementary, not mutually exclusive. The first model would book only one of the two and ship the case as “planned”.

4.10 Triggers

Note (Trigger decoration in WoPeD). The transition-properties dialog in WoPeD carries a second panel below the branching modes: the *trigger*. Four trigger types are available (AUTOMATIC, RESOURCE, MESSAGE, and TIME), each with its own icon. Their formal meaning follows.²⁴

Definition (Triggers). Execution constraints can depend on the *environment* in which a process is enacted. In a workflow net, transitions can be decorated with information on **who** or **what** is responsible for firing them; such annotations are called **triggers**.

Note (Kinds of triggers). A trigger can be one of three kinds:

- a **human interaction** (the activity is started by a user);
- the **receipt of a message** (the activity is started by an external event);
- the **expiration of a time-out** (the activity is started when a timer elapses).

A transition with no trigger fires *automatically* as soon as it is enabled: this is the default behaviour assumed throughout the chapter so far.

Note (Symbols for triggers). The four trigger types each carry a small icon placed on top of the transition box:

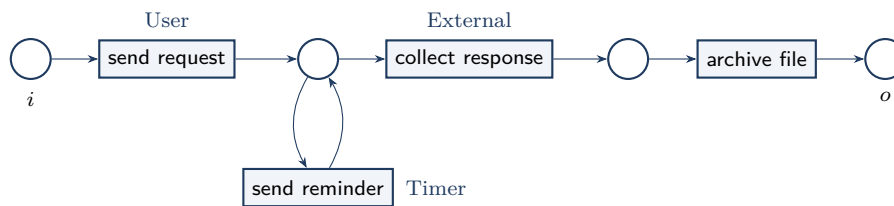
²⁴<http://woped.dhbw-karlsruhe.de/woped/>

Symbol	Meaning
(empty)	Automatic: task enacted automatically.
↓	User: a human user takes initiative.
⊠	External: external event required (e.g. message).
○ <i>clock</i>	Timer: activity starts when timer elapses.

The shapes match the BPMN event-type catalogue: an empty box for automatic activities, a small “user” arrow, an envelope for external/message events, a clock face for time events.²⁵

Example (Trigger example: request handling). A small workflow net illustrates the three non-automatic triggers in action:

- SEND_REQUEST carries a *User* trigger: a clerk pushes the request out;
- COLLECT_RESPONSE carries an *External* trigger: it fires only when an incoming message arrives from the counterpart;
- SEND_REMINDER carries a *Timer* trigger and loops back to the shared waiting place: if no response arrives within a timeout, a reminder is sent and the wait restarts;
- ARCHIVE_FILE is undecorated: it fires automatically once the response has been collected.



Example (Realistic trigger example: credit-card order handling). A larger workflow net for credit-card order handling: nine places ($i, A, \dots, G, o$) and nine transitions, several of them decorated. PACK COMPLETE OR INCOMPLETE, SHIP, RECEIVE and BACKORDER carry User triggers; CANCEL ORDER carries a Timer trigger (the order is cancelled if not packed within the time-out); UPDATE BILLING INFO UPDATED OR CANCELLED carries an External trigger (it waits for the bank’s message); START, SPAM CUSTOMER and the XOR-split CHARGE CREDIT CARD SUCCESS OR FAILURE fire automatically. Real processes mix every trigger type on a single net.

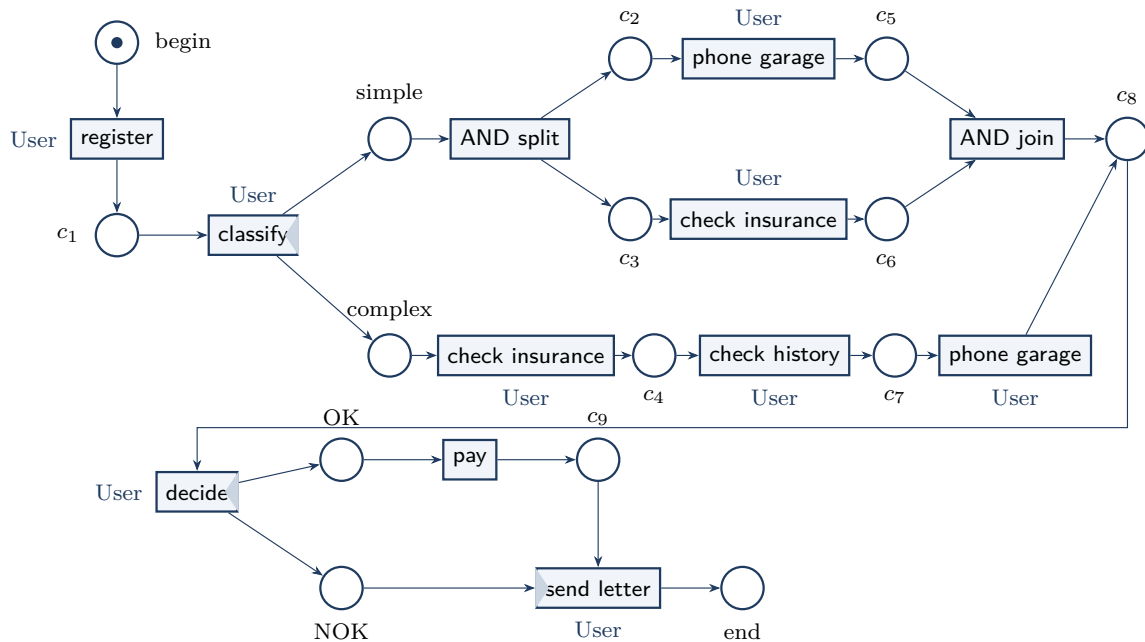
²⁵Weske M., *Business Process Management*, Springer, 2007.

- in any event, the company **sends a letter** to the customer.

Design the workflow net.

Example (Car Damage: solution). The target workflow net combines all the patterns of the chapter.

- **Register** (mandatory): a single transition right after the source place BEGIN, producing into the control place c_1 .
- **Classify** as XOR-split: from c_1 the token is routed either into the place SIMPLE or into the place COMPLEX.
- **Simple branch**: an AND-split from SIMPLE into $\{c_2, c_3\}$ feeds the two independent tasks PHONE_GARAGE (into c_5) and CHECK_INSURANCE (into c_6); an AND-join synchronises $\{c_5, c_6\}$ into c_8 .
- **Complex branch**: a strict sequence COMPLEX $\rightarrow$ CHECK_INSURANCE $\rightarrow c_4 \rightarrow$ CHECK_HISTORY $\rightarrow c_7 \rightarrow$ PHONE_GARAGE $\rightarrow c_8$. The place c_8 acts as the XOR-join of the two branches.
- **Decide** as XOR-split from c_8 : the token goes either to OK or to NOK.
- **Pay** on the OK branch only: OK $\rightarrow$ PAY $\rightarrow c_9$; the XOR-join SEND_LETTER accepts the token from c_9 or from NOK, so a letter is sent in any event.
- **End**: SEND_LETTER produces into the sink place END. All human tasks carry User triggers; PAY and the AND-split/join fire automatically.



5 Process Mining

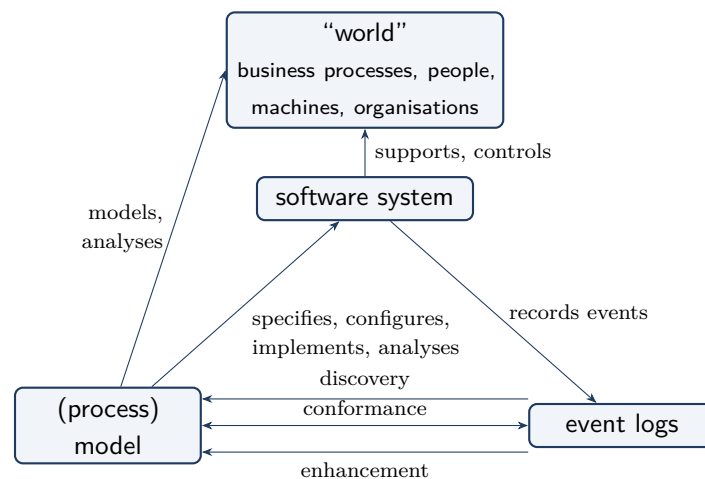
Process mining is a young discipline sitting between machine learning and data mining on one side and process modelling and analysis on the other. Its techniques confront an **event log**, the recorded behaviour of a real process, with a **process model**, in order to discover, monitor, and improve processes by extracting knowledge from data that today's information systems already produce, often unintentionally.²⁷

²⁷van der Aalst W., *Process Mining: Data Science in Action*, 2nd ed., Springer, 2016, Ch. 1 and 6.

5.1 The process-mining framework

Four artefacts and their connections frame every process-mining technique. The “world” is the informal cloud of business processes, people, machines, and organisations whose behaviour we wish to understand. The **process model** models and analyses the world, and at the same time specifies, configures, implements, and analyses the **software system** that enacts the process inside the organisation. The software system supports and controls the world; while doing so, it *records events* (messages, transactions, task completions) into the **event log**. The log lives next to the model but is initially disconnected from it: reconnecting the two is exactly the job of process mining, through three kinds of techniques.

- **Discovery:** log $\rightarrow$ model. Build a model from the log alone.
- **Conformance:** log $\leftrightarrow$ model. Compare log and model to spot discrepancies.
- **Enhancement:** log $\rightarrow$ model. Improve the model using information in the log.



5.2 Processes, cases, events, attributes

Process-mining vocabulary is fixed by a short hierarchy:

- each **process** defines a set of **cases**;
- each case consists of a series of **events**, each a unique instance of a task bound to exactly one case;
- events are ordered in time through their timestamps;
- events carry **attributes**: case identifier, activity name, timestamp, and possibly duration, cost, and the resource that performed them.

The central assumption is that some software system records every event of interest. Two attributes are mandatory for the recording to be useful: a **case identifier** relating the event to one process instance, and a well-defined **activity** of the process. The store of such records is the **event log**.

5.3 Event log example

A fragment of a realistic event log is a table with one row per event:

Case id	Event id	Timestamp	Activity	Resource	Cost
1	35654423	30-12-2010:11.02	Register request	Pete	50
2	35654483	30-12-2010:11.32	Register request	Mike	50
2	35654485	30-12-2010:12.12	Check ticket	Mike	100
2	35654487	30-12-2010:14.16	Examine casually	Pete	400
1	35654424	31-12-2010:10.06	Examine thoroughly	Sue	400
2	35654488	05-01-2011:11.22	Decide	Sara	200
1	35654425	05-01-2011:15.12	Check ticket	Mike	100
1	35654426	06-01-2011:11.18	Decide	Sara	200
1	35654427	07-01-2011:14.24	Reject request	Pete	200
2	35654489	08-01-2011:12.05	Pay compensation	Ellen	200

The table can be read along two axes. Ordered by timestamp, the events form the chronological stream of everything that happened: the natural ordering of a raw log file. Grouped by case id and ordered by timestamp inside each group, they expose **traces**: the per-case sequences of activities that a discovery algorithm actually consumes. Case 1 produces the trace

⟨Register request, Examine thoroughly, Check ticket, Decide, Reject request⟩

while case 2 produces

⟨Register request, Check ticket, Examine casually, Decide, Pay compensation⟩.

5.4 Discovery

Definition (Discovery). A **discovery** technique takes an event log as input and produces a process model as output, without using any a-priori information.

The simplest output is a control-flow model capturing the activities and their ordering. If the log also records resources, discovery can extract complementary models: for instance a **social network** of handovers between individuals. Discovery is the *play-in* of process mining.

5.5 Conformance

Definition (Conformance checking). **Conformance checking** measures how reality, as recorded in the log, conforms to the process model, and vice versa.

Unlike discovery, conformance takes *both* an event log and an existing model and compares them: which transitions are never observed, which logged sequences the model cannot reproduce, how severe the deviations are. It is an audit tool: it diagnoses the model without changing it. The standard technique is *replay*: each trace is followed step by step on the model, tokens are pushed through the net, and every break point (a transition firing without being enabled, or expected but absent) is recorded. Replay also decorates the model with timing and frequency data, which supports performance analysis, bottleneck detection, predictive models, and operational support at runtime.

5.6 Enhancement

Definition (Enhancement). Whereas conformance checking measures the alignment between a model and reality, **enhancement** aims to extend or improve existing models and systems using information about the actual process recorded in some event log.

Enhancement changes the model: additively (decorate it with frequencies, durations, costs) or structurally (**model repair**: e.g. replace a modelled sequence with a parallel pattern when the log shows the two activities in either order). A divergence between log and model can be read from two opposite viewpoints. Under the *descriptive* viewpoint the model should describe what the process does, so “the model is wrong; how do we improve it?”. Under the *normative* viewpoint the model prescribes how the process must run, so “the event log is wrong; how do we control execution?”. The same divergence leads to opposite actions; choosing is a managerial decision, not a technical one.

The dual of play-in is **play-out**: the model is fed to an engine that generates traces. Play-out covers four uses: a workflow engine driving real instances, a simulation engine producing synthetic data for what-if analysis, trace generation for testing, and model checking of the generated traces against temporal-logic properties.

5.7 A discovery example

The running example starts from the event log above. For control-flow discovery only the case id and the ordering of activities matter; timestamps fix the order and can then be dropped, and the other columns are optional. Each activity gets a one-letter code:

<i>a</i>	register request
<i>b</i>	examine thoroughly
<i>c</i>	examine casually
<i>d</i>	check ticket
<i>e</i>	decide
<i>f</i>	reinitiate request
<i>g</i>	pay compensation
<i>h</i>	reject request

A *trace* is then a finite word over the alphabet of activities, written in angle brackets, and an event log is a multiset of traces. A larger fragment of the same process gives six cases:

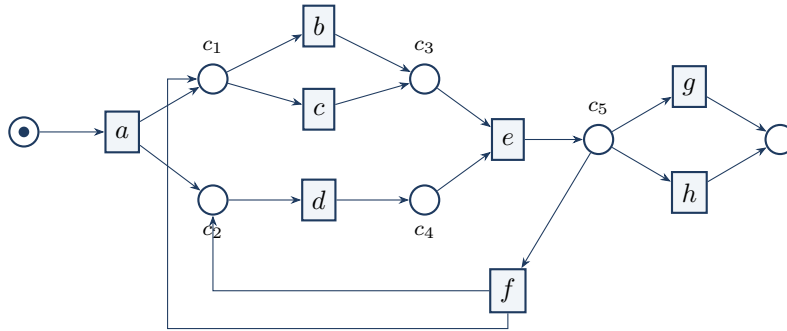
Case id	Trace
1	$\langle a, b, d, e, h \rangle$
2	$\langle a, d, c, e, g \rangle$
3	$\langle a, c, d, e, f, b, d, e, g \rangle$
4	$\langle a, d, b, e, h \rangle$
5	$\langle a, c, d, e, f, d, c, e, f, c, d, e, h \rangle$
6	$\langle a, c, d, e, g \rangle$

Long traces (cases 3 and 5) come from iterations of the loop through f ; short traces run directly from a to g or h depending on the decision outcome.²⁸

The discovery task is to produce a model that explains the six traces. Five structural observations do the job.

1. All cases *start* with a : the model opens with a fed by the source place.
2. All cases *end* with g or h : the model closes with the two alternatives converging on the sink.
3. Every e is preceded by d and by one of the examinations b or c : the model joins an exclusive choice $\{b, c\}$ and the check d into an AND-join before e .
4. The orderings bd , db , cd , dc all occur, so the relative order of examination and check is free: a **parallel split** after a opens two concurrent branches, one with the XOR $\{b, c\}$ and one with d .
5. e is always followed by f , g , or h , and in long traces f re-enters the examine/check/decide block: an XOR-split after e branches to g , h , and f , with f looping back to the parallel split.

The resulting net N :



Replaying each of the six traces shows that all of them belong to the behavioural language $L(N)$: the log is adequately captured by the net.

The example used a Petri net because nets are compact and have unambiguous yet intuitive semantics, but the same algorithmic skeleton applies to other formalisms: the discovered process can be redrawn mechanically in BPMN, with gateways replacing the split/join structure, exactly as in the visual-notation chapter.

²⁸van der Aalst W., *Process Mining*, Springer, 2016, Table 1.2.

5.7.1 Generalisation, overfitting, underfitting

The net N accepts *more* traces than those in the log: $\langle a, d, c, e, f, b, d, e, g \rangle$, for instance, is executable in N but never observed. A model that enumerated exactly the sample traces would be useless on any new case; discovery must **generalise** and return the most likely underlying model consistent with the past. Generalisation lives between two failure modes:

- **overfitting**: the model is too specific, allowing only the accidental behaviour observed in the log, and rejects new traces of the same process;
- **underfitting**: the model is too general, accepting behaviour unrelated to what was observed.

On the six-trace log, N strikes a good balance: it replays all observed traces and admits a controlled set of plausible variations. The judgement is qualitative on such a small log; real-world logs require the quantitative criteria below.

Two deliberately bad alternatives show the extremes. A *too restrictive* net ($a \rightarrow \text{AND-split into } \{b, d\} \rightarrow \text{join} \rightarrow e \rightarrow h$) rejects every trace containing c or f or ending in g : only cases 1 and 4 fit. A *too permissive* net, the **flower net** (a followed by a single central place on which b, c, d, e, f all loop, closed by g or h), accepts any sequence of intermediate activities in any order, hence every observed trace but also countless traces with no relation to the process. It delivers essentially no information about the underlying structure.

5.8 A conformance example

How well does a log pair with a net? Replaying a ten-trace log on the discovered net N :

- traces 1–6, the ones used for discovery, all fit ($\checkmark$);
- trace 7 = $\langle a, b, e, g \rangle$ fails: e fires without d ($\times$);
- trace 8 = $\langle a, b, d, e \rangle$ fails: the run is truncated before reaching g or h ($\times$);
- trace 9 = $\langle a, d, c, e, f, d, c, e, f, b, d, e, h \rangle$ fits ($\checkmark$);
- trace 10 = $\langle a, c, d, e, f, b, d, g \rangle$ fails: g fires without the second DECIDE ($\times$).

Seven traces out of ten are reproduced. The restrictive net introduced above replays only traces 1 and 4 (2 out of 10), while the flower net accepts all ten vacuously. Conformance checking formalises these per-trace verdicts into quantitative metrics.

5.9 Quality criteria

Four orthogonal criteria capture what a good process-mining result should look like:

- **fitness**: the model is able to replay the event log;
- **precision**, the model does not *underfit*: it disallows behaviour unrelated to what was observed;
- **generalisation**, the model does not *overfit*: it allows reasonable variations beyond the observed traces;
- **simplicity**: the model has a simple structure, in line with Occam's razor.

A discovery algorithm must optimise the four simultaneously, which is why no single number summarises the quality of its output. On a benchmark log of 1391 cases distributed over 21 trace variants of the running process, four candidate models score as follows:

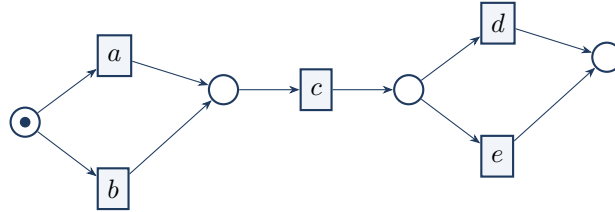
Model	fit.	prec.	gen.	simp.
N_1 : the discovered net N	+	+	+	+
N_2 : sequential net of the most frequent variant only	−	+	−	+
N_3 : flower net	+	−	+	+
N_4 : one linear sub-net per observed variant	+	+	−	−

N_2 hard-codes the single most frequent trace $\langle a, c, d, e, h \rangle$ as a sequence: *some events are missing in the model* (b, f, g never appear), so most of the log cannot be replayed. N_3 replays everything but constrains nothing. N_4 memorises the 21 variants side by side: *some events are repeated in the model*, the structure is large, and no unseen variant is accepted: overfitting via memorisation. The conformance chapter shows how fitness is quantified into a number between 0 and 1, replacing these $+/-$ labels.

5.10 Question time

Example (A five-transition net). Given an event log with 6 occurrences of $\langle a, c, d \rangle$ and 3 occurrences of $\langle b, c, e \rangle$, which Petri net with exactly the five transitions a, b, c, d, e would you infer?

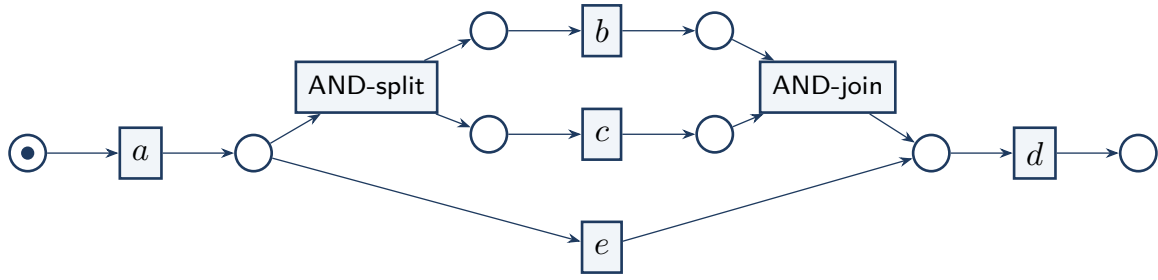
Three observations narrow the search: each trace starts with a or b , each trace ends with d or e , and c always runs in between. A natural net puts a and b as alternatives feeding a middle place, then c , then the alternatives d and e :



This net replays the two observed traces but *also* allows $\langle a, c, e \rangle$ and $\langle b, c, d \rangle$: the Cartesian product $\{a, b\} \times \{d, e\}$. The model generalises; whether that is desirable depends on the modelling intent. A more conservative net keeps the endpoint choices correlated: a reaches c and then d through private places, b reaches c and then e through another private pair, the two paths sharing only the transition c . That net replays exactly the two observed traces: high precision, possible overfitting.

Example (Another five-transition net). A second log: 3 occurrences of $\langle a, b, c, d \rangle$, 1 of $\langle a, e, d \rangle$, 2 of $\langle a, c, b, d \rangle$. Which Petri net with exactly the five transitions a, b, c, d, e would you infer?

Every trace starts with a and ends with d ; the middle is either b and c in some order, or e alone. A first candidate realises the choice with an XOR at the place after a : one branch enters an AND-split running b and c in parallel and re-joins, the other branch runs e ; both rejoin before d :



The net captures the three observed traces, but the auxiliary AND-split and AND-join transitions *do not correspond to any activity of the log*: the model contains events that were never observed. A different solution avoids them with a small lattice of places in which either both b and c fire, in any order, or e alone fires by consuming both tokens, without auxiliary transitions. That lattice is exactly the net the α -algorithm discovers on this log, constructed below.

5.11 Process discovery: the α -algorithm

Definition (Process discovery algorithm). A **process discovery algorithm** is a function that maps an event log L onto a process model M such that M is *representative* of the behaviour observed in L .

Definition (Simple event log). Let A be a set of activities. A **simple trace** σ over A is a finite sequence of activities. A **simple event log** L over A is a multiset of traces, each tagged with a multiplicity counting how many cases produced it (multiplicity 1 is implicit).

Two logs that recur in the analysis:

$$\begin{aligned} L_1 &= [\langle a, b, c, d \rangle^3, \langle a, c, b, d \rangle^2, \langle a, e, d \rangle], \\ L_2 &= [\langle a, b, c, d \rangle^3, \langle a, c, b, d \rangle^4, \langle a, b, c, e, f, b, c, d \rangle^2, \\ &\quad \langle a, b, c, e, f, c, b, d \rangle, \langle a, c, b, e, f, b, c, d \rangle^2, \langle a, c, b, e, f, b, c, e, f, c, b, d \rangle]. \end{aligned}$$

The α -algorithm was one of the first discovery algorithms able to handle concurrency. It has known limitations, but it is simple and its core ideas live on inside modern techniques. The strategy: scan the log for specific patterns (the **log-based ordering relations**), pack them into a **footprint matrix**, and reconstruct the net from the matrix.

5.11.1 Log-based ordering relations

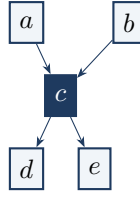
Definition (Directly-follows relation $>_L$). For an event log L over A and activities $a, b \in A$,

$$a >_L b \iff \exists \sigma = \langle t_1, \dots, t_n \rangle \in L, i \in \{1, \dots, n-1\} : t_i = a \wedge t_{i+1} = b.$$

In words, a is (sometimes) immediately followed by b .

Definition (Directly-follows graph (DFG)). The **directly-follows graph** of L has one node per activity and one edge $a \rightarrow b$ for every pair with $a >_L b$, optionally weighted by frequency.

For $L = \{\langle a, c, d \rangle, \langle b, c, e \rangle\}$ the relations are $a >_L c$, $b >_L c$, $c >_L d$, $c >_L e$ (and not, say, $a >_L d$: no a is *immediately* followed by d). The DFG has four edges meeting at c :



The DFG is the visual fingerprint of the log; many commercial tools (Disco, for one) stop there.

Definition (Derived ordering relations). Built on top of $>_L$:

- **causality:** $a \rightarrow_L b$ iff $a >_L b$ and $b \not>_L a$ (one-way arc in the DFG);
- **mutual exclusion:** $a \#_L b$ iff $a \not>_L b$ and $b \not>_L a$ (no arc);
- **concurrency:** $a \parallel_L b$ iff $a >_L b$ and $b >_L a$ (arcs both ways: the pair occurs in both orders, the log-level fingerprint of concurrency).

For every pair of activities exactly one of $x \rightarrow_L y$, $y \rightarrow_L x$, $x \#_L y$, $x \parallel_L y$ holds.

On $L_1 = [\langle a, b, c, d \rangle^3, \langle a, c, b, d \rangle^2, \langle a, e, d \rangle]$:

$$>_{L_1} = \{(a, b), (a, c), (a, e), (b, c), (c, b), (b, d), (c, d), (e, d)\}$$

$$\parallel_{L_1} = \{(b, c), (c, b)\}$$

$$\rightarrow_{L_1} = \{(a, b), (a, c), (a, e), (b, d), (c, d), (e, d)\}$$

$$\#_{L_1} = \{(a, a), (a, d), (b, b), (b, e), (c, c), (c, e), (d, a), (d, d), (e, b), (e, c), (e, e)\}$$

The only concurrency is between b and c : both orderings appear in the log.

5.11.2 Footprint matrix

The four relations pack into a single **footprint matrix**: one row and one column per activity, and in cell (x, y) the relation holding between x and y . For L_1 (filling concurrencies first, then causalities, then $\#$ everywhere else):

	a	b	c	d	e
a	$\#$	$\rightarrow$	$\rightarrow$	$\#$	$\rightarrow$
b	$\leftarrow$	$\#$	$\parallel$	$\rightarrow$	$\#$
c	$\leftarrow$	$\parallel$	$\#$	$\rightarrow$	$\#$
d	$\#$	$\leftarrow$	$\leftarrow$	$\#$	$\leftarrow$
e	$\leftarrow$	$\#$	$\#$	$\rightarrow$	$\#$

The matrix is symmetric around the diagonal up to direction: diagonal entries are $\#_L$ (unless an activity loops on itself in the log), $a \rightarrow_L$ at (x, y) mirrors a $\leftarrow_L$ at (y, x) , and $\#_L$ and $\parallel_L$ mirror themselves.

5.11.3 Patterns from the footprint

The footprint tells which control-flow pattern to place around every group of activities:

- **sequence**, $a \rightarrow_L b$: a single place between a and b ;

- **XOR-split**, $a \rightarrow_L b$, $a \rightarrow_L c$, $b \#_L c$: one output place after a feeding both b and c ; only one fires per case;
- **XOR-join**, $a \rightarrow_L c$, $b \rightarrow_L c$, $a \#_L b$: one input place before c receiving from the alternative predecessors;
- **AND-split**, $a \rightarrow_L b$, $a \rightarrow_L c$, $b \parallel_L c$: two distinct output places after a , so firing a feeds both branches;
- **AND-join**, $a \rightarrow_L c$, $b \rightarrow_L c$, $a \parallel_L b$: two distinct input places before c , which fires only when both hold a token.

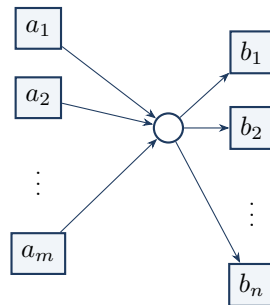
Each pattern is a specific combination of $\rightarrow_L$, $\#_L$, $\parallel_L$ entries; the α -algorithm assembles the net from this dictionary.

5.11.4 The algorithm

Definition (α -algorithm). Let L be an event log over the set of activities A . Define:

1. $T_L = \{t \in A \mid \exists \sigma \in L, t \in \sigma\}$, one transition per activity occurring in L ;
2. $T_I = \{t \in A \mid \exists \sigma \in L, t = \text{first}(\sigma)\}$, the activities starting some trace;
3. $T_O = \{t \in A \mid \exists \sigma \in L, t = \text{last}(\sigma)\}$, the activities ending some trace;
4. $X_L = \{(A, B) \mid A, B \subseteq T_L, A, B \neq \emptyset, \forall a \in A, b \in B : a \rightarrow_L b, \forall a_1, a_2 \in A : a_1 \#_L a_2, \forall b_1, b_2 \in B : b_1 \#_L b_2\}$, the candidate decision points;
5. $Y_L = \{(A, B) \in X_L \mid \forall (A', B') \in X_L : A \subseteq A' \wedge B \subseteq B' \Rightarrow (A', B') = (A, B)\}$, the *maximal* decision points;
6. $P_L = \{p_{(A, B)} \mid (A, B) \in Y_L\} \cup \{i_L, o_L\}$, one place per maximal pair plus source and sink;
7. $F_L = \{(a, p_{(A, B)}) \mid (A, B) \in Y_L, a \in A\} \cup \{(p_{(A, B)}, b) \mid (A, B) \in Y_L, b \in B\} \cup \{(i_L, t) \mid t \in T_I\} \cup \{(t, o_L) \mid t \in T_O\}$, the arcs;
8. $\alpha(L) = (P_L, T_L, F_L, i_L)$, the discovered net.

Steps 1–3 are bookkeeping: the transitions, the start activities, the end activities. The heart is steps 4–5. A pair $(A, B) \in X_L$ marks a potential *decision point*: every activity in A causes every activity in B , the activities in A are pairwise mutually exclusive, and so are those in B . The construction then inserts one place per pair, fed by all of A and feeding all of B :



This single device subsumes every XOR/AND split and join of the pattern dictionary. On the footprint matrix the three conditions are a geometric test: (A, B) is a candidate iff the $A \times A$ and $B \times B$ blocks are entirely $\#_L$ and the $A \times B$ block is entirely $\rightarrow_L$. Shrinking a candidate preserves the test, so X_L is closed under non-empty subsets and contains many redundant pairs; a place generated from a subset pair would duplicate behaviour already captured by the larger one.

Step 5 therefore keeps only the maximal pairs: for instance $p_{(\{a\},\{b\})}$ is subsumed by $p_{(\{a\},\{b,c\})}$, which already feeds both b and c .

Example (The α -algorithm on L_1 , steps 1–5). For $L_1 = [\langle a, b, c, d \rangle^3, \langle a, c, b, d \rangle^2, \langle a, e, d \rangle]$:

$$T_{L_1} = \{a, b, c, d, e\}, \quad T_I = \{a\}, \quad T_O = \{d\}.$$

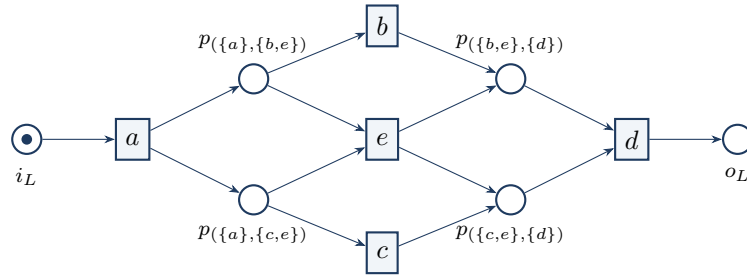
Scanning the footprint matrix for uniform rectangles gives the candidates

$$X_{L_1} = \{ (\{a\}, \{b\}), (\{a\}, \{c\}), (\{a\}, \{e\}), (\{a\}, \{b, e\}), (\{a\}, \{c, e\}), \\ (\{b\}, \{d\}), (\{c\}, \{d\}), (\{e\}, \{d\}), (\{b, e\}, \{d\}), (\{c, e\}, \{d\}) \}.$$

A sample check, $(\{a\}, \{b, e\})$: the cell (a, a) is $\#$, the $A \times B$ cells (a, b) and (a, e) are both $\rightarrow$, and the $B \times B$ block $\{b, e\}^2$ is all $\#$: candidate confirmed. No pair may contain both b and c on the same side, because $b \parallel_{L_1} c$ violates mutual exclusion. Dropping dominated pairs $((\{a\}, \{b\})$ and $(\{a\}, \{e\})$ are inside $(\{a\}, \{b, e\})$, $(\{b\}, \{d\})$ and $(\{c\}, \{d\})$ inside $(\{b, e\}, \{d\})$, and so on) leaves the maximal pairs

$$Y_{L_1} = \{ (\{a\}, \{b, e\}), (\{a\}, \{c, e\}), (\{b, e\}, \{d\}), (\{c, e\}, \{d\}) \}.$$

Steps 6–8 assemble the net: one internal place per maximal pair, the source i_L feeding every transition in T_I , every transition in T_O feeding the sink o_L , and the arcs dictated by the pairs. For L_1 :



Firing a marks both internal input places; then either b and c fire, one from each place and in any order, or e fires alone consuming both tokens. Either way both right-hand places end up marked and d closes the case: exactly the behaviour “ b and c in any order, or just e ”, with no auxiliary transitions. The result is a sound workflow net: one source, one sink, every node on a path between them.

5.12 A second worked example

A larger log stress-tests the algorithm:

$$L_5 = [\langle a, b, e, f \rangle^2, \langle a, b, e, c, d, b, f \rangle^3, \langle a, b, c, e, d, b, f \rangle^2, \\ \langle a, b, c, d, e, b, f \rangle^4, \langle a, e, b, c, d, b, f \rangle^3].$$

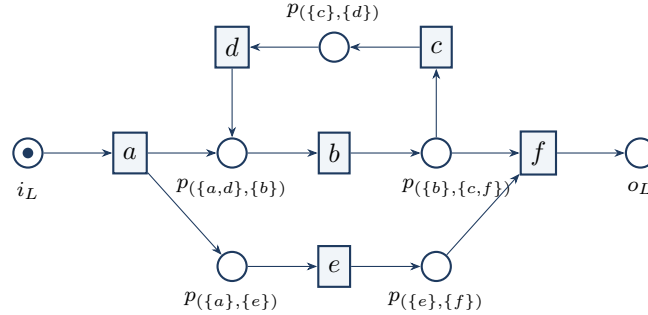
Here $T_{L_5} = \{a, b, c, d, e, f\}$, $T_I = \{a\}$, $T_O = \{f\}$, and e is concurrent with b , c , and d . The footprint yields nine candidate pairs,

$$X_{L_5} = \{ (\{a\}, \{b\}), (\{a\}, \{e\}), (\{b\}, \{c\}), (\{b\}, \{f\}), (\{c\}, \{d\}), \\ (\{d\}, \{b\}), (\{e\}, \{f\}), (\{a, d\}, \{b\}), (\{b\}, \{c, f\}) \},$$

of which five are maximal:

$$Y_{L_5} = \{(\{a\}, \{e\}), (\{c\}, \{d\}), (\{e\}, \{f\}), (\{a, d\}, \{b\}), (\{b\}, \{c, f\})\}.$$

The discovered net runs the branch through e concurrently with a loop in which b leads either to another round through c and d or to the exit f :



5.13 Exercises

Example (Apply the α -algorithm to L_4). Let $L_4 = [\langle a, c, d \rangle^{45}, \langle b, c, d \rangle^{42}, \langle a, c, e \rangle^{38}, \langle b, c, e \rangle^{22}]$. Verify that the footprint matrix is

	a	b	c	d	e
a	#	#	$\rightarrow$	#	#
b	#	#	$\rightarrow$	#	#
c	$\leftarrow$	$\leftarrow$	#	$\rightarrow$	$\rightarrow$
d	#	#	$\leftarrow$	#	#
e	#	#	$\leftarrow$	#	#

and that the discovered net is $i_L \rightarrow \{a, b\} \rightarrow p(\{a, b\}, \{c\}) \rightarrow c \rightarrow p(\{c\}, \{d, e\}) \rightarrow \{d, e\} \rightarrow o_L$.

Solution sketch. The directly-follows pairs are $a >_{L_4} c$, $b >_{L_4} c$, $c >_{L_4} d$, $c >_{L_4} e$ and no others, so $\|_{L_4} = \emptyset$ and the causalities are exactly those four pairs. The maximal decision points are $(\{a, b\}, \{c\})$ (an XOR-join collecting the two alternative starts) and $(\{c\}, \{d, e\})$ (an XOR-split routing the case to either end). The rest follows mechanically.

Example (Apply the α -algorithm to L_3). Let

$$L_3 = [\langle a, b, c, d, e, f, b, d, c, e, g \rangle, \langle a, b, d, c, e, g \rangle^2, \langle a, b, c, d, e, f, b, c, d, e, f, b, d, c, e, g \rangle].$$

Check that $c \|_{L_3} d$ is the only concurrency and that the discovered net is

$$i_L \rightarrow a \rightarrow p(\{a, f\}, \{b\}) \rightarrow b \rightarrow \text{parallel branches } c, d \rightarrow p\text{-pair joining at } e \rightarrow \{f, g\},$$

with f looping back to $p(\{a, f\}, \{b\})$ and $g \rightarrow o_L$. The pair $(\{a, f\}, \{b\})$ encodes entry-or-iteration into the loop body.

Example (Apply the α -algorithm to L_2). For the log L_2 defined earlier, check that the relevant relations are $a \rightarrow_{L_2} b$, $a \rightarrow_{L_2} c$, $b \|_{L_2} c$, $b \rightarrow_{L_2} d$, $c \rightarrow_{L_2} d$, $b \rightarrow_{L_2} e$, $c \rightarrow_{L_2} e$, $e \rightarrow_{L_2} f$, $f \rightarrow_{L_2} b$, $f \rightarrow_{L_2} c$, and that the maximal pairs are

$$p_1 = p(\{a, f\}, \{b\}), \quad p_2 = p(\{a, f\}, \{c\}), \quad p_3 = p(\{b\}, \{d, e\}), \quad p_4 = p(\{c\}, \{d, e\}), \quad p_5 = p(\{e\}, \{f\}).$$

The net runs b and c in parallel (p_1, p_2); the places p_3 and p_4 jointly choose between closing the case with d and iterating with e , whose place p_5 feeds f , which re-enters the parallel block.

5.14 Limitations of the α -algorithm

Implicit dependencies. On the log

$$L_6 = [\langle a, c, e, g \rangle^2, \langle a, e, c, g \rangle^3, \langle b, d, f, g \rangle^2, \langle b, f, d, g \rangle^4]$$

the algorithm produces a net with two places that turn out to be **redundant**: removing them does not change the behaviour, but the footprint alone gives the α -algorithm no way to detect the redundancy. Successors (α^+ , α^{++} , $\alpha\#$, region-based and heuristic miners) address this kind of imperfection.

Short loops. On

$$L_7 = [\langle a, c \rangle^2, \langle a, b, c \rangle^3, \langle a, b, b, c \rangle^2, \langle a, b, b, b, b, c \rangle]$$

the activity b repeats, so $b >_{L_7} b$ and hence $b \parallel_{L_7} b$. No candidate pair (A, B) may contain b , because the sets must be *pairwise* mutually exclusive, including each element with itself. The expected net is a self-loop on b between a and c ; the algorithm instead leaves b **disconnected** from the path $a \rightarrow c$. Length-two loops fail similarly: on

$$L_8 = [\langle a, b, d \rangle^3, \langle a, b, c, b, d \rangle^2, \langle a, b, c, b, c, b, d \rangle]$$

both bc and cb occur, so the footprint diagnoses $b \parallel_{L_8} c$ (wrongly, since the two alternate rather than running in parallel) and the discovered net leaves c disconnected instead of looping b and c around an intermediate place.

5.15 Noise

Process-mining literature uses **noise** for *rare and infrequent* behaviour, not for logging errors. Frequency carries information: a trace observed in many cases is more representative than a one-off variant. The α -algorithm ignores frequencies entirely: every trace, however rare, contributes equally to the footprint. A more robust miner could filter infrequent behaviour, but whether rare traces are outliers to discard or genuine alternative paths is an open modelling question. Picture the log as a cloud of dots (traces), most clustered in a “frequent behaviour” core, a few stray in a surrounding noise band; the target model is a region laid over that picture. Three pathological outcomes:

- **non-fitting model**: the region covers only a corner of the frequent core; many observed traces fall outside (low fitness);
- **overfitting model**: the region hugs each individual dot; only the exact observed traces are accepted, with no generalisation;
- **underfitting model**: the region swells beyond the noise band, accepting behaviour even more permissive than the noisy log itself.

Good discovery covers the frequent core, slightly more than the observed dots (to generalise) and less than the noise band (to keep precision): the trade-off that the four quality criteria quantify.

6 Why process mining: purpose, goals, and requirements

Process mining closes a gap. When a business process is documented in a process model (BPMN, Petri net, EPC), that model reflects *design intent*: what the process should do. When the same process runs on a software system, the actual behaviour is recorded in an *event log*: what the process *actually* does. These two rarely match. Process mining reconnects them and answers three categories of questions: What does reality show? Is reality aligned with design? How can we improve?

6.1 Three functional goals

Process mining addresses concrete problems through the three techniques formalized in the previous section; here each is restated from the practitioner's angle: what it delivers and when to reach for it:

Definition (Discovery). Build a process model from recorded behaviour alone. Start with an event log; end with a model that explains the observed traces. Use when: the model is missing, outdated, or unknown.

Discovery reveals the *as-is* process: which activities happen, in what order, and where people deviate.

Definition (Conformance). Compare an event log against a known model to measure alignment. Pinpoint deviations, quantify severity, diagnose root causes. Use when: a reference model exists; you need to audit, monitor, or debug it.

Conformance answers: Does reality match the design? If not, where and by how much? It detects both errors (model too strict, reality too loose) and intentional workarounds. Conformance does not change the model; it measures the gap.

Definition (Enhancement). Extend or refine an existing model using insights from the log. Add timing, cost, resource patterns, or discover sub-processes missed in the original design. Use when: the model is correct but incomplete.

Enhancement enriches a known model with quantitative data, performance metrics, and hidden flows extracted from traces.

6.2 From question to insight: the workflow

A typical process-mining workflow has four stages:

1. **Problem scoping.** Identify which gap you want to close: missing model (discovery), misalignment (conformance), or incomplete picture (enhancement)? Fix the scope: which process, which time window, which cases.
2. **Data preparation.** Extract or access the event log from the software system. Ensure it contains case identifiers, activity names, and timestamps. Clean, filter, and verify coverage (does the log span the process instances of interest?).

3. **Model application.** Run the appropriate algorithm (discovery, conformance, or enhancement) on the cleaned log. For discovery or conformance, you may need a reference model; for enhancement, you start with one.
4. **Interpretation and action.** Read the results, translate them into process changes, resource decisions, or design fixes. Often you iterate: refine the log (step 2) or adjust the algorithm parameters (step 3).

6.3 Prerequisites: data, tools, and knowledge

Process mining is a data science discipline. Three prerequisites are non-negotiable:

Definition (Event logs). The log is your only source of truth. It must record every event of interest, with case identifier, activity name, and timestamp. Attributes like resource, cost, or status deepen the analysis but are optional. The log lives in the software system; extracting it cleanly is often the hardest step.

If a log is missing, incomplete, or incorrectly structured, no algorithm will recover it. Pre-processing (cleaning, deduplication, filtering, schema alignment) typically consumes 70–80 % of a real project. Budget for data curation from the start.

Definition (Process models and notations). For conformance and enhancement, you need a reference model in a notation that the tool understands: Petri nets (mathematical), BPMN (visual, industry-standard), or EPC (German standard). For discovery, the algorithm produces a model; understanding the output requires knowledge of the chosen notation.

Definition (Software tools). Process mining is not done by hand. Choose a tool:

- **Disco:** Visual, intuitive, minimal learning curve. Good for exploration and quick diagnostics. Limited to discovery and simple conformance.
- **PM4Py:** Python library. Powerful, flexible, scriptable. Steeper learning curve; requires programming. Recommended for detailed analysis and custom workflows.
- **ProM:** Academic tool, extensive, includes cutting-edge algorithms. Complex to use; suited for researchers and specialists.

Different tools excel at different tasks; practitioners often switch tools mid-project.

6.4 Knowledge and skills

Three competencies overlap in process mining:

Domain knowledge. You must understand the process: what activities are important, what constitutes a deviation, what metrics matter. A credit card fraud analyst asks different questions than a hospital administrator.

Process modelling. You need to read, write, and critique process models in BPMN or Petri nets. This is a skill: understanding control flow, parallelism, choice, synchronisation. Learn by reading other models and building simple ones.

Data science. Event logs are tabular data; filtering, grouping, and aggregating them requires comfort with data frames, SQL-like queries, or Python. You need to think in terms of cases,

traces, frequencies, and distributions.

Few people hold all three; most work in teams. The data engineer handles extraction and cleaning. The process analyst reads and validates models. The domain expert interprets results and recommends changes.

6.5 When process mining makes sense

Process mining is powerful but not a cure-all. Use it when:

- A software system records the process (ERP, workflow engine, healthcare information system).
- The process is complex or poorly understood.
- Compliance, performance, or continuous improvement is critical.
- A model exists but may be outdated.
- You have access to domain experts to validate findings.

Avoid it when:

- The log is missing or unreliable.
- The process is trivial or fully documented.
- Extracting the log requires rewriting the software system.
- No one on the team has data or process-modelling skills.

7 Orchestration and collaboration

7.1 Why diagrams?

Graphical languages are first of all a *communication* medium. Their value comes from a careful selection of symbols (shapes, colours, arrows) assembled into an alphabet shared by the stakeholders: intuitive enough for a domain expert without mathematical training, precise enough to be unambiguous. Expressivity does not justify clutter, though: an auto-generated flowchart that wires dozens of decorative boxes (START, DECISION, DATABASE, DOCUMENT, ...) into a baroque graph is unreadable. A good process diagram uses *few* graphical elements and a small number of clear connections.

The three notations seen in the course share the ingredient catalogue but disagree on the symbols:

	BPMN	EPC	WfN
start / end	circle	hexagon	circle (place)
join / split	diamond	circle	square (transition)
tasks	rounded rect.	rounded rect.	square (transition)
links	solid arrow	dashed arrow	solid arrow

The insurance-claim process of the previous chapters, redrawn in EPC syntax with tools such

as ARIS Express²⁹ or VP Online,³⁰ keeps the same semantic content as its BPMN rendering: activities (RECORDING, TYPE, POLICY, PREMIUM, . . . , FILING) alternate with events, and small AND/XOR connector circles realise the splits and joins. EPC layouts run conventionally top to bottom and come out taller than their BPMN counterparts.

7.2 Event-driven process chains (EPC)

The EPC method was developed in the early 1990s by August-Wilhelm Scheer as the control-flow component of the **ARIS framework** (Architecture of Integrated Information Systems), a holistic approach to enterprise modelling covering organisation, data, function, and control views.³¹ EPC is used to configure ERP implementations, to drive the modelling of business processes, and to support their analysis and redesign. The notation is informal (no legend is needed to read a diagram) and serialises into the XML-based **EPC Markup Language** (.epml).

Definition (Event-driven Process Chain (EPC)). An **Event-driven Process Chain** (EPC) is an ordered graph of **events** and **functions**. It provides various **connectors** that allow alternative and parallel execution of processes; the connectors are further qualified by the usage of **logical operators** such as OR, AND, and XOR.

The claimed pedagogical strength of EPC is simplicity: regardless of formal expressiveness, the easy-to-understand look of the diagrams made it a widely accepted industrial notation.

7.2.1 Ingredients



- **Events** (hexagons) are the passive elements: they describe the circumstances under which a function works or the state it results in: pre- and post-conditions in a Hoare-style reading. Every EPC must *start and end with events*, and along the flow events and functions *alternate*: between any two functions sits at least one event, and vice versa.
- **Functions** (rounded rectangles) are the active elements: the tasks of the business process. A function can be refined into another EPC, supporting hierarchical modelling as in workflow nets.
- **Connectors** (circles, octagons in some tools) carry one of three logical operators, often abbreviated by the propositional symbols $\wedge = \text{AND}$, $\times = \text{XOR}$, $\vee = \text{OR}$: AND synchronises parallel branches, XOR routes an exclusive choice, OR expresses inclusive choice (one or more branches, possibly all).
- **Control-flow connections** (dashed arrows) link events to functions or connectors and vice versa, expressing causal dependency: the source must occur before the target.

A connector with one incoming arc and several outgoing arcs is a **split**; with several incoming arcs and one outgoing arc it is a **join**. Combined with the three operators this gives six patterns

²⁹<https://ariscommunity.com/aris-express>

³⁰<https://online.visual-paradigm.com/>

³¹Scheer A.-W., *ARIS: Vom Geschäftsprozess zum Anwendungssystem*, Springer, 1998; *ARIS: Business Process Modeling*, 2nd ed., Springer, 2000.

(AND/XOR/OR-split, AND/XOR/OR-join); the arrow geometry alone disambiguates split from join, so the glyph does not change.³²

Note (OR-join: intended meaning). The OR-join is the most subtle connector:

- if only one input arrives, it releases the flow;
- if both inputs arrive, it releases exactly one output (not two);
- if one input arrives, it must wait until the other arrives *or* until it is guaranteed that the other will never arrive.

The last clause is what makes OR-joins formally hard: deciding that “the other will never arrive” requires reasoning about the global state, which a local firing rule cannot do. This is the source of the well-known semantic difficulties around OR-joins.

The OR-join is not a luxury. When two upstream branches may both produce a token, both omit it, or only one of them produces one, the OR-join is the only construct that merges them correctly: an XOR-join would expect exactly one input and break on the simultaneous arrival of both, an AND-join would wait forever when one branch stays silent.

7.2.2 A taste of .epml

EPML is the XML interchange format for EPC.³³ A file is rooted at an `<epml:epml>` element; a `<directory>` holds `<epc>` containers carrying the events, functions, connectors, and arcs. An extract of an “Online Shopping” EPC:

```
<?xml version="1.0" encoding="UTF-8"?>
<epml:epml xmlns:epml="http://www.epml.de">
  <directory name="Root">
    <epc epcId="1" name="Online Shopping">
      <event id="1"><name>Start Online Shopping</name></event>
      <arc id="10"><flow source="1" target="2"/></arc>
      <function id="2"><name>Determine List of Products</name></function>
      <arc id="11"><flow source="2" target="3"/></arc>
      <event id="3"><name>Not all Products in Cart</name></event>
      <arc id="12"><flow source="3" target="4"/></arc>
      <xor id="4"/>
      ...
      <processInterface id="111">
        <name>Continue with Order Process</name>
      </processInterface>
    </epc>
  </directory>
</epml:epml>
```

³²Weske M., *Business Process Management*, Springer, 2007.

³³Mendling J., Nüttgens M., “EPC markup language (EPML): an XML-based interchange format for event-driven process chains (EPC)”, *Information Systems and e-Business Management*, 4(3):245–263, Springer, 2006.

The event–arc–function alternation is preserved in the XML, and a `<processInterface>` pointer lets a process call another process: the EPML counterpart of a sub-process.³⁴

7.3 Orchestration

The rest of the chapter develops a buyer–reseller pair taken from Weske.³⁵

Definition (Orchestration). Business process models are by definition enacted within a *single* organisation. The ordering of activities can therefore be controlled by a **business process management system** run *centrally* by that organisation. This kind of control is called **orchestration**.

The name comes from the conductor of an orchestra. An orchestration is a *single-viewpoint* model: one organisation owns the process and dictates the flow.

The reseller’s orchestration contains RECEIVE ORDER, followed by an AND-split into a billing branch (SEND INVOICE → RECEIVE PAYMENT) and a shipping branch (SHIP PRODUCTS), synchronised before completion (in the EPC rendering, before an ARCHIVE ORDER function). The same single-viewpoint process can be drawn in all three notations (EPC with alternating events, as a workflow net whose places hold the case state, or as a BPMN pool) and the reseller uses it to configure its own BPM system: every instance then runs as specified, with billing and shipping concurrent. Symmetrically, the buyer owns an orchestration: PLACE ORDER, then concurrently RECEIVE PRODUCTS and RECEIVE INVOICE → SETTLE INVOICE, synchronised before completion. In both diagrams no message to or from the other participant appears.

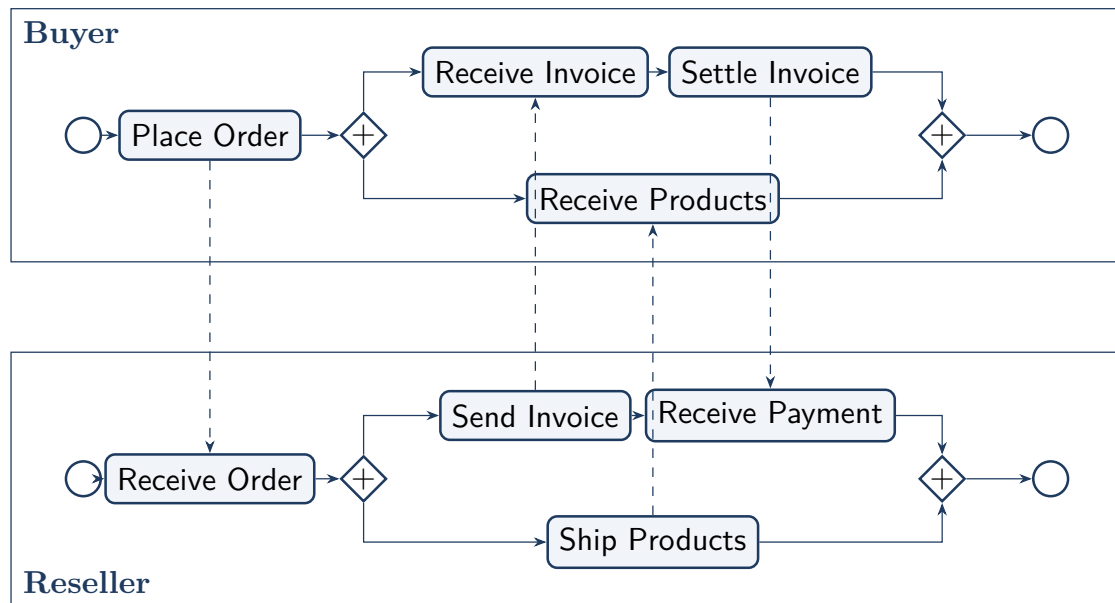
7.4 Collaboration

Definition (Collaboration). Each business process is enacted by one organisation, but cross-organisational business processes can *interact* with each other. Linking the activities of single business processes that are related is called **collaboration**. A collaboration describes a *multi-point, decentralised* viewpoint: it comprises the activities of autonomous business processes and how they interact to achieve a common goal.

There is no longer a single conductor: separately developed processes must communicate, exchanging information of any kind: electronic messages, physically transported objects (the shipped products), documents (the invoice), and the model must make the exchanges explicit so that the global behaviour can be reasoned about. In BPMN the pools sit side by side and the exchanges become **message flows**, drawn as dashed arcs. Four of them weave the two orchestrations into one collaboration:

³⁴<http://www.epml.de>

³⁵Weske M., *Business Process Management*, Springer, 2007, Sect. 1.1.



The order, the invoice, the products, and the payment each cross the pool boundary exactly once; what one pool sends must match what the other expects.

7.5 Choreography

Definition (Choreography). The interactions of a set of business processes can be specified also as a process **choreography**: a *global-viewpoint* model that contains only the activities related to interactions between participants.

The choreography is distinguished by what it removes: unlike an orchestration it has *no central agent* controlling the activities, and unlike a collaboration it contains *only* the interaction activities, not the internal ones. The name comes from dancers: each behaves autonomously but follows the prescribed steps and timing. For the buyer–reseller pair the choreography lists only four interactions: BUYER PLACES THE ORDER TO RESELLER, RESELLER SHIPS PRODUCTS TO BUYER, RESELLER SENDS THE INVOICE TO BUYER, BUYER SETTLES THE INVOICE TO RESELLER. A choreography is a sort of *contract*: it states how actors *should* interact, not how each realises its part. The three viewpoints do not subsume one another:

- **orchestration**, single viewpoint: one organisation, one pool (for the implementation team);
- **collaboration**, multiple viewpoints: several pools, each running its own orchestration, wired by message flows (for the integration team);
- **choreography**, global viewpoint: only the message exchanges, read as a contract (for inter-organisation negotiation).

7.6 Compatibility

Different orchestrations of the same participant are possible, but do they all make sense, and do they interact correctly with the counterpart? Two reseller and four buyer variants set up the question.³⁶

³⁶Weske M., *Business Process Management*, Springer, 2007.

- R_1 : RECEIVE ORDER $\rightarrow$ AND-split into {SEND INVOICE $\rightarrow$ RECEIVE PAYMENT} and {SHIP PRODUCTS}.
- R_2 : RECEIVE ORDER $\rightarrow$ SEND INVOICE $\rightarrow$ RECEIVE PAYMENT $\rightarrow$ SHIP PRODUCTS (fully sequential: payment before shipping).
- B_1 : PLACE ORDER $\rightarrow$ AND-split into {RECEIVE INVOICE $\rightarrow$ SETTLE INVOICE} and {RECEIVE PRODUCTS}.
- B_2 : PLACE ORDER $\rightarrow$ AND-split into {RECEIVE PRODUCTS $\rightarrow$ SETTLE INVOICE} and {RECEIVE INVOICE}: pays only after the goods arrive.
- B_3 : PLACE ORDER $\rightarrow$ AND-split into {RECEIVE PRODUCTS} and {RECEIVE INVOICE}, AND-join, then SETTLE INVOICE: pays only after *both* goods and invoice.
- B_4 : PLACE ORDER $\rightarrow$ RECEIVE INVOICE $\rightarrow$ RECEIVE PRODUCTS $\rightarrow$ SETTLE INVOICE (strict sequence).

Example (Exercise: compatibility matrix). Build the 2×4 matrix marking every pair (R_i, B_j) whose interaction may run into problems: an activity fired in an order the counterpart is not ready to accept. Optionally extend the matrix with further variants.

Pairwise reasoning fills the matrix:

- $R_1 + B_1$: **ok**: both sides handle billing and shipping concurrently; every message finds its receiver.
- $R_1 + B_2$: **ok**: R_1 ships without waiting for payment, so B_2 gets the products, settles, and R_1 collects the payment.
- $R_1 + B_3$: **ok**: R_1 sends invoice and products concurrently; B_3 receives both, then settles.
- $R_1 + B_4$: **ok**: the AND-split of R_1 is permissive enough to serve the buyer's strict receive-invoice-then-products sequence.
- $R_2 + B_1$: **ok**: B_1 settles upon receiving the invoice without waiting for the products, so R_2 gets its payment and ships last.
- $R_2 + B_2$: **no**: R_2 ships only after the payment, B_2 pays only after the products: each side waits for the other and the collaboration deadlocks.
- $R_2 + B_3$: **no**: same deadlock: B_3 needs the products before settling, R_2 needs the payment before shipping.
- $R_2 + B_4$: **no**: B_4 wants the products before settling; R_2 again waits for the payment first.

	B_1	B_2	B_3	B_4
R_1	ok	ok	ok	ok
R_2	ok	no	no	no

The pattern is informative. R_1 , the AND-split reseller, is *permissive* and composes with all four buyers; R_2 , the strict-sequence reseller, only works with buyers willing to pay before the goods arrive. Two locally-correct orchestrations can compose into a deadlocking collaboration: compatibility is a genuinely global property, formalised later in the course.

7.7 Exercises: the travel agency

Example (Travel-agency orchestration (single viewpoint)). Design a travel-agency orchestration in BPMN, EPC, and WfN. The tasks handle a travel request and include booking a flight, booking a hotel, changing dates, cancelling the booking, and confirming the booking. Draw a process diagram relating the tasks.

Solution sketch.

In EPC: start event TRAVEL REQUEST flows into an XOR connector that also collects the CHANGE DATES return branch and passes to the function RECEIVE DATES. An AND connector then forks BOOK HOTEL and BOOK FLIGHT and joins them again. A downstream XOR lets the customer CHANGE DATES (loop back to the first XOR), CONFIRM (end event SUCCESS), or CANCEL (end event FAILURE).

The BPMN version replaces connectors with gateways (XOR diamonds for the decisions, AND diamond for the fork) inside a single pool with two end events.

The WfN version realises the same structure: competing transitions on a shared input place for each XOR, one transition with two output places for the AND, the change branch looping back, and the sink reached through either the confirm or the cancel transition. More verbose, but unambiguous.

Example (Travel choreography (global viewpoint)). Model the same scenario (booking flight and hotel with the possibility to change dates, cancel, or confirm) as a BPMN choreography between TOURIST and TRAVEL AGENCY.

Solution sketch. Only the message exchanges appear: TRAVEL REQUEST (tourist → agency), TRAVEL PLANNED (agency → tourist), ASK CONFIRMATION (agency → tourist), then the choice between CHANGE DATES (loop back), CONFIRM (tourist → agency, success), and CANCEL (failure). No internal activity of either side is shown.

Example (Tourist orchestration and full collaboration). Draw the tourist's own orchestration (BPMN, EPC, WfN), then assemble the BPMN collaboration of tourist and travel agency.

Solution sketch. The tourist's pool: ISSUE TRAVEL REQUEST, a message-receive task for the planned trip, then an XOR decision among CONFIRM, CANCEL, and CHANGE DATES (looping back to wait for a new plan), ending in a success or failure event. The collaboration places the two pools side by side, each running its orchestration, with one message flow per choreography interaction: TRAVEL REQUEST and CONFIRM/CANCEL flow tourist → agency, TRAVEL PLANNED and ASK CONFIRMATION flow agency → tourist, and the CHANGE DATES message returns the agency to its booking phase, restarting the negotiation.

8 EPC and BPMN

Two high-level notations share much with workflow nets but were designed for human readability rather than formal analysis: **EPC** (Event-driven Process Chains), introduced in the early 1990s within the SAP/ARIS framework, and **BPMN** (Business Process Model and Notation),

standardised by the OMG.³⁷ Their vocabularies partition along the same axes: start/end events, gateways vs connectors, tasks vs functions, sequence flows vs control-flow arcs. EPC comes first; BPMN mirrors it with a richer alphabet.

8.1 EPC requirements

EPC elements can be combined freely (cycles are allowed, the graph has no prescribed shape) but a well-formed diagram must satisfy structural requirements:

- the graph is **weakly connected**: no isolated nodes, no disconnected sub-diagrams (two valid fragments drawn side by side without a shared node are *one* ill-formed EPC);
- each **event** has at most one incoming and at most one outgoing arc, and at least one incident arc; there is at least one start event (no incoming arc) and at least one end event (no outgoing arc);
- each **function** has exactly one incoming and exactly one outgoing arc: a function may neither start nor terminate the flow, neither split nor join it;
- each **connector** has either one input and several outputs (a *split*) or several inputs and one output (a *join*); a connector with one of each is degenerate, one with several of each is ill-formed.

A worked example exercising the whole alphabet: START → TRAVEL REQUEST → XOR-join → AND-split → {BOOK FLIGHT, BOOK HOTEL} → AND-join → TRAVEL PLANNED → ASK CONFIRMATION → XOR-split → {CONFIRM → END OK, CANCEL → END NULL, CHANGE REQUEST looping back to the XOR-join}. The XOR-join after TRAVEL REQUEST is what makes the loop possible. Realistic diagrams scale the same ingredients up: a purchase-order EPC for an online bookshop covers the path from order receipt through stock check, payment, packaging, shipping, and archival, with branches for out-of-stock items and rejected payments.³⁸

8.2 EPC guidelines

Beyond the mandatory syntax, common *guidelines* ease reading: a unique start and a unique end event; no direct flow between two events (mediate with a function); no direct flow between two functions (announce with an event): so events and functions *alternate*; no event flowing directly into an (X)OR-split, because events cannot decide. The travel example above already violates two of them (function-to-connector-to-function flow without an intermediate event, and two end events) while remaining syntactically valid.

Empirical studies on real EPC repositories find the guidelines too restrictive: enforcing them clutters diagrams with redundant nodes. The practical resolution is to drop them at design time and *repair* violations with implicit dummy nodes:

- two consecutive events → insert a dummy *function* between them;
- two consecutive functions → insert a dummy *event*;

³⁷Weske M., *Business Process Management: Concepts, Languages, Architectures*, Springer, Ch. 4.3, 4.7, 5.7; Dumas M. et al., *Fundamentals of Business Process Management*, Springer, Ch. 3, 4.

³⁸Drawn with ARIS Express, <http://www.ariscommunity.com/aris-express>.

- event followed by an (X)OR-split → insert a dummy function that observes the event and takes the decision;
- multiple start events → assume an implicit *XOR*: each instance enters through exactly one of them;
- multiple end events → assume an implicit join, but *the kind of join is not standardised*: XOR, AND, and OR readings all circulate. Unlike the start-event case there is no unanimity: make the join explicit when the choice matters.

EPC also supports *annotations* on functions, none of which affects control flow: **organization units** (an ellipse with a vertical bar: who is responsible), **information/material objects** (a rectangle: data or resources produced/consumed), and **supporting systems** (a rectangle with two vertical bars: the ERP module or software that enacts the function).

Example (Which connectors?). *Airport security*: JOIN THE LINE → ??-split → {PERSONAL SCREENING, LUGGAGE SCREENING} → ??-join. Both screenings must happen: AND-split, AND-join.

Query-response loop: RECEIVE QUERY → ??-join → PREPARE RESPONSE → EVALUATE RESPONSE → ??-split → {redo, done}. Re-entering is exclusive with terminating: both connectors are XOR: the join collects entry and loop-back, the split decides redo or stop.

What's wrong? A login/registration diagram joins its two start events without an explicit connector, routes them through a central connector with two inputs *and* two outputs (ill-formed), and merges parallel paths with an OR-join that leaves the synchronisation unspecified: three violations of the rules above.

8.3 EPC sample diagrams

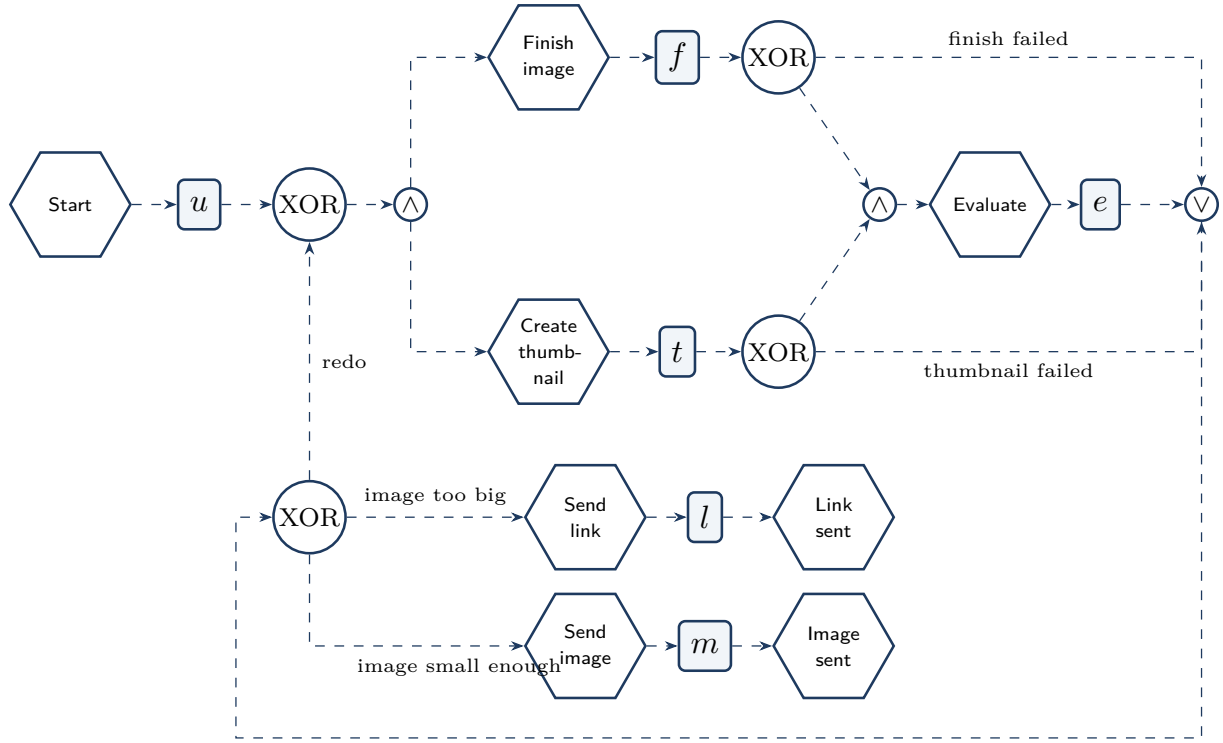
Published EPCs make good comment exercises. The waterfall software-engineering model appears as an EPC in the paper introducing EPML, with each phase followed by a verification step and possible loop-backs.³⁹ Weske's order-processing EPC (ANALYZE ORDER → XOR accepted/rejected; CHECK STOCK → XOR in stock: billing via XOR GET PAYMENT/SEND BILL; or out of stock: AND-split PURCHASE RAW MATERIAL/PLAN PRODUCTION, then MANUFACTURE PRODUCTS → SHIP PRODUCTS) deliberately simplifies a real scenario to exercise every construct,⁴⁰ and tool galleries offer more.⁴¹ One published image-processing EPC repays a closer look:⁴²

³⁹Mendling J., Nüttgens M., "EPC markup language (EPML)", *ISeB* 4:245–263, 2006.

⁴⁰Weske M., *Business Process Management*, Springer, 2007, Fig. 4.35; the extended variant (Fig. 4.38–4.39) adds organisational lanes, data objects, and responsible units, answering "what happens, who does it, on what data" in one picture.

⁴¹<https://online.visual-paradigm.com>

⁴²From a survey on Petri-net transformations of business processes, Fig. 4.



Comment material: the diagram mixes all three connector kinds; the failure arcs feed an OR-join whose firing rule is contested (it merges two exclusive failure paths with the success path out of e); and failure handling is interleaved with the main flow rather than separated. The issues anticipate the semantics question below.

8.4 EPC semantics

Informally, a process *starts* when one of its initial events occurs, activities run in the order prescribed by the connectors, and the process *finishes* when only final events remain pending; an EPC is correct if every execution ends that way, with no unhandled obligations.

Definition (Folder-passing semantics). A formal-but-intuitive semantics places *folders* on the diagram: marks of process state, like Petri-net tokens on places. A **transition relation** explains how folders move from one configuration to the next: following an arc, firing a function, activating a connector. The relation is in general *nondeterministic*.

Events and functions are one-in, one-out: a folder on the incoming arc passes to the outgoing arc (the function performing its work on the way).

The connectors:

- **AND-split:** duplicates the folder, one copy per outgoing arc (parallel continuation);
- **AND-join:** waits for a folder on *every* incoming arc, consumes them all, emits one (synchronisation);
- **XOR-split:** moves the folder to *exactly one* outgoing arc, chosen nondeterministically.

Note (XOR-join: the contested meaning). The intended rule: fire when *exactly one* incoming arc carries a folder. Two edge cases make it hard. If folders arrive on both inputs the join should block:

inconsistent with an exclusive upstream choice. If a folder arrives on one input, the join cannot fire immediately: it must be *informed* that no folder will ever arrive on the other. Inferring the *absence* of folders, and propagating that information, is precisely what a local firing rule cannot do: unlike Petri nets, EPCs need non-local analysis.

The standard answer makes absence explicit: when an XOR-split fires onto one branch, the other branches are stamped with an *absence marker*: a token meaning “no folder will ever reach this point”. Markers travel like folders: a function receiving one does not execute and propagates the absence downstream; events, splits, and joins propagate analogously.

- **OR-split**: places a folder on a nondeterministically chosen *non-empty subset* of the outgoing arcs and stamps the rest with absence markers;
- **OR-join**: if one input receives a folder, release one downstream; if several do, still release *only one*; but on a single arrival the join must *wait* until either another folder arrives or the absence of every other input is guaranteed.

Decorated EPC, next, is what tames the OR-join’s waiting rule.

8.5 Decorated EPC

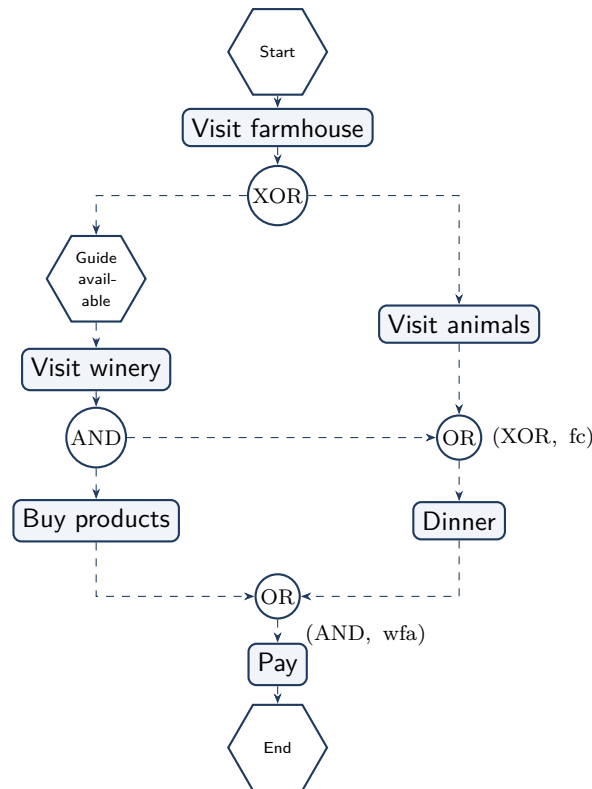
Definition (Decorated EPC). A **decorated EPC** resolves the join ambiguities by attaching to each join two pieces of information: the *corresponding split* (which upstream split separated the flows the join now merges) and the *applicable policy* (how the join triggers its outgoing folder). Given a join j , a **candidate split** for j is a split node whose distinct outgoing branches reach distinct incoming arcs of j along some control-flow path; the **corresponding split** is the candidate chosen and recorded on j ; the corresponding split is **matching** when its connector type coincides with the join’s type.

The policy of an OR-join is constrained by the decoration:

- with a *matching* split, the policy is forced to **wait-for-all** (wfa): wait for every activated path, then emit a single folder;
- otherwise the modeller picks **every-time** (emit one folder downstream per incoming folder) or **first-come** (fc: emit on the first arrival, ignore later ones).

Every OR-join in a decorated EPC carries an explicit policy tag; some authors propose distinct trapezoid symbols per policy.

Example (Decorations at work: the farm visit).



The diagram contains two OR-joins but no OR-split, so neither join can have a matching split. The first OR-join merges the VISIT ANIMALS branch with one output of the AND-split; only the XOR-split is a *candidate* for it, because the AND-split sends just one of its two branches there (the other goes to BUY PRODUCTS) and so separates nothing that this join merges. The second OR-join merges BUY PRODUCTS and DINNER; both the XOR-split and the AND-split are candidates, and the modeller must choose. Declaring the XOR-split as corresponding to the first join (not matching: XOR vs OR) suggests policy **fc**: at most one input ever fires, so triggering on the first arrival is safe. Declaring the AND-split as corresponding to the second join (again not matching) suggests **wait-for-all**: when the guide is available both branches fire and the join must synchronise them.

8.6 BPMN

BPMN is designed to be readily understandable by three audiences at once: *business analysts* who draft processes, *technical developers* who implement them, and *business people* who manage and monitor them. Before its standardisation the field was crowded with partial overlapping proposals (BPMI's BPML, BPD, OASIS's BPEL, Microsoft's XLANG, UML2 Activity Diagrams, W3C's WS-CDL, IBM's WSFL), none of which unified the three audiences. BPMN's standardisation reduced the fragmentation, consolidated the best ideas, and enabled inter-operable BPM systems.

Note (Short history). 2000: BPMI.org founded to study open specifications for e-business process management.⁴³ 2005: BPMI merges into the OMG (BMI Domain Task Force). 2006: BPMN 1.0 approved; 2007: 1.1; 2009: 1.2 and 2.0 Beta 1; 2011: **BPMN 2.0** final. Version 2.0 updated events, activities, gateways, and artefacts, and added choreographies as a diagram type, a full metamodel, an XML serialisation, diagram interchange, and a (verbal) execution semantics: BPMN's answer to EPC's contested folder-passing rules. No major rewrite has followed since.

⁴³<http://www.bpmi.org/>

BPMN defines **Business Process Diagrams** based on the flowcharting technique, with four element categories: **swimlanes** (pools, lanes), **flow objects** (events, activities, gateways), **connecting objects** (sequence flow, message flow, association), and **artefacts** (data objects, groups, annotations). Reference posters catalogue the full vocabulary on a single sheet.⁴⁴ A first-order mapping to EPC: pools/lanes $\leftrightarrow$ organisation-unit annotations, events $\leftrightarrow$ events, activities $\leftrightarrow$ functions, gateways $\leftrightarrow$ connectors, sequence flow $\leftrightarrow$ control-flow arcs; while *message flow* has no native EPC counterpart. BPMN's vocabulary is larger: typed events (timer, error, signal, ...), transactions and compensation, conversations, and the choreography viewpoint.

Note (BPMN pros and cons). *Strengths*: simplicity (a small set of basic symbols covers most cases); extensibility by decoration; intuitive graphics; generality (orchestration, collaboration, choreography in one notation); wide tool support with a .bpmn XML interchange format. *Weaknesses*: over 100 graphical elements; a verbose specification (~500 pages); hard to learn comprehensively, with different readings of the same diagram sometimes possible; execution support differs (and is partial) across tools.

8.6.1 Swimlanes

Definition (Swimlane, pool, lane). A **swimlane** organises activities into separate visual categories representing capabilities, organisational units, or responsibilities. BPMN has two swimlane objects: a **pool** represents a *participant* (organisation, department, role), drawn as a labelled rectangle containing that participant's activities; a **lane** is a hierarchical sub-partition spanning the length of its pool.

A pool can be drawn *collapsed*, hiding its internal process: the participant becomes a black box, useful in collaborations where only its external interactions matter. Sequence flow crosses freely between lanes of the same pool (lanes partition responsibility, not control flow) but **sequence flow between two pools is forbidden** in every variant: autonomous participants connect only by message flows.

8.6.2 Flow objects

BPMN deliberately fixes only three core shapes, and obtains the rich vocabulary by *decorating* them rather than adding shapes:

- **event** (circle): something that “happens” during the process. The border style encodes the position: thin = *start*, double = *intermediate*, thick = *end*; internal pictograms add types (message, timer, error, signal, ...).
- **activity** (rounded rectangle): a unit of work. A **task** is atomic; a **sub-process** is compound, drawn collapsed with a + marker or expanded inline with its own start and end events. Collapsed sub-processes support top-down design (black boxes refined later) and bottom-up reading.
- **gateway** (diamond): splits or merges the sequence flow. The marker selects the semantics: empty (or ×) = *exclusive* XOR, route to exactly one branch / await one; + = *parallel* AND, activate all / await all; o = *inclusive* OR, activate one or more / await all active; * = *complex*, condition-driven, used sparingly.

⁴⁴Polančič G., Rozman T., *BPMN poster*, University of Maribor, 2008, <http://bpmn.itposter.net>; BPMN 2.0 poster, <http://bpmb.de/poster> (also available in Italian translation).

Note (Naming conventions). Processes and pools: a noun, possibly with an adjective, often a nominalised verb (“order fulfilment”). Activities: imperative verb + noun, optionally with adjective or complement (APPROVE ORDER, RENEW DRIVER LICENSE VIA OFFLINE AGENCIES). Events: noun + past-participle verb, signalling that something just happened (INVOICE EMITTED, ORDER RECEIVED). Everywhere: short labels, no articles.

8.6.3 Connecting objects

Three kinds of connections answer three questions: in what order? who talks to whom? what data?

- **sequence flow** (solid line, solid arrowhead): the order of activities; both endpoints in the *same* pool. BPMN deliberately avoids the term “control flow”;
- **message flow**: communication between *different* pools (developed with collaborations);
- **association**: links a flow object to an artefact.

The sequence-flow requirements echo EPC: events have at most one incoming and one outgoing flow, activities exactly one of each, and only gateways may branch or merge.

Note (Implicit gateways: avoid them). When an activity does have multiple incoming sequence flows, BPMN reads them as an implicit *XOR-join* (mutually exclusive arrivals); multiple outgoing flows as an implicit *AND-split* (all activated in parallel; with conditions attached, an OR-split). The combination is treacherous: an activity with two outgoing flows feeding two tasks that re-enter a third activity reads as AND-split + XOR-join: a notorious lack-of-synchronisation pattern where the XOR-join forwards each of the two parallel tokens, so the downstream activity runs twice instead of once. The dual mistake, XOR-split + AND-join, deadlocks instead. Draw gateways explicitly; in the course’s final project, implicit gateways are rejected.

8.7 Typical patterns

Example (Sequence: order fulfilment). The simplest pattern chains PURCHASE ORDER RECEIVED (start) → CONFIRM ORDER → GET SHIPMENT ADDRESS → SHIP PRODUCT → EMIT INVOICE → RECEIVE PAYMENT → ARCHIVE ORDER → ORDER FULFILLED (end). A token born in the start event walks every flow and dies in the end event.⁴⁵

Example (Repetition: ministerial correspondence). A repetition block starts with an XOR-join and ends with an XOR-split: the join collects the entry branch and the loop-back branch, the split exits or loops. XOR, not AND, because the merged branches are mutually exclusive: the first iteration arrives from outside, later ones from the loop-back, never both at once. An AND-join would deadlock; an OR-join would work but is overkill.

Example (Parallelism: airport security). After PROCEED TO SECURITY CHECK an AND-split sends tokens to PASS PERSONAL SECURITY SCREENING and PASS LUGGAGE SCREENING; an AND-join synchronises both before PROCEED TO DEPARTURE LEVEL. Canonical parallel split + synchronisation.

Example (Multiple start and end events). Order fulfilment with two triggers, PURCHASE ORDER RECEIVED and REVISED ORDER RESUBMITTED (the latter passing through RETRIEVE ORDER DETAILS), merges them with an XOR-join before CONFIRM ORDER: mutually exclusive triggers of the same process.

⁴⁵Dumas et al., *Fundamentals of Business Process Management*, Springer, Fig. 3.1 and following.

Adding a stock check yields two end events, ORDER FULFILLED and ORDER REJECTED, selected by an XOR-split.

Note (Implicit termination). A process instance ends only when *every* token in the model has reached an end event; unlike the TERMINATE end event, which kills all remaining tokens at once. With mutually exclusive end events the difference is invisible; it matters when concurrent branches finish at different times.

Example (Exclusive decisions: invoice checking). CHECK INVOICE FOR MISMATCHES feeds an XOR-split with three mutually exclusive outcomes: no mismatches → POST INVOICE; correctable → RE-SEND INVOICE TO CUSTOMER; not correctable → BLOCK INVOICE. Every outgoing flow of an (X)OR-split should be labelled with the condition selecting it, and the labels must be mutually exclusive; a *default flow* (slash decoration) reads “otherwise” and catches the case where every other condition is false.

8.7.1 Inclusive decisions

A company ships orders from warehouses in Amsterdam and Hamburg; an order may need one or both. An XOR-split over the three cases {Amsterdam, Hamburg, both} is correct but does not scale: each added warehouse doubles the duplicated activities. An AND-split into two XOR-decisions (“send sub-order or skip”) scales but is wrong: it admits the forbidden case in which both branches skip. The right tool is the **inclusive (OR) gateway**: the OR-split forks a token onto every branch whose condition holds (at least one), and the OR-join waits for *all active* incoming branches before emitting a single token. Inclusive joins inherit EPC’s contested semantics: the wait depends on upstream decisions, and tools implement it differently; so **use OR gateways only when strictly necessary**. A full order-fulfilment model combines the three kinds: XOR for confirm/reject, AND for shipping || billing, OR for the manufacturing branches of partially-stocked products. Balance your gateways: every split matched by a join of the same kind.

8.8 Lanes and sub-processes

Placement of flow objects in lanes follows ownership: events and activities go in the lane of the participant that triggers or performs them; (X)OR-splits go in the lane of the preceding decision activity, because that participant decides; the placement of AND-splits, of joins of any kind, and of data objects is semantically irrelevant: readability decides. In the order-fulfilment example the lanes SALES CLERK, WAREHOUSE WORKER, FINANCIAL OFFICER, RECORDS OFFICER each own their activities while the sequence flow weaves across lanes within the single pool.

Fragments sharing a goal can be grouped into sub-processes (ACQUIRE RAW MATERIALS, SHIP AND INVOICE), drawn expanded (a large rounded rectangle with its own start/end events) or collapsed to a +-marked activity.⁴⁶

Note (When to decompose). Decompose when the model grows hard to read: empirically, models with more than ~30 flow objects per level show a sharp increase in errors and reader confusion. Keep each level under 30 and push the rest into sub-processes.

⁴⁶Dumas et al., *op. cit.*, Fig. 4.1.

8.9 Exercises: translating between BPMN and EPC

Example (BPMN $\rightarrow$ EPC). Redraw the BPMN diagrams of this chapter (invoice checking, order fulfilment, order fulfilment with manufacturing) as EPCs. The mapping is mechanical (start/end events to EPC events, tasks to functions, gateways to connectors of the same kind) plus three care points: (i) insert dummy events between functions wherever BPMN had direct sequence flow, so the alternation rule survives; (ii) label the events on XOR branches with the outcome they represent (NO MISMATCHES, MISMATCHES CORRECTABLE, ...); (iii) attach an explicit decoration (corresponding split and policy) to every OR-join. The last point is the catch: BPMN leaves the OR-join semantics as “wait for active branches”, while a decorated EPC forces the implicit choice to become explicit.

Example (From text to EPC, and back to BPMN). (a) Model as an EPC the customer-order process of a small e-commerce company: the process starts when a customer places an order online and ends when the order is delivered; it must include checking stock availability, notifying the customer if items are unavailable, preparing the shipment, and processing the payment. A reasonable answer: ORDER PLACED $\rightarrow$ CHECK STOCK $\rightarrow$ XOR-split; unavailable $\rightarrow$ NOTIFY CUSTOMER $\rightarrow$ ORDER CANCELLED; available $\rightarrow$ PREPARE SHIPMENT $\parallel$ PROCESS PAYMENT (AND-split/join) $\rightarrow$ ORDER DELIVERED.

(b) Reverse direction: given the travel-request EPC (RECEIVE DATES, optional BOOK CAR via XOR, AND fork BOOK FLIGHT/BOOK HOTEL, then XOR among CONFIRM/CANCEL/CHANGE DATES looping back), draw the BPMN diagram: drop the dummy events, connect tasks directly with sequence flow, preserve the gateway kinds, and absorb the loop-back with an XOR-join before RECEIVE DATES. Solutions can be sent to bruni@di.unipi.it.

9 Other BPMN features

The rest of the BPMN vocabulary (artefacts, typed events, task markers, message flows, call activities, choreography diagrams) has no native EPC counterpart and motivates the choice of BPMN for serious process modelling.⁴⁷

9.1 Artefacts

Definition (Artefact, association). BPMN lets modellers extend the basic notation with any kind of **artefact** appropriate to the domain; three types are pre-defined: **data objects** (information consumed or produced by activities), **groups** (a dashed rounded rectangle visually grouping elements for documentation or analysis, with no effect on flow), and **text annotations** (free-form notes for the reader). An **association** (a dotted line, with optional arrowheads) connects a flow object to an artefact. Artefacts never affect the sequence flow.⁴⁸

9.2 Task types and activity markers

The body of an activity carries two orthogonal decorations: a *task type* pictogram in the upper-left corner stating the **nature** of the action, and an *activity marker* on the lower border stating **how** it executes. Five activity markers exist: sub-process (+), loop ($\odot$), multi-instance (parallel or

⁴⁷Weske M., *Business Process Management*, Springer, Ch. 4.3, 4.7, 5.7; Dumas M. et al., *Fundamentals of Business Process Management*, Springer, Ch. 3, 4.

⁴⁸White S.A., *Introduction to BPMN*, BPTrends, July 2004.

sequential), ad-hoc ($\sim$), and compensation. The two decorations can coexist (“user task with a loop”). The task types:

- **send** (filled envelope) / **receive** (empty envelope): the task sends, or blocks waiting for, a message from another participant; ordinary intra-pool tasks stay unmarked;
- **user** (silhouette at a screen): a person interacts through a user interface, aided by software; **manual** (hand): performed entirely without computer assistance. The distinction matters for automation: user tasks are candidates for workflow support, manual tasks are not;
- **service** (cogs): automated interaction with an external service or API; **script** (scroll): automated work executed by a script inside the BPMN engine itself. Both run without human intervention; they differ in where the work happens. A **business rule** task evaluates a decision table or rule set.

Example (Loop marker: ministerial correspondence). The explicit repetition block (XOR-join + body + XOR-split) collapses into a single activity decorated with the loop marker $\odot$. The exit condition is essential (the activity iterates until it holds) and is declared in the activity’s attributes (**testBefore/testAfter** for while vs do-while semantics). When the loop wraps a small internal process its start/end events may be left implicit, and at the extreme the whole body becomes a black box named MINISTERIAL RESPONSE: the idiomatic way to keep an orchestration short while preserving the rework semantics.

9.3 Typed events: catching and throwing

An event either **catches** a trigger (it waits, and proceeds when the trigger fires) or **throws** a result observable by the rest of the world. Start events are always catching, end events always throwing, intermediate events come in both flavours: a *white* (unfilled) marker means catching, a *black* (filled) marker means throwing. The internal marker gives the kind:

- **none** (untyped);
- **message** (envelope);
- **timer** (clock), catching only;
- **error** (lightning);
- **cancel** (X-arrow);
- **compensation**;
- **conditional** (document), catching only;
- **signal** (triangle), a broadcast;
- **multiple** (pentagon);
- **link** (off-page connector);
- **terminate**, an end event that kills all in-flight tokens immediately, unlike normal ends under implicit termination.

An intermediate catching message event is a *process break*: the flow pauses until the message arrives.

Example (Timer events: small claims tribunal). A **timer start event** creates an instance whenever a temporal condition occurs (“every Friday morning”); a **timer intermediate event** suspends the flow

until a time-out elapses (“wait one week”). The small-claims-tribunal process chains one timer start and two timer intermediates for the three milestones of a callover: three weeks before, one week before, the day itself. Timers are catching only: a process reacts to time, it does not produce it.⁴⁹

9.4 Message passing in collaborations

Definition (Message flow). A **message flow** represents the sending and receiving of messages between two *separate* participants: a dashed line with an open arrowhead whose endpoints lie in different pools.

The syntactic rules enforce the orchestration/collaboration distinction: sequence flow cannot cross a pool boundary (but crosses lanes freely), and message flow cannot stay inside one pool. On the message side the envelope convention applies uniformly: a start event with a white envelope creates an instance upon message receipt; an end event with a black envelope sends a message on completion; intermediate events and tasks carry either flavour. A receive task and a catching message event are operationally interchangeable, as are a send task and a throwing message event: tasks emphasise the work attached to the message, events the synchronisation point.

Example (From processes to collaborations: the doctor’s office). A stand-alone PATIENT pool chains SEND DOCTOR REQUEST → RECEIVE APPOINTMENT → SEND SYMPTOMS → RECEIVE PRESCRIPTION → SUBMIT MEDICINE REQUEST → RECEIVE MEDICINE: the messages are annotated, the partner is not modelled. Adding a *collapsed* DOCTOR pool and six dashed message flows makes the partner explicit as a black box. Expanding the doctor pool into its own orchestration (RECEIVE REQUEST → SEND APPOINTMENT → ...) pairs every send task with a receive task across the boundary: a full collaboration. A final refinement splits the doctor pool into lanes FRONT DESK (administrative exchanges) and DOCTOR (medical reasoning).

Example (Seller, customer & suppliers). A seller pool with lanes ERP SYSTEM, WAREHOUSE & PRODUCTION, SALES runs the extended order-fulfilment orchestration (stock check, raw-materials acquisition from two suppliers in parallel, manufacturing, shipping, billing). Adding a collapsed CUSTOMER pool introduces five message flows (purchase order, confirmation, shipping address, invoice, payment) without touching the orchestration; the boundary send/receive tasks form the *interface* that a choreography would extract. Two collapsed SUPPLIER pools add the raw-materials request/response exchanges. A **message data object** (an envelope drawn on a message flow, white for incoming, grey for outgoing) documents *what* each flow carries (PURCHASE ORDER [CONFIRMED], INVOICE, ...).

9.5 Deferred choice

The fifth gateway kind, the **event-based exclusive gateway** (diamond with a pentagon-in-circle marker), selects its branch not by evaluating a data condition but by *which catching event fires first*: a message arrives, a timer elapses. It must be followed only by catching events or receive tasks; the first to fire wins, the others are cancelled. Data-based decisions choose internally; event-based decisions defer the choice to the environment.

Example (Travel agency: from data-based to event-based). The travel-agency orchestration (TRAVEL REQUEST → XOR-join → AND fork BOOK FLIGHT/BOOK HOTEL → TRAVEL PLANNED → ASK

⁴⁹Dumas et al., *op. cit.*, Fig. 4.12.

CONFIRMATION → choice among CONFIRM/CANCEL/CHANGE DATES looping back) first models the choice as a data-based XOR on the customer's reply. Replacing it with an event-based gateway followed by three catching message events moves the decision *outside*, to the customer; adding a timer branch bounds the wait with an automatic cancellation on time-out. In the two-pool collaboration with the TOURIST, the three replies become message flows and the agency simply waits at the gateway for whichever arrives first.

Example (Annotation exercises). (a) *Sushi negotiation.* Two pools (SUSHI LOVER, SUSHI OBSESSED) exchange a chain of proposal/reply messages about where to eat. Annotate boundary events and tasks with the correct types (catching vs throwing, message vs timer, event vs send/receive task) under the constraints: white envelope = catching, black = throwing, event-based gateways followed by catching elements only.

(b) *Alice sells her car.* Alice asks for a quote; Bob can accept, refuse, or make a counteroffer. Two pools: Alice sends the quote, then an event-based gateway waits for ACCEPT (success), REFUSE (failure), or COUNTEROFFER (loop back, closed by a data-based XOR-join re-entering the quote phase); Bob's pool is symmetric with the deferred choice on the sending side. The annotation exercise is the same: every send matched by a receive on the other side.

9.6 Data objects and data stores

Definition (Data object). A **data object** represents information flowing through the process (documents, emails, records) drawn as a dog-eared page icon attached to a flow object by an association. Directed associations distinguish inputs (read) from outputs (written); a bidirected association marks read-and-write. A state annotation in brackets tracks the document's life-cycle: PURCHASE ORDER [RECEIVED] is read by CHECK STOCK AVAILABILITY and written back as PURCHASE ORDER [CONFIRMED].

The same logical object may appear in multiple states on one diagram (the state changes, not the identity) turning the data side into a small state-transition system, useful for compliance documentation. Data objects may equally be attached to a sequence flow (a hand-over between consecutive activities) or to a message flow (becoming message data objects); an activity may read several inputs and write several outputs, each through its own association.

Definition (Data store). A **data store** (drawn as a cylinder) is a place where the process reads or writes *persistent* data: a database, a filing cabinet, a catalog. Unlike a data object, which lives within a single process instance, a data store outlives instances and is shared across them.

In the order-fulfilment example, SUPPLIERS CATALOG is read by CHECK RAW MATERIALS AVAILABILITY. WAREHOUSE DB and PRODUCTS WAREHOUSE are read and written by the stock activities. ORDERS DB is written by ARCHIVE ORDER.

Note (Artefacts and readability). Artefacts provide additional information, but excessive use compromises readability: the fully annotated order-fulfilment diagram (four data stores, state-annotated data objects, message data objects) is informative but dense. Add artefacts only when they document something the reader cannot infer from the activity labels.

Example (Question time: which connections?). Given the order-fulfilment diagram stripped of its data connections, restore them: each link between an activity and RAW MATERIALS REQUEST / RAW MATERIALS PROVIDED / PRODUCT is an *association* (directed, undirected, or bidirected; use the state

transitions as a guide) when it stays inside the pool, and a *message flow* with a message data object when it crosses to the supplier or customer pools.

9.7 Call activities

Sub-processes nest: a home-loan process refines CHECK LIABILITY and SIGN LOAN into expanded sub-processes one level below, SIGN LOAN itself nesting SCHEDULE LOAN DISBURSEMENT: each level shows just the activities relevant at its abstraction.⁵⁰ When a sub-process such as SIGN LOAN is defined as a *separate model*, any process can reuse it through a **call activity**: a rounded rectangle with a *thick border* that invokes the globally defined sub-process. Home-loan and student-loan applications both end with the same call SIGN LOAN. Global processes buy readability (smaller parents), maintainability (one update propagates to all callers), and consistency (no divergent re-implementations) at the price of indirection, which the thick border flags at a glance.

9.8 Error events

The error event exercises the catching/throwing distinction on exceptional flow. A sub-process can carry an intermediate *catching* error event on its *boundary* (double border, unfilled lightning): when the sub-process throws an error, control diverts from the boundary onto the recovery branch, bypassing the normal flow. In the image-manipulation process the evaluation sub-process catches a FAIL on its boundary; in order fulfilment, ACQUIRE RAW MATERIALS fires an internal *throwing* error event MATERIALS UNAVAILABLE (filled lightning), caught on the boundary by a handler that runs NOTIFY UNAVAILABILITY TO CUSTOMER and ends in ORDER UNFULFILLED. The same recovery can be drawn inline in the main flow instead: boundary handlers keep the normal path clean, inline handling makes the recovery visible.

9.9 Choreographies

Blindfolded observers each touching one part of an elephant describe a snake (trunk), a fan (ear), a wall (side), a rope (tail): locally correct, globally insufficient. Likewise each participant of a collaboration sees only its own exchanges; the choreography is the global view tying the partial views together.

Definition (Choreography, choreography task). A **choreography** defines the sequence of *interactions* between participants: no central control flow, no internal logic: only the messages, read as a global contract. A **choreography task** is an atomic choreography activity representing one interaction (a coherent set of message exchanges) between exactly *two* participants: a rounded rectangle in three bands: initiating participant on top, interaction name in the middle, receiving participant below, with band fill distinguishing initiator from receiver.⁵¹ Choreography tasks are wired by ordinary sequence flow, and all gateway kinds (XOR, AND, OR, event-based) apply.

A multi-party supply chain shows the scale: SUPPLIER, RETAILER, SHIPPER, and CONSIGNEE chain planned orders, delivery variations and checkpoints, a parallel block of provide/deliver-item

⁵⁰Dumas et al., *op. cit.*, Fig. 4.3.

⁵¹OMG, *Business Process Model and Notation*, v.2.0, §12.3.1, §12.4.1.

sub-choreographies, and the final update/accept/confirm sequence on the purchase order and delivery schedule.

Example (Collaborations vs choreographies, both ways). *Projection.* The patient–doctor collaboration projects onto a choreography of six tasks, one per message (DOCTOR REQUEST, SET APPOINTMENT, EXPLAIN SYMPTOMS, GIVE PRESCRIPTION, MEDICINE REQUEST, GIVE MEDICINE); the internal flows disappear. In an exam choreography between STUDENT and TEACHER, the student’s projection is REGISTER FOR THE EXAM, GET EXERCISE, SOLVE EXERCISE, GET RATE; the teacher’s is the matching receive/send sequence.

Composition. Going the other way (filling in internal flows that realise the projections) is non-trivial. A naïve exam collaboration gives each pool an XOR choice between entering the EXAM sub-process and skipping straight to the rating: if the two pools resolve their choices differently, they disagree on which message comes next: deadlocks and unspecified receptions that the choreography alone does not prevent. Deriving a correct collaboration from a choreography is the *realisability* problem, studied at length in the literature.

Exercise. The naïve collaboration also allows zero exercises. Modify it to enforce at least one: enter EXAM unconditionally once, then loop through an XOR-split (another exercise / proceed to rating); the choreography gains the constraint that GET EXERCISE/SOLVE EXERCISE precede GET RATE.

Example (Exercise: survey of EPC/BPMN editors). Search for EPC and BPMN editors (yEd, ARIS Express, Visual Paradigm Online, Camunda Modeler, Bizagi Modeler, Signavio, ...) and annotate each with: supported operating systems; pricing (free or not); support for the .epml/.bpmn interchange formats; and, if installed, a 1–5 usability rating. Findings can be sent to `bruni@di.unipi.it`.

10 From Automata to Nets

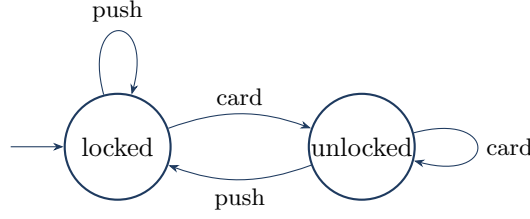
Petri nets are a graphical and formal model of concurrent systems.⁵² Petri nets fit business-process analysis because they are *visual* (practitioners can read them), *formal* (machines can analyse them), and *tool-supported*. On that basis they support validation (testing behaviour on scenarios), verification (proving properties), and performance analysis (planning, dimensioning, optimisation). Automata model *sequential* behaviour: one current state, one transition at a time. They serve protocol analysis, parsing, game-character logic, CPU control, speech recognition, vending machines, elevators, and traffic lights. What they cannot express directly is *concurrency*: independent actions must be interleaved explicitly, exploding the state space. A Petri net is to a parallel system what a finite automaton is to a sequential one.

10.1 Finite automata

To define an automaton: identify the admissible *states* (optionally marking absorbing error states), add *transitions*, fix the *initial* state, optionally mark *final* states.

Example (Turnstile). A coin-operated turnstile has states LOCKED and UNLOCKED and actions CARD and PUSH. In LOCKED a push has no effect and a card unlocks; in UNLOCKED a push lets one person through and re-locks, a further card does nothing.

⁵²A deeper treatment of the tractable subclass of *free-choice nets*: <https://www7.in.tum.de/~esparza/bookfc.html>



The same machinery scales to real controllers: a query-recognition automaton whose states are syntactic positions and whose transitions are labelled by words (accepting complete sentences such as “*I want some information please*”); an ATM whose states are the screens of the dialogue (PIN check looping on failure, then deposit/withdrawal branches re-converging on card return); video-game characters (STANDING/JUMPING/DUCKING/DIVING driven by buttons, or an NPC cycling WANDER/ATTACK/EVADE on player proximity and health). One state at a time, one transition fired, one symbol consumed.

10.2 Preliminaries

Definition (Sets, functions, relations). Standard set notation is assumed: $\emptyset$, $a \in A$, $A \cap B$, $A \cup B$, $A \setminus B$, $\overline{A}$, $A \subseteq B$, $A \subset B$, $A \times B$, the powerset $\wp(A)$, and the numeric domains $\mathbb{N}, \mathbb{Z}, \mathbb{Q}, \mathbb{R}, \mathbb{B} = \{0, 1\}$. A function $f : A \rightarrow B$ corresponds to the relation $R_f = \{(a, f(a)) \mid a \in A\} \subseteq A \times B$; conversely a set $N \subseteq S$ is encoded by its *characteristic function* $f_N : S \rightarrow \mathbb{B}$ with $f_N(x) = 1$ iff $x \in N$: sets and characteristic functions are two views of the same object.

Definition (Logic). We use *true/false*, $\wedge$, $\vee$, $\neg$, $\exists$, $\forall$, $\Rightarrow$, $\Leftrightarrow$. Implication is truth-functional, not causal: $P \Rightarrow Q \equiv Q \vee \neg P \equiv \neg Q \Rightarrow \neg P$ (the contrapositive justifies proof by contradiction). De Morgan: $\neg(P \wedge Q) = \neg P \vee \neg Q$, $\neg(P \vee Q) = \neg P \wedge \neg Q$, $\neg \exists x. P = \forall x. \neg P$, $\neg \forall x. P = \exists x. \neg P$. Quantifier order matters: $\forall n. \exists m. n < m$ is true over $\mathbb{N}$ (pick $m = n + 1$), $\exists m. \forall n. n < m$ is false: $\forall \exists$ lets the witness depend on the universal variable, $\exists \forall$ demands a uniform one.

Definition (Sequences and Kleene star). A^n is the set of sequences of exactly n elements of A (with $A^0 = \{\epsilon\}$, the empty sequence); the *Kleene star* collects all finite sequences:

$$A^* \triangleq \bigcup_{n \geq 0} A^n,$$

countably infinite even when A is finite (e.g. $\{0, 1\}^* = \{\epsilon, 0, 1, 00, 01, \dots\}$).

Definition (Inductive (recursive) definitions). An inductive definition gives *base cases* and *inductive cases* building new elements from previously defined ones: a natural number is 0 or a successor $n + 1$; a sequence over A is ϵ or a juxtaposition wa with $a \in A$. Functions on inductive sets follow the same pattern, one clause per case:

$$\begin{aligned} 0! &\triangleq 1, & (n+1)! &\triangleq n! \cdot (n+1); & |\epsilon| &\triangleq 0, & |wa| &\triangleq |w| + 1; \\ \exp(k, 0) &\triangleq 1, & \exp(k, n+1) &\triangleq \exp(k, n) \times k. \end{aligned}$$

Inductive and *recursive* name the same device; what makes it well-founded is a measure that strictly decreases from the complex case to the simpler one. Without such a measure termination is not guaranteed: the Collatz function $f(n) = 1$ if $n \leq 1$, $f(n/2)$ if n even, $f(3n+1)$ otherwise *looks* recursive (e.g. $f(12) = f(6) = f(3) = f(10) = f(5) = f(16) = f(8) = f(4) = f(2) = f(1) = 1$), but whether it is total on $\mathbb{N}$ is an open problem.

10.3 DFA, formally

Definition (Deterministic finite automaton (DFA)). A **DFA** is a tuple $\mathcal{A} = \langle Q, \Sigma, \delta, q_0, F \rangle$: a finite set Q of states, a finite alphabet Σ of input symbols, a *transition function* $\delta : Q \times \Sigma \rightarrow Q$, an initial state $q_0 \in Q$, and a set $F \subseteq Q$ of final (accepting) states. *Deterministic* means δ is a total function: in any state, each symbol leads to exactly one next state.

The turnstile is the DFA with $Q = \{\text{LOCKED}, \text{UNLOCKED}\}$, $\Sigma = \{\text{PUSH}, \text{CARD}\}$, $q_0 = \text{LOCKED}$, $F = \{\text{LOCKED}\}$ (an interaction is accepted when the turnstile returns to the locked position), and δ read off the four arcs of the diagram.

Definition (Extended transition function $\hat{\delta}$). To process whole words, δ extends inductively to $\hat{\delta} : Q \times \Sigma^* \rightarrow Q$:

$$\hat{\delta}(q, \epsilon) \triangleq q, \quad \hat{\delta}(q, wa) \triangleq \delta(\hat{\delta}(q, w), a).$$

$\hat{\delta}(q, w)$ is the state reached from q reading w left to right; the recursion terminates because each unfolding shortens the word.

Example (Unfolding $\hat{\delta}$ on the turnstile).

$$\begin{aligned} \hat{\delta}(\text{LOCKED}, \text{CARD CARD PUSH}) &= \delta(\hat{\delta}(\text{LOCKED}, \text{CARD CARD}), \text{PUSH}) = \delta(\delta(\hat{\delta}(\text{LOCKED}, \text{CARD}), \text{CARD}), \text{PUSH}) \\ &= \delta(\delta(\delta(\text{LOCKED}, \text{CARD}), \text{CARD}), \text{PUSH}) = \delta(\delta(\text{UNLOCKED}, \text{CARD}), \text{PUSH}) \\ &= \delta(\text{UNLOCKED}, \text{PUSH}) = \text{LOCKED}. \end{aligned}$$

The trajectory ends in a final state: the word **CARD CARD PUSH** is accepted.

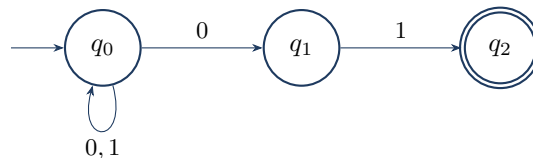
Definition (Language $L(\mathcal{A})$). $\mathcal{A}$ *accepts* $w \in \Sigma^*$ iff $\hat{\delta}(q_0, w) \in F$; the **language recognised** by $\mathcal{A}$ is

$$L(\mathcal{A}) \triangleq \{ w \in \Sigma^* \mid \hat{\delta}(q_0, w) \in F \}.$$

A DFA is drawn as a *transition diagram*: nodes are states, one labelled arc $q \xrightarrow{a} q'$ per $\delta(q, a) = q'$, a START arrow on q_0 , double circles on final states. Acceptance becomes reachability: w is accepted when some path from q_0 to a final state spells out w . Alternatively a DFA is a *transition table*: rows are states ($\rightarrow$ marks the start row, $*$ the accepting rows), columns are symbols, entries the next states. For the turnstile, the accepted words are exactly the well-balanced interaction logs (every card eventually consumed by a push) described by the regular expression $(\text{PUSH}^* \text{CARD CARD}^* \text{PUSH})^* \text{PUSH}^*$.

Definition (Non-deterministic finite automaton (NFA)). An **NFA** is defined like a DFA except that the transition function maps into the *powerset* of states, $\delta : Q \times \Sigma \rightarrow \wp(Q)$: on a given symbol the automaton may move to several next states, or to none. Determinism is the special case where every $\delta(q, a)$ is a singleton.

Example (A three-state NFA).



Here $\delta(q_0, 0) = \{q_0, q_1\}$, $\delta(q_0, 1) = \{q_0\}$, $\delta(q_1, 1) = \{q_2\}$, and $\delta(q_1, 0) = \delta(q_2, 0) = \delta(q_2, 1) = \emptyset$: neither uniqueness nor totality of the next state holds, so this is not a DFA.

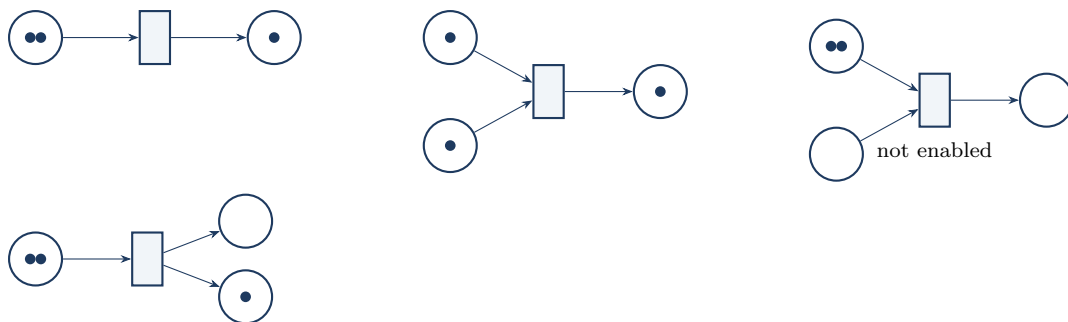
10.4 Reshaping automata into nets

Six conceptual steps turn a DFA diagram into a Petri net:

1. **get a token**: instead of “the automaton is in state q ”, say “ q holds a token”: same diagram, semantics shifted from control-flow position to distributed marking;
2. **drop the start arrow**: the token’s position identifies the current state;
3. **transitions as boxes**: reify each labelled arc $q \xrightarrow{a} q'$ into a box with one arc in and one arc out; the graph becomes bipartite;
4. **forget final states**: nets focus on reachable markings, not language acceptance;
5. **add more tokens**: any number of tokens, distributed over the places: the configuration is the *marking*;
6. **allow more arcs**: a box with several input arcs synchronises, with several output arcs forks.

Definition (Petri net terminology). The resulting object has four kinds of entity: **places** (circles, passive, hold tokens), **transitions** (boxes, active, fire by consuming and producing tokens), **arcs** (directed edges place $\rightarrow$ transition or transition $\rightarrow$ place), and **tokens** (dots inside places; their distribution is the *marking*). Three facts: the graph is *bipartite* (no place–place or transition–transition arcs); the structure is *static*: only the marking evolves; tokens do not flow preserving identity: firing *removes* tokens from input places and *creates* fresh ones in output places.

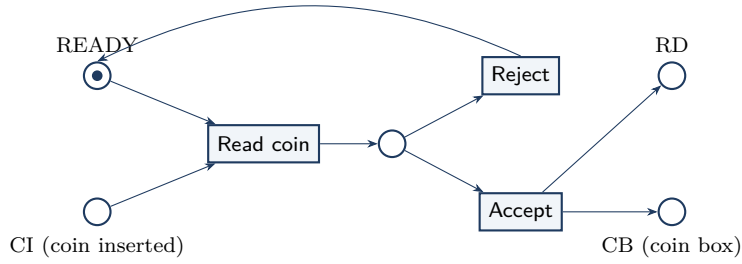
Example (Token game). The smallest steps of the token game. Left: a transition with one input (two tokens) and one output (one token) fires, consuming one and producing one. Centre: a transition with *two* inputs synchronises them: it fires only when both hold a token, consuming one from each. Right: if one input place is empty the transition is *not enabled*, however many tokens the other input has. Bottom: a transition with two outputs forks, producing one token in each.



Definition (Firing rule). A transition is *enabled* at a marking when each of its input places contains at least one token. Firing an enabled transition **consumes** one token from each input place and **produces** one token in each output place, atomically. Several transitions may be enabled at once; which fires is a non-deterministic choice: the freedom of an external scheduler.

Example (Coin handling). The coin-handling sub-process of a vending machine. READ COIN consumes the machine’s readiness (READY, initially marked) together with an inserted coin (CI) and moves to an internal staging place, from which REJECT returns control to READY (the coin goes back to the customer) or ACCEPT drops the coin into the coin box (CB) and signals RD (ready to dispense). Both REJECT and

ACCEPT are enabled by the same staging token: the net models a non-deterministic valid/invalid decision that a DFA could not represent without auxiliary state.



10.5 Multisets

Sets and multisets both ignore order; they differ on repetition: in a set each element appears at most once, in a **multiset** the number of occurrences is part of the data: the right type for markings, where a place may hold many tokens.

Definition (Multisets, formal sums). $\mu(S)$ (also $S^\oplus$) is the set of multisets over S . A multiset M corresponds to a multiplicity function $f_M : S \rightarrow \mathbb{N}$ ($x \in M$ iff $f_M(x) > 0$): the generalisation of the characteristic function $S \rightarrow \mathbb{B}$ of a set, which is the special case with multiplicities in $\{0, 1\}$. A multiset is conveniently written as a *formal sum*

$$M = k_1x_1 + k_2x_2 + \cdots + k_nx_n = \sum_{i=1}^n k_ix_i$$

with positive multiplicities k_i (omitted when 1); equality is up to reordering of summands.

Definition (Multiset operations). Pointwise on multiplicity functions: the **empty** multiset ($\emptyset(x) = 0$); **containment** $M \subseteq M'$ iff $M(x) \leq M'(x)$ for all x (strict if also $M \neq M'$); **sum** $(M + M')(x) = M(x) + M'(x)$; **difference** $M - M'$, *defined only when* $M \supseteq M'$, with $(M - M')(x) = M(x) - M'(x)$. The partiality of difference is exactly the enabledness condition of a transition.

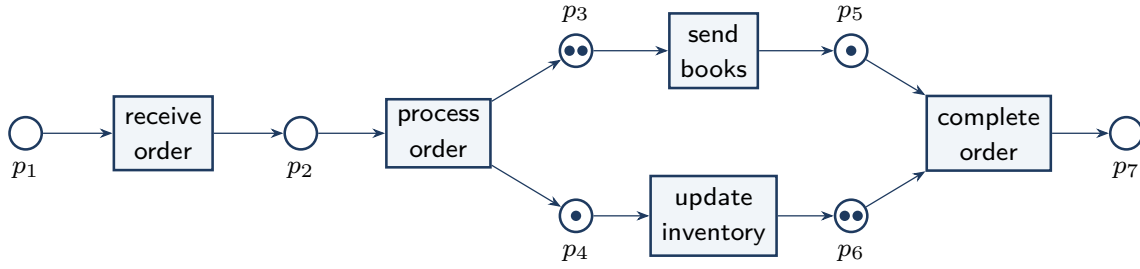
Example (Multiset arithmetic).

- $3a + 2b \subseteq 2a + 3b + c$? *No*: the a -multiplicity 3 exceeds 2.
- $3a + 2b \supseteq 2a + 3b + c$? *No*: the c -multiplicity 0 falls below 1.
- $a + 2b \subset 2a + 3b$? *Yes*: dominated pointwise and not equal.
- $(a + 2b) + (2a + c) = 3a + 2b + c$.
- $(2a + 3b) - (2a + b) = 2b$.
- $(2a + 2b) - (a + c)$: *not defined*: the subtrahend has c -multiplicity 1, the minuend 0.

10.6 Petri nets, formally

Definition (Petri net). A **Petri net** is a tuple (P, T, F, M_0) where P is a finite set of *places*, T a finite set of *transitions* with $P \cap T = \emptyset$, $F \subseteq (P \times T) \cup (T \times P)$ the *flow relation* (bipartite arcs), and $M_0 : P \rightarrow \mathbb{N}$ (equivalently $M_0 \in \mu(P)$) the *initial marking*. In general a **marking** $M : P \rightarrow \mathbb{N}$ gives the number of tokens in each place: the state of the net at an instant.

Example (The order-fulfilment net).



Formally:⁵³

$$\begin{aligned}
 P &= \{p_1, \dots, p_7\}, & T &= \{t_1, \dots, t_5\}, \\
 F &= \{(p_1, t_1), (t_1, p_2), (p_2, t_2), (t_2, p_3), (t_2, p_4), (p_3, t_3), (t_3, p_5), \\
 &\quad (p_4, t_4), (t_4, p_6), (p_5, t_5), (p_6, t_5), (t_5, p_7)\}, \\
 M_0 &= 2p_3 + p_4 + p_5 + 2p_6.
 \end{aligned}$$

Definition (Pre-set, post-set, enabling, firing). For a node x (place or transition), the **pre-set** and **post-set** are

$$\bullet x \triangleq \{y \mid (y, x) \in F\}, \quad x^\bullet \triangleq \{y \mid (x, y) \in F\}.$$

A transition t is **enabled** at M iff $\bullet t \subseteq M$ (the pre-set read as a multiset with unit multiplicities), written $M \xrightarrow{t}$ or $M[t]$. An enabled transition can **fire**, producing

$$M' = M - \bullet t + t^\bullet, \quad \text{written } M \xrightarrow{t} M',$$

well-defined because enabledness guarantees $M \supseteq \bullet t$.

On the order-fulfilment net: $\bullet p_1 = \emptyset$, $p_1^\bullet = \{t_1\}$; $\bullet t_2 = \{p_2\}$, $t_2^\bullet = \{p_3, p_4\}$; $\bullet t_5 = \{p_5, p_6\}$, $t_5^\bullet = \{p_7\}$, and so on mechanically from F . Firing is atomic; only one transition fires at a time (*interleaving* semantics); and the total number of tokens may grow or shrink with $|t^\bullet| - |\bullet t|$: non-conservation lets nets model creation and destruction of resources without bookkeeping.

Note (WoPeD). WoPeD⁵⁴ (Workflow Petri Net Designer) is the free reference tool of the course: editing, token-game simulation (click enabled transitions to fire them), reachability and soundness analysis, standard import/export formats.

Definition (Notation for firing).

- $M \rightarrow$: some transition is enabled at M ;
- $M \rightarrow M'$: some transition t has $M \xrightarrow{t} M'$;
- $M \not\xrightarrow{t}$: t is not enabled at M ;
- $M \not\rightarrow$: no transition is enabled: M is a **deadlock**.

On the order-fulfilment net with $M_0 = 2p_3 + p_4 + p_5 + 2p_6$: $M_0 \rightarrow$ (for instance t_3 is enabled); $M_0 \rightarrow 2p_3 + p_4 + p_6 + p_7$ (by firing t_5); $M_0 \not\rightarrow (p_1 \text{ is empty})$; and $3p_7 \not\rightarrow$: with all tokens in the terminal place nothing is enabled, a deadlock.

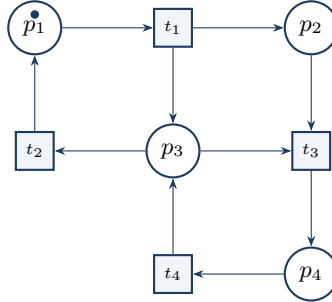
⁵³Weske M., *Business Process Management*, Springer, 2007.

⁵⁴<http://woped.dhbw-karlsruhe.de/woped/>

10.7 Firing sequences

Definition (Firing sequence). For $\sigma = t_1 t_2 \cdots t_{n-1} \in T^*$, write $M \xrightarrow{\sigma} M'$ if there are markings $M = M_1 \xrightarrow{t_1} M_2 \xrightarrow{t_2} \cdots \xrightarrow{t_{n-1}} M_n = M'$. A firing sequence is the run-time analogue of an automaton's input word.

Example (The cycle net).



Transitions: $t_1 : p_1 \rightarrow p_2 + p_3$ (fork), $t_3 : p_2 + p_3 \rightarrow p_4$ (synchronise), $t_4 : p_4 \rightarrow p_3$, $t_2 : p_3 \rightarrow p_1$. From $M_0 = p_1$ only t_1 is enabled; firing it yields $p_2 + p_3$, where t_2 and t_3 compete (t_2 consumes from p_3 , t_3 from both p_2 and p_3). Firing t_2 gives $p_1 + p_2$, where t_1 is enabled again but t_3 is not; a second t_1 gives $2p_2 + p_3$: a single place accumulating two tokens (markings are multisets), with the behaviour branching non-deterministically wherever several transitions are enabled.

Definition (Reachable markings). Write $M \xrightarrow{*} M'$ if $M \xrightarrow{\sigma} M'$ for some $\sigma \in T^*$ (the empty sequence gives $M \xrightarrow{\epsilon} M$). The set of markings reachable from M_0 is denoted $\text{reach}(M_0)$ or $[M_0]$: the state space of the net seen as a transition system.

Example (Question time: which sequences are enabled?). On the cycle net, from $M_0 = p_1$:

- $p_1 \xrightarrow{t_1 t_2 t_1 t_3} ?$ Yes: $p_1 \xrightarrow{t_1} p_2 + p_3 \xrightarrow{t_2} p_1 + p_2 \xrightarrow{t_1} 2p_2 + p_3 \xrightarrow{t_3} p_2 + p_4$.
- $p_1 \xrightarrow{t_1 t_3 t_4 t_2} ?$ Yes: $p_1 \xrightarrow{t_1} p_2 + p_3 \xrightarrow{t_3} p_4 \xrightarrow{t_4} p_3 \xrightarrow{t_2} p_1$: back to the initial marking.
- $p_1 \xrightarrow{t_1 t_3 t_4 t_3} ?$ No: after $t_1 t_3 t_4$ the marking is p_3 , where t_3 is not enabled.
- $p_1 \xrightarrow{t_1 t_3 t_3} ?$ No: after $t_1 t_3$ the marking is p_4 , where t_3 is not enabled.

Definition (Infinite firing sequences, prefixes, occurrence sequences). For an infinite $\sigma = t_1 t_2 \cdots \in T^\omega$, $M \xrightarrow{\sigma}$ holds if there are markings $M = M_1 \xrightarrow{t_1} M_2 \xrightarrow{t_2} \cdots$ forever. Concatenation $\sigma_1 \sigma_2$ is defined for finite σ_1 (and finite or infinite σ_2); σ_1 is a *prefix* of σ if $\sigma = \sigma_1$ or $\sigma = \sigma_1 \sigma_2$ with $\sigma_2 \neq \epsilon$ (*proper* in the latter case). A finite or infinite σ enabled at M is an **occurrence sequence**; an infinite one is conveniently a map $\sigma : \mathbb{N} \rightarrow T$ returning the i -th transition.

On the cycle net the loop $t_1 t_3 t_4 t_2$ returns to p_1 in four steps, so the infinite sequence $p_1 \xrightarrow{t_1 t_3 t_4 t_2 t_1 t_3 t_4 t_2 \cdots}$ is well-defined: the net is not terminating. Every prefix of an enabled infinite sequence ($\epsilon, t_1, t_1 t_3, \dots$) is itself a valid finite firing sequence, and the converse characterisation holds:

Definition (Proposition: enabledness via prefixes). $M \xrightarrow{\sigma}$ holds iff $M \xrightarrow{\sigma'}$ holds for every prefix σ' of σ .

Proof. ($\Rightarrow$) Immediate: if the whole sequence is enabled, each prefix has already been fired along the way. ($\Leftarrow$) Trivial for finite σ (a prefix of itself). For infinite σ , fix $i \in \mathbb{N}$: the finite prefix $t_1 \cdots t_{i-1} t_i$ is enabled by hypothesis, so t_i is enabled after $t_1 \cdots t_{i-1}$; since i is arbitrary, the infinite chain of firings exists. $\square$

10.8 Exercises

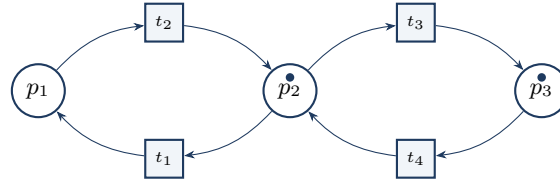
Example (DFA: double parity). A DFA with states q_0, q_1, q_2, q_3 over $\{0, 1\}$, start and only accepting state q_0 :

	0	1
$\rightarrow * q_0$	q_2	q_1
q_1	q_3	q_0
q_2	q_0	q_3
q_3	q_1	q_2

Does it accept 100? $\hat{\delta}(q_0, 100) = q_1 \notin F$: no. Does it accept 1010? $\hat{\delta}(q_0, 1010) = q_0$: yes. The language: label each state by the parities of the counts of 0s and 1s read so far: $q_0 = (\text{even}, \text{even})$, $q_2 = (\text{odd } \#0)$, $q_1 = (\text{odd } \#1)$, $q_3 = (\text{odd}, \text{odd})$; a 0 flips the first parity, a 1 the second. Hence

$$L(\mathcal{A}) = \{w \in \{0, 1\}^* \mid |w|_0 \text{ and } |w|_1 \text{ both even}\}.$$

Example (Petri net: enabledness and reachability). A net with places p_1, p_2, p_3 and transitions $t_1 : p_2 \rightarrow p_1$, $t_2 : p_1 \rightarrow p_2$, $t_3 : p_2 \rightarrow p_3$, $t_4 : p_3 \rightarrow p_2$; initial marking $M_0 = p_2 + p_3$.



Pre/post-sets: $\bullet t_1 = \{p_2\}$, $t_1^\bullet = \{p_1\}$; $\bullet t_2 = \{p_1\}$, $t_2^\bullet = \{p_2\}$; $\bullet t_3 = \{p_2\}$, $t_3^\bullet = \{p_3\}$; $\bullet t_4 = \{p_3\}$, $t_4^\bullet = \{p_2\}$; for places, $\bullet p_2 = \{t_2, t_4\}$, $p_2^\bullet = \{t_1, t_3\}$, etc.

Enabled at M_0 : t_1 , t_3 (token in p_2) and t_4 (token in p_3); t_2 is not (p_1 empty). The firings:

$$p_2 + p_3 \xrightarrow{t_1} p_1 + p_3, \quad p_2 + p_3 \xrightarrow{t_3} 2p_3, \quad p_2 + p_3 \xrightarrow{t_4} 2p_2.$$

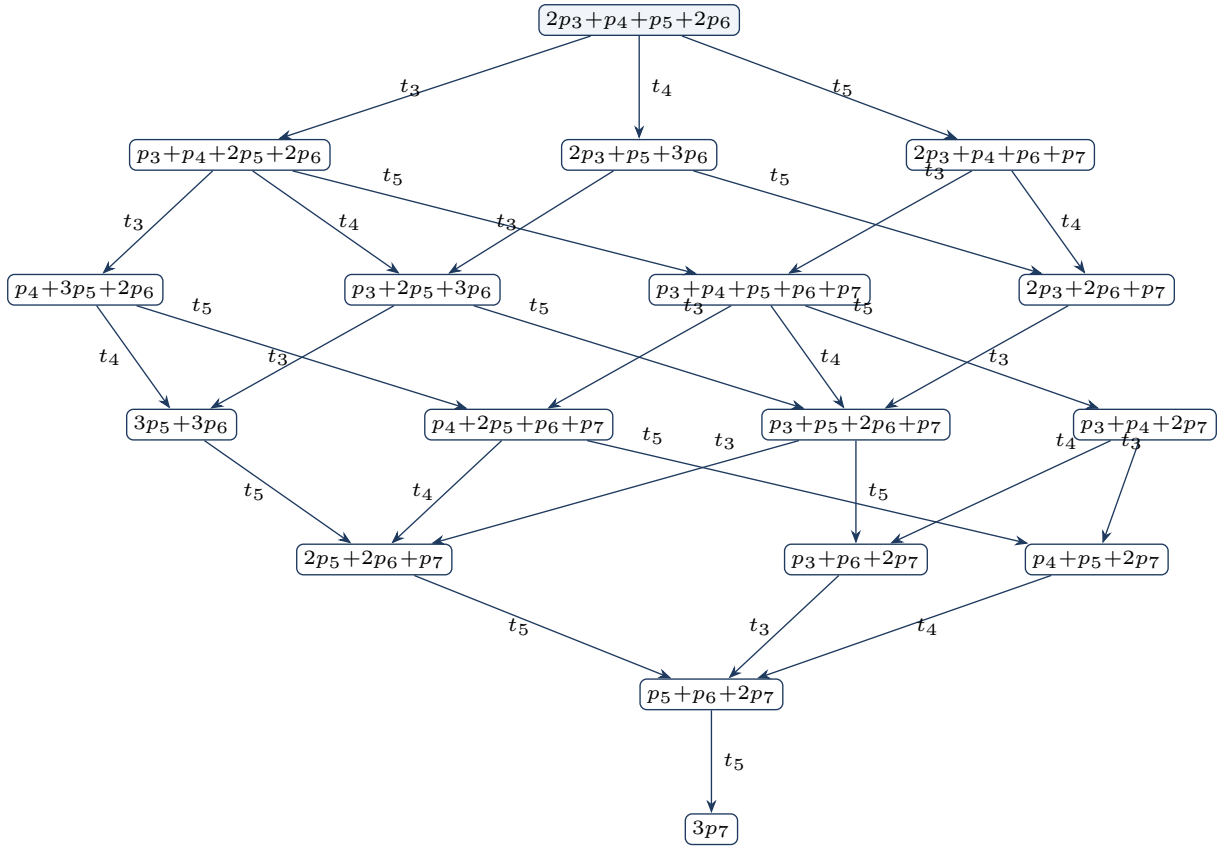
Reachable set: every transition has one input and one output, so the two tokens are conserved; breadth-first exploration reaches all six two-token markings,

$$[M_0] = \{p_2 + p_3, p_1 + p_3, 2p_2, 2p_3, p_1 + p_2, 2p_1\}.$$

Example (Order-fulfilment net: reachability). Same three questions on the order-fulfilment net with $M_0 = 2p_3 + p_4 + p_5 + 2p_6$. Enabled: t_3 , t_4 , and t_5 (p_5 and p_6 both marked); t_1, t_2 are not. The firings:

$$M_0 \xrightarrow{t_3} p_3 + p_4 + 2p_5 + 2p_6, \quad M_0 \xrightarrow{t_4} 2p_3 + p_5 + 3p_6, \quad M_0 \xrightarrow{t_5} 2p_3 + p_4 + p_6 + p_7.$$

Exploring breadth-first until no new marking appears yields a finite reachability graph that converges to the deadlock $3p_7$: each of the three orders in flight is eventually completed by t_5 , which consumes two tokens and produces one:



The next chapter studies this graph as the *occurrence graph* (reachability graph).

11 Occurrence graphs

The *occurrence graph* formalises the behaviour of Petri nets; it then defines boundedness and, via *coverability*, handles unbounded nets.⁵⁵

Definition (Occurrence graph (reachability graph)). The *occurrence graph* of a Petri net N represents all its occurrence sequences: nodes are the reachable markings, arcs the firings,

$$OG(N) = ([M_0], A), \quad A \subseteq [M_0] \times T \times [M_0], \quad (M, t, M') \in A \iff M \xrightarrow{t} M'.$$

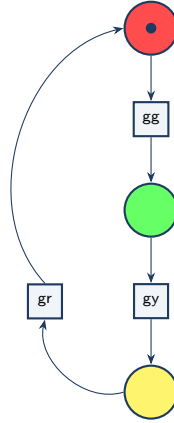
It is computed by a worklist-style breadth-first construction:

1. $Nodes = \emptyset$, $Arcs = \emptyset$, $Todo = \{M_0\}$;
2. pick M from $Todo$; move it to $Nodes$;
3. collect all firings $Firings = \{(M, t, M') \mid M \xrightarrow{t} M'\}$ and add them to $Arcs$;
4. add to $Todo$ every produced M' not already in $Nodes \cup Todo$;
5. repeat from 2 until $Todo$ is empty.

The algorithm terminates iff the reachable set is finite; the unbounded case is handled by the coverability construction at the end of the chapter.

⁵⁵Desel J., Esparza J., *Free Choice Petri Nets*, Cambridge University Press, <https://www7.in.tum.de/~esparza/bookfc.html>.

Example (Traffic light). A single traffic light: three places **red**, **yellow**, **green** and three transitions in a cycle, $M_0 = \text{red}$.



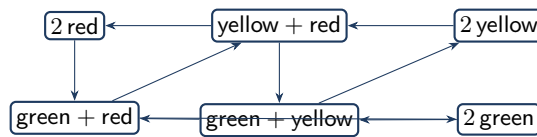
(gg = go-green, gy = go-yellow, gr = go-red; the cycle order is $\text{red} \rightarrow \text{green} \rightarrow \text{yellow} \rightarrow \text{red}$.) The worklist run is short: exploring **red** adds the arc $\text{red} \xrightarrow{gg} \text{green}$; exploring **green** adds $\text{green} \xrightarrow{gy} \text{yellow}$; exploring **yellow** adds $\text{yellow} \xrightarrow{gr} \text{red}$, whose target is already a node. *Todo* empties and the OG is a 3-node directed cycle.

Example (Two independent traffic lights). Two disjoint copies of the net (places **red**, $\dots$ and **red'**, $\dots$), $M_0 = \text{red} + \text{red'}$. At every marking the enabled transitions of *both* lights compete, so each exploration step enqueues up to two successors ($\text{red} + \text{red'}$ branches to $\text{green} + \text{red'}$ and $\text{red} + \text{green'}$, and so on). The construction stabilises on the full cartesian product: nine reachable markings

$$\{\text{red}, \text{yellow}, \text{green}\} \times \{\text{red'}, \text{yellow'}, \text{green'}\},$$

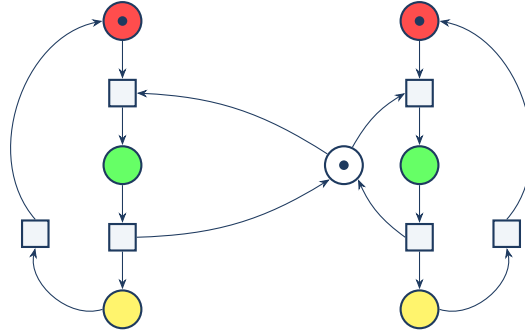
with arcs given by the moves of either light independently. The two lights never synchronise.

Example (One net, two tokens). Same single-light net, but $M_0 = 2 \text{ red}$: markings are multisets of two tokens over three places. From 2 red only **go-green** is enabled (transitions fire one at a time; firing once removes a token, so the transition is not fired “twice in parallel”), giving $\text{green} + \text{red}$; from there **go-green** again gives 2 green , while **go-yellow** gives $\dots$ and the exploration closes after six nodes:

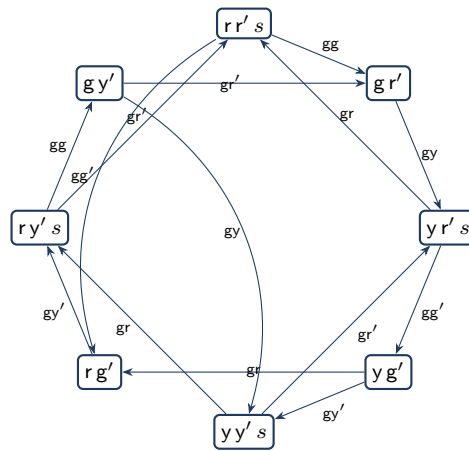


Six nodes against the nine of the two independent lights: same number of tokens, smaller state space, because the two tokens share the same places: the OG is the quotient of the product graph under the symmetry exchanging the tokens.

Example (Mutual exclusion: a shared semaphore place). At a crossroad the two lights must never be **green** together. Add a *semaphore* place s with one initial token, input of both **go-green** transitions and output of both **go-yellow** transitions: whichever light turns green removes the token, blocking the other until the green phase ends.



Example (Exercises on the mutex net). (a) Draw the OG of the mutex net from $\text{red} + \text{red}' + s$. The invariant: s is empty exactly when one light is green. Reachable markings: the four “neither-green” combinations of $\{\text{red}, \text{yellow}\} \times \{\text{red}', \text{yellow}'\}$ with s marked, plus the four “exactly-one-green” markings with s empty: eight in total, against the nine of the unconstrained product.

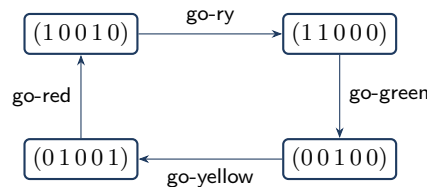


(b) Make the greens *alternate*: after one light has been green, only the other may turn green next. A single mutex place cannot remember *who* held the token last; use *two* control places c_1, c_2 , exactly one initially marked: **go-green** of light 1 consumes from c_1 and the corresponding **go-red** produces on c_2 , symmetrically for light 2. The OG becomes a single six-state *cycle*: the alternation removes every residual choice.

(c) Play the token games in WoPeD,⁵⁶ which also draws the reachability graph automatically.

Example (German traffic lights). German lights show red+yellow just before green: four phases, $\text{red} \rightarrow \text{red}+\text{yellow} \rightarrow \text{green} \rightarrow \text{yellow} \rightarrow \text{red}$. Tasks: model the four-state automaton, then a Petri net with three places for the lamps such that no impossible phase change is allowed. The WoPeD solution adds control places and a transition **go-ry** between **go-red** and **go-green**; with token counts listed as $(\text{red}, \text{yellow}, \text{green}, \text{control}_1, \text{control}_2)$ the reachability graph is the four-state cycle

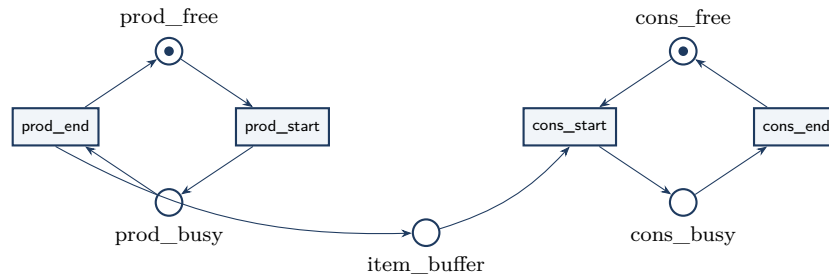
$$(10010) \xrightarrow{\text{go-ry}} (11000) \xrightarrow{\text{go-green}} (00100) \xrightarrow{\text{go-yellow}} (01001) \xrightarrow{\text{go-red}} (10010).$$



⁵⁶<http://www.woped.org>

An equivalent design with a single control place is a state machine: every transition consumes and produces exactly one token, so the total token count is invariant.

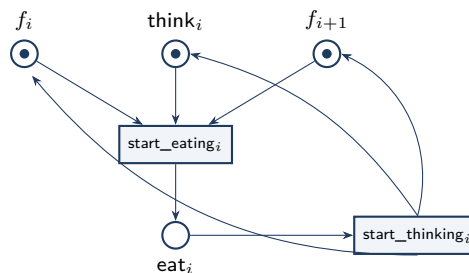
Example (Producer and consumer). Producer and consumer each alternate between *free* and *busy*; every production cycle pushes one product into a shared buffer, every consumption cycle pops one (when available):



prod_end pushes into the buffer on completion of each production cycle, cons_start pops when starting a consumption cycle. Variants:

- *four producers, three consumers*: same structure, initial markings $4 \text{ prod_free} + 3 \text{ cons_free}$: up to four production and three consumption cycles in flight, buffer unbounded;
- *bounded buffer (capacity 2)*: add a complementary place free_slot with two initial tokens, input of prod_end and output of cons_start; the invariant $\text{item_buffer} + \text{free_slot} = 2$ holds at every reachable marking;
- *reachability graphs*: finite in the bounded variants (their size grows polynomially with producers, consumers, and capacity, and tool-generated drawings quickly become too dense to read); infinite when the buffer is uncapped.

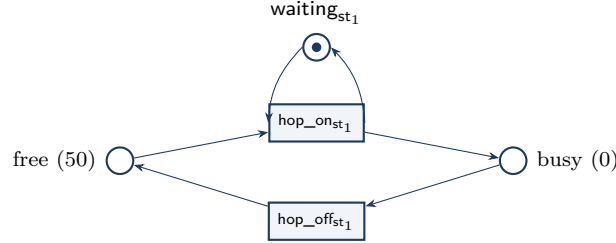
Example (Dining philosophers). Dijkstra posed the problem (five computers competing for five shared tape drives) as an exam question; Hoare popularised the dinner-table telling.⁵⁷ Five philosophers alternate between thinking and eating; the spaghetti requires two forks, one fork sits between each pair of neighbours, so *no two neighbours may eat simultaneously*. Per philosopher i : places think_i (marked) and eat_i ; start_eating_i consumes think_i and the two adjacent forks $f_i, f_{i+1 \bmod 5}$; start_thinking_i returns them:



The fragment is replicated five times around the table, each fork shared by two neighbours. Compute the reachability graph in WoPeD: it is finite, symmetric under the cyclic shift of the philosophers, and, because start_eating_i takes *both* forks atomically, deadlock-free: an eating philosopher can always stop eating. The classical deadlock of the problem (every philosopher holding her left fork, waiting for the right one) appears only when fork-taking is split into two separate transitions; the textbook remedy is to have one philosopher pick up her forks in the opposite order.

⁵⁷Dijkstra E. W., “Hierarchical ordering of sequential processes”, *Acta Informatica*, 1971; Hoare C. A. R., *Communicating Sequential Processes*, 1985.

Example (Circular railway). A circular railway has four stations and one train of capacity 50; passengers hop on/hop off only while the train is parked. The train part is a pure cycle: four **waiting** places (token = train parked at station i) and, between consecutive stations, an in-transit place entered by leave_i and exited by arrive_{i+1} . Passengers are modelled by two places with $\text{free} + \text{busy} = 50$; at each station the hop transitions test the train's presence with a self-loop:

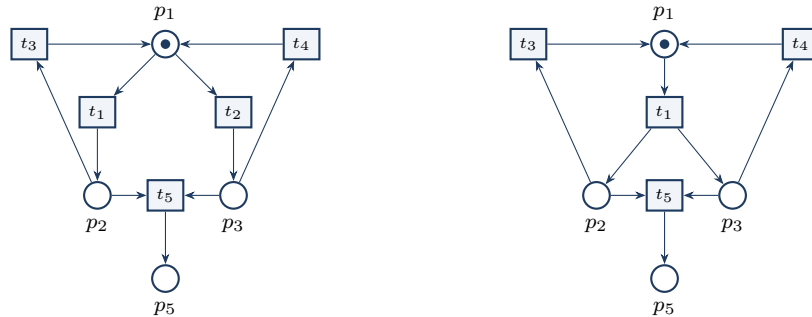


(hop_off carries the symmetric self-loop on the waiting place, omitted for legibility; the fragment repeats at all four stations.) State count: 51 passenger configurations $\times$ 8 train positions (4 parked + 4 in transit) = **408** reachable states: counting only the parked positions would give $51 \times 4 = 204$ and miss the in-transit half.

11.1 Boundedness

Definition (k -bounded, safe, bounded). A place p is **k -bounded** ($k \in \mathbb{N}$) if no reachable marking puts more than k tokens in it: $\forall M \in [M_0]. M(p) \leq k$. A place is **safe** if it is 1-bounded, and **bounded** if it is k -bounded for some k . A net is k -bounded / safe / bounded when all its places are; the least such k is the *bound* of the net. k -boundedness guarantees that an implementation with capacity k per place never overflows; in a safe net markings are plain *sets* of places.

Example (Choice vs fork). Two nets, identical but for t_1 and t_2 :



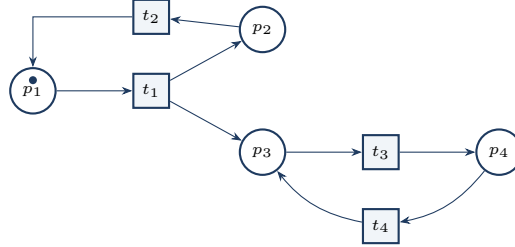
Left: $t_1 : p_1 \rightarrow p_2$ and $t_2 : p_1 \rightarrow p_3$ form an exclusive *choice*; the single token shuttles between p_1 and one of p_2, p_3 , so t_5 (which needs both) never fires and p_5 stays empty. All places are bounded and safe; the net is safe. Right: $t_1 : p_1 \rightarrow p_2 + p_3$ is a *fork*; firing t_1 , returning one branch via t_3 or t_4 , and forking again accumulates tokens ($p_1 + p_3 \xrightarrow{t_1} p_2 + 2p_3 \xrightarrow{t_3} p_1 + 2p_3 \xrightarrow{t_1} \dots$), and t_3, t_4 can even pile tokens on p_1 itself ($p_2 + p_3 \xrightarrow{t_3} p_1 + p_3 \xrightarrow{t_4} 2p_1$). No place is bounded: the net is neither bounded nor safe.

Definition (Theorem: bounded $\iff$ finite OG). A Petri net system is bounded iff its reachability graph is finite.

Proof. ($\Rightarrow$) If the net is k -bounded with $n = |P|$ places, every reachable marking is a function $P \rightarrow \{0, \dots, k\}$, so $|[M_0]| \leq (k+1)^n$ is finite, and the arcs, a subset of $[M_0] \times T \times [M_0]$, are finite too. ($\Leftarrow$) If the OG is finite, $k = \max\{\max_p M(p) \mid M \in [M_0]\}$ is well defined and the net is k -bounded. $\square$

Contrapositive reading: a net is unbounded iff its reachability graph is infinite: an infinite OG witnesses an unbounded place, and vice versa.

Example (Exercise: decide boundedness). Places $p_1, \dots, p_4$, $M_0 = p_1$; transitions $t_1 : p_1 \rightarrow p_2 + p_3$, $t_2 : p_2 \rightarrow p_1$, $t_3 : p_3 \rightarrow p_4$, $t_4 : p_4 \rightarrow p_3$.



Every return to p_1 through t_2 allows another t_1 , each firing of which deposits one more token in the $\{p_3, p_4\}$ cycle: the reachable markings include $p_2 + p_3$, $p_1 + p_3$, $p_2 + p_3 + p_4$, $p_2 + 2p_3$, $\dots$: an infinite ascending chain. p_1 and p_2 are safe; p_3 and p_4 are unbounded; the net is unbounded and its OG infinite.

Example (Question time: bounded or not?). *Producer/consumer (unbounded buffer)*: all role places are safe, but the producer can push faster than the consumer pops: `item_buffer` is unbounded, hence so is the net. *Bounded-buffer variant*: `free_slot` caps the buffer (both 2-bounded); with four producers and three consumers the net is 4-bounded: bounded, not safe. *Dining philosophers*: every place is safe (philosopher tokens cycle between two places, fork tokens are never duplicated): the net is safe. *Railway*: the eight train-position places are safe, `free` and `busy` are 50-bounded: the net is 50-bounded, not safe.

11.2 Coverability

Definition (Coverability graph, extended bags). The *coverability graph* is a *finite over-approximation* of the reachability graph for (possibly) unbounded nets. Its nodes are *extended bags* $B : P \rightarrow \mathbb{N} \cup \{\omega\}$, where $B(p) = \omega$ records that tokens on p can be pumped without bound. Every reachable marking is *covered* by some node; not every node corresponds to a reachable marking.

The pumping argument that justifies ω : if the construction finds a path $M_0 \xrightarrow{*} M \xrightarrow{*} M'$ with $M \subset M'$, let $L = M' - M > \emptyset$; by monotonicity the segment can be repeated, $M \xrightarrow{*} M + L \xrightarrow{*} M + 2L \xrightarrow{*} \dots$, so every place with $L(p) > 0$ is unbounded. The construction therefore replaces such an M' by the extended bag with ω on exactly those places. The witness pair must lie on a single forward path: $M, M' \in [M_0)$ alone does not justify pumping. And $M \subset M'$ is strict multiset inclusion, $\leq$ everywhere and $<$ somewhere.

Definition (Operations on extended bags). For $B, B' : P \rightarrow \mathbb{N} \cup \{\omega\}$ and a marking $M : P \rightarrow \mathbb{N}$:

- **inclusion**: $B \subseteq B'$ iff for every p , either $B'(p) = \omega$, or both are finite and $B(p) \leq B'(p)$;
- **sum**: $(B + B')(p) = \omega$ if either summand is ω at p , otherwise the ordinary sum;
- **difference** $B - M$ (defined only for $M \subseteq B$ with M a plain marking): ω stays ω , finite components subtract. Subtracting a bag containing ω is undefined: $\omega - \omega$ would be ambiguous.

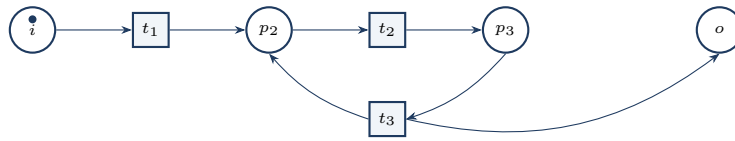
For example $2a + b + \omega c \subseteq 2a + 2b + \omega c \subseteq 2a + \omega b + \omega c$, but $2a + b + \omega c \not\subseteq 2a + \omega b + 3c$; $(3a + 2b + \omega c) + (2a + \omega b + \omega c) = 5a + \omega b + \omega c$; $(5a + \omega b + \omega c) - (3a + 2b + 4c) = 2a + \omega b + \omega c$, while subtracting $3a + 2b + \omega c$ is undefined.

Definition (Coverability graph: algorithm). The worklist of the OG algorithm, with one extra step injecting ω :

1. $N = \{M_0\}$, $A = \emptyset$;
2. take $B \in N$ and $t \in T$ such that B enables t and no t -labelled arc leaves B ;
3. let $B' = B - \bullet t + t \bullet$;
4. build B'_c : for each p , set $B'_c(p) = \omega$ if some node $B'' \in N$ has (i) a directed path from B'' to B in (N, A) , (ii) $B'' \subset B'$, and (iii) $B''(p) < B'(p)$; otherwise $B'_c(p) = B'(p)$;
5. add B'_c to N and (B, t, B'_c) to A ; repeat from 2 until no new arc can be added.

The ancestor clause (i) is the forward-path requirement of the pumping argument. The construction *always* terminates.

Example (Coverability: worked example). Four places and three transitions, $M_0 = i$:



with $t_1 : i \rightarrow p_2$, $t_2 : p_2 \rightarrow p_3$, $t_3 : p_3 \rightarrow p_2 + o$: the restock-and-produce shape of t_3 is the trademark of an unbounded place. The construction: $i \xrightarrow{t_1} p_2 \xrightarrow{t_2} p_3 \xrightarrow{t_3} p_2 + o$. The new bag $p_2 + o$ has the ancestor p_2 on its forward path with $p_2 \subset p_2 + o$, difference o : insert $p_2 + \omega o$ instead. From it, t_2 gives $p_3 + \omega o$ (ω survives the arithmetic), and t_3 from there returns $p_2 + \omega o$: already a node, only the arc is added. The coverability graph is finite:

$$i \xrightarrow{t_1} p_2 \xrightarrow{t_2} p_3 \xrightarrow{t_3} p_2 + \omega o \xrightleftharpoons[t_3]{t_2} p_3 + \omega o.$$

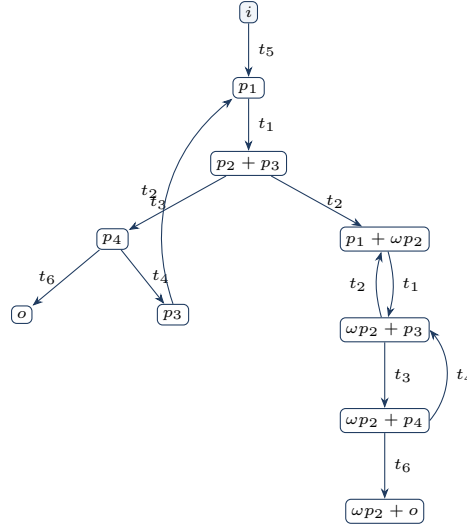
Five nodes; o is unbounded, every other place safe; the actual reachability graph is infinite (one extra token in o per loop).

Note (Properties of coverability graphs).

- A coverability graph is *always finite*, but *not* uniquely defined: it depends on the order of selections at step 2.
- Every firing sequence of the net has a corresponding path in the graph; the converse fails in general: paths through ω components may correspond to no real run.
- A path visiting only finite markings (no ω) does correspond to a real firing sequence.
- If the net is bounded, no ω is ever introduced and the coverability graph *is* the reachability graph.

WoPeD computes a coverability graph by default: exact for bounded nets, conservative otherwise.

Example (Coverability: a richer net). Six places and six transitions: $t_5 : i \rightarrow p_1$, $t_1 : p_1 \rightarrow p_2 + p_3$, $t_2 : p_3 \rightarrow p_1$, $t_3 : p_2 + p_3 \rightarrow p_4$, $t_4 : p_4 \rightarrow p_3$, $t_6 : p_4 \rightarrow o$; $M_0 = i$. The loop $t_1 t_2$ (fork, then give back only p_3 's token) pumps p_2 . The coverability graph:



The ω first appears at $p_1 + \omega p_2$ (t_2 from $p_2 + p_3$ has the ancestor p_1 with $p_1 \subset p_1 + p_2$) and propagates along the right-hand loop; the nodes o and $\omega p_2 + o$ are terminal: the first a genuine reachable deadlock, the second its unbounded counterpart.

12 Liveness

Two net properties, written $N \vdash \psi$ (“the net N satisfies ψ ”), are studied formally: *boundedness*, already met, caps the tokens on every place; *liveness* guarantees that no transition becomes permanently disabled.⁵⁸ Two logical tools recur. To disprove an implication $P \Rightarrow Q$ one exhibits a witness with $P \wedge \neg Q$: small Petri nets serve as counterexamples throughout. And reachability composes: $M \in [M_0] \wedge M' \in [M] \Rightarrow M' \in [M_0]$, so “for any reachable marking” always means “reachable from M_0 ” without ambiguity.

12.1 Live and dead nodes

Definition (Liveness and deadness). Let $N = (P, T, F, M_0)$, $t \in T$, $p \in P$. The four predicates for transitions, and their placewise analogues (replace “enabled” by “marked”, i.e. $M(p) > 0$):

$$\text{Live}(t) \equiv \forall M \in [M_0], \exists M' \in [M] : M' \xrightarrow{t} \quad \text{Live}(p) \equiv \forall M \in [M_0], \exists M' \in [M] : M'(p) > 0$$

$$\text{Dead}(t) \equiv \forall M \in [M_0] : M \not\xrightarrow{t} \quad \text{Dead}(p) \equiv \forall M \in [M_0] : M(p) = 0$$

Non-live and *non-dead* are the negations; pushing $\neg$ through the quantifiers flips $\forall\exists$ into $\exists\forall$:

$$\text{NonLive}(t) \equiv \exists M \in [M_0], \forall M' \in [M] : M' \not\xrightarrow{t}, \quad \text{NonDead}(t) \equiv \exists M \in [M_0] : M \xrightarrow{t}.$$

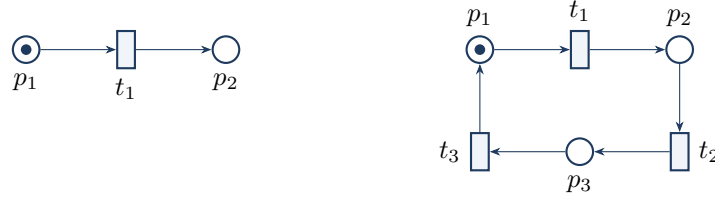
A net is **live** when all its transitions are live, **place-live** when all its places are.

Intuitively, t is live when at no point of any computation can we exclude that t will fire again: from every reachable marking some continuation re-enables it. The $\forall\forall\exists$ alternation (over every transition, then every reachable marking, then some continuation) is what gives the property bite: it must not be confused with the much weaker $\exists M : M \xrightarrow{t}$, which only says t is not dead.

⁵⁸Desel J., Esparza J., *Free Choice Petri Nets*, Cambridge University Press, 1995, <https://www7.in.tum.de/~esparza/bookfc.html>.

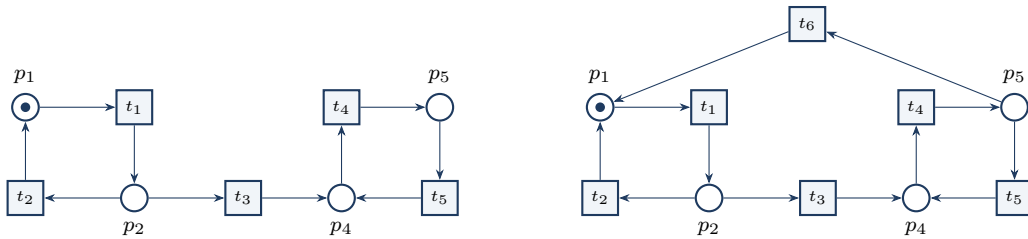
Live and dead are extremes, not complements: a transition can be non-live and non-dead at once: able to fire now, yet able to lose that ability forever.

Example (Non-live and non-dead; a live cycle).



Left: $[M_0] = \{p_1, p_2\}$ and from p_2 nothing is enabled. t_1 is non-dead (enabled at M_0) and non-live (after firing, disabled forever): both negative properties at once. Right: closing the cycle with $t_2 : p_2 \rightarrow p_3$ and $t_3 : p_3 \rightarrow p_1$ makes every transition live (from each of p_1, p_2, p_3 the marking enabling any chosen transition is reachable along the cycle) so the net is live.

Example (Which transitions and places are live?).



Left net. The token can loop on $p_1 \rightleftharpoons p_2$, but once t_3 moves it to the $p_4 \rightleftharpoons p_5$ loop there is no way back: from any marking in $\{p_4, p_5\}$ the transitions t_1, t_2, t_3 are forever disabled. Live: t_4, t_5 (the right loop is reachable from everywhere and cycles); non-live: t_1, t_2, t_3 ; dead: none (each fires at least once from M_0). The net is *not* live. Placewise, identically: p_4, p_5 are live, p_1, p_2 non-live, none dead: not place-live.

Right net. The back transition $t_6 : p_5 \rightarrow p_1$ closes the loop: every transition and every place can be re-engaged from every reachable marking. The net is live and place-live.

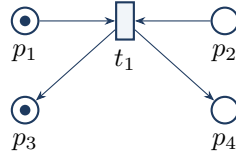
Note (Liveness on the occurrence graph). On the OG the quantifier alternation becomes reachability: t is *live* iff from every node some node with an outgoing t -labelled arc is reachable; t is *dead* iff no t -labelled arc appears at all. Placewise: p is live iff from every node some node marking p is reachable; dead iff no node marks p . In the left net above the labels t_1, t_2, t_3 appear only in the initial region of the OG, unreachable from the $\{p_4, p_5\}$ cycle: the diagnosis is immediate.

Definition (Dead at a marking; working assumptions). For a marking M not necessarily initial: t is **dead at M** when $\forall M' \in [M] : M' \not\xrightarrow{t}$, and p is **dead at M** when $\forall M' \in [M] : M'(p) = 0$; taking $M = M_0$ recovers the global notions. Three facts: a node dead at some reachable M is non-live; conversely, non-liveness *means* the existence of such an M ($\text{NonLive} \Leftrightarrow \exists M \text{ Dead}(\cdot, M)$); and deadness is *monotone*, once dead at M , dead at every marking in $[M]$: the set of dead nodes can only grow during a run. Throughout, nets have no *isolated* nodes ($\bullet n = n^\bullet = \emptyset$), which would be forever disabled or unmarked by construction.

Example (True or false? Deadness and adjacency). *Every transition adjacent to a dead place is dead. True.* Let p be dead. Transitions in $p^\bullet$ never see a token on their input p , so they are never enabled.

Transitions in $\bullet p$, if they ever fired, would put a token in p , contradicting its deadness, so they never fire either. Deadness of a place percolates to its whole neighbourhood.

Every place adjacent to a dead transition is dead. **False:**

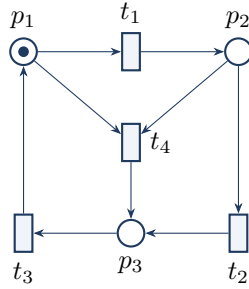


t_1 is dead (it needs tokens on both p_1 and p_2 , and p_2 is never marked) yet p_1 and p_3 hold the initial tokens and are not dead. The asymmetry is structural: a transition dies because of a *single* empty input, leaving its other neighbours untouched.

Definition (Liveness implies place-liveness). **Proposition.** Every live net is place-live.

Proof. Take $p \in P$ and $M \in [M_0]$. Since no node is isolated, some transition $t \in \bullet p \cup p^\bullet$ exists. By liveness of t at M there are $M'', M''' \in [M]$ with $M \xrightarrow{*} M'' \xrightarrow{t} M'''$. If $t \in p^\bullet$ then p is an input of t , so enabling forces $M''(p) > 0$; if $t \in \bullet p$ then firing produces $M'''(p) > 0$. Either way a reachable marking marks p . $\square$

Example (Place-live but not live). The converse fails. Witness:



The token cycles $p_1 \rightarrow p_2 \rightarrow p_3 \rightarrow p_1$ through t_1, t_2, t_3 , so every place is live. But t_4 requires tokens on *both* p_1 and p_2 , which a single circulating token can never provide: t_4 is dead, hence the net is not live.

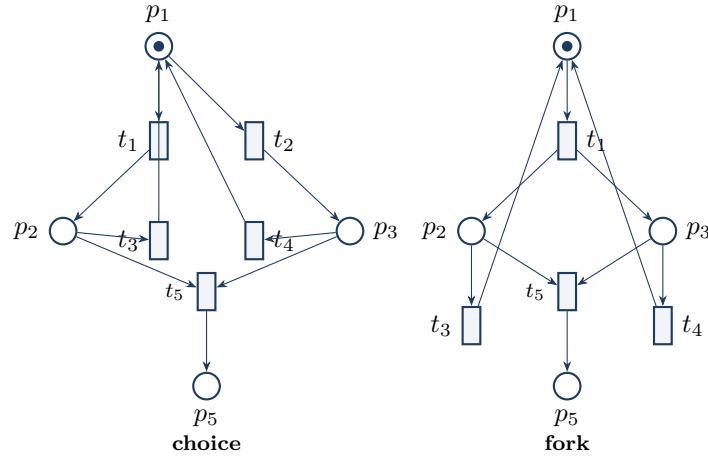
12.2 Deadlock-freedom

Definition (Deadlock-freedom). N is **deadlock-free** when every reachable marking enables at least one transition:

$$\forall M \in [M_0], \exists t \in T : M \xrightarrow{t}.$$

The structure $\forall M \exists t$ is far weaker than liveness' $\forall t \forall M \exists M'$: one immediate step suffices, and the witness transition may depend on M . On the OG: a net is deadlock-free iff no node is a *sink*: detecting deadlocks is one linear scan for nodes without outgoing arcs.

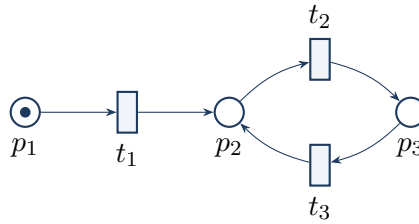
Example (Choice vs fork, again). Two nets identical but for t_1 : places p_1 (marked), p_2, p_3, p_5 ; back transitions $t_3 : p_2 \rightarrow p_1$, $t_4 : p_3 \rightarrow p_1$; and $t_5 : p_2 + p_3 \rightarrow p_5$, with p_5 a dead end. In the *choice* version ($t_1 : p_1 \rightarrow p_2$, $t_2 : p_1 \rightarrow p_3$) the single token is never on p_2 and p_3 simultaneously: t_5 is dead, $[M_0] = \{p_1, p_2, p_3\}$, and each marking enables something: **deadlock-free**. In the *fork* version ($t_1 : p_1 \rightarrow p_2 + p_3$) the marking $p_2 + p_3$ enables t_5 , which leads to p_5 , a sink, since p_5 has no consumers: **not deadlock-free**.



Definition (Liveness implies deadlock-freedom). Lemma. A live net is deadlock-free (a live, non-deadlock-free net does not exist).

Proof by contradiction. Suppose $M \in [M_0]$ is a deadlock, $M \not\rightarrow$. Pick any $t \in T$. By liveness there is $M' \in [M]$ with $M' \xrightarrow{t}$. But a deadlock has $[M] = \{M\}$ (nothing fires, nothing moves) so $M' = M$ and $M \xrightarrow{t}$, contradiction. $\square$

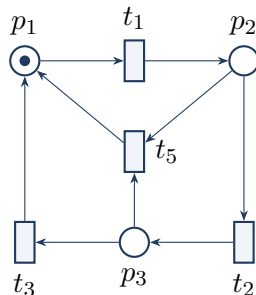
The converse fails: deadlock-freedom does not imply liveness. Witness: $t_1 : p_1 \rightarrow p_2$ followed by a loop $t_2 : p_2 \rightarrow p_3, t_3 : p_3 \rightarrow p_2$, with $M_0 = p_1$. Once the token enters the loop it cycles forever (no deadlock) but t_1 has become dead: the net is not live.



12.3 The lattice of implications

Contraposition ($P \Rightarrow Q \equiv \neg Q \Rightarrow \neg P$) extends the proved implications for free, and small witnesses settle the rest.

- *Not place-live $\Rightarrow$ not live*: **true**: the contrapositive of $\text{live} \Rightarrow \text{place-live}$.
- *Not live $\Rightarrow$ not place-live*: **false**. In the place-live-but-not-live witness above (or its variant where $t_5 : p_2 + p_3 \rightarrow p_1$ replaces t_4), the dead transition does not stop every place from being live: deadness of a transition does not propagate to its neighbour places.

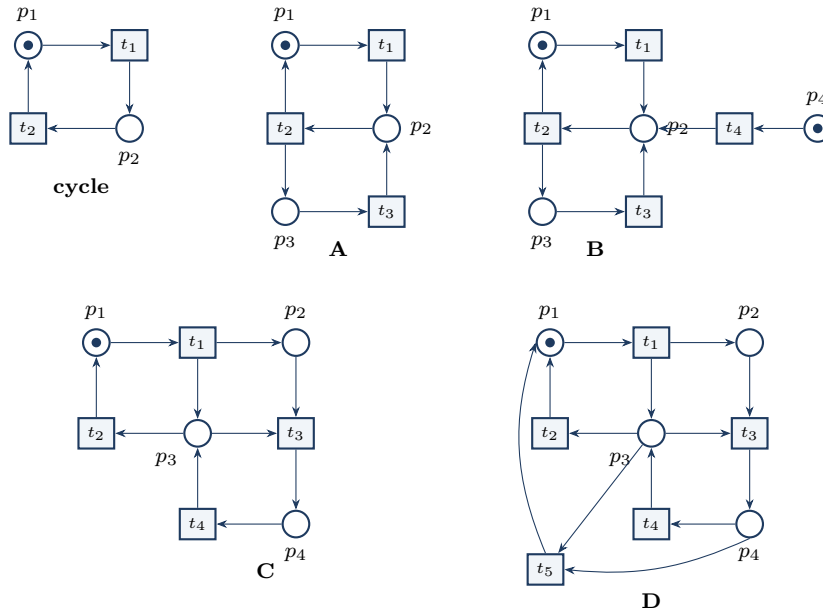


- *Place-live* $\Rightarrow$ *deadlock-free*: **true**. Take $M \in [M_0]$. If every place is marked at M , any transition is enabled. Otherwise pick p with $M(p) = 0$: by place-liveness some $M' \in [M]$ has $M'(p) > 0$, so $M' \neq M$ and the non-empty firing sequence from M to M' starts with a transition enabled at M : no deadlock. $\square$
- *Deadlock-free* $\Rightarrow$ *place-live*: **false**. Extend the choice net with a dead-end place p_5 fed only by the dead transition $t_5 : p_2 + p_3 \rightarrow p_5$: the dynamics cycles deadlock-freely and ignores p_5 , which stays dead: not place-live.

12.4 Exercises: the four-property checklist

For each net decide: bounded? safe? live? deadlock-free?

Example (Five nets).



Cycle ($t_1 : p_1 \rightarrow p_2, t_2 : p_2 \rightarrow p_1$): the token bounces between two markings. **A** adds $t_2 : p_2 \rightarrow p_1 + p_3$ and $t_3 : p_3 \rightarrow p_2$: each firing of t_2 leaves a token in the lower loop while restarting the upper one, so tokens accumulate without bound, yet every transition stays reachable forever. **B** extends A with a one-shot $t_4 : p_4 \rightarrow p_2$: nothing refills p_4 , so t_4 dies after its single firing. **C** ($t_1 : p_1 \rightarrow p_2 + p_3, t_2 : p_3 \rightarrow p_1, t_3 : p_2 + p_3 \rightarrow p_4, t_4 : p_4 \rightarrow p_3$): the $t_1 t_2$ loop pumps p_2 , and every transition remains re-enabable. **D** extends C with $t_5 : p_3 + p_4 \rightarrow p_1$. A token-count argument shows the lower region $\{p_3, p_4\}$ never holds more than one token. Adding a token there takes t_1 , which also consumes p_1 . Only t_2 (consuming p_3) or t_5 (consuming $p_3 + p_4$) can replenish p_1 , and each removes a token from the lower region before another can enter. So p_3 and p_4 never coexist and t_5 is **dead**.

	bounded	safe	live	deadlock-free
cycle	yes	yes	yes	yes
A	no	no	yes	yes
B	no	no	no (t_4 one-shot)	yes
C	no	no	yes	yes
D	no	no	no (t_5 dead)	yes

The patterns to remember: unbounded nets can be live and deadlock-free (A, C); a one-shot or structurally dead transition kills liveness without ever blocking the firing (B, D); only the cycle is bounded, and only bounded nets have finite OGs.

Note (Recap: the implications of this chapter).

- live $\Rightarrow$ place-live, hence non-place-live $\Rightarrow$ non-live;
- t dead $\Rightarrow$ non-live; p dead $\Rightarrow$ non-place-live (hence non-live);
- live $\Rightarrow$ deadlock-free, hence possible deadlock $\Rightarrow$ non-live;
- place-live $\Rightarrow$ deadlock-free, hence possible deadlock $\Rightarrow$ non-place-live.

Liveness is the strongest property, deadlock-freedom the weakest; none of the converses holds, and every gap is witnessed by one of the small nets of this chapter.

13 Incidence matrices

13.1 The unroll-once pattern

An exercise from the BPMN chapters finds its closure here. The exam collaboration (student/teacher) is modified so that *at least one* exercise is assigned before the rate is computed: pull the first iteration out of the loop (a mandatory GET EXERCISE/SOLVE EXERCISE round on the student side, matched by ASSIGN EXERCISE/GET SOLUTION on the teacher side) and only then let an XOR gateway decide between repeating and exiting to GET RATE/ASSIGN RATE. This *unroll-once* pattern is the standard encoding of at-least-one loops.

Example (Dining philosophers, revisited in WoPeD). The net of the occurrence-graph chapter, per philosopher, places $\text{think}_i/\text{eat}_i$ and transitions start_eating_i (consuming both adjacent forks) and start_thinking_i (returning them), assembles into the full ring by sharing each fork place between two neighbours: $5 \cdot 2 + 5 = 15$ places and $5 \cdot 2 = 10$ transitions. WoPeD computes its reachability graph: finite (the net is safe), strongly connected, heavily symmetric (the five rotations of any marking are reachable), and *without deadlocks*: the atomic two-fork grab makes mutual exclusion of neighbours structural, and an eating philosopher can always put the forks back. Nodes are labelled by bit-vectors recording who is eating; at most two non-adjacent philosophers eat simultaneously.

13.2 Vectors and matrices

Linear algebra encodes Petri nets and their computations: the net becomes an integer matrix, markings become vectors, firing becomes vector addition, and a firing sequence is summarised by an integer vector counting its transitions. The structural-analysis machinery of the following chapters (invariants, soundness conditions) is built on this encoding.⁵⁹

Definition (Vectors, products, linear systems). For a finite set $E = \{e_1, \dots, e_n\}$, any map $v : E \rightarrow \mathbb{Q}$ (or $\mathbb{N}$, $\mathbb{Z}$) is a vector indexed by E ; we use the same symbol for the row and the column form, the orientation being clear from the multiplication performed. The *scalar product* of two equal-length vectors is $\mathbf{x} \cdot \mathbf{y} = \sum_i x_i y_i$ (row $\times$ column); a matrix-by-vector product computes one scalar product per row, a vector-by-matrix product one per column. A *homogeneous linear system* is a matrix equation $A \cdot \mathbf{x} = \mathbf{0}$;

⁵⁹Desel J., Esparza J., *Free Choice Petri Nets*, Cambridge University Press, 1995, <https://www7.in.tum.de/~esparza/bookfc.html>.

its non-zero solutions are the invariants studied from the next chapter on, with place-invariants appearing as left null-vectors and transition-invariants as right null-vectors of the same matrix.

Definition (Markings as vectors; orderings). A marking $M : P \rightarrow \mathbb{N}$ is a vector indexed by the places: $M = [M(p_1) \cdots M(p_n)]$. For vectors over E :

$$\mathbf{v} \leq \mathbf{w} \iff \forall e. v(e) \leq w(e); \quad \mathbf{v} < \mathbf{w} \iff \mathbf{v} \leq \mathbf{w} \wedge \exists e. v(e) < w(e); \quad \mathbf{v} \prec \mathbf{w} \iff \forall e. v(e) < w(e).$$

$\mathbf{0}$ is the all-zeros vector of the appropriate length. The two strict orders differ: $<$ allows ties, $\prec$ forbids them: e.g. $[1\ 2\ 0] < [2\ 3\ 0]$ holds, but $[0\ 0\ 0] \prec [2\ 0\ 1]$ fails on the tied component, while $[0\ 0\ 0] \prec [2\ 3\ 1]$ holds. Neither $[3\ 2\ 0] \leq [2\ 3\ 1]$ nor $[3\ 2\ 0] \geq [2\ 3\ 1]$ holds: vector orders are partial.

13.3 The incidence matrix

Can net computations be expressed by matrix multiplication? Yes, with a caveat: the *change* of marking caused by firing t is structural (it depends only on the arcs of t , not on the marking at which t fires) so a single integer per (place, transition) pair captures it; but the *enabling* condition is not linear and will be lost. Four connection cases exhaust the possibilities:

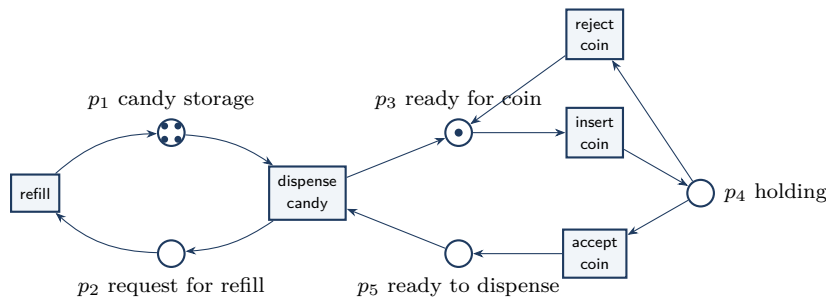


Definition (Incidence matrix). For a net $N = (P, T, F)$ the **incidence matrix** $\mathbf{N} : (P \times T) \rightarrow \{-1, 0, +1\}$ is

$$\mathbf{N}(p, t) = \begin{cases} -1 & \text{if } (p, t) \in F \wedge (t, p) \notin F, \\ +1 & \text{if } (p, t) \notin F \wedge (t, p) \in F, \\ 0 & \text{otherwise (no arc, or self-loop).} \end{cases}$$

With $|P| = n$, $|T| = m$, $\mathbf{N}$ is an $n \times m$ integer matrix: sparse, since each transition touches few places. The third case lumps “no connection” and “self-loop” together: they have the same *net effect*, but different enabling (a self-loop requires $M(p) \geq 1$). This loss of information is the price of the encoding and resurfaces as the limit of structural analysis. The column $\mathbf{t}_j = \mathbf{N}(\cdot, t_j)$ is the effect-vector of t_j on every place; the row $\mathbf{p}_i = \mathbf{N}(p_i, \cdot)$ is the input/output profile of p_i across all transitions.

Example (Running example: the vending machine). Five places (p_1 candy storage, p_2 request for refill, p_3 ready for coin, p_4 holding, p_5 ready to dispense) and five transitions: t_1 refill ($p_2 \rightarrow p_1$), t_2 dispense candy ($p_1 + p_5 \rightarrow p_2 + p_3$), t_3 insert coin ($p_3 \rightarrow p_4$), t_4 accept coin ($p_4 \rightarrow p_5$), t_5 reject coin ($p_4 \rightarrow p_3$). Initial marking $M_0 = 4p_1 + p_3$.



Filling the matrix column by column from the arc structure (e.g. *refill* consumes a refill request and produces a unit of stock):

$$\mathbf{N} = \begin{bmatrix} +1 & -1 & 0 & 0 & 0 \\ -1 & +1 & 0 & 0 & 0 \\ 0 & +1 & -1 & 0 & +1 \\ 0 & 0 & +1 & -1 & -1 \\ 0 & -1 & 0 & +1 & 0 \end{bmatrix} \quad \begin{array}{l} \text{rows: } p_1, \dots, p_5 \\ \text{columns: } t_1 \text{ refill, } t_2 \text{ dispense, } t_3 \text{ insert,} \\ \quad t_4 \text{ accept, } t_5 \text{ reject} \end{array}$$

Reading the matrix: the $+1/-1$ pair on rows p_3, p_4 in column t_3 records the coin moving from *ready* to *holding*; columns t_4 and t_5 both have -1 on p_4 : accept and reject compete for the same holding token, a structural XOR. Every column sums to zero: each transition moves tokens without creating or destroying them, a property the next chapter names *conservativeness*.

Definition (Firing in vector notation). Firing t_j at M produces $M' = M + \mathbf{t}_j$, the j -th column added to the marking vector. On the vending machine, the sequence $M_0 \xrightarrow{t_3} M_1 \xrightarrow{t_4} M_2 \xrightarrow{t_2} M_3$ unrolls as

$$\begin{bmatrix} 4 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \xrightarrow{+\mathbf{t}_3} \begin{bmatrix} 4 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} \xrightarrow{+\mathbf{t}_4} \begin{bmatrix} 4 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \xrightarrow{+\mathbf{t}_2} \begin{bmatrix} 3 \\ 1 \\ 1 \\ 0 \\ 0 \end{bmatrix} :$$

one candy sold, a refill request raised, the machine back at *ready for coin*. The final marking is $M_0 + \mathbf{t}_3 + \mathbf{t}_4 + \mathbf{t}_2$: the order of the summands is irrelevant, only their multiset matters. That multiset is the Parikh vector.

A scalar-product warm-up that will matter later: dotting the row of p_3 with the column $\mathbf{y} = [1, 1, 2, 0, 1]^T$,

$$\begin{bmatrix} 0 & 1 & -1 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 1 \\ 2 \\ 0 \\ 1 \end{bmatrix} = 0 + 1 - 2 + 0 + 1 = 0,$$

and computing the full product $\mathbf{N} \cdot \mathbf{y} = [0, 0, 0, 1, -1]^T$: the first three rows cancel, the last two are opposite. Vectors $\mathbf{y}$ for which such products vanish are the seeds of the invariants of the next chapter.

13.4 Parikh vectors

Definition (Parikh vector). For $\sigma \in T^*$, the **Parikh vector** $\vec{\sigma} : T \rightarrow \mathbb{N}$ maps each transition to its number of occurrences in σ . Inductively: $\vec{\varepsilon} = \mathbf{0}$; $\vec{t} = \mathbf{e}_t$ (the standard basis vector with a single 1 at position t); $\vec{\sigma t} = \vec{\sigma} + \vec{t}$. Hence $\sigma_1 \vec{\sigma}_2 = \vec{\sigma}_1 + \vec{\sigma}_2$: the Parikh map forgets order and retains multiplicity: permutations share the same vector.

On the vending machine, $M_0 \xrightarrow{\sigma} 3p_1 + p_2 + p_3$ with $\sigma = t_3 t_5 t_3 t_4 t_2$ (insert, reject, re-insert, accept, dispense) has $\vec{\sigma} = [0, 1, 2, 1, 1]$; the longer run $M_0 \xrightarrow{\sigma'} 2p_1 + 2p_2 + p_4$ with $\sigma' = t_3 t_4 t_2 t_3 t_4 t_2 t_3 t_5 t_3$

has $\vec{\sigma}' = [0, 2, 4, 2, 1]$: twice the dispense events, the same single rejection, with $t_2 : t_3 : t_4$ in the proportion the loop structure dictates.

13.5 The marking equation

Since $\vec{t}_j = \mathbf{e}_j$ and multiplying a matrix by $\mathbf{e}_j$ selects its j -th column, $\mathbf{N} \cdot \vec{t}_j = \mathbf{t}_j$, so the single-step rule reads $M \xrightarrow{t} M' \Rightarrow M' = M + \mathbf{N} \cdot \vec{t}$. The generalisation to sequences is the central lemma of the chapter.

Definition (Marking equation lemma). If $M \xrightarrow{\sigma} M'$ then $M' = M + \mathbf{N} \cdot \vec{\sigma}$.

Proof. Induction on $|\sigma|$. Base $\sigma = \varepsilon$: $M' = M$ and $\mathbf{N} \cdot \mathbf{0} = \mathbf{0}$. Step $\sigma = \sigma' t$ with $M \xrightarrow{\sigma'} M'' \xrightarrow{t} M'$:

$$M' = M'' + \mathbf{N} \cdot \vec{t} = M + \mathbf{N} \cdot \vec{\sigma}' + \mathbf{N} \cdot \vec{t} = M + \mathbf{N} \cdot (\vec{\sigma}' + \vec{t}) = M + \mathbf{N} \cdot \vec{\sigma' t}. \quad \square$$

On the vending machine, $M_0 + \mathbf{N} \cdot [0, 1, 2, 1, 1]^T = [4, 0, 1, 0, 0]^T + [-1, +1, 0, 0, 0]^T = [3, 1, 1, 0, 0]^T$: the marking $3p_1 + p_2 + p_3$ reached by the step-by-step simulation, recovered without simulating. Two consequences: the reached marking depends *only on the number* of occurrences of each transition, *not on their order*. The qualifier is essential, the lemma is a one-way implication: it computes the effect of a sequence but says nothing about whether the sequence was enabled.

13.6 Monotonicity

An infinite firing sequence has no last transition, so “ $M \xrightarrow{\sigma} M'$ ” has no target marking to speak of; enabledness must be reduced to the finite case.

Definition (Enabledness of infinite sequences (prefix characterisation)). For $\sigma \in T^\omega$, write $M \xrightarrow{\sigma}$ (“ σ is enabled at M ”) when the net can fire σ forever from M . Then

$$M \xrightarrow{\sigma} \iff M \xrightarrow{\sigma'} \text{ for every finite prefix } \sigma' \text{ of } \sigma.$$

Proof. ($\Rightarrow$) Firing σ fires each prefix. ($\Leftarrow$) Every transition of σ occurs in some finite prefix, so if all prefixes are enabled the whole sequence fires step by step. $\square$

Definition (Monotonicity lemma). If $M \xrightarrow{\sigma} M'$ then $M + L \xrightarrow{\sigma} M' + L$ for every marking $L \geq \mathbf{0}$; the same holds for infinite σ (if $M \xrightarrow{\sigma}$ then $M + L \xrightarrow{\sigma}$).

Proof. Finite case by induction on $|\sigma|$: the empty sequence is enabled anywhere; for $\sigma = \sigma' t$, the induction gives $M + L \xrightarrow{\sigma'} M'' + L$, where t is still enabled (extra tokens only help) and fires to $M'' + L + \mathbf{N} \cdot \vec{t} = M' + L$. Infinite case: by the prefix characterisation it suffices that every finite prefix be enabled at $M + L$, which the finite case provides. $\square$

Two readings: what can be done with fewer resources can be done with more; and the surplus survives untouched. On the vending machine, $M_0 \xrightarrow{\sigma} [3, 1, 1, 0, 0]^T$ with $\sigma = t_3 t_5 t_3 t_4 t_2$ lifts to $M_0 + L \xrightarrow{\sigma} [4, 2, 1, 0, 0]^T$ for $L = [1, 1, 0, 0, 0]^T$: the two extra tokens are spectators.

Definition (Corollary (repeatable prefix) and two lemmas). Corollary. If $M \xrightarrow{\sigma} M'$ with $M \leq M'$, then $M \xrightarrow{\sigma\sigma\sigma\cdots}$. *Proof.* Every prefix of σ^ω has the form $\sigma^n \sigma'$ with σ' a prefix of σ ; induction on n with $L = M' - M \geq \mathbf{0}$: the monotonicity lemma turns $M \xrightarrow{\sigma^n \sigma'}$ into $M' = M + L \xrightarrow{\sigma^n \sigma'}$, and prefixing one σ gives $M \xrightarrow{\sigma^{n+1} \sigma'}$. $\square$

Boundedness lemma. In a bounded system, $M \in [M_0]$ with $M \geq M_0$ forces $M = M_0$. *Proof.* Let $M = M_0 + L$ and $M_0 \xrightarrow{\sigma} M$. Iterating the monotonicity lemma, $M_0 \xrightarrow{\sigma} M_0 + L \xrightarrow{\sigma} M_0 + 2L \xrightarrow{\sigma} \dots$, so every $M_0 + kL$ is reachable; boundedness forces this set to be finite, hence $L = \mathbf{0}$. $\square$

Contrapositive diagnostic: a reachable $M > M_0$ proves unboundedness.

Repetition lemma. If $M \xrightarrow{\sigma} M'$ and σ can be fired arbitrarily many times from M , then $M \leq M'$. *Proof.* Otherwise some place has $M'(p) = M(p) - k$ with $k > 0$, i.e. $(\mathbf{N} \cdot \vec{\sigma})(p) = -k$; firing σ $n = M(p) + 1$ times would give $M(p) - nk < 0$, absurd. $\square$

A repeatable sequence satisfies $\mathbf{N} \cdot \vec{\sigma} \geq \mathbf{0}$: it is a *cycle of resources*, producing on every place at least what it consumes. The Parikh vectors with $\mathbf{N} \cdot \vec{\sigma} = \mathbf{0}$ form the kernel of $\mathbf{N}$: the *T-invariants* of the next chapter.

13.7 Exercises

Example (Vending machine: diagnosing sequences by matrix algebra). With $M_0 = 4p_1 + p_3$:

(a) $\sigma = t_3t_4t_2t_3t_5t_3t_4t_1t_2t_1t_3t_5t_2$ has Parikh vector $\vec{\sigma} = [2, 3, 4, 2, 2]$. The marking equation gives

$$M_0 + \mathbf{N} \cdot \vec{\sigma} = \begin{bmatrix} 3 \\ 1 \\ 2 \\ 0 \\ \boxed{-1} \end{bmatrix},$$

a negative entry on p_5 : impossible. Hence *no* firing sequence with this Parikh vector is enabled at M_0 : non-fireability proved without simulating a single prefix.

(b) $\sigma' = t_3t_4t_2t_1$ (insert, accept, dispense, refill) has $\vec{\sigma}' = [1, 1, 1, 1, 0]$ and

$$\mathbf{N} \cdot \vec{\sigma}' = \mathbf{0}, \quad \text{so } M_0 + \mathbf{N} \cdot \vec{\sigma}' = M_0.$$

Direct simulation shows σ' is fireable at M_0 (each step enabled in turn), and it returns exactly to $M_0 = M_0$: by the repeatable-prefix corollary, $M_0 \xrightarrow{\sigma'\sigma'\dots}$: the insert/accept/dispense/refill cycle runs forever. Its Parikh vector lies in the kernel of $\mathbf{N}$: a T-invariant, the object of the next chapter.

14 Analysis of Workflow Nets

Soundness is the property that characterises a well-formed business process; the Petri-net machinery developed so far studies it.⁶⁰ Boundedness and liveness, as defined earlier, are the right global properties for an *ongoing* system; a workflow instead has a definite start and end per case, and soundness adapts them to that setting. Since BPMN, EPC and workflow nets share the same building blocks (start/end markers, splits/joins, tasks, links), the analysis is developed on workflow nets and transfers to the other notations through the usual mappings; the reseller process of the orchestration chapter (drawn in all three notations) is the running motivation.

⁶⁰Weske M., *Business Process Management*, Springer, 2007, Ch. 6.

14.1 Workflow nets, structurally

Definition (Workflow net). A Petri net (P, T, F) is a *workflow net* (WF net) if:

1. there is a distinguished *initial place* i with $\bullet i = \emptyset$;
2. there is a distinguished *final place* o with $o^\bullet = \emptyset$;
3. every other node lies on a path from i to o .

Places hold tokens representing *cases*; transitions are *activities*; a doubled border marks an activity refined by another workflow net. The canonical initial marking is one token on i .

The definition is purely structural, yet it already rejects several malformed shapes: a net with *no* source or sink place (when would the case start or end?), a net with *several* sources or sinks (which one starts or completes the case?), and *dangling tasks* without inputs (when should they run?) or outputs (they contribute nothing to completion): condition 3 excludes them all. What it does not catch are vacuous decorations (a “split” with one output, a “join” with one input): syntactically legal, semantically misleading, left to later analysis. Terminology: a *net system* is a net with an initial marking, the setting of behavioural properties; a *net* alone is the setting of structural ones. For WF nets the two nearly coincide thanks to the canonical marking.

14.2 Behavioural pathologies

Structural correctness does not imply behavioural correctness. Four small, structurally impeccable workflow nets exhibit four distinct failure modes:

- **dead transition:** a synchronising transition whose input places can never be simultaneously marked (e.g. an AND-join downstream of an XOR-split) never fires: a path no case can travel;
- **pending tokens upon completion:** a run marks the final place o while leaving tokens behind on intermediate places: the case is reported complete, but the net is polluted for the next case;
- **pending activities upon completion:** a back-loop keeps upstream transitions enabled after o is marked: the case is closed, yet work continues;
- **deadlock:** a reachable marking enables nothing and o is unmarked: the case is irrevocably stuck. A larger net can “look fine” and still hide one: only replaying the dynamics (rolling back in time through the firing sequence) exposes the marking where the mismatched split/join strands its tokens.

Definition (Livelock). A *livelock* traps a case in a cycle with no opportunity to end: from some reachable marking, every firing sequence keeps moving tokens around a sub-region without ever marking o . The system is not deadlocked (work happens) but the case progresses in circles. Livelocks are a failure class distinct from dead transitions and residual tokens.

All these flaws are detectable *without knowing the goal* of the business process: they are intrinsic to the net’s shape and dynamics, hence amenable to automatic model-level checking. This is **verification**: qualitative questions about the model (can it deadlock? do all cases terminate? can this task ever run?). It must be kept distinct from **validation**: the relation between model and reality (how does the model fit the event logs?), which is the business of process mining.

The mismatched splits and joins behind these defects could often be prevented at design time; the structural conditions that guarantee prevention (free-choiceness, S/T-decompositions) come in the following chapters.

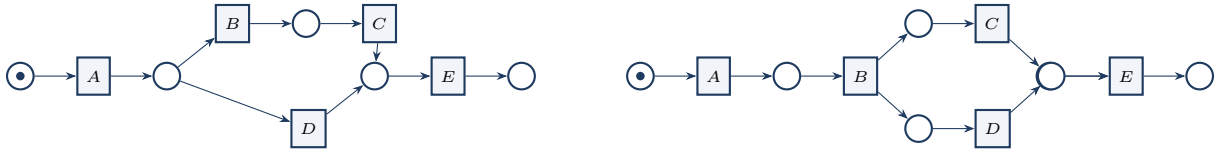
14.3 The language of a workflow net

Definition (Language of a WF net). For a workflow net N with initial place i and final place o ,

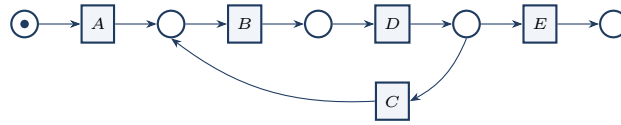
$$L(N) = \{ \sigma \mid i \xrightarrow{\sigma} o \}$$

is the set of firing sequences leading from the marking i to the marking o : the *admissible traces*, the legitimate complete runs of a case.

Example (Question time: languages of small nets).



Left, a choice: after A , either the long path BC or the shortcut D , then E : $L(N) = \{ABCE, ADE\}$. Right, a parallel split: B forks C and D , joined by E ; the branches interleave freely: $L(N) = \{ABCDE, ABDCE\}$. A loop produces a Kleene-star factor:



$L(N) = \{ABD(CBD)^k E \mid k \geq 0\}$. Further variants raise three separate points.

A structurally valid net can have $L(N) = \emptyset$: if its only synchronising join requires two tokens that no run can provide simultaneously, *zero* traces complete, and the workflow admits no legitimate case at all.

Matching a regular expression against $L(N)$ needs care with the boundary cases. For a net that runs A , then B , then either exits with C or iterates $R_1 R_2$ back into B , the correct descriptions are $A B (R_1 R_2 B)^k C$ and the equivalent $A (B R_1 R_2)^k B C$ ($k \geq 0$); the tempting $A (B R_1 R_2)^k C$ is wrong because $k = 0$ would skip the mandatory B .

Counting questions (how often can a task fire? can two occurrences be adjacent? can B outnumber A ?) are answered by token-flow conservation (each B consumes a token some A produced, so B s never outnumber A s), which the invariants of the next chapter make precise.

14.4 Simulation, reachability, and their limits

Simulation: playing the token game, recording runs, randomising choices (WoPeD supports all three), finds defects but cannot prove their absence: how to be sure all runs were examined? *Reachability analysis* is the exhaustive answer: all runs live in the reachability graph (coverability graph if unbounded), end states are evident, dangerous states can be searched. Its limit is *state explosion*: the graph can be exponentially larger than the net, and a moderate workflow already produces an unreadable mesh. Worked diagnoses on a claim-handling net show what the analysis

finds: an AND-join whose second input is never fed (deadlock); a place fed by both parallel branches and consumed once (tokens left after completion); checks still enabled after the case closed (pending activities); and a model covering only the success paths of its checks: failure transitions missing altogether, a business-completeness defect rather than a soundness one.

14.5 Soundness

Definition (Soundness). Informally, a business process is *sound* if it contains no unnecessary tasks, every started case can always run to completion, and nothing is left pending when it completes. Formally, a workflow net N with initial place i and final place o is **sound** if:

1. **no dead task:** $\forall t \in T, \exists M \in [i] : M \xrightarrow{t}$;
2. **option to complete:** $\forall M \in [i], \exists M' \in [M] : M'(o) \geq 1$;
3. **proper completion:** $\forall M \in [i] : M(o) \geq 1 \Rightarrow M = o$.

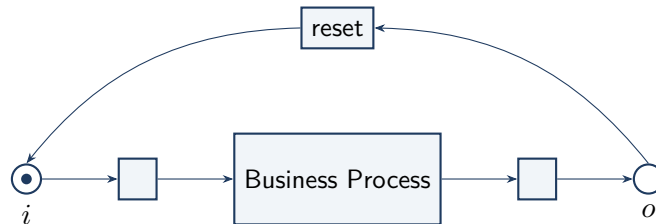
Here i and o also denote the markings with a single token on the respective place. The three conditions kill the whole pathology catalogue: dead transitions (1), deadlocks and livelocks (2), residual tokens and activities (3).

Condition 2 is the subtle one: it does not force every run to terminate, only that completion remains *possible* from every reachable marking: loops are fine. To make this operational on looping nets one adopts the **fairness assumption**: a transition enabled infinitely often eventually fires, so no task is postponed forever and every fair run of a sound net eventually takes the exit.

Checking the definition naively is expensive: condition 1 is one reachability question *per transition*; condition 2 quantifies over *all reachable markings*; condition 3 is a *non-reachability* claim over the infinitely many markings $M > o$. The brute-force procedure works on the reachability graph: first check the net is a WF net (structural); then build the RG from i and verify that every transition labels some arc (condition 1), that the RG has exactly one sink, equal to o , reachable from every node (conditions 2–3), and that no other node marks o . Automatic, but limited by state explosion, unhelpful for *repairing* an unsound model, and inapplicable verbatim when the RG is infinite (the coverability graph is finite but loses exactness). Hence the structural strategy: translate soundness into liveness and boundedness, which have well-studied algorithms.

14.6 The auxiliary net N^*

A sound workflow plays a case from i to o ; if it could be *reset* (a token reaching o moved back to i) it could play again, any number of times, turning the open workflow into a cyclic system:



Definition (N^* and its strong connectedness). For a workflow net $N : i \rightarrow o$, let N^* be N plus one new transition *reset* with $\bullet \text{reset} = \{o\}$ and $\text{reset} \bullet = \{i\}$.

Proposition. N^* is strongly connected.

Proof. Take nodes x, y of N^* ; we exhibit a directed path from y to x . If neither is reset: by WF-net condition 3, y lies on a path $i \rightarrow^* y \rightarrow^* o$ and x on a path $i \rightarrow^* x \rightarrow^* o$; compose $y \rightarrow^* o \xrightarrow{\text{reset}} i \rightarrow^* x$. If $x = \text{reset}$: any path $y \rightarrow^* o$ extends by the arc $o \rightarrow \text{reset}$. If $y = \text{reset}$: the arc $\text{reset} \rightarrow i$ extends by a path $i \rightarrow^* x$. $\square$

Definition (Main theorem (van der Aalst)). N is **sound** $\iff N^*$ is **live and bounded**.

Definition (Proof ($\Leftarrow$): N^* live and bounded $\Rightarrow N$ sound). Liveness of N^* gives, for each $t \in T$, a reachable marking enabling t : condition 1. Now take any $M \in [i]$ enabling reset; then $M \supseteq o$, and firing reset gives $M' \supseteq i$ with $M' \in [i]$. If $M' \neq i$, the surplus $M' - i$ could be pumped by repeating the run-and-reset cycle, contradicting boundedness of N^* ; hence $M' = i$ and $M = o$. So whenever o is marked the rest of the net is empty (condition 3), and liveness of reset means a marking enabling it (i.e. reaching o) is reachable from every reachable marking (condition 2). $\square$

Definition (Technical lemma). If N is sound, then M is reachable in N iff M is reachable in N^* .

Proof. ($\Rightarrow$) trivial: N^* contains N . ($\Leftarrow$) Let $i \xrightarrow{\sigma} M$ in N^* ; induct on the number r of resets in σ . If $r = 0$, σ runs in N . If $r > 0$, split $\sigma = \sigma' \text{reset} \sigma''$ at the first reset: σ' is reset-free, so it runs in N and, since reset was enabled after it, reaches a marking $\supseteq o$; by soundness (proper completion) that marking is o , and firing reset returns exactly to i . Then $i \xrightarrow{\sigma''} M$ in N^* with $r - 1$ resets, and the inductive hypothesis closes. $\square$ Modulo soundness, reset adds no new states.

Definition (Proof ($\Rightarrow$): N sound $\Rightarrow N^*$ bounded and live). *Bounded.* Suppose not: some $i \rightarrow^* M \rightarrow^* M'$ in N^* with $M \subset M'$; let $L = M' - M > \mathbf{0}$. By the technical lemma M, M' are reachable in N ; by option to complete, $M \xrightarrow{\sigma} o$ in N for some σ ; by monotonicity, $M' = M + L \xrightarrow{\sigma} o + L$. So $o + L$ is reachable, marks o , and differs from o : contradicting proper completion.

Live. Take any transition t of N^* and any reachable M ; by the technical lemma M is reachable in N . Option to complete gives $M \xrightarrow{\sigma} o$; no-dead-task gives $i \xrightarrow{\sigma'} M'' \xrightarrow{t}$ for some M'' . Then in N^* : $M \xrightarrow{\sigma} o \xrightarrow{\text{reset}} i \xrightarrow{\sigma'} M'' \xrightarrow{t}$: from any reachable marking, t can be re-enabled. (For $t = \text{reset}$, stop at o .) $\square$

14.7 Strong connectedness

Definition (Paths, circuits, connectedness). In a net (P, T, F) , a *path* is a non-empty node sequence $x_1 \cdots x_k$ with $(x_i, x_{i+1}) \in F$; a *circuit* is a path with distinct nodes and $(x_k, x_1) \in F$: at least two nodes, since the flow relation has no self-arcs. An *undirected path* allows steps in $F \cup F^{-1}$. A net is *weakly connected* if any two nodes are joined by an undirected path (equivalently: it does not split into separated components), and *strongly connected* if any two are joined by a directed path (equivalently, for weakly connected nets: every arc lies on a directed cycle). Strong implies weak; the converse fails, and “strongly but not weakly” is vacuous.

The producer–consumer family illustrates the cases: with a one-way buffer the net is weakly but not strongly connected (nothing flows back); with producer and consumer disjoint it is not even weakly connected; closing the loop with a return arc makes it strongly connected. Disconnected nets decompose, so weak connectedness is assumed implicitly from now on.

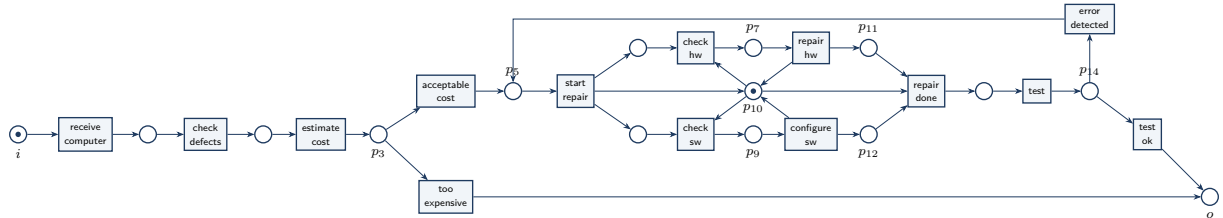
Definition (Strong connectedness theorem). If a weakly connected system is *live and bounded*, then it is *strongly connected*. (Proof omitted.)

The contrapositive is a cheap structural filter: a weakly-but-not-strongly connected net cannot be both live and bounded: if live then unbounded, if bounded then not live, and possibly neither (all three combinations occur on small examples). This is also why the N^* construction makes sense: N itself is never strongly connected (i has no inputs), so it can never be live and bounded: only its closure N^* can.

Note (Tool support: WoPeD diagnostics). WoPeD⁶¹ implements the soundness check through the N^* construction: it verifies the workflow-net property, builds N^* internally, and checks boundedness and liveness. **Reading caveat:** the boundedness/liveness entries of the analysis pane refer to N^* , not to the open net N : a transition flagged non-live in N^* signals a soundness violation of N . The diagnostics localise failures: a *possible deadlock* lists the non-live transitions and highlights the join whose inputs cannot be marked together; an *unbounded place* (“boundedness fails because the case can end leaving a token in c_8 ”) pinpoints residual tokens after completion. On sound nets the pane is all green (zero unbounded places, zero dead or non-live transitions) as on the claim-handling workflow (register, classify, simple/complex, parallel insurance and garage checks, decide, pay/send letter), which additionally passes the structural battery (free-choiceness, S-components, well-structuredness) of the next chapters.

14.8 Worked example: the Computer Repair Service

Example (CRS workflow: from text to a sound net). *Specification:* a customer brings a defective computer; the CRS checks the defect and produces a repair-cost estimate. If the cost is unacceptable the customer takes the computer home unrepaired; otherwise the repair proceeds with two activities in arbitrary order (check/repair the hardware, check/configure the software) after which the system is tested: on error the repair repeats, otherwise the computer is returned.



The construction proceeds clause by clause: a linear intake (*receive computer*, *check defects*, *estimate cost*); the customer’s XOR at p_3 (*too expensive* exits to o , *acceptable cost* continues); the two repair activities; and the test loop: *test ok* exits, *error detected* returns to p_5 for another repair round. The specification is precise about the repair: the hardware and software activities run “sequentially but in an arbitrary order”: not in parallel. A pure AND-split would let them interleave; instead *start repair* marks all three of p_6 , p_8 and the **shared place** p_{10} , and each *check* consumes p_{10} while the matching *repair/configure* returns it. That single token is a **mutex**: whichever branch starts first holds p_{10} until its *repair/configure* completes, so the other branch cannot begin meanwhile. Order is free, overlap is forbidden: exactly “sequential in arbitrary order”. *Repair done* then joins p_{11} , p_{12} and p_{10} . The result is a WF net by construction (unique i , unique o , all nodes on i – o paths). WoPeD confirms **soundness**: boundedness and liveness of N^* hold, no dead or non-live transitions. The structural pane reports one free-choice violation and three PT-handles (with three TP-handles): not soundness defects, but patterns that block the deeper structural theorems (free-choice nets, S/T-components) of the next chapters.

⁶¹<http://woped.dhbw-karlsruhe.de/woped/>

15 Sound by construction

The previous chapter's pipeline (build the net, derive N^* , check liveness and boundedness) is *verification*: design first, check later. The opposite philosophy is *construction*: assemble the workflow from pre-validated pieces with composition rules that preserve the desired properties, so the result is sound *by construction*. Two ingredients: a catalogue of **building blocks**, small WF nets easily proved sound and *safe* (1-bounded), and **composition patterns** under which sound-and-safe is closed. Together they form a calculus: every net produced by repeated application inherits the properties for free.

15.1 Composition by refinement

Let N and N' be safe and sound WF nets with interfaces i, o and i', o' , and let t be a task of N with **exactly one input and one output place** ($|\bullet t| = |t\bullet| = 1$).

Definition (Refinement $N[N'/t]$ and composition theorem). $N[N'/t]$ is obtained by *replacing t with N'* : the unique input place of t is fused with i' , the unique output place with o' , and t with its two arcs is removed: macro-expansion of a task into a full sub-procedure.

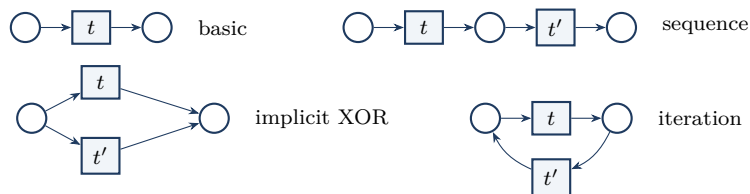
Theorem (proof sketched below). If N and N' are sound and safe, so is $N[N'/t]$.

Definition (Initial and final transitions). A transition t is *initial* if $i \in \bullet t$ and *final* if $o \in t\bullet$. In a *sound* WF net:

- every initial transition has $\bullet t = \{i\}$: were there another input place p , no token could ever reach p before t fires, so t would be dead;
- every final transition has $t\bullet = \{o\}$: were there another output place, its token would either survive completion (violating proper completion) or never be consumable.

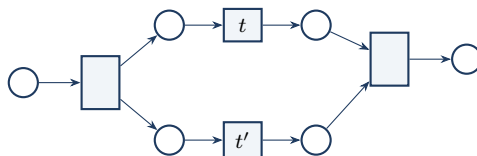
The initial transitions of N' take over the consuming role of t at its input place, the final transitions its producing role at the output place. *Proof idea of the theorem*: a sound WF net behaves like a transition whose consumption and production are not atomic: one token in, eventually one token out, nothing else across the interface. Formally one exhibits a bijection between the reachable markings of $N[N'/t]$ and the interface-respecting pairs of reachable markings of N and N' ; safeness and soundness of the composite follow from those of the parts.

15.2 Building blocks



All four are trivially sound and safe. The *implicit XOR* branches on a free-choice place (the token enables both alternatives, the chosen one consumes it); the two internal places of the *iteration* block must not coincide with i or o of the host net.

A second family reifies branching as transitions. Here one caution: a single transition never makes a choice: firing it marks *all* its output places at once, which is an AND, not an XOR. The choice must sit on a place. So the *explicit XOR-split* is an entry transition feeding one internal place from which two alternative transitions compete (the free-choice place carries the decision, exactly as in the implicit XOR); the *explicit XOR-join* is the dual; and the combined *XOR block* chains an explicit split, two alternative tasks and an explicit merge. The *AND block* is drawn like the XOR block but its entry transition produces *both* internal places simultaneously: t and t' then interleave freely and the exit transition consumes both. Split as concurrency, not as choice: that single difference in the entry transition's post-set separates AND from XOR. The catalogue is open: any net proved sound and safe may join it.



15.3 Refinement and abstraction at work

Refinement grows nets top-down. Start from the basic block. Replace its one task by a sequence. Replace a task of that result by another sequence. Replace still others by iteration and XOR blocks. After a few such rounds the net is a structured combination of sequences, a loop and an exclusive branch: sound and safe at every intermediate step, with no global verification ever run.

Abstraction is the dual, and the practical proof technique: to certify a given net, recognise sub-patterns matching catalogue blocks, collapse each to a single task, repeat. A worked reduction: an iteration block collapses first, then an explicit XOR block along the bottom, then a sequence in the upper branch, another iteration, another sequence; the two residual branches between the AND-split and AND-join form a parallel block, which collapses to the *basic* block: the structural certificate that the original net was sound and safe.

The word *abstraction* carries three distinct senses across the course, and naming them apart keeps this reduction calculus from being confused with the others. **Horizontal abstraction** moves *along* the control flow at one level: collapse a whole block (a sequence, an XOR-choice, an AND-parallelism, an iteration) into a single node, the inverse of refinement, which is exactly the calculus above. **Vertical abstraction** moves *across* hierarchy levels: a task hides an entire sub-net, and collapsing or expanding it changes the level of detail: the sub-process construction of the Petri-nets chapter (a transition realised by a whole workflow net) and the collapsed/expanded sub-processes of BPMN. **Aggregated abstraction** folds *many elements into one* summary rather than a block or a level: many measurements collapse into one aggregate value, as with the KPI aggregation functions of the quantitative-analysis chapter, or a discovered process map is read at a coarser level of detail in process mining. Horizontal and vertical abstraction are structure-preserving and reversible: collapse then expand recovers the original; aggregation is lossy, since a single summary cannot in general be resolved back into the elements it condensed.

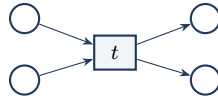
Example (Design example: Car Damage). The car-damage claim workflow (drawn as a workflow net in the Petri-nets chapter) was built deliberately by composition, and the abstraction calculus certifies it: the *simple* branch contains a textbook AND block (AND-split, *phone garage* || *check insurance*, AND-join);

the *complex* branch is a plain sequence (*check insurance ok*, *check history*, *phone garage*); *classify* with the two branches and their merge forms an explicit XOR; and *decide* routing to *pay* or *send letter*, merged at *end*, forms a second one. Every pattern is in the catalogue, the net reduces to the basic block: **sound and safe by construction**.

Example (Desugaring first, then reduce). A more intricate exercise net does not match any catalogue block as drawn: its XOR and AND branch points are still gateway shorthands, not explicit transitions. The fix is *desugaring*: expand each gateway into the Petri-net transitions it abbreviates (an XOR-split into two alternative entry transitions on a shared place, an AND-split into one forking transition, and so on). Once desugared, the net is a plain Petri net of sequences, XOR blocks, an AND block and an iteration, and abstraction certifies it clause by clause: a sequence collapses, an implicit XOR collapses, the lower-branch iteration collapses, the two parallel branches fold into the AND block, and the last step reaches the *basic* block: **sound and safe**. A second exercise looks different but, after the same desugaring, *is the same net*, and reduces identically. The moral is not that the calculus stalls: it is that syntax must be normalised before the block patterns become visible. Desugaring is the method's first degree of freedom; the second (extending the catalogue with independently verified blocks) is the generalisation below.

15.4 Generalised substitution

The substitution rule so far requires $|\bullet t| = |t\bullet| = 1$. The generalisation targets transitions with arbitrary fan-in and fan-out:



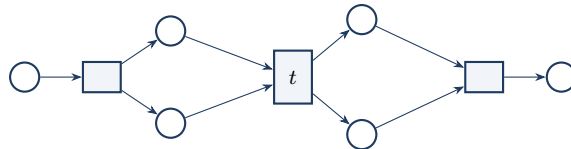
Definition (General replacement $N[N'/t]$). Let t be any transition of N , and N' a WF net with initial place i' , final place o' ,

$$T_{i'} = \{u \mid \bullet u = \{i'\}\} \quad (\text{initial transitions}), \quad T_{o'} = \{v \mid v\bullet = \{o'\}\} \quad (\text{final transitions}).$$

$N[N'/t]$ removes t from N and i', o' from N' , then wires: for every input arc (p, t) of t and every $u \in T_{i'}$, an arc (p, u) ; for every output arc (t, q) and every $v \in T_{o'}$, an arc (v, q) .

Theorem (proof omitted). If N and N' are sound and safe, so is $N[N'/t]$.

The wiring replicates locally the connection pattern t had towards the rest of N ; the single-input-single-output rule is the special case of singleton $\bullet t$ and $t\bullet$. A fifth catalogue block exploits the new freedom, *AND with sync*: an AND-split forks two places, a central transition consumes *both* and produces *both* the places consumed by the final AND-join:



Example (Generalised refinement and a full certification). *Refinement with fan-in/fan-out*: a central transition t_2 with two inputs (p_2, p_3) and two outputs (p_4, p_5) is replaced by a sequence $u \rightarrow q \rightarrow v$; being the unique initial and final transitions of the inserted net, u inherits both input arcs and v both output arcs: the interface is preserved, the behaviour expands.

A complete reduction: a larger multi-arc net is certified by abstraction: an implicit XOR in the upper branch collapses, then two sequences; the lower branch contains a non-canonical sub-net (two parallel multi-arc transitions plus a feedback transition) that is promoted to an *ad-hoc building block* after an independent proof of its soundness and safeness; with it in the catalogue the lower branch reduces to a sequence, then to a single transition; the two residual branches between i and o form an implicit XOR; one last collapse reaches the basic block. The method's two degrees of freedom (desugar the syntax, extend the catalogue with independently verified blocks) make the calculus practical beyond the four primitive patterns.

16 Invariants

An **invariant** of a dynamic system is an assertion that holds at every reachable state. Invariants are negative oracles: a state violating the invariant is certainly unreachable, with no enumeration of the reachable set. This chapter develops two structural classes for Petri nets: *place invariants* (S-invariants) and *transition invariants* (T-invariants), computable from the incidence matrix alone.⁶²

Example (The mutilated chessboard). Remove two opposite corners from a chessboard (62 cells remain) and take 31 dominoes, each covering two adjacent cells. The arithmetic fits ($31 \times 2 = 62$), yet *no tiling exists*: the removed corners share a colour, leaving 32 cells of one colour and 30 of the other, while every domino covers one of each: so any tiling covers equal numbers. The conserved quantity (covered-black minus covered-white) discriminates reachable from unreachable configurations without trying a single tiling.

An invariant is always relative to a set of admissible moves. For a polygon under {rotate, move, mirror} the perimeter and area are invariant.

Adding *scale* kills both, but spares the combinatorial quantities (number of vertices and sides, vertex degrees, convexity) and the label (colour).

Adding *stretch* (non-uniform scaling) still preserves colour, convexity, sides and vertices, since affine maps send edges to edges; degenerate stretches, however, can merge vertices and break degrees. Choosing the move set is part of stating the invariant.

For a Petri net the moves are the firings of enabled transitions. Trivial invariants: the labels and the static skeleton (P, T, F) (firing changes only the marking), the token count of places no reachable firing touches. The token count of an arbitrary place is *not* invariant: any transition consuming from it changes it. More interestingly, the system-level properties themselves are invariants:

Note (Lemma: properties closed under reachability). If (P, T, F, M_0) is live (resp. deadlock-free, bounded, cyclic) and $M \in [M_0]$, then (P, T, F, M) is live (resp. deadlock-free, bounded, cyclic). Here *cyclic* means $\forall M \in [M_0]. M_0 \in [M]$: no point of no return.

Proof shape (same for all four). Any $M' \in [M]$ is also in $[M_0]$ by transitivity of reachability, since $[M] \subseteq [M_0]$. For live and deadlock-free (purely universal properties) the claim instantiates to every such M' directly. For boundedness, which is $\exists k \forall M'. M'(p) \leq k$, the same bound k that witnesses $[M_0]$ also bounds the smaller set $[M]$. For cyclicity, concatenate the return path $M' \rightarrow^* M_0$ with $M_0 \rightarrow^* M$. $\square$

⁶²Desel J., Esparza J., *Free Choice Petri Nets*, Cambridge University Press, <https://www7.in.tum.de/~esparza/bookfc.html>.

16.1 S-invariants

Structural invariants are vectors of rationals computed from the net's skeleton, independent of the initial marking; a basis is computable in polynomial time. Reading a model through its invariants exposes the conservation laws that govern every reachable marking.

Definition (S-invariant and its fundamental property). An **S-invariant** (place invariant) of $N = (P, T, F)$ with incidence matrix $\mathbf{N}$ is a row vector $\mathbf{I}$ of length $|P|$ with

$$\mathbf{I} \cdot \mathbf{N} = \mathbf{0}.$$

$\mathbf{I}$ assigns a weight to each place; the equation says every transition preserves the weighted token count. Linear combinations of S-invariants are S-invariants (the solution set of a homogeneous system is a vector space, so a basis describes them all).

Fundamental property. For every S-invariant $\mathbf{I}$ and every $M \in [M_0]$: $\mathbf{I} \cdot M = \mathbf{I} \cdot M_0$.

Proof. $M = M_0 + \mathbf{N} \cdot \vec{\sigma}$ for some firing sequence σ (marking equation), so $\mathbf{I} \cdot M = \mathbf{I} \cdot M_0 + (\mathbf{I} \cdot \mathbf{N}) \cdot \vec{\sigma} = \mathbf{I} \cdot M_0$. $\square$

On the vending-machine net of the incidence-matrices chapter ($M_0 = 4p_1 + p_3$), the system $\mathbf{I} \cdot \mathbf{N} = \mathbf{0}$ expands to $x_1 = x_2$, $x_3 = x_4 = x_5$: every S-invariant has the form $[n, n, m, m, m]$, a two-dimensional space with basis $\{[1, 1, 0, 0, 0], [0, 0, 1, 1, 1]\}$: the four stock-side tokens stay in $\{p_1, p_2\}$, the single control token stays in $\{p_3, p_4, p_5\}$.

Note (A cheap necessary condition for reachability). Reachability is decidable but EXPSpace-hard. Two cheap necessary tests: if some S-invariant has $\mathbf{I} \cdot M \neq \mathbf{I} \cdot M_0$, then $M \notin [M_0]$; and if $\mathbf{N} \cdot \mathbf{x} = M - M_0$ has no rational solution, then $M \notin [M_0]$. Neither is sufficient. Example: in the producer/consumer net, the consumer-loop invariant $\mathbf{I} = \text{cons_free} + \text{cons_busy}$ has a fixed value determined by M_0 ; any candidate marking with a different consumer count (say one consumer-state token where M_0 had none) is rejected as unreachable without exploring a single state.

The conservation reading: tokens are coins, places are denominations, $\mathbf{I}(p)$ is the value of denomination p ; every firing is a change-making operation that preserves the total value $\mathbf{I} \cdot M$. The same intuition (transitions redistribute but never create or destroy the weighted substance) yields a local test:

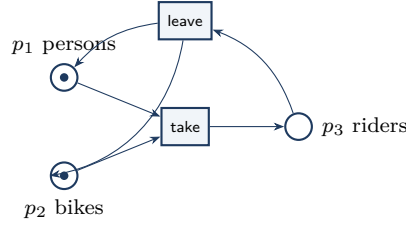
Note (Alternative characterisation). $\mathbf{I} : P \rightarrow \mathbb{Q}$ is an S-invariant iff for every transition t :

$$\sum_{p \in \bullet t} \mathbf{I}(p) = \sum_{p \in t \bullet} \mathbf{I}(p) \quad (\text{flow-in weight} = \text{flow-out weight}).$$

Proof. $\mathbf{I} \cdot \mathbf{N} = \mathbf{0}$ holds iff $\mathbf{I} \cdot \mathbf{t} = 0$ for every column $\mathbf{t}$; expanding with $\mathbf{N}(p, t) = +1$ on $t \bullet$, -1 on $\bullet t$, 0 elsewhere gives $\sum_{p \in t \bullet} \mathbf{I}(p) - \sum_{p \in \bullet t} \mathbf{I}(p) = 0$. $\square$

One balance per transition, checkable on the picture without the matrix: the workhorse both for verifying and for *constructing* invariants.

Example (Bike rental: which candidates are S-invariants?).



With **take**: $p_1 + p_2 \rightarrow p_3$ and **leave**: $p_3 \rightarrow p_1 + p_2$, the local balance reduces to one equation: $\mathbf{I}(p_1) + \mathbf{I}(p_2) = \mathbf{I}(p_3)$. Testing candidates $[p, b, r]$: $[1, 0, 1]$ **yes** (persons free or riding: constant), $[0, 1, 1]$ **yes** (bikes free or ridden), $[1, -1, 0]$ **yes** (persons-minus-bikes surplus), $[1, 1, 2]$ **yes** (a rider counts as person + bike); $[1, 1, -1]$, $[1, 1, 1]$, $[1, 2, 2]$ **no** (balance fails; $[1, 1, 1]$ double-counts the rider). The four winners are not independent: $[1, 1, 2] = [1, 0, 1] + [0, 1, 1]$ and $[1, -1, 0] = [1, 0, 1] - [0, 1, 1]$: a basis is $\{[1, 0, 1], [0, 1, 1]\}$, the two natural conservation laws.

Example (Reading invariants off the picture). *Traffic lights with mutex*: per light, the three state places form a uniform invariant (“each light shows exactly one colour”); a third invariant weights green and yellow of both lights together with the mutex place: the safety law “at most one light is away from red”. Sums of these are again invariants.

Producer/consumer: by the linear system, or directly on the picture: weight the producer loop $\{\text{prod_free}, \text{prod_busy}\}$ with 1; the balance at **prod_end** then forces the buffer weight to 0; zero-weight places decouple the picture, and the consumer loop independently takes weight 1. Basis: $[1, 1, 0, 0, 0]$ and $[0, 0, 0, 1, 1]$, the producer and the consumer never disappear, whatever the buffer does. Their sum is the combined law “one producer-state plus one consumer-state token, always”.

Dining philosophers: each pair $(\text{think}_i, \text{eat}_i)$ is a two-place loop, giving the uniform invariant $\text{think}_i + \text{eat}_i = 1$ (five of them). For the resources, the naive “ eat_i + both adjacent forks” with unit weights fails the balance at **start_eating_i** (flow-in 2, flow-out 1); the corrected versions are either $2\text{eat}_i + f_{\text{left}} + f_{\text{right}}$, or (cleanest) one invariant *per fork*: $f + \text{eat}_A + \text{eat}_B = 1$ for the two neighbours A, B of f , i.e. each fork is free or held by exactly one neighbour. Sums build global laws (“five philosophers, each in some state”); the picture-level method (one local balance per transition, small loops first, weight 0 for outsiders) scales where solving the system would be tedious.

16.2 S-invariants and system properties

Definition (Semi-positive, support, positive, uniform). An S-invariant $\mathbf{I}$ is **semi-positive** ($\mathbf{I} > \mathbf{0}$) when $\mathbf{I} \geq \mathbf{0}$ and $\mathbf{I} \neq \mathbf{0}$; its **support** is $\langle \mathbf{I} \rangle = \{p \mid \mathbf{I}(p) > 0\}$; it is **positive** ($\mathbf{I} \succ \mathbf{0}$) when $\langle \mathbf{I} \rangle = P$; **uniform** when semi-positive with coefficients in $\{0, 1\}$ (a plain token count). Every semi-positive S-invariant satisfies $\bullet \langle \mathbf{I} \rangle = \langle \mathbf{I} \rangle \bullet$: the transitions feeding the support are exactly those draining it (symmetrically for T-invariants).

Note (Theorem: positive S-invariant $\Rightarrow$ bounded). If the system admits a positive S-invariant $\mathbf{I}$, it is bounded; more precisely every $p \in \langle \mathbf{I} \rangle$ of a semi-positive invariant satisfies

$$M(p) \leq \frac{\mathbf{I} \cdot M_0}{\mathbf{I}(p)} \quad \text{for every } M \in [M_0].$$

Proof. $\mathbf{I}(p)M(p) \leq \mathbf{I} \cdot M = \mathbf{I} \cdot M_0$ since the other terms are non-negative; divide by $\mathbf{I}(p) > 0$. $\square$

On the bike rental with one person and one bike, $\mathbf{I} = [1, 1, 2]$ is positive with $\mathbf{I} \cdot M_0 = 2$, so $M(p_3) \leq 2/2 = 1$: at most one rider, ever. The sharper $\mathbf{I} = [1, 0, 1]$ gives the same bound from

1/1: different invariants give different bounds, and hunting the tightest is a linear-programming question. On the vending machine, $[1, 1, 1, 1, 1]$ (sum of the basis) is positive with $\mathbf{I} \cdot M_0 = 5$: every place is 5-bounded.

Note (Theorem: a necessary condition for liveness). If the system is live, then every semi-positive S-invariant satisfies $\mathbf{I} \cdot M_0 > 0$.

Proof. Pick $p \in \langle \mathbf{I} \rangle$ and a transition t adjacent to p . By liveness some reachable M fires t : if t produces into p the successor marks p ; if t consumes from p then M itself marks p . Either way some reachable M^* has $\mathbf{I} \cdot M^* \geq \mathbf{I}(p)M^*(p) > 0$, and the fundamental property transports the value back to M_0 . $\square$

Contrapositive certificate: a semi-positive S-invariant with $\mathbf{I} \cdot M_0 = 0$ proves the system non-live. To find one, look for an invariant whose support avoids all initially marked places: on the bike rental started with bikes only, $[1, 0, 1]$ has value 0: **take** can never fire; on a seven-place exercise net marked only on p_5, p_6 , the invariant supported on $\{p_2, p_3, p_4\}$ does the job; on the producer/consumer started without consumer tokens, the consumer-loop invariant $[0, 0, 0, 1, 1]$ kills liveness.

Two markings *agree on all S-invariants* when $\mathbf{I} \cdot M = \mathbf{I} \cdot M'$ for every $\mathbf{I}$: equivalently, when $\mathbf{N} \cdot \mathbf{y} = M' - M$ has a rational solution. Agreement is necessary for mutual reachability, not sufficient: unreachable look-alikes exist. S-invariants are negative oracles: positive invariant $\Rightarrow$ bounded; $\mathbf{I} \cdot M_0 = 0$ (semi-positive) $\Rightarrow$ non-live; $\mathbf{I} \cdot M \neq \mathbf{I} \cdot M_0 \Rightarrow M$ unreachable, and none of the converses holds. The producer/consumer with unbounded buffer shows the first gap concretely: the balance at `prod_end` forces buffer weight $x = 0$ in every invariant ($y = y + x$), so no positive S-invariant exists: consistently with the unbounded buffer.

16.3 T-invariants

Definition (T-invariant and its fundamental property). A **T-invariant** of N is a column vector $\mathbf{J}$ of length $|T|$ with

$$\mathbf{N} \cdot \mathbf{J} = \mathbf{0}.$$

$\mathbf{J}(t)$ is an occurrence count for t : firing every transition $\mathbf{J}(t)$ times causes zero net change everywhere.

Fundamental property. For $M \xrightarrow{\sigma} M'$: $\vec{\sigma}$ is a T-invariant iff $M = M'$. *Proof:* immediate from the marking equation $M' = M + \mathbf{N} \cdot \vec{\sigma}$, both directions. $\square$

Alternative characterisation (dual of the S case): $\mathbf{J}$ is a T-invariant iff at every *place* p , $\sum_{t \in \bullet p} \mathbf{J}(t) = \sum_{t \in p \bullet} \mathbf{J}(t)$: weighted production equals weighted consumption, one balance per place, readable off the picture.

T-invariants are the reproducibility certificates: multisets of firings that, when executable, return the marking to its starting point: cycles of the reachability graph up to permutation. On the bike rental, $[1, 1]$ (one **take**, one **leave**) is a T-invariant. A richer exercise net: $t_1 : p_1 \rightarrow p_2 + p_3$, $t_2 : p_1 \rightarrow p_4 + p_5$, $t_4 : p_2 \rightarrow p_4$, $t_3 : p_5 \rightarrow p_3$, $t_5 : p_3 + p_4 \rightarrow p_1$: a token leaves p_1 along one of two routes, splits, and t_5 reassembles it. Testing candidates $[t_1, \dots, t_5]$ against the per-place balance: $[1, 0, 0, 1, 1]$, $[1, 1, 1, 1, 2]$ and $[0, 1, 1, 0, 1]$ are T-invariants; $[1, 1, 2, 1, 2]$ is not (at p_3 , in-flow $\mathbf{J}(t_1) + \mathbf{J}(t_3) = 3$ against out-flow $\mathbf{J}(t_5) = 2$), nor is $[1, 1, 1, 0, 2]$ (the zero on t_4 breaks the balance at p_2).

16.4 T-invariants and system properties

Note (Reproduction lemma and the bounded-live theorem). *Pigeonhole*: a path through more states than exist must revisit one.

Lemma. In a bounded system, if $M_0 \xrightarrow{\sigma}$ for an infinite σ , there is a semi-positive T-invariant supported on transitions of σ . *Proof.* The reachable set is finite, so the infinite trajectory revisits a marking: $M_i = M_j$ with $i < j$; the Parikh vector of the segment $t_{i+1} \cdots t_j$ is a non-zero T-invariant by the fundamental property. $\square$

Theorem. A bounded *and* live system has a *positive* T-invariant. *Proof.* By liveness, from any marking a sequence firing *every* transition at least once exists; chain $k + 1$ such sequences ($k = |[M_0]|$): among the $k + 2$ intermediate markings two coincide, and the enclosed segment is a T-invariant containing at least one full sweep: positive on every transition. $\square$

The contrapositive chain is the diagnostic: no positive T-invariant $\Rightarrow$ not both live and bounded; combined with liveness it proves unboundedness, with boundedness (e.g. via a positive S-invariant) it proves non-liveness. The implication does *not* reverse: a positive T-invariant alone certifies nothing:

- two dead transitions over two empty places admit $\mathbf{J} = [1, 1]$, yet nothing ever fires;
- the producer/consumer with an unmarked consumer side has $\mathbf{J} = [1, 1, 1, 1]$ positive, yet is non-live (dead consumer) *and* unbounded (the producer pumps the buffer);
- variants where the buffer is never consumed are live but unbounded, with the same positive $\mathbf{J}$.

16.5 Strong connectedness via invariants

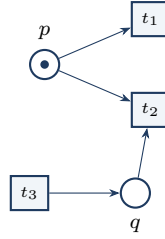
Note (Theorem (proof omitted)). A weakly connected net with a positive S-invariant *and* a positive T-invariant is strongly connected. Contrapositive: a weakly-but-not-strongly connected net cannot have both.

The producer/consumer illustrates one direction: not strongly connected, positive T-invariant $[1, 1, 1, 1]$, but no positive S-invariant (the buffer is stuck at weight 0). A mirror example with a positive S-invariant (weights ≥ 1 everywhere, place p_1 at 2) but no positive T-invariant is bounded yet non-live: two invariant computations pin down the qualitative behaviour with no simulation at all.

17 S-systems and T-systems

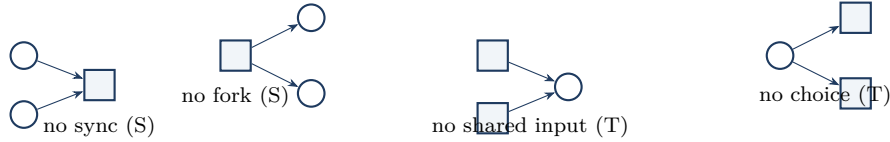
Structural properties depend only on the net's topology; behavioural ones on the token game and M_0 . A *structural characterisation* of a behavioural property (the prize of this chapter) yields checks that are far cheaper than reachability analysis, valid for a whole class of nets.⁶³ The classes are carved by forbidding the troublesome interaction between *conflict* and *synchronisation*:

⁶³Desel J., Esparza J., *Free Choice Petri Nets*, Cambridge University Press, <https://www7.in.tum.de/~esparza/bookfc.html>.



Initially t_1 and t_2 are not in conflict: t_2 also needs the token that only t_3 can produce on q . Once t_3 fires, the two compete for the token on p : a local choice invalidated by an unrelated, uncontrollable synchronisation elsewhere. The two net classes below each outlaw one half of the phenomenon.

Definition (S-nets and T-nets). A net is an **S-net** (*Stellen* = places) if every transition has exactly one input and one output place: $\forall t. |\bullet t| = 1 = |t\bullet|$: *synchronisation and forking are ruled out*; a transition only moves one token. A net is a **T-net** if every place has exactly one input and one output transition: $\forall p. |p\bullet| = 1 = |\bullet p|$: *choice and conflict are ruled out*; concurrency remains, but no transition ever steals a token from another. An **S-system** (**T-system**) is a system whose net is an S-net (T-net).



The classes overlap without nesting: a plain cycle of alternating places and transitions is both; a net with a free-choice place (two consumers) is S-only; one with a synchronising transition is T-only; nets violating both constraints are neither: one example per region of the Euler–Venn diagram.

17.1 S-systems

Note (Fundamental property of S-systems). Write $M(P) = \sum_{p \in P} M(p)$ for the total token count (e.g. $M = 2p_1 + 3p_2$ over three places has $M(P) = 5$). Every firing in an S-system removes exactly one token and inserts exactly one, so:

Proposition. In an S-system, $M \in [M_0]$ implies $M(P) = M_0(P)$.

Proof. Induction on the firing sequence: the inductive step fires one t with $|\bullet t| = |t\bullet| = 1$, so $M'(P) = M(P) - 1 + 1$. $\square$

Corollary. Every S-system is bounded: $M(p) \leq M(P) = M_0(P)$ for every place.

Proposition. In a *connected* S-net, $\mathbf{I}$ is an S-invariant iff $\mathbf{I} = [k, \dots, k]$: the per-transition balance $\mathbf{I}(p^t) = \mathbf{I}(p_t)$ forces equal weights across every transition, and connectedness propagates the constant. (On general nets, the uniform vectors are always invariants; non-uniform candidates must pass the balance at the bridging transitions.)

The negative oracle in action: on a five-place S-net with $M_0 = p_1 + 2p_2 + 3p_5$ ($M_0(P) = 6$), the marking $p_2 + 4p_4 + 2p_5$ has total 7: unreachable by arithmetic alone.

Note (Liveness theorem for S-systems). An S-system is live **iff** its net is strongly connected and M_0 marks at least one place.

Proof ($\Rightarrow$). Live by hypothesis, bounded because S-system; by the strong-connectedness theorem (live + bounded $\Rightarrow$ strongly connected on weakly connected nets), the net is strongly connected. Liveness also gives some enabled t at some reachable marking, whose unique input place must eventually be marked, so $M_0(P) \geq 1$.

Proof ($\Leftarrow$). Take $M \in [M_0]$ and $t \in T$. Some p_1 has $M(p_1) \geq 1$ (totals are conserved and positive). Strong connectedness gives a directed path $p_1 t_1 p_2 t_2 \cdots p_n t_n = t$; since each t_i has exactly $\bullet t_i = \{p_i\}$ and $t_i^\bullet = \{p_{i+1}\}$, firing $t_1 \cdots t_{n-1}$ walks the token to the unique input place of t , enabling it. $\square$

The constructive proof doubles as an operating manual: to enable a chosen transition, route any token along a path to it. On the question-time nets the verdicts follow instantly: strongly connected and marked: **live**; any one-way bottleneck between components, however large the net: **not live**.

Note (Reachability in live S-systems). Lemma. In a strongly connected S-net, $M(P) = M'(P)$ implies M' reachable from M .

Proof. Induction on $M(P)$. Base 0: both markings are empty. Step: pick p with $M(p) > 0$ and p' with $M'(p') > 0$; set $K = M - p$, $K' = M' - p'$. By induction $K \xrightarrow{\sigma} K'$. Strong connectedness gives a path from p to p' , and the S-net constraint makes its transition sequence σ' slide one token from p to p' . By monotonicity, $M = K + p \xrightarrow{\sigma} K' + p \xrightarrow{\sigma'} K' + p' = M'$. $\square$

Theorem. In a *live* S-system, M is reachable iff $M(P) = M_0(P)$. ($\Rightarrow$ is the fundamental property; $\Leftarrow$: liveness gives strong connectedness, then the lemma applies.)

So for live S-systems reachability *is* token counting: $p_2 + 4p_4 + p_5$ (total 6) is reachable from $M_0 = p_1 + 2p_2 + 3p_5$, while any total-7 marking is not.

Note (Workflow S-nets are safe and sound). A workflow net N is an S-net iff N^* is (the reset transition has one input, o , and one output, i). **Theorem:** a workflow net that is an S-net is *safe and sound*.

Proof. N^* is an S-system, hence bounded; N^* is strongly connected (workflow short-circuit); $M_0(P) = 1 > 0$, so by the liveness theorem N^* is live; live + bounded $N^* \iff N$ sound. Safeness: the fundamental property keeps $M(P) = 1$ forever, so no place ever holds two tokens. $\square$

Two visual checks (workflow shape; every transition one-in-one-out) settle soundness with no behavioural analysis at all.

17.2 T-systems

A workflow net is never a T-net (i lacks an input, o an output), but its short-circuit N^* can be: the reset transition supplies exactly the missing arcs. The behavioural theory of T-systems rests on *circuits*: closed directed loops γ with place set P_γ and token count $M(\gamma) = \sum_{p \in P_\gamma} M(p)$; γ is *marked* when $M(\gamma) > 0$.

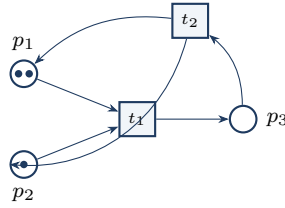
Note (Fundamental property of T-systems). Proposition. For every circuit γ of a T-system and every $M \in [M_0]$: $M(\gamma) = M_0(\gamma)$.

Proof. Fire any t . If t is not on γ , it touches no place of γ : a place of γ adjacent to t would have t as its unique producer or consumer, putting t on the circuit. If t is on γ , exactly one place of $\bullet t$ and one of $t^\bullet$

lie on γ (its circuit predecessor and successor): one token leaves a γ -place, one enters another. Either way $M(\gamma)$ is unchanged. $\square$

Dually to S-nets: $\mathbf{J}$ is a T-invariant of a connected T-net iff $\mathbf{J} = [k, \dots, k]$ (the per-place balance forces equal counts on each place's unique producer and consumer). Computation in T-systems is concurrent but *deterministic*: firing one enabled transition can never disable another; no place has two consumers.

Example (Circuit counts as reachability oracles). On the bounded-buffer producer/consumer T-system the three circuits are the producer loop, the consumer loop, and the buffer loop through `free_slot` and `item_buffer`; each carries its own conserved count. A candidate marking that alters any single circuit count (say the buffer loop from 2 to 3) is unreachable, even when the *global* token count matches: the per-circuit law is strictly sharper than the S-system criterion. Smallest example:



Two circuits: $\gamma_1 = p_1 t_1 p_3 t_2$ and $\gamma_2 = p_2 t_1 p_3 t_2$, with $M_0(\gamma_1) = 2$ and $M_0(\gamma_2) = 1$. The marking $p_1 + 2p_2$ keeps the global total (3) but raises $M(\gamma_2)$ to 2: **unreachable**.

Note (Boundedness, safeness, liveness of T-systems). **Lemma.** A strongly connected T-system is bounded: every place lies on some circuit γ_p , and $M(p) \leq M(\gamma_p) = M_0(\gamma_p) \leq \max_{\gamma} M_0(\gamma)$. $\square$

Corollaries. If moreover $M_0(P) = 1$, or more generally $M_0(\gamma) \leq 1$ for every circuit, the system is *safe*: many tokens are fine as long as no circuit carries two.

Liveness theorem. A T-system is live **iff** every circuit is marked at M_0 . *Proof* ($\Rightarrow$). An empty circuit stays empty (fundamental property), so any transition on it waits forever for its circuit-predecessor place: dead. ($\Leftarrow$, *sketch*.) For a target t and marking M , consider the places from which unmarked paths lead to t ; the marked-circuit hypothesis always allows a firing that shrinks this set, and induction empties it, enabling t . $\square$

The verdicts are pure circuit checks: token positions within a circuit are irrelevant, only coverage counts. A place is bounded iff it lies on some circuit: in the *unbounded*-buffer producer/consumer, both loops are marked (so the system is live) but `item_buffer` sits on no circuit and grows without bound.

Note (Workflow T-nets). **Theorem.** If N is a workflow net and N^* is a T-system, then N is safe and sound **iff** every circuit of N^* is marked.

Proof. N^* is strongly connected (workflow short-circuit), hence bounded; with $M_0(P) = 1$ it is safe; marked circuits $\iff N^*$ live; live + bounded $N^* \iff N$ sound. $\square$

Worked checks: a fork-join workflow whose N^* has three circuits, all threaded through the initially marked i : safe and sound at a glance. A variant with a small internal loop on one branch fails instantly: that loop is a circuit of N^* carrying *no* token in M_0 , so the marked-circuit hypothesis breaks and the net is **not sound**: one unmarked closed loop, visible by eye, is enough.

Note (Recap). S-systems: bounded always; live $\iff$ strongly connected + marked; reachability = token count (when live); S-invariants constant. T-systems: per-circuit conservation; strongly connected $\Rightarrow$ bounded, and live + bounded $\Rightarrow$ strongly connected; place-bounded $\iff$ on a circuit; safe under unit

circuit counts; live $\iff$ all circuits marked; T-invariants constant. Workflow nets: S-net $\Rightarrow$ safe and sound; N^* T-net $\Rightarrow$ safe and sound $\iff$ circuits marked.

18 EPC analysis

EPCs are graphically clean but semantically loose: the XOR-join is disputed when several inputs arrive (fire once per input? block?), and for the OR-join *no* semantics in the literature works on every diagram: every published definition fails somewhere.⁶⁴ Multiple start and end events are allowed, but the diagram does not record which combinations are intended. The strategy is to *translate* the EPC into a workflow net, declare the EPC **sound** iff the net is, and reuse the verification machinery. The translation is not canonical (especially around the OR-join) and the soundness verdict depends on the choices made.

18.1 The translation pipeline

Three steps, supported by WoPeD: **Step 1** replaces each EPC node by a Petri-net fragment (event $\rightsquigarrow$ place, function $\rightsquigarrow$ transition, connector $\rightsquigarrow$ a small fragment depending on kind and split/join role); **Step 2** wires the fragments along the EPC arcs, either in *dummy style* (insert a dummy place/transition at each seam: verbose but mechanical) or in *fusion style* (merge compatible endpoints: tighter, falls back to dummies at irregular junctions); **Step 3** enforces the workflow shape, funnelling multiple start events from a fresh **start** place (via an XOR-, AND-, or mixed split, depending on whether the starts are exclusive, joint, or independent: information the EPC does not record) and dually funnelling the sinks into a fresh **end** place. Three strategies instantiate the pipeline:

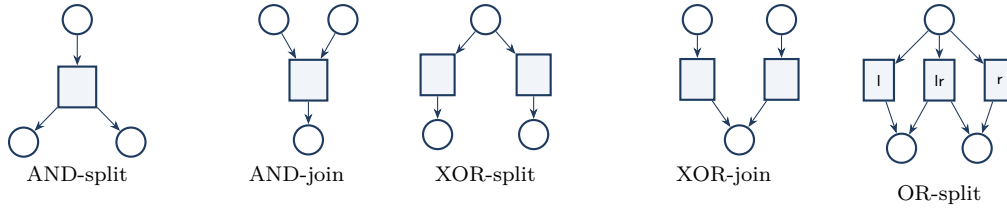
	ingenuity / style	applicability	outcome
1st	easy, fusion	any EPC	likely unsound; relaxed soundness
2nd	medium, direct (dummy opt.)	no OR, strict alternation	free-choice net, fast analysis
3rd	hard, dummy	decorated EPC (pairings, policies)	accurate analysis

18.2 First approach: straight translation and relaxed soundness

Following Dehnert and Rittgen,⁶⁵ each connector is replaced by a fragment that preserves *all* possibly intended behaviours: EPCs are agreed upon without committing to a semantics, and the translation keeps every interpretation alive, judging the result under a weaker criterion. The fragments:

⁶⁴*Petri Net Transformations for Business Processes: A Survey*, §4.2; Weske M., *Business Process Management: Concepts, Languages, Architectures*, Springer, Ch. 6.

⁶⁵Dehnert J., Rittgen P., “Relaxed Soundness of Business Processes”, *CAiSE 2001*, LNCS 2068, Springer.



The AND fragments are deterministic (one synchronising transition). The XOR-split is the free-choice conflict on a shared place; the XOR-join lets the first arrival through. The OR connectors are the awkward ones: one transition *per non-empty subset* of branches (left, right, both: growing exponentially with the arity), so that every interpretation remains executable; the OR-join mirrors the split with a subset-consuming fan-in. Fusion then merges compatible endpoints across each EPC arc (element fusion when a place meets a transition; arc fusion identifying duplicated seam places).

Example (Worked example: goods reception). The running EPC: goods arrived triggers an AND-split into a left branch: check goods, then an XOR (ok directly, or not ok $\rightarrow$ complaint $\rightarrow$ data revised rejoining via an XOR-join), then store goods producing stored, and a right branch through an OR-join into record receipts of goods, producing goods recorded. The translation expands every connector (the OR-join becoming its multi-transition subset fan-in), fuses the seam places, and closes Step 3 with an AND-join into a fresh end (the upstream AND-split guarantees both sinks fire).

Straight translation verdict: not sound. WoPeD flags four unbounded places, all around the OR-join fragment (three inside it, plus goods recorded immediately downstream): with tokens on both incoming branches, the intended both-inputs variant is enabled; but so are the single-input variants, and firing one of those consumes one token and strands the other, which can then multiply. Proper completion fails; the net is unbounded.

Repair 1: replace the OR-join by an AND-join (keep only the both-inputs transition): now **not sound the other way**: twelve non-live transitions. When the XOR takes the ok arm, the branch that should feed the join's second input never fires, and the AND-join deadlocks. Unboundedness has been traded for deadlock.

Repair 2: AND-join plus an ad-hoc arc feeding the missing input from the ok arm: WoPeD reports **sound**, all panels green. But the patch lives in the net world: the result is no longer the translation of any EPC, the audit trail to the modeler is broken, and reverse-engineering an EPC that translates to the patched net yields a diagram more complex and less readable than the original: whose analysis would have to restart from scratch. The first approach is a quick diagnostic, not a repair loop.

18.3 Relaxed soundness

Strict soundness is too demanding for early-design EPCs, which legitimately contain under-specified control flow. The *sound behaviours* of a net are its complete runs, $L(N) = \{\sigma \mid i \xrightarrow{\sigma} o\}$; unsound runs (deadlocks, leaked tokens) are not garbage: they pinpoint candidate mistakes.

Definition (Relaxed soundness). A workflow net N is **relaxed sound** iff every transition participates in at least one sound run:

$$\forall t \in T. \exists \sigma \in L(N). \vec{\sigma}(t) > 0.$$

Bad runs may exist, but no transition is structurally dead with respect to sound behaviour.

The goods-reception net (straight translation) is relaxed sound: one sound run takes the **not ok** arm and the both-inputs OR variant, another takes the **ok** arm and a single-input variant, and together they cover every transition. The AND-join repair is *not* relaxed sound as a net (one fragment transition joins no sound run) yet it *is* relaxed sound *as an EPC*: every event, function, and connector appears in some sound run, since several net transitions collapse into one EPC connector. Granularity matters. Two caveats close the topic: relaxed soundness is verified by *enumerating* sound runs until all transitions are covered, and (open problem) no cheaper structural characterisation is known.

18.4 Second approach: simplified EPCs and free-choice nets

Following van der Aalst,⁶⁶ restrict the language so the translation lands in the *free-choice* class, where soundness is decidable in polynomial time. Two syntactic constraints: strict **event/function alternation** along every path (enforced beforehand by a Step 0 that inserts dummy events and functions on broken arcs, including on direct connector-to-connector arcs), and **no OR connectors** at all.

The connector translation becomes *context-dependent*: the surrounding nodes already provide places (events) or transitions (functions), and only the missing kind is materialised.

- **AND-split / AND-join**: between a function and events (or events and a function), the function's transition simply gains multiple output (input) arcs: no new nodes; between an event and functions, a fresh synchronising transition is inserted after the place (dually for the join).
- **XOR-split**: out of an event, the place feeds the competing function-transitions directly: the free-choice conflict; out of a function, a fresh place plus one fresh transition per branch are inserted.
- **XOR-join**: into an event, the branch transitions feed the place directly; into a function, fresh per-branch transitions funnel into a fresh place feeding it.

The uniform principle: two adjacent events need an intermediate transition, two adjacent functions an intermediate place. The result is always a free-choice workflow net. Free choice buys fast analysis, not correctness: the farmhouse-visit EPC, normalised into the sub-class, still translates to a net that WoPeD rejects: two places of the AND-join fragment (its internal place and a dummy) left uncovered by any S-component, ten non-live transitions, a reachable deadlock. The structural defect of the diagram survives the translation.

18.5 Third approach: decorated EPCs

Following Rittgen,⁶⁷ keep the full connector alphabet (OR included) but require the designer to *decorate* the diagram, as introduced in the EPC chapter: every (X)OR-join is paired with a **corresponding split**, and every OR-join carries a **policy**: **wfa** (wait-for-all: synchronise every activated branch; the safe default, mandatory when the corresponding split matches in type), **et** (every-time: each arriving token triggers the output independently), or **fc** (first-come: first

⁶⁶van der Aalst W.M.P., “Formalization and Verification of Event-driven Process Chains”, Eindhoven University of Technology, 1999.

⁶⁷Rittgen P., “Modified EPCs and Their Formal Semantics”, Universität Koblenz-Landau, 1999.

arrival triggers, later ones are ignored). XOR-joins have no policy: their semantics (block if both inputs activate, fire on the unique one) is guaranteed by the required matching split (the implicit start split counts).

Step 1 then picks the join fragment by *policy and corresponding-split type*: with a matching OR-split, the subset choice is routed from split to join, which consumes exactly the activated subset; *wfa* under an AND-split degenerates to a plain AND-join; *wfa/fc/et* under an XOR-split all coincide (only one input ever arrives), realised by two routing lanes carrying the split's commitment to the join. The dangerous combinations are explicit in the fragments: *et* under an AND-split lets *both* tokens through: multiple tokens in the target, safeness broken by design; *fc* under an AND-split lets one through and *strands* the other: pending tokens by design. Step 2 proceeds dummy-style (direct conversion on compatible seams; dummy transition between two places, dummy place between two transitions).

On the farmhouse example (upper OR-join decorated *fc* with the XOR-split as corresponding split, lower OR-join *wfa* with the AND-split) the fully determined translation still yields a net that WoPeD rejects: a free-choice violation, two places outside every S-component, thirteen non-live transitions. The verdict is the deepest of the three: the diagram itself contains a genuine semantic mismatch between its XOR choice and its AND synchronisation, and *no* policy can paper over it. The modeler must revisit the EPC.

Note (EPC analysis: pros and cons of the three approaches). *Leave complete freedom* (1st): accepts any EPC, but most are unsound and the diagnosis is shallow: relaxed soundness only. *Constrain the diagrams* (2nd): polynomial-time precision on the free-choice image, but people like flexible syntax and ignore guidelines. *Require decorations* (3rd): the most accurate analysis, but people are lazy: decorations get skipped or misread. There is no universally good answer; each approach is a point on the trade-off between expressiveness, precision, and modeler discipline.

Example (Exercise: customer-order EPC). A customer-order process (compose order data, accept/reject, check availability, produce if needed, ship, check payment, complete) uses only XOR and AND connectors. It fits the simplified sub-class, so the second technique applies: fusion-style translation, Step 3 closing the two mutually exclusive end events (*rejected*, *completed*) with an XOR end pattern. The resulting free-choice workflow net passes every WoPeD check, two S-components covering all places, well-structured handles, bounded, live: **sound**. When the EPC fits the sub-class, the second technique is decisive and fast.

19 BPMN analysis

BPMN is analysed like EPC (by reduction to Petri nets) but the language is far richer: orchestrations *and* collaborations, typed events, compensation, transactions, five gateway types (exclusive, event-based, parallel, inclusive, complex).⁶⁸ The core control-flow alphabets line up with EPC one-to-one (events/events, activities/functions, gateways/connectors, sequence flow/control flow); what BPMN adds is the organisational scope (swimlanes) and, crucially, *message flow* across pools. The literature has formalised BPMN through many targets (abstract state machines, rewriting, process algebras, temporal logics); the Petri-net route inherits the OR-join ambiguity seen for EPC. BPMN *process* diagrams translate to workflow nets; *collaboration* diagrams trans-

⁶⁸Weske M., *Business Process Management*, Springer, 2007, Ch. 5.7.

late to **workflow systems**: workflow nets composed through shared message places; a diagram is **sound** iff its net (or system) is.

19.1 The Dijkman–Dumas–Ouyang translation

The translation follows Dijkman, Dumas and Ouyang,⁶⁹ and operates on *simplified BPMN* in normal form: start/exception events have one outgoing and no incoming flow; end events the dual; every activity and intermediate event has exactly one incoming and one outgoing flow; every gateway is purely a split or purely a join. The constraints cost nothing: mechanical rewrites restore them (insert an XOR-join before multi-input nodes, an AND-split after multi-output ones, decompose mixed gateways into join-then-split, add missing dummy start/end events). BPMN’s merged multi-in/multi-out gateway is shorthand for a join followed by a split of the same kind; the course recommendation is stronger: *never draw merged gateways*, always write the join and the split separately, so every node has one unambiguous role. Three policy restrictions complete the sub-class: **no OR gateways** (every EPC OR-join problem carries over; encode inclusive behaviour via XOR+AND), limited sub-processing (no multi-instance/ad-hoc/event-driven beyond the Step-0 macros below), no transactions or compensation.

Note (The twist: an inverted dictionary). The BPMN-to-net correspondence inverts the EPC one:

- BPMN **event** $\rightsquigarrow$ **transition**;
- BPMN **activity** $\rightsquigarrow$ **transition**;
- BPMN **sequence flow** or **message flow** $\rightsquigarrow$ **place**.

The place/transition alternation is read across the *flows*, not across the nodes. Size mnemonic: one place per flow arc, one transition per event or activity, one or two transitions per gateway, and no dummies, since the normal form guarantees direct mapping.

Three steps, supported by WoPeD: **Step 1** converts every sequence and message flow into a place; **Step 2** converts every flow object into transitions: events and activities one each; an AND gateway one synchronising transition; an XOR-split one transition *per outgoing branch* (all consuming the single input place: the free-choice conflict) and the XOR-join dually one per incoming branch; the *event-based* gateway disappears by **place fusion**: its input place merges with the upstream output place, and the downstream message/timer event-transitions race on it; **Step 3** enforces unique initial and final places, funnelling multiple starts from a fresh **start** place through an XOR pattern (one transition per original start: independent triggers) and multiple ends into a fresh **end** place dually (AND when the topology guarantees joint completion).

Example (Two unsound orchestrations). *Order process*: a message start fires an AND-split: upper branch Check credit card then an XOR ok?; lower branch Prepare products into an AND-join; the yes arm meets the AND-join, then Ship products, then an XOR-join with the no arm into the end event. Translation and analysis: **not sound**: two unbounded places. The defect: the no arm bypasses the AND-join entirely, so the lower branch’s token has no consumer; WoPeD’s reachability graph shows mostly dead-end leaves, and highlighting the no-arm transition shows every marking it contaminates. The repair

⁶⁹Dijkman R.M., Dumas M., Ouyang C., “Semantics and analysis of business process models in BPMN”, *Information and Software Technology* 50(12):1281–1294, 2008. The formalisation effort itself uncovered deficiencies in the BPMN 1.x standard.

belongs at the BPMN level: the **no** branch must cancel the production task explicitly (or an inclusive join would be needed, which the sub-class forbids).

Travel itinerary: Draft itinerary, an XOR **ok?** choosing Confirm itinerary or a Discuss/Change loop with a second **ok?**, converging on an AND-join before the end. **Not sound**: two dead and thirteen non-live transitions; when the first **ok?** answers **yes**, the loop branch never feeds the AND-join. The loop-until-confirmed intent contradicts the join's synchronisation requirement; the net faithfully inherits the contradiction.

Example (Always sushi: a sound collaboration). The two-pool negotiation: the *sushi lover* proposes (send sushi), offers Thai or Indian alternatives through XOR branches, and waits at an event-based gateway for the reply; the *sushi obsessed* partner either accepts immediately ("Yes, I love sushi") or routes through its own XOR choices, answering along message flows. Translating each pool in isolation, both turn out to be **S-nets** (every transition one-in, one-out) so the workflow-S-net theorem applies and each pool is *safe and sound by structure alone*, no behavioural analysis needed. The composed system (the two nets sharing the message-flow places) is no longer an S-net (message places cross pools), but WoPeD confirms it **sound**: the negotiation always terminates, fittingly always in sushi. The reachability graph is dense but finite: exactly the case where the tool, not the eye, settles the verdict.

Example (Buyer 3 + Reseller 2: locally sound, globally deadlocked). The B2B order collaboration: *Buyer 3* places the order, then concurrently waits for products and invoice, and finally settles; *Reseller 2* receives the order, sends the invoice, *waits for the payment*, and only then ships. Four message flows connect them. Each pool in isolation: **safe and sound** (the reseller is even a plain sequence, an S-net). The joint system: **not sound**, eight dead and sixteen non-live transitions, with the reachability graph ending in a deadlock. The dependency is circular: receive products waits for ship products, which waits for receive payment, which waits for settle invoice, which waits for receive products. This is the compatibility failure of the orchestration chapter ($R_2 + B_3$ in the compatibility matrix), now diagnosed mechanically: *soundness of the parts does not imply soundness of the whole* when message flows close synchronisation cycles across pools.

19.2 Step 0: preprocessing macros

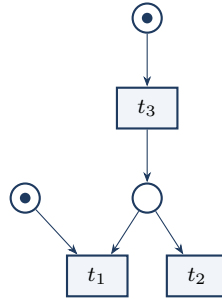
A preprocessor expands the richer BPMN constructs into simplified-BPMN fragments before the main translation:

- **looping activities**: *while-do* (**TestTime=Before**: an XOR-split at entry checks the condition, the **c**-branch runs the task and loops back, **!c** exits) and *do-until* (**TestTime=After**: task first, then the XOR decides repeat-or-exit);
- **multi-instance activities** with design-time cardinality n and parallel ordering: unrolled into n task copies between one AND-split and one AND-join;
- **sub-processes**: a collapsed inline sub-process becomes a call transition, the child's inner fragment, and a return transition; a top-level invoked sub-process is routed through guard transitions that dispatch the call and synchronise the return;
- **exception handling**: for an *atomic* task with a boundary error event, the exception transition shares the task's input place (a free-choice steal: either the task completes or the exception diverts the token to the handler, never both); for a *sub-process*, an activation-token place tracks the running child, the exception transition can steal it at any point during execution, and the construction keeps the fragment 1-safe while routing the diverted token to the handler.

After Step 0, every supported construct (typed start/intermediate/end events, tasks, sub-process invocations, loops, multi-instances, AND/XOR/event-based gateways, normal/exception/message flows) lies in the simplified sub-class, and the three-step translation applies unchanged.

20 Free-choice nets

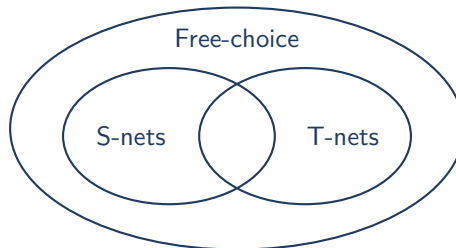
For arbitrary Petri nets the analysis algorithms are expensive when they exist at all, and no general theory links structure to behaviour. The obstacle, in one phrase, is the *interference between conflict and synchronisation*.⁷⁰



t_1 and t_2 are not initially in conflict (t_2 still waits for t_3 's token) but once t_3 fires they compete on the shared place: an uncontrollable upstream synchronisation silently induces a downstream conflict. Free-choice nets rule this out structurally.

Definition (Free-choice nets). *Restrictive formulation:* a net is **free-choice** if for every arc $(p, t) \in F$ either $p^\bullet = \{t\}$ (no conflict on p) or ${}^\bullet t = \{p\}$ (no synchronisation at t): conflict and synchronisation never touch. *Standard formulation* (adopted here, slightly more general): for any two transitions, ${}^\bullet t_1 \cap {}^\bullet t_2 = \emptyset$ or ${}^\bullet t_1 = {}^\bullet t_2$: pre-sets are disjoint or equal (equivalently on post-sets of places). A system is free-choice if its net is. The choice between conflicting transitions is then genuinely *free*: made locally on the shared pre-set, uninfluenced by the rest of the system.

The check is visual: whenever two transitions share an input place, do they share *all* their inputs? A net where t_1 needs $\{p_1, p_3\}$ while t_2 needs only $\{p_3\}$ is not free-choice; making both consume exactly the same pre-set repairs it. Every **S-net** is free-choice (all pre-sets are singletons: equal or disjoint), and so is every **T-net** (all post-sets of places are singletons). The inclusions are strict, and the four regions of the diagram are all inhabited: a pure sequence (S and T), a free choice between two transitions (S only), an AND-join (T only), and a net combining conflicts and synchronisations kept apart (free-choice, neither S nor T):



⁷⁰Desel J., Esparza J., *Free Choice Petri Nets*, Cambridge University Press, <https://www7.in.tum.de/~esparza/bookfc.html>.

Note (Fundamental property; N^*). Proposition. In a free-choice system, if a reachable M enables some $t \in p^\bullet$, it enables *every* $t' \in p^\bullet$ (free-choice gives ${}^\bullet t = {}^\bullet t'$, and all those places are marked). Enabledness is a property of the conflict set, not of the single transition: all or none.

Proposition. A workflow net N is free-choice iff N^* is: the reset transition's pre-set $\{o\}$ is disjoint from every other pre-set (o is a sink in N), so adding or removing it cannot break the condition.

Note (Liveness = place-liveness on free-choice systems). A place p is **live** when from every reachable marking some further reachable marking puts a token on p ($\forall M \in [M_0] \exists M' \in [M]. M'(p) \geq 1$); the system is **place-live** when every place is live. Equivalently, p is dead when it can permanently run dry, and then every transition in ${}^\bullet p \cup p^\bullet$ is dead too, since none can ever fire again through p . Place-liveness is the token-level analogue of liveness (which asks the same of every *transition*).

In any system liveness implies place-liveness. On free-choice systems the converse holds too:

Theorem. A free-choice system is live iff it is place-live.

Proof of the non-trivial direction. Take any reachable M and target t with ${}^\bullet t = \{p_1, \dots, p_n\}$. Place-liveness yields $M \rightarrow^* M_1$ marking p_1 ; then $M_1 \rightarrow^* M_2$ marking p_2 , and so on. The crux: the token on p_1 cannot vanish while reaching M_2 : any transition t' consuming it has $t' \in p_1^\bullet$, hence ${}^\bullet t' = {}^\bullet t$ by free choice, so if t' was enabled then t was *already* enabled and we are done. Iterating, after n steps all of ${}^\bullet t$ is marked and t is enabled. $\square$

Liveness, a property of transitions, becomes checkable on places: the first payoff of the restriction, and the stepping stone to:

Theorem (Commoner). A free-choice system is live iff every proper siphon contains an initially marked trap.

Three questions remain: what is a siphon, what is a trap, and is the check feasible?

20.1 Stable sets of markings

Definition (Stable set). A set of markings $\mathbf{M}$ is **stable** if $M \in \mathbf{M} \Rightarrow [M] \subseteq \mathbf{M}$: once inside, firing cannot leave. By induction it suffices to check closure under *one* step: $(M \in \mathbf{M} \wedge M \xrightarrow{t} M') \Rightarrow M' \in \mathbf{M}$. The reachability set $[M_0]$ is the least stable set containing M_0 .

Worked candidates, all stable: $\{\mathbf{0}\}$ (the empty marking enables nothing); the set of all markings (the universe); the live markings (liveness is closed under reachability); the deadlock markings (vacuously: nothing fires); the level set $\{M \mid \mathbf{I} \cdot M = \mathbf{I} \cdot M_0\}$ of any S-invariant (the fundamental property of invariants). A non-example: $\{M \mid M(P) = 1\}$, the one-token markings: a single transition $t : p_1 \rightarrow p_2 + p_3$ jumps from one token to two.

20.2 Siphons

Definition (Siphon). $R \subseteq P$ is a **siphon** if ${}^\bullet R \subseteq R^\bullet$: every transition that feeds R also consumes from R ; *proper* if non-empty. Consequence: **once empty, always empty**: the set $\{M \mid M(R) = 0\}$ is stable, since with R empty no transition of ${}^\bullet R$ is enabled.

Worked check on the producer/consumer net, with $R = \{\text{prod_busy}, \text{prod_free}, \text{item_buffer}\}$:

$${}^\bullet R = \{\text{prod_start}, \text{prod_end}\} \subseteq R^\bullet = \{\text{prod_start}, \text{prod_end}, \text{cons_start}\},$$

so R is a proper siphon. Dropping `prod_free` breaks it: `prod_start` still feeds `prod_busy` but no longer consumes from the set: the three-place set was the minimal siphon around those places.

Note (Siphons against liveness). Proposition. In a live system every proper siphon is marked at M_0 . *Proof:* pick $p \in R$ with $p^\bullet \neq \emptyset$ (a siphon that ever holds a token has one, else it could never be emptied) and $t \in p^\bullet$; liveness fires t somewhere reachable, which requires $M(p) \geq 1$, so R is marked there, and a siphon empty at M_0 would have stayed empty. $\square$ Contrapositive: an initially unmarked proper siphon is a purely structural witness of non-liveness.

20.3 Traps

Definition (Trap). $R \subseteq P$ is a **trap** if $R^\bullet \subseteq {}^\bullet R$: every transition draining R also refills it; *proper* if non-empty. Dual consequence: **once marked, always marked**: $\{M \mid M(R) > 0\}$ is stable.

Dual worked check: $R = \{\text{item_buffer}, \text{cons_busy}, \text{cons_free}\}$ has $R^\bullet = \{\text{cons_start}, \text{cons_end}\} \subseteq {}^\bullet R = \{\text{prod_end}, \text{cons_start}, \text{cons_end}\}$: a trap. Dropping `cons_free` breaks it (`cons_end` then drains without refilling).

Note (Maximal traps and Commoner, operational form). Lemma. The union of traps is a trap $((X_1 \cup X_2)^\bullet = X_1^\bullet \cup X_2^\bullet \subseteq {}^\bullet X_1 \cup {}^\bullet X_2 = {}^\bullet (X_1 \cup X_2))$. Hence every set of places contains a unique *maximal* trap, and a siphon contains a marked trap iff its maximal trap is marked.

Commoner, restated. A free-choice system is live iff every proper siphon has its maximal trap initially marked; *non-live* iff some proper siphon has an unmarked maximal trap: a single-witness formulation, the basis of the algorithmics.

20.4 Complexity of liveness: NP-complete

Non-liveness of a free-choice system is in NP: *guess* $R \subseteq P$ (non-deterministic, polynomial); *check* ${}^\bullet R \subseteq R^\bullet$ (polynomial); *compute* the maximal trap $Q \subseteq R$: polynomial by the routine

```
Q := R
while (exists p in Q, exists t in p^bullet, t notin ^bullet Q):
    Q := Q \ {p}
return Q
```

(each iteration removes a place whose consumer never refills Q ; the residue is the largest trap inside R), and *answer* non-live iff $M_0(Q) = 0$. A deterministic algorithm would face $2^{|P|}$ candidate subsets.

Note (NP-hardness: encoding CNF-SAT). Given a CNF formula ϕ over variables $x_1, \dots, x_n$ (a conjunction of clauses, each a disjunction of literals), build a free-choice net from the *negation* $\neg\phi$ (a disjunction of conjunctions, by De Morgan):

- one initially marked place L_i per variable; two transitions $x_i, \bar{x}_i$ both consuming exactly $\{L_i\}$ (equal pre-sets: free choice preserved): firing one commits the truth value;
- one place per *literal occurrence* in a clause of $\neg\phi$, fed by the literal's transition;
- one transition C_j per clause of $\neg\phi$, consuming all its occurrence places: enabled iff the chosen assignment satisfies that conjunct;

- a place **True** fed by every C_j , and a transition **Back** consuming **True** and refilling every L_i , closing the cycle.

The net is free-choice, of size polynomial in $|\phi|$. If the chosen assignment satisfies ϕ , it falsifies $\neg\phi$: no C_j is enabled, **True** never marks, **Back** dies, the L_i are never refilled: the system is **not live**. Conversely a dead run traces back to a falsifying assignment of $\neg\phi$, i.e. a satisfying one of ϕ :

$$\phi \text{ satisfiable} \iff \text{the net is non-live.}$$

Verdict: non-liveness of free-choice systems is NP-complete; no polynomial decision procedure for liveness exists unless $P = NP$.

20.5 Liveness and boundedness: the Rank Theorem

Definition (Cluster). For a node x of $N = (P, T, F)$, the **cluster** $[x]$ is the least set with: (1) $x \in [x]$; (2) if a place is in the cluster, *all its consumers* are ($p \in [x] \Rightarrow p^\bullet \subseteq [x]$); (3) if a transition is in the cluster, *all its feeders* are ($t \in [x] \Rightarrow {}^\bullet t \subseteq [x]$). Forward from places, backward from transitions, to closure. Clusters partition the nodes; each cluster gathers a maximal block of intertwined conflicts. Example: in a net where $p_2^\bullet = \{t_2, t_4\}$, $p_3^\bullet = \{t_3, t_4\}$, ${}^\bullet t_3 = \{p_3, p_4\}$, the closure of p_2 is $[p_2] = \{p_2, p_3, p_4, t_2, t_3, t_4\}$, while an untangled pair forms its own cluster $\{p_1, t_1\}$.

Note (Rank Theorem (proof omitted)). A free-choice system is **live and bounded** iff: (1) it has at least one place and one transition; (2) it is connected; (3) M_0 marks every proper siphon; (4) it has a positive S-invariant; (5) it has a positive T-invariant; (6) $\text{rank}(N) = |C_N| - 1$, with C_N the set of clusters.

Each condition is checkable in polynomial time: (1)–(2) trivially; (4)–(5) are linear-programming feasibility; (6) is a matrix rank plus a graph closure; and (3) reduces to the dual of the trap routine: start from the initially *unmarked* places R and extract the maximal siphon:

```
Q := R
while (exists p in Q, exists t in ^bullet p, t notin Q^bullet):
    Q := Q \ {p}
return Q
```

M_0 marks every proper siphon iff this returns $\emptyset$. Hence:

Note (Live and bounded is in P). Deciding whether a free-choice system is *live and bounded* takes polynomial time: in sharp contrast with plain liveness, which is NP-complete. Adding the boundedness requirement *collapses* the complexity. The four headline results of the chapter: liveness = place-liveness; non-liveness $\iff$ a proper siphon empty at some reachable marking; Commoner's siphon-trap characterisation of liveness; the Rank Theorem with its polynomial decision procedure.

Example (Exercises). (a) *SAT via nets.* For $\phi = x_2 \wedge (x_1 \vee \overline{x_3} \vee \overline{x_4}) \wedge (x_1 \vee \overline{x_2}) \wedge (\overline{x_1} \vee x_4) \wedge (\overline{x_2} \vee \overline{x_4})$, negate, build the free-choice net of $\neg\phi$, and check liveness. The constructed net turns out *live*, so ϕ is **not satisfiable**.

(b) *Witnessing siphon.* A given free-choice system is non-live; find a proper siphon with no marked trap. Candidate $R = \{p_1, p_2, p_3, p_4, p_5, p_8\}$: the siphon condition checks out, and the maximal-trap routine strips the set step by step: p_2, p_3 go first (their consumers t_2, t_3 never refill the candidate), then p_1 (drained by t_1), then p_8 (drained by t_9), finally p_4, p_5 , terminating at $Q = \emptyset$. The only trap inside R is empty: by Commoner, non-live, with R as the witness.

(c) *Mirror direction.* A live free-choice system has exactly two proper siphons, $R_1 = \{p_1, \dots, p_5, p_7, p_8\}$ and $R_2 = \{p_1, \dots, p_5, p_6, p_8\}$. Running the routine on R_1 removes only p_2 ; the residue is a trap, initially marked. Symmetrically for R_2 . Every proper siphon contains a marked trap, in agreement with Commoner and the system's liveness.

21 P and NP problems

The complexity classes P and NP recur in the analysis of workflow nets, notably in the NP-completeness results of the free-choice chapter.

Definition (Problem and instance). A **problem** defines a family of related questions parameterised by an input. A **problem instance** is one specific question in that family, obtained by fixing the input.

The *factorization problem*, “given a number n , return all its prime factors”, is a problem; “return all prime factors of 18” is one of its instances.

Definition (Decision problem). A **decision problem** is a problem whose answer is always boolean: yes or no.

The primality problem, “given a number n , is n prime?”, is a decision problem; “is 18 prime?” is an instance with answer **no**. Complexity classes are defined over decision problems: reducing every output to a single bit makes algorithms directly comparable.

Computational complexity theory measures the *resources* an algorithm consumes: how many basic operations it executes (**time**) and how much memory it uses (**space**), both as functions of the input size, so that problems can be compared independently of the speed of any particular machine.

Definition (The class P). P is the set of decision problems solvable by a *deterministic* algorithm in a number of steps polynomial in the size of the input.

Problems in P count as **solved effectively**: a polynomial-time algorithm scales gracefully with input size.

Example (Eulerian circuit is in P). Given an undirected graph G , an *Eulerian circuit* traverses each edge of G exactly once and returns to its starting vertex. By Euler's classical result (the theorem proved in the Königsberg digression of the introduction) such a circuit exists if and only if every vertex of G has even degree (and the graph is connected on its edges); checking the degrees takes time linear in the number of edges, so the decision problem “does G admit an Eulerian circuit?” is in P.

Definition (The class NP). NP is the set of decision problems solvable by a *non-deterministic* algorithm in a polynomial number of steps. Equivalently: a problem is in NP when a candidate solution can be **verified** by a deterministic algorithm in polynomial time.

The second formulation is the one that matters in practice: problems in NP may be hard to solve from scratch, but their solutions can be **checked effectively**: the candidate solution acts as a *certificate*.

Example (Hamiltonian circuit is in NP). A *Hamiltonian circuit* of an undirected graph G is a circuit visiting each vertex of G exactly once. Given a candidate sequence of vertices, verifying that it visits every vertex once and uses only edges of G is linear in the size of G ; so the decision problem is in NP. The contrast with the Eulerian case: no structural characterisation analogous to “every vertex has even degree” is known, so the certificate *is* the circuit itself. Without one, the natural procedure extends a partial path vertex by vertex and backtracks at every dead end; the tree of partial solutions grows combinatorially with the number of vertices. Classic test cases are the skeletons of highly symmetric polyhedra (a hexagonal prism, the dodecahedron, the icosahedron): each admits a Hamiltonian circuit, easy to check once exhibited, laborious to find by hand.

21.1 P vs NP and NP-completeness

Note (The P versus NP question). Whether $P = NP$ is the most important open question in computer science.

Intuition and daily computational experience support $P \neq NP$: solving from scratch feels much harder than checking. No proof of either direction is known. If $P = NP$, every efficiently verifiable problem would also be efficiently solvable, and Hamiltonicity, satisfiability and a vast catalogue of optimisation problems would collapse into the polynomial-time class.

Definition (NP-completeness). A problem $Q \in NP$ is **NP-complete** when every problem in NP reduces to Q in polynomial time.

NP-complete problems are the *hardest* in NP: a polynomial algorithm for any one of them would, via the reductions, solve all of NP in polynomial time. Under the conjecture $P \neq NP$, the landscape has P strictly inside NP with the NP-complete problems forming a frontier disjoint from P ; under $P = NP$ all three classes coincide.

21.2 The SAT decision problem

The **Boolean satisfiability problem** (SAT) is the prototypical NP-complete problem. Vocabulary: **variables** $x_1, \dots, x_n$; **literals**, the variables and their negations $\bar{x}_i$; a **clause**, a disjunction of literals; a **formula** in *conjunctive normal form*, a conjunction of clauses. The decision question: given a formula ϕ , is there an assignment of boolean values to its variables making ϕ true? For instance,

$$\phi = (x_1 \vee \bar{x}_3) \wedge (x_1 \vee \bar{x}_2 \vee x_3) \wedge (x_2 \vee \bar{x}_3)$$

is satisfied by $x_1 = x_2 = x_3 = \mathbf{true}$. Verifying a candidate assignment is linear (evaluate each clause) so SAT is in NP; the deep direction is that every problem in NP reduces to SAT in polynomial time (Cook–Levin theorem), making SAT NP-complete. To feel the combinatorial blow-up first-hand, Olivier Roussel’s online SAT game poses instances with just 9 variables across five difficulty levels.

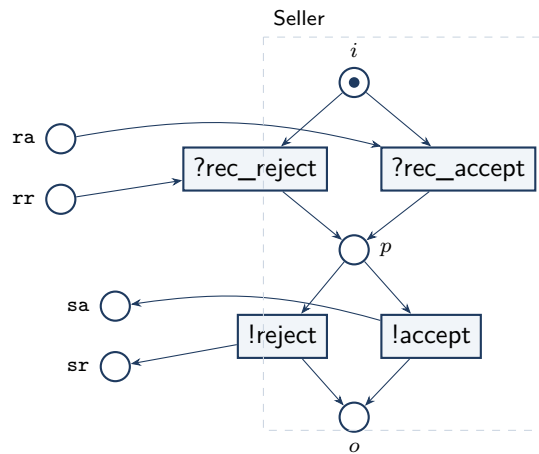
22 Workflow modules

Note (Object). *Workflow modules* are the device for modelling interaction between separately developed workflows. A workflow net describes one process in isolation; a workflow module exposes an interface

through which it exchanges messages with peer modules.⁷¹

Not every task of a workflow net is automatic: some fire when a message arrives, others fire to send one. The plain formalism has no syntax for this: transitions touch internal places only.

A first, informal annotation tags the communicating transitions: $?-$ transitions (e.g. $?rec_accept$) fire when the corresponding message arrives, $!-$ transitions (e.g. $!reject$) produce a message visible to peers. Making the annotation structural turns the tags into *interface places*: a set P^I of **input places** feeding the $?-$ transitions and a set P^O of **output places** collecting tokens from the $!-$ transitions. Interface places sit outside the workflow net but inside the module; interaction becomes ordinary token flow through them. The Seller module:



The receiving block $P^I = \{ra, rr\}$ feeds the $?-$ transitions, the sending block $P^O = \{sa, sr\}$ collects from the $!-$ transitions; everything inside the dashed boundary except the interface places is the original (sound) workflow net. Adding the interface places breaks the workflow-net shape (the net acquires sources besides i and sinks besides o) and the enriched object gets its own name.

Definition (Workflow module). A **workflow module** consists of

- a (sound) workflow net (P, T, F) ;
- a set P^I of *incoming places* with incoming arcs $F^I \subseteq P^I \times T$;
- a set P^O of *outgoing places* with outgoing arcs $F^O \subseteq T \times P^O$;

where each transition $t \in T$ has *at most one* interface arc incident to it (in $F^I \cup F^O$): one message received or sent per firing; chains of messages are modelled by chains of transitions.

Definition (Structural compatibility). A set of workflow modules is **structurally compatible** if every message that can be sent has exactly one module that can receive it, and every message that can be received has exactly one module that can send it (matching data formats assumed). Structurally: output places and input places match up one-to-one across modules.

22.1 Workflow systems

Compatibility becomes composition by *fusing* the matching interface places: the sender's output place and the receiver's input place collapse into one shared place, and the modules become a

⁷¹Ch.6 of *Business Process Management: Concepts, Languages, Architectures*, Weske, 2007.

single Petri net in which interaction is visible token flow. Closing the construction restores the workflow-net shape:

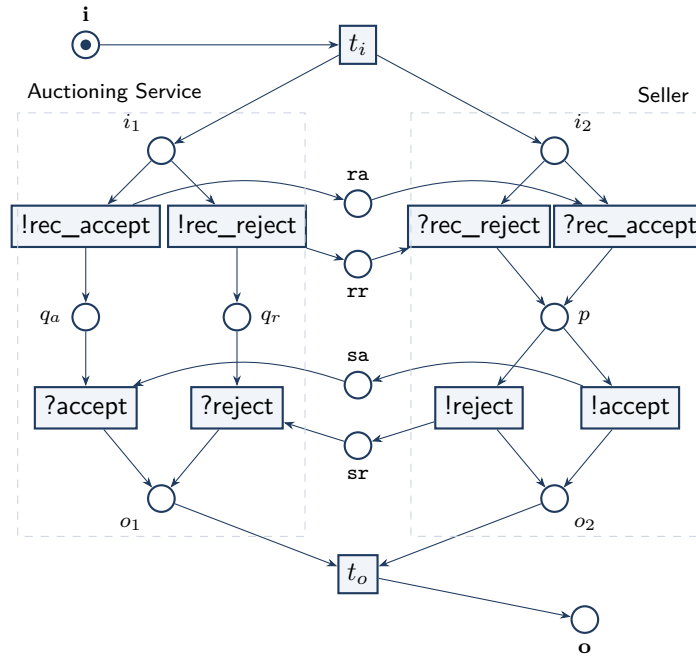
Definition (Workflow system). A **workflow system** is the workflow net obtained from n structurally compatible workflow modules with initial places $i_1, \dots, i_n$ and final places $o_1, \dots, o_n$ by adding

- a fresh initial place $\mathbf{i}$ and a transition t_i with $\bullet t_i = \{\mathbf{i}\}$, $t_i^\bullet = \{i_1, \dots, i_n\}$;
- a fresh final place $\mathbf{o}$ and a transition t_o with $\bullet t_o = \{o_1, \dots, o_n\}$, $t_o^\bullet = \{\mathbf{o}\}$;

with initial marking $M_0 = \mathbf{i}$.

A workflow system is an ordinary workflow net (unique source, unique sink, every node on a path between them) so its soundness is checked with the machinery already in place (reachability graph, Commoner, free-choice results).

Example (Auctioning Service + Seller). The Seller module above composes with an Auctioning Service exposing the mirror interface ($P^O = \{\mathbf{ra}, \mathbf{rr}\}$, $P^I = \{\mathbf{sa}, \mathbf{sr}\}$):



Both modules are sound in isolation, yet WoPeD flags the system **not sound**: a free-choice violation and 10 non-live transitions. The defect is a cross-module conflict. The Service *pre-commits*: choosing **!rec_accept** routes its token to q_a , where only **?accept** (needing **sa**) can consume it. The Seller, having received the message, still chooses *freely* between **!accept** and **!reject**; if it answers **sr**, the Service waits forever on **sa**: deadlock. The fix is internal to one module: the revised Service' merges q_a and q_r into a single place, so whichever answer arrives can be consumed. With Service' the system is **sound**. The lesson: composing sound modules does not guarantee a sound system; each composition must be checked.

Example (Exercise: the order collaboration, two wirings). A seller module (Receive Order consuming R0, Send Invoice producing SI, Receive Payment consuming BP, Ship Products producing SP, in sequence) composes with a buyer module (Place Order producing R0, then an AND-split into an invoice branch and a products branch, AND-join, end). Two buyer variants differ only in where Settle Invoice sits. *Variant one*: the invoice branch is Receive Invoice then Settle Invoice (producing BP), in parallel with

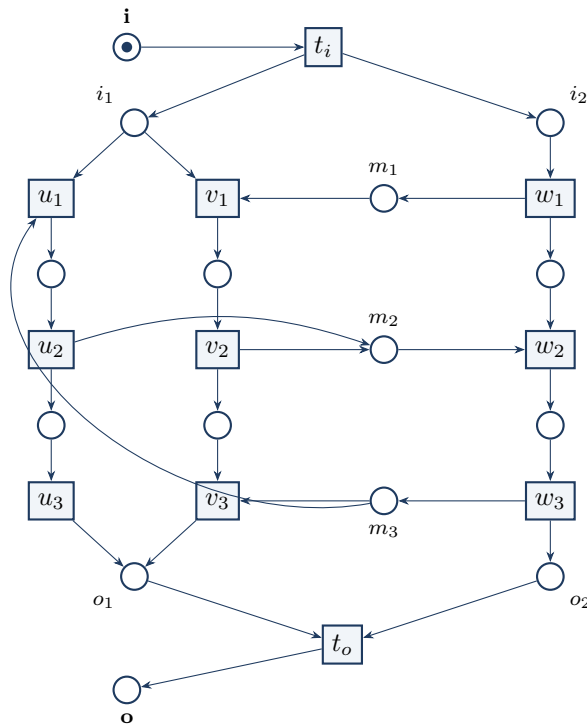
Receive Products; WoPeD reports the composed system **sound**. *Variant two*: Settle Invoice moves after the AND-join: the buyer pays only once products *and* invoice are in hand. Now the waits are circular: the seller ships only after payment, the buyer pays only after shipment. WoPeD: 6 dead transitions (Ship Products, Settle Invoice, Receive Payment, Receive Products, the AND-join, the end transition), 12 non-live: **not sound**. This is the $R_2 + B_3$ compatibility failure of the orchestration chapter, now diagnosed mechanically: same modules, same interface signature, different wiring, different verdict.

22.2 Weak soundness

Full soundness is often too strict for composed systems. Modules are designed for reuse: a generic module exposes branches (cancellation, refund, alternative protocols) that a particular composition never triggers. Forbidding dead transitions penalises exactly this modularity.

Definition (Weak soundness). A workflow net is **weakly sound** if it satisfies *option to complete* (from every reachable marking the final marking is reachable) and *proper completion* (when a token reaches $\mathbf{o}$, no token lingers elsewhere). Dead tasks are allowed. Weak soundness is decidable on the reachability graph and guarantees deadlock freedom and proper termination of all modules in the composition.

Example (Sound + sound = not sound, but weakly sound). The abstract witness. The left module offers two alternative branches from i_1 ; the right module is a plain sequence; three fused places m_1, m_2, m_3 carry the messages:



Each module in isolation is sound: the left one freely chooses branch u or branch v , the right one runs its sequence. In the composition the choice is no longer free: u_1 needs m_3 besides i_1 . But m_3 is produced only by w_3 , the *last* step of the right module, and reaching w_3 requires m_2 , which only u_2 or v_2 can send; u_2 sits behind u_1 itself, so the only way forward is branch v . The unique run: t_i ; w_1 sends m_1 ; v_1 consumes i_1 and m_1 ; v_2 sends m_2 ; w_2 consumes it; w_3 sends m_3 and completes the right module; v_3 consumes m_2 and m_3 ; t_o fires. The final marking is exactly $\mathbf{o}$: *option to complete* and *proper completion* hold in every reachable marking. Yet u_1, u_2, u_3 never fire: three dead transitions. Verdict: **not sound**,

but weakly sound. The dropped clause (no dead transitions) is exactly what weak soundness relaxes, which makes it the right criterion of behavioural compatibility for composed workflow systems.

23 Conformance checking

Conformance checking asks a single question: given an event log produced by the real execution of a process and a model of how the process should behave, do the two agree?⁷² The answer comes in two complementary forms: *global conformance measures*, scalar scores summarising the agreement (“85% of the cases can be replayed”), and *local diagnostics*, pinpointing where the disagreement lives: nodes of the model where the log deviates, cases of the log the model rejects. Both readings matter: the model can be wrong, or the cases can be.

Definition (Fitness). **Fitness** measures the proportion of behaviour in the event log possible according to the model: of the four quality criteria for process models (fitness, precision, generalization, simplicity), the one closest to conformance. The naïve version counts the fraction of cases that can be *replayed*: a workflow net N replays a trace σ iff $\sigma \in L(N)$, i.e. σ is a firing sequence from **[start]** to **[end]**. A trace with $\sigma \notin L(N)$ is *non-fitting*.

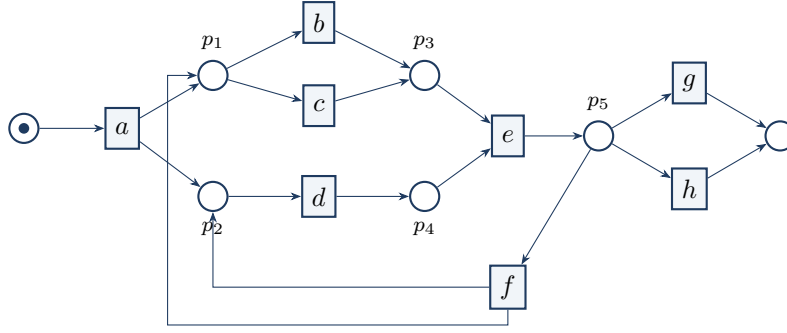
Example (Running example: event log L_{full}). The log records 1391 cases of the request-handling process of the process-mining chapter (a register request, b examine thoroughly, c examine casually, d check ticket, e decide, f reinitiate request, g pay compensation, h reject request), in 21 distinct traces:

#	freq	trace	#	freq	trace
σ_1	455	$\langle a, c, d, e, h \rangle$	σ_{12}	11	$\langle a, c, d, e, f, d, b, e, g \rangle$
σ_2	191	$\langle a, b, d, e, g \rangle$	σ_{13}	9	$\langle a, d, c, e, f, c, d, e, h \rangle$
σ_3	177	$\langle a, d, c, e, h \rangle$	σ_{14}	8	$\langle a, d, c, e, f, d, b, e, h \rangle$
σ_4	144	$\langle a, b, d, e, h \rangle$	σ_{15}	5	$\langle a, d, c, e, f, b, d, e, g \rangle$
σ_5	111	$\langle a, c, d, e, g \rangle$	σ_{16}	3	$\langle a, c, d, e, f, b, d, e, f, d, b, e, g \rangle$
σ_6	82	$\langle a, d, c, e, g \rangle$	σ_{17}	2	$\langle a, d, c, e, f, d, b, e, g \rangle$
σ_7	56	$\langle a, d, b, e, h \rangle$	σ_{18}	2	$\langle a, d, c, e, f, b, d, e, f, b, d, e, g \rangle$
σ_8	47	$\langle a, c, d, e, f, d, b, e, h \rangle$	σ_{19}	1	$\langle a, d, c, e, f, d, b, e, f, b, d, e, h \rangle$
σ_9	38	$\langle a, d, b, e, g \rangle$	σ_{20}	1	$\langle a, d, b, e, f, b, d, e, f, d, b, e, g \rangle$
σ_{10}	33	$\langle a, c, d, e, f, b, d, e, h \rangle$	σ_{21}	1	$\langle a, d, c, e, f, d, b, e, f, c, d, e, f, d, b, e, g \rangle$
σ_{11}	14	$\langle a, c, d, e, f, b, d, e, g \rangle$			

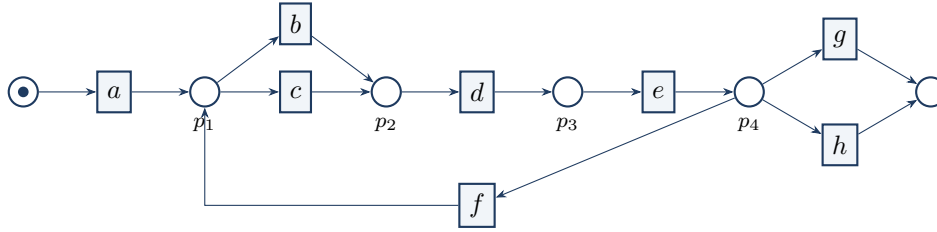
23.1 Four models, naïve fitness

Four candidate WF-nets are checked against L_{full} (the gallery differs from the quality-criteria gallery of the process-mining chapter, which used the same names for different nets). N_1 is the α -discovered net: after a , the examination (b or c) runs in parallel with the ticket check d ; e joins them; then pay g , reject h , or reinitiate f , which loops back into both branches:

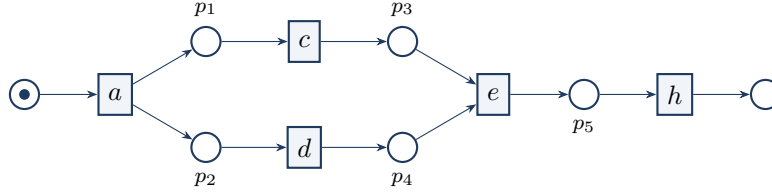
⁷²Ch.1 and Ch.6 of *Process Mining: Data Science in Action*, Wil van der Aalst, Springer 2016.



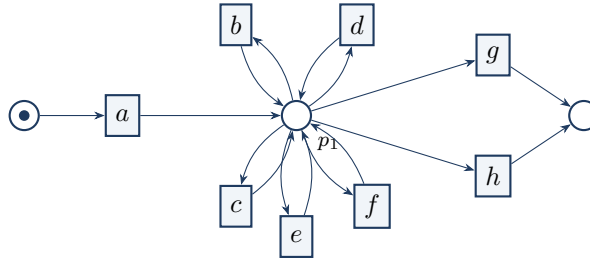
N_1 accepts all 1391 traces: $\text{fitness}_{\text{naïve}}(N_1, L_{\text{full}}) = \frac{1391}{1391} = 1$. N_2 forces the examination before the ticket check:



Every trace starting $\langle a, d, \dots \rangle$ violates the ordering: 443 of the 1391 cases are not firing sequences, so $\text{fitness}_{\text{naïve}}(N_2, L_{\text{full}}) = \frac{948}{1391} = 0.6815$. N_3 removes choice: examination always casual, decision always reject (b, f, g do not exist):



759 cases are non-fitting: $\text{fitness}_{\text{naïve}}(N_3, L_{\text{full}}) = \frac{632}{1391} = 0.4543$. N_4 is the *flower model*: one central place from which b, c, d, e, f fire in any order, any number of times, between a mandatory a and a final g or h :



A poorly structured model that constrains nothing; yet $\text{fitness}_{\text{naïve}}(N_4, L_{\text{full}}) = 1$. Naïve fitness does not discriminate between a faithful model and a model that allows everything.

Note (Almost-fitting traces). Naïve fitness is also too strict: a trace of 100 events of which 99 are replayable counts exactly as much as one with 10: zero. The fix is to move from whole traces to individual events.

23.2 Token replay: missing and remaining tokens

Instead of stopping at the first problem, replay the trace to the end, forcing transitions to fire even when not enabled, and keep books on the damage.

Definition (Four counters and token-replay fitness). Replaying σ on a WF-net N maintains

- p : *produced* tokens (every firing produces into its output places; the environment produces one token in **start** at the beginning);
- c : *consumed* tokens (every firing consumes from its input places; the environment consumes one token from **end** at the end);
- m : *missing* tokens, injected because a transition was forced to fire while not enabled (the place gets an **m-tag**);
- r : *remaining* tokens, left in places other than **end** when the replay terminates (**r-tags**).

The **token-replay fitness** of σ on N is

$$\text{fitness}(\sigma, N) = \frac{1}{2} \left(1 - \frac{m}{c}\right) + \frac{1}{2} \left(1 - \frac{r}{p}\right) \in [0, 1],$$

penalising equally the fabricated tokens (consumption side) and the never-consumed ones (production side); the maximum 1 is reached exactly when $m = r = 0$.

Example (Three replays). $\sigma_1 = \langle a, c, d, e, h \rangle$ on N_1 . Every step is enabled: the environment feeds **start** ($p=1$); a consumes it and fills p_1, p_2 ($p=3, c=1$); c, d, e, h fire normally (e consumes two, from p_3 and p_4); the environment drains **end**. Final counters $p = c = 7, m = r = 0$: $\text{fitness}(\sigma_1, N_1) = 1$.

$\sigma_3 = \langle a, d, c, e, h \rangle$ on N_2 . After a (token in p_1), the trace demands d , but d consumes from p_2 , which is empty: no examination has run. Inject the missing token into p_2 ($m=1$, **m-tag** on p_2) and fire. Then c fires legally ($p_1 \rightarrow p_2$), e and h complete, the environment drains **end**. Final: $p = c = 6, m = 1$, and the token c put in p_2 is never consumed: $r = 1$, **r-tag** on p_2 .

$$\text{fitness}(\sigma_3, N_2) = \frac{1}{2} \left(1 - \frac{1}{6}\right) + \frac{1}{2} \left(1 - \frac{1}{6}\right) \approx 0.8333.$$

The tags read causally: the **m-tag** says “ d occurred in the log but was not possible in the model here”; the **r-tag** says “ d was supposed to happen at this point in the model but did not in the log”.

$\sigma_2 = \langle a, b, d, e, g \rangle$ on N_3 , with *event removal*. Activities absent from the model (b and g do not exist in N_3) are first removed from the trace; the residual $\langle a, d, e \rangle$ is replayed. a fills p_1, p_2 ; d fires ($p_2 \rightarrow p_4$); e needs $p_3 + p_4$ but p_3 is empty (the skipped c would have filled it): $m=1$, force, produce into p_5 . No h fires, so **end** stays empty and the environment’s final consumption costs a second missing token ($m=2$). Tokens strand in p_1 (the examination that never ran) and p_5 (the rejection that never followed): $r = 2$. Final: $p = c = 5, m = r = 2$:

$$\text{fitness}(\sigma_2, N_3) = \frac{1}{2} \left(1 - \frac{2}{5}\right) + \frac{1}{2} \left(1 - \frac{2}{5}\right) = 0.6,$$

with three localised diagnoses: c was expected but did not happen, e happened where not possible, h was expected but did not happen.

Definition (Token-replay fitness of a log). For a log L with multiplicities $L(\sigma)$, apply the same formula to the log-wide totals $M = \sum_{\sigma} L(\sigma) m_{N, \sigma}$ and likewise C, R, P :

$$\text{fitness}(L, N) = \frac{1}{2} \left(1 - \frac{M}{C}\right) + \frac{1}{2} \left(1 - \frac{R}{P}\right).$$

On the running models:

$$\text{fitness}(L_{full}, N_1) = 1, \quad \text{fitness}(L_{full}, N_2) = 0.9504, \quad \text{fitness}(L_{full}, N_3) = 0.8797, \quad \text{fitness}(L_{full}, N_4) = 1.$$

The event-level score sits well above the naïve case-level one (0.9504 vs 0.6815 for N_2): cases that fail as wholes still have most events replayable. It is read informally as “about 95% of the events replay correctly”; the link with the actual proportion of replayable events is indirect (a swap of two adjacent events costs one missing plus one remaining token), but the intuition is robust.

23.3 Diagnostics and drill-down

Replaying the whole log aggregates the per-trace tags into a global picture. On N_2 , besides the flow counts on the arcs (930 rejections, 461 payments, 146 reinitiations), the replay flags 443 tokens missing in p_2 (d happened although the model forbade it) and 443 remaining in p_2 (d was due but had already happened): both numbers point at the same systematic deviation, the sequential ordering violated 443 times. On N_3 the problems multiply: 430 tokens remain in p_1 (c expected, not happened) and 10 are missing there (c happened when not possible: repeat examinations after a reinitiation); 146 missing in p_2 (the d re-executions of the removed f -loop); 566 missing in p_3 (e without examination); 607 remaining in p_5 (decision without rejection); 461 of the 1391 cases never reach **end** (they ended with the non-existent g).

Note (Drill down). Fitness also partitions the log: L splits into L_{fit} (cases the model replays) and L_{dev} (deviators). The deviating sublog is a starting point in itself: discover a model for it (what shape does the rebellious behaviour have?), check who handled those cases, whether they took longer or cost more, and, if fraud is suspected, build the social network of the deviators.

23.4 Comparing footprints (optional reading)

The *play-out* technique generates from a workflow net a locally complete set of firing traces: the smallest set exercising every directly-follows pair the net can produce. Reading that set as a log yields the directly-follows relation $>$ and from it the **footprint matrix** of the *model*, comparable cell by cell with the footprint of an event log (which is *complete* when every pair of activities that can follow one another does so at least once).

Definition (Footprint-based conformance). Let n be the number of cells of the footprint matrix ($n = |A|^2$) and d the number of cells on which the two footprints differ. The **footprint-based conformance** is

$$1 - \frac{d}{n} \in [0, 1],$$

symmetric and applicable to log-vs-model, model-vs-model (similarity of candidate models) and log-vs-log comparisons (*concept drift*: did the process change between early and late portions of the log?).

Example (L_{full} vs N_2 via footprints). Playing out N_1 gives four traces

$$\langle a, b, d, e, g \rangle, \quad \langle a, d, b, e, f, d, c, e, h \rangle, \quad \langle a, c, d, e, f, b, d, e, g \rangle, \quad \langle a, d, b, e, f, c, d, e, h \rangle,$$

whose footprint coincides with the footprint of L_{full} (N_1 was discovered from it; conformance = 1):

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>	<i>h</i>
<i>a</i>	#	→	→	→	#	#	#	#
<i>b</i>	←	#	#		→	←	#	#
<i>c</i>	←	#	#		→	←	#	#
<i>d</i>	←			#	→	←	#	#
<i>e</i>	#	←	←	←	#	→	→	→
<i>f</i>	#	→	→	→	←	#	#	#
<i>g</i>	#	#	#	#	←	#	#	#
<i>h</i>	#	#	#	#	←	#	#	#

Playing out the sequential N_2 (e.g. $\langle a, b, d, e, f, b, d, e, g \rangle$, $\langle a, c, d, e, f, c, d, e, h \rangle$) yields a different matrix: a is never directly followed by d , and b/c are causal predecessors of d rather than parallel. The cells where the two footprints disagree, in the format log: model:

pair	L_{full}	N_2	pair	L_{full}	N_2
(a, d)	→	#	(d, b)		←
(b, d)		→	(d, c)		←
(b, e)	→	#	(d, f)	←	#
(c, d)		→	(e, b)	←	#
(c, e)	→	#	(e, c)	←	#
(d, a)	←	#	(f, d)	→	#

Twelve of the 64 cells differ:

$$1 - \frac{12}{64} = 0.8125.$$

The measure is meaningful only when the log is complete with respect to $>_L$; completeness can be assessed by cross-validation (build the footprint from a portion of the log, check whether more cases add new pairs). Since both objects have a footprint, the same construction serves log-to-log similarity (concept drift) and model-to-model benchmarking without changing the formula.

24 Quantitative analysis

The chapters so far asked *qualitative* questions about process models; this one turns models into numbers: throughput, waiting times, utilisation, cost. The exposition follows Ch. 7 of *Fundamentals of Business Process Management*.⁷³ Three concerns are easy to conflate. **Validation** relates the model to reality: does it capture the process the organisation actually carries out? **Verification** asks qualitative questions about the model itself (can it deadlock, can every case terminate, can two tasks run in either order?) the soundness-style questions of the previous chapters. **Performance analysis** asks quantitative ones: how many cases per hour, what average flow time, how many extra resources to reach a given throughput.

Definition (Performance dimensions, measures, KPI). Companies want processes *faster, cheaper, better*: the three performance dimensions **time, finance, quality**. A **process performance measure** is

⁷³M. Dumas, M. La Rosa, J. Mendling, H. A. Reijers, *Fundamentals of Business Process Management*, Ch. 7; course material at <https://courses.cs.ut.ee/2014/bpm/>.

a quantity that can be unambiguously determined for a given business process; the abstract dimensions are refined into concrete **Key Performance Indicators** (KPI): scalar quantities computable from process executions and usable as optimisation targets.

Definition (Cycle, processing, waiting time). The time dimension decomposes into three measures: **cycle time**, the time needed to handle one case from start to end; **processing time** (or *service time*), the time resources spend actually handling the case; **waiting time**, the time the case spends idle, including queueing caused by resource unavailability. Cycle time is the sum of the other two.

The split matters because the two parts respond to different interventions: processing time is hard to compress without changing what a task does, waiting time often yields to better scheduling or extra resources. A cycle-time goal can take three shapes: reduce the *average*, reduce the *maximum*, or meet a deadline *negotiated with the customer* (a service-level agreement), and they drive different redesigns: averages hide outliers, a maximum or a contract forces attention to the worst cases.

The finance dimension covers *cost*, *turnover*, *yield* and *revenue*; a yield increase moves profit as much as a cost decrease, but redesign is most often framed in terms of cost. Costs split into **fixed** (overhead unaffected by processing intensity: infrastructure, maintenance), **variable** (positively correlated with sales volume, purchases, hires) and **operational** (directly related to the output of the process, labour cost above all). Redesign typically targets operational cost. Automating a task cuts labour cost but introduces *incidental* cost (developing the replacing application) and fixed *maintenance* cost for it: automation pays only when case volume amortises both over the saved labour.

Quality has two viewpoints. **External quality** is the client's satisfaction with the product or with how the process ran; key factors are the amount, relevance, quality and timeliness of the information the client receives as the process progresses. **Internal quality** is the participants': level of control over the work, variation experienced, challenges faced. External quality is often measured through time (average cycle time, percentage of missed deadlines); by convention, any measure involving time is classified under the *time* dimension, keeping the three dimensions disjoint at the measurement level.

Each measure is paired with an **aggregation function** turning a population of executions into one scalar: count, average, variance, minimum, maximum. "Average delivery cost per item" pairs cost-per-item with the average; "maximal time of delivery" pairs delivery time with the maximum. Deriving a KPI from a management goal takes three steps: *formulate* the performance objective (the desirable state of the process), *identify* the relevant dimension and aggregation function and derive the KPI, *set* a target value.

Example (Restaurant KPI). A restaurant is losing customers to poor service, and the management team focusses on meal delivery. Customer interviews give two anchors: half the customers want their meal within 15 minutes, and all agree that 30 minutes or more is unacceptable. The same goal supports three different KPI:

	Objective	KPI (time dimension)	Target
v1	served within 30'	$ST_{30} = \% \text{ of customers served in } < 30'$	$\geq 97\%$
v2	served in about 15'	$ST_{15} = \% \text{ of customers served in } < 15'$	$\geq 85\%$
v3	served in about 15'	$AMDT = \text{average meal delivery time}$	$\leq 15'$

v1 guards the tail (almost nobody waits unacceptably long, nothing promised about typical service); v2 tightens the threshold but relaxes the target, tolerating a long tail; v3 collapses the distribution into an average.

The canonical measures in practice: for time, cycle time and waiting time (time in non-value-added tasks); for cost, cost per execution and resource utilisation; for quality, error rates and missed promises.

24.1 Cycle time analysis

Definition (Flow analysis). **Flow analysis** estimates the overall performance of a process from the performance of its individual activities: cycle time of the process from the cycle times of the activities, average cost from costs per execution, error rate from activity error rates. It trades realism for tractability, assuming that activities are independent and that the model's structure dictates how their numbers compose.

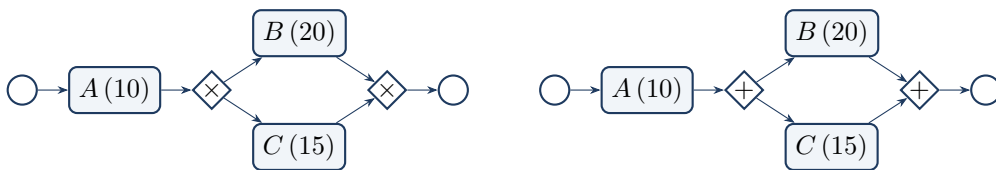
Definition (Cycle time analysis). The **cycle time** of a case is the difference between its end time and its start time; **cycle time analysis** computes the average cycle time of a process or fragment from the cycle times of its activities, assuming an average *activity time* (waiting + processing) is known for every activity.

Notation: CT is the average cycle time of the process; $CT_A, CT_B, \dots$ those of activities $A, B, \dots$. In diagrams the activity time appears in parentheses under the name, $A(10)$. Composition proceeds by *flow patterns*: sequence, alternative paths (XOR-block), parallel paths (AND-block), rework loops.

Sequence. Time composes additively along a sequence: a case reaching B has already spent CT_A in A . For $A(10)$ followed by $B(20)$, $CT = 10 + 20 = 30$. In general

$$CT = \sum_{i=1}^n CT_i.$$

XOR-block and AND-block. The fragment between an XOR-split and its matching XOR-join is an **XOR-block**; between an AND-split and its AND-join, an **AND-block**.



In the XOR-block each case takes exactly one branch, so its cycle time is 30 (via B) or 25 (via C); where the *average* sits depends on how often each branch is taken. The **branching probability** p_i of branch i is the frequency with which it is taken, with $\sum_i p_i = 1$, estimated from historical data or domain knowledge: without the p_i the cycle time of an XOR-block is undefined. The rule is the weighted average

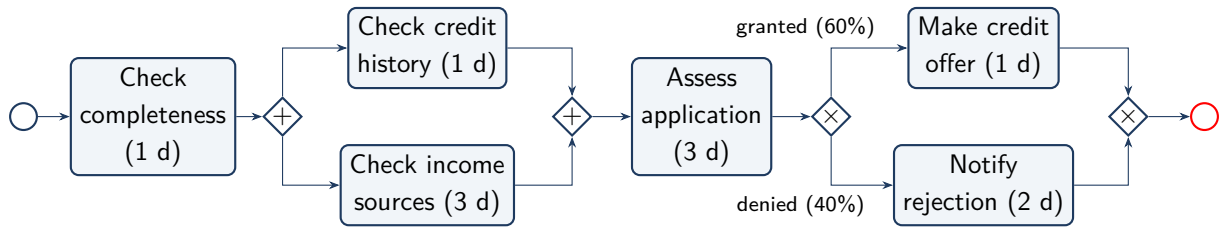
$$CT = \sum_{i=1}^n p_i \cdot CT_i,$$

giving $0.5 \cdot 30 + 0.5 \cdot 25 = 27.5$ for a 50/50 split and $0.9 \cdot 30 + 0.1 \cdot 25 = 29.5$ for 90/10. In the AND-block both branches run concurrently, so summing all three activity times (45) overestimates: the AND-join waits for the *slowest* branch, $CT = 10 + \max\{20, 15\} = 30$. The rule is

$$CT = \max_i CT_i,$$

and the synchronisation cost is the time the faster branch wastes waiting.

Example (Loan application).



Decompose left to right and apply each rule: completeness check 1 d; AND-block $\max\{1, 3\} = 3$ d; assessment 3 d; XOR-block $0.6 \cdot 1 + 0.4 \cdot 2 = 1.4$ d. The sequence rule sums the four fragments:

$$CT = 1 + \max\{1, 3\} + 3 + (0.6 \cdot 1 + 0.4 \cdot 2) = 8.4 \text{ days.}$$

Rework loops. Cyclic structures break the naïve sum: if $B(20)$ were always repeated, $CT = 10 + 20 + 20 + \dots$ diverges. The missing ingredient is the **rework probability** r : the frequency with which the post- B XOR-split sends the case back; with probability $1 - r$ the case exits. Two variants, “1 or more” (left) and “0 or more” (right):



In the left variant the body runs at least once; the case reworks $n - 1$ times and then exits with probability $r^{n-1}(1 - r)$, a geometric distribution over the iteration count. Weighting each count by its probability and writing CT_P for the cycle time of the loop body:

$$CT = \sum_{i=1}^{\infty} i \cdot CT_P \cdot r^{i-1}(1 - r) = CT_P(1 - r) \sum_{i=1}^{\infty} i r^{i-1} = \frac{CT_P(1 - r)}{(1 - r)^2} = \frac{CT_P}{1 - r},$$

using the identity $\sum_{i \geq 1} i x^{i-1} = (1 - x)^{-2}$. The factor $1/(1 - r)$ inflates the body's time by the expected number of iterations.

In the right variant the case may skip B entirely: exactly n executions have probability $r^n(1-r)$ for $n = 0, 1, 2, \dots$, and the same computation gives

$$CT = \sum_{i=0}^{\infty} i \cdot CT_P \cdot r^i(1-r) = CT_P(1-r) \cdot \frac{r}{(1-r)^2} = \frac{r \cdot CT_P}{1-r}.$$

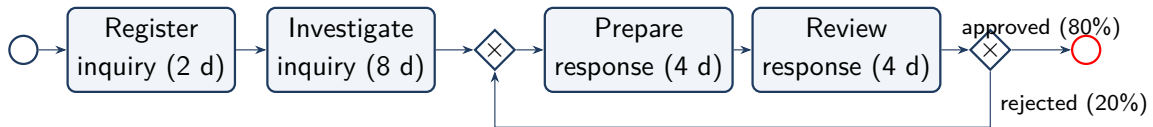
Note (Relating the two rules). The 0-or-more formula is the 1-or-more one weighted by the probability r that the body executes at all: a case skips P with probability $1-r$, contributing 0, so $CT = \frac{CT_P}{1-r} \cdot r$.

Example (Loan application with rework). Refine the loan process: the completeness check (2 h) is repeated until the application is complete, with rework probability $r = 0.2$ (the XOR-split after it exits with “done” 80%, loops back with “repeat” 20%); the parallel checks take 0.5 h (credit history) and 3 h (income sources); assessment 2 h; credit offer 2 h (60%), rejection notice 0.5 h (40%). The four patterns compose:

$$CT = \frac{2}{1-0.2} + \max\{0.5, 3\} + 2 + (0.6 \cdot 2 + 0.4 \cdot 0.5) = 2.5 + 3 + 2 + 1.4 = 8.9 \text{ hours.}$$

The loop inflates the 2 h check to 2.5 h; the rest composes as before.

Example (Inquiry handling: cycle time).



The rework loop covers *both* Prepare response and Review response: a rejected response is re-prepared and re-reviewed until approved, so $CT_P = 4 + 4 = 8$ d with $r = 0.2$. The rest is a sequence:

$$CT = 2 + 8 + \frac{4 + 4}{1 - 0.2} = 10 + 10 = 20 \text{ days.}$$

The loop turns the 8-day body into 10 effective days: each case visits the prepare–review pair 1.25 times on average.

24.2 Theoretical cycle time and efficiency

In most real processes waiting time dominates the cycle time. Two causes recur: *batched processing* (paperwork accumulated and handled in chunks) and *actor unavailability* (approvals or prescriptions needing a specific person). Waiting time is therefore where redesign pays off most, and a pair of measures quantifies the opportunity.

Definition (Theoretical cycle time, cycle time efficiency). When the *processing* time of every activity is known, the **theoretical cycle time** TCT is computed by the same flow-analysis rules as CT but with processing times in place of activity times: the cycle time the process would have if no case ever waited. The **cycle time efficiency** is

$$CTE = \frac{TCT}{CT} \in [0, 1].$$

$CTE \approx 1$ leaves little room to improve cycle time without radical changes; $CTE \approx 0$ signals a process dominated by idle time, improvable by cutting waits.

Example (Inquiry handling: TCT and CTE). The inquiry process has $CT = 20$ d = 160 h (at 8 h per business day). Processing times: Register inquiry 30 min, Investigate inquiry 12 h, Prepare response 4 h, Review response 2 h. The same composition with processing times gives

$$TCT = 0.5 + 12 + \frac{4 + 2}{1 - 0.2} = 12.5 + 7.5 = 20 \text{ hours}, \quad CTE = \frac{20}{160} = 12.5\%.$$

One eighth of the cycle time is actual work; the other seven eighths are waiting: a textbook candidate for waiting-time reduction.

Note (Limitations of flow analysis). Three structural limits. The equations cover only *block-structured* models (sequence, XOR-, AND-blocks, rework loops); goto-style jumps or unmatched gateways need ad-hoc reasoning. Activity times must be *known*: estimating them, by interviews or logs, is its own measurement problem. And the formulas are *load-insensitive*: they treat activity times as constants, while in reality waiting grows when arrivals increase against fixed resources.

24.3 Little's law

Two process-level measures accompany cycle time: the **arrival rate** λ , the average number of new cases created per unit of time, and the **Work-In-Process** WIP , the average number of cases active (started but not completed) at a given time. Both are averages over a stable window: λ measures inflow, WIP the active population.

Definition (Little's law). In 1954 the operations-research professor John Little postulated (without proof) that in a *stable* system⁷⁴ the long-term average number of customers equals the long-term average effective arrival rate times the average time a customer spends in the system:

$$WIP = \lambda \cdot CT.$$

The identity holds with no assumption on arrival or service distributions and no knowledge of the queueing discipline; Little proved it in 1961,⁷⁵ with simpler proofs following in 1967, 1969 and 1972. Two readings matter for redesign. First, WIP grows with either CT or λ : a slowing process accumulates cases, a faster inflow swells the active population; if λ grows and WIP must stay constant, CT has to shrink. Second, since WIP and λ are easy to estimate by sampling, the law computes cycle time from two black-box observations, no flow analysis needed: $CT = WIP/\lambda$.

Example (Cycle time by observation). Over the past year (250 business days) 2500 applications arrived: $\lambda = 2500/250 = 10$ per day. Weekly sampling found on average 200 applications concurrently active: $WIP = 200$. Hence $CT = 200/10 = 20$ days: matching the flow-analysis result for the inquiry process.

Example (Restaurant). A restaurant open 12 hours serves 1200 customers per day. During peak times (6 hours in total) it receives 900 customers with on average 90 present at any time; off-peak (the other 6 hours) 300 customers with 30 present. Time spent by a customer in each regime:

$$CT_{\text{peak}} = \frac{90}{900/6} = 0.6 \text{ h}, \quad CT_{\text{off}} = \frac{30}{300/6} = 0.6 \text{ h} = 36 \text{ minutes}.$$

⁷⁴Stable: the number of customers in the system does not grow unboundedly.

⁷⁵J. D. C. Little, *A proof of the queueing formula $L = \lambda W$* , Operations Research 9 (1961), 383–387; the classical statement reads $L = \lambda \cdot W$.

The two regimes coincide: peak customers see a fuller room, but the inflow is proportionally higher. Now the manager expects peak demand to grow while the room is already at capacity. A higher λ with *WIP* capped by the building forces *CT* down: customers must spend less time inside: shorten serving, redesign order-taking and payment, turn tables faster. Once physical capacity is fixed, process redesign is the only lever.

24.4 Cost analysis

Flow analysis applies to cost with one change. Sequence, XOR-block and rework loops use the *same* formulas as cycle time; the AND-block uses the *sum* of the branch costs, not the maximum, because every branch incurs its cost regardless of which is slower: time runs in parallel, money does not. Activity cost itself has two components: **human-resource cost**, the hourly cost of the resource times the processing time, and **other cost**, fixed fees incurred per execution independently of the time spent.

Example (Loan application: cost). The refined loan process (rework on the completeness check, $r = 0.2$), with activities annotated by processing time. Clerk activities cost 25 e/h : Check completeness (2 h), Check credit history (0.5 h), Check income sources (3 h). Credit-officer activities cost 50 e/h : Assess application (2 h), Make credit offer (2 h), Notify rejection (0.5 h). Each credit-history check also costs a 1 e fee.

Activity	Resource	Resource cost	Other	Total
Check completeness	clerk	$2 \cdot 25 = 50$	0	50
Check credit history	clerk	$0.5 \cdot 25 = 12.5$	1	13.5
Check income sources	clerk	$3 \cdot 25 = 75$	0	75
Assess application	officer	$2 \cdot 50 = 100$	0	100
Make credit offer	officer	$2 \cdot 50 = 100$	0	100
Notify rejection	officer	$0.5 \cdot 50 = 25$	0	25

Composing with the cost rules (rework: divide by $1 - r$; AND: sum; XOR: weighted average):

$$\text{Cost} = \frac{50}{1 - 0.2} + 13.5 + 75 + 100 + 0.6 \cdot 100 + 0.4 \cdot 25 = 62.5 + 188.5 + 70 = 321 e.$$

The AND-block contributes $13.5 + 75 = 88.5 e$ (the sum) where the cycle-time computation took only $\max\{0.5, 3\} = 3$ hours from the same block.

Example (Inquiry handling: cost). The inquiry process again, with resources and rates: Register inquiry by a clerk (25 e/h , 30 min), Investigate inquiry by an advisor (50 e/h , 12 h), Prepare response by a senior advisor (75 e/h , 4 h), Review response by a counselor (100 e/h , 2 h). No AND-block; the rework loop covers the prepare–review pair with $r = 0.2$:

$$\text{Cost} = 25 \cdot 0.5 + 50 \cdot 12 + \frac{75 \cdot 4 + 100 \cdot 2}{1 - 0.2} = 12.5 + 600 + 625 = 1237.50 e.$$

The loop sits on the costliest pair of activities and dominates the bill: the 500 e body becomes 625 e at 1.25 visits per case.

25 Diagnosis for workflow nets

This chapter studies diagnosis techniques for *unsound* workflow nets. Knowing that a WF-net is broken is not enough: the analyst needs to locate the place, transition or structural fragment responsible. The tools of the trade (WoPeD, Woflan, ProM) implement the techniques below and report the diagnosis directly on the net.⁷⁶ WoPeD’s semantical-analysis report, for instance, counts the *S-components* of a net and lists the places *not covered* by any of them; a single uncovered place compromises structural soundness and is flagged as a diagnosis hint.

25.1 S-coverability

Note (Rank Theorem (main result, proof omitted)). A free-choice system (P, T, F, M_0) is *live and bounded* iff all six conditions hold:

1. it has at least one place and one transition;
2. it is connected;
3. M_0 marks every proper siphon;
4. it has a positive S-invariant;
5. it has a positive T-invariant;
6. $\text{rank}(N) = |C_N| - 1$, where C_N is the set of clusters.

Each condition is a checkable structural test: the tools implement them and report violations one by one, turning the abstract characterisation into a concrete diagnosis report.

Condition 4 admits a constructive search. A workflow case is typically a bundle of parallel threads of control, each imposing an order on its tasks; decompose the net into *S-nets* so that every place belongs to at least one of them (overlaps allowed), let each S-net contribute the uniform S-invariant that weights its places with 1, and sum: the result is a positive S-invariant of the whole net. The objects that make this precise are subnets and S-components.

Definition (Subnet). Let $N = (P, T, F)$ be a net and $\emptyset \subset X \subseteq P \cup T$. The **subnet of N induced by X** is

$$N' = (P \cap X, T \cap X, F \cap (X \times X)) :$$

keep the nodes in X , forget every arc with an endpoint outside X .

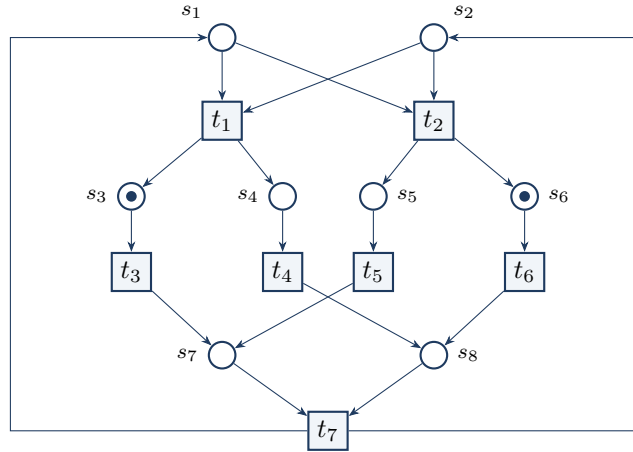
Definition (S-component). The subnet N' induced by X is an **S-component** of N when

1. N' is a strongly connected S-net (every transition of N' has exactly one input and one output place in N'), and
2. for every place $p \in X \cap P$: $\bullet p \cup p\bullet \subseteq X$.

The second clause is a closure property: taking a place drags in *all* its surrounding transitions, pre-set and post-set. Places cannot leave attached transitions behind, while transitions may drop input and output places that fall outside X : that asymmetry is what makes finding S-components a non-trivial selection problem.

Example (S-components and an S-cover).

⁷⁶H. M. W. Verbeek, T. Basten, W. M. P. van der Aalst, *Diagnosing workflow processes using Woflan*, optional reading: <http://wwwis.win.tue.nl/~wvdaalst/publications/p135.pdf>.



Both t_1 and t_2 consume from both s_1 and s_2 ; t_1 produces s_3, s_4 and t_2 produces s_5, s_6 ; the middle transitions cross (t_4 feeds s_8 , t_5 feeds s_7); t_7 joins s_7, s_8 and refills s_1, s_2 . A first attempt at an S-component picks s_1 and walks down both outer branches: $X = \{s_1, t_1, t_2, s_3, s_6, t_3, t_6, s_7, s_8, t_7\}$. Closure forced the whole selection: s_1 drags in t_1, t_2, t_7 ; landing in both s_7 and s_8 drags in t_7 's two inputs, and now t_7 has *two* input places in the subnet: not an S-net, hence no S-component. The fix is to land on one shared place per side. The selection $X_1 = \{s_1, t_1, t_2, s_3, s_5, t_3, t_5, s_7, t_7\}$ works: in the subnet t_1 keeps only $s_1 \rightarrow s_3$, t_2 only $s_1 \rightarrow s_5$, both branches converge on s_7 , and t_7 keeps the single input s_7 and single output s_1 : a strongly connected S-net satisfying closure. Symmetrically $X_2 = \{s_2, t_1, t_2, s_4, s_6, t_4, t_6, s_8, t_7\}$ is an S-component through the even places. Together they cover all of $s_1, \dots, s_8$.

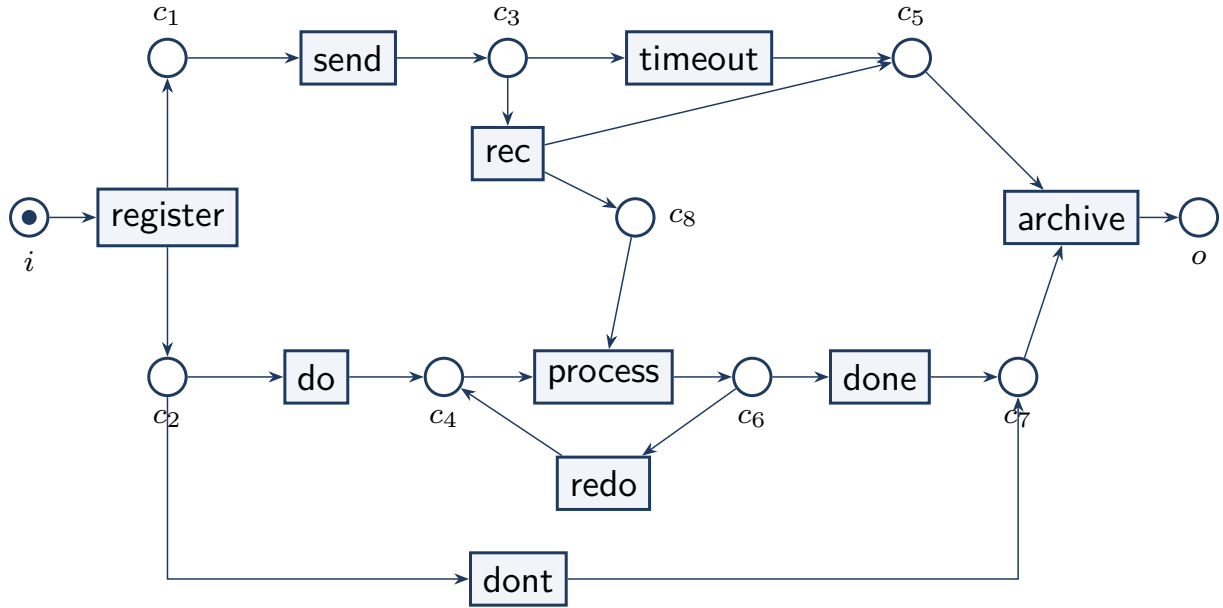
Definition (S-cover, S-coverability). An **S-cover** of N is a set of S-components of N such that every place of N belongs to at least one of them; N is **S-coverable** when an S-cover exists.

An S-invariant of an S-component lifts to the whole net: keep its weights on the covered places and assign 0 elsewhere. Uncovered places contribute nothing to the dot product, so $M_0 \cdot I = M \cdot I$ still holds for every reachable M . In the example the uniform invariants of the two components lift to $I_1 = [10101010]$ and $I_2 = [01010101]$, and their sum $[11111111]$ is strictly positive: an S-cover always yields the positive S-invariant that condition 4 of the Rank Theorem asks for.

Definition (S-coverability theorem 1). If a free-choice system is live and bounded, then it is S-coverable. (Proof omitted.)

The contrapositive is the diagnosis tool: a free-choice net that is not S-coverable cannot be live and bounded. Applied to the short-circuited net N^* of a WF-net N (reset transition from sink to source, reducing soundness of N to liveness+boundedness of N^*): if N^* is free-choice but not S-coverable, N is not sound.

Example (The registration workflow).



A registration case forks into a correspondence thread (c_1 : *send* the form, then either *receive* it back or *timeout*) and a processing thread (c_2 : *do* the work, or *dont*); *rec* also deposits the returned form in c_8 , which *process* consumes together with c_4 ; *redo* loops the work back; *archive* synchronises c_5 and c_7 into o . Can N^* (with the implicit reset $o \rightarrow i$) be S-covered? WoPeD answers: N^* is free-choice, finds two S-components ($\{i, c_1, c_3, c_5, o\}$ through the correspondence thread and $\{i, c_2, c_4, c_6, c_7, o\}$ through the processing thread, each closed by the reset) and reports **one place not covered**: c_8 . No S-component can contain c_8 : closure drags in *rec* and *process*; *process* then needs its output c_6 in the selection; closure on c_6 drags in *redo*, which needs its output c_4 ; closure on c_4 drags *process* back in with a *second* input: never an S-net. By theorem 1, N is not sound.

Note (Reading WoPeD output). The reset transition is implicit: absent from the canvas, used by the analysis. Everything in the report (S-components, uncovered places, the handles below) refers to N^* , not to N ; WoPeD also numbers places with its own internal names, which need not match the labels on the slide or the model.

25.2 Well-structuredness

Below the S-components section, WoPeD's report counts *PT-handles* and *TP-handles* (on the registration net's N^* : 4 and 5). Both are structural mismatches between gateway types, and both harm soundness.

Definition (TP-handle, PT-handle). A transition t and a place p form a **TP-handle** if there are two distinct elementary paths from t to p sharing only t and p . A place p and a transition t form a **PT-handle** if there are two distinct elementary paths from p to t sharing only p and t .

The two-disjoint-paths geometry captures the two flawed patterns. A TP-handle is an AND-split whose branches are merged by a XOR-join: both branches deliver a token, the merge place expects one, so it hosts two: the net stops being safe, the same case appears twice downstream. A PT-handle is a XOR-split whose branches are synchronised by an AND-join: only one branch ever delivers, the join waits for all, deadlock. On the registration net, *register* and c_7 close a TP-handle (π_1 through c_1 , *send*, c_3 , *rec*, c_8 , *process*, c_6 , *done*; π_2 through c_2 , *dont*); c_3 and *archive* close a PT-handle (π_1 through *timeout*, c_5 ; π_2 through *rec*, c_8 , *process*, c_6 , *done*, c_7).

Definition (Well-handled, well-structured; S-coverability theorem 2). A net is **well-handled** when it has neither TP- nor PT-handles; a WF-net N is **well-structured** when N^* is well-handled. **Theorem:** if N is sound and well-structured, then N^* is S-coverable. (Proof omitted.)

The contrapositive again does the work: N well-structured + N^* not S-coverable $\Rightarrow N$ not sound. Theorem 1 required free-choice; theorem 2 trades it for well-structuredness, widening the family of nets the technique can diagnose. Since handles live on N^* , a handle reported by WoPeD may route one of its paths through the implicit reset: on the registration net, WoPeD pairs c_3 with *process*, closing π_1 inside N (through *rec* and c_8) and π_2 through *timeout*, c_5 , *archive*, o , *reset*, i , *register*, c_2 , *do*, c_4 : a path invisible on the canvas because the reset is implicit.

25.3 Error sequences

Woflan (WOrkFLow ANalyzer)⁷⁷ is the historical diagnosis tool for workflow nets, today available as a ProM⁷⁸ plugin. Given N it checks: is N a workflow net? is N^* bounded? is N^* live? On failure it does not stop at “unsound”: it lists the unbounded places, the dead transitions and the non-live transitions of N^* . Those static lists say *that* something is wrong, not *when* or *how*; to exhibit the scenario, Woflan adds behavioural witnesses.

Definition (Error sequence). An **error sequence** is a firing sequence σ such that (1) every continuation of σ leads to an error (no future firing recovers) and (2) no proper prefix of σ already has property (1).

The two soundness clauses that can break, option to complete and proper completion, give two species: *non-live* and *unbounded* sequences.

25.4 Non-live sequences

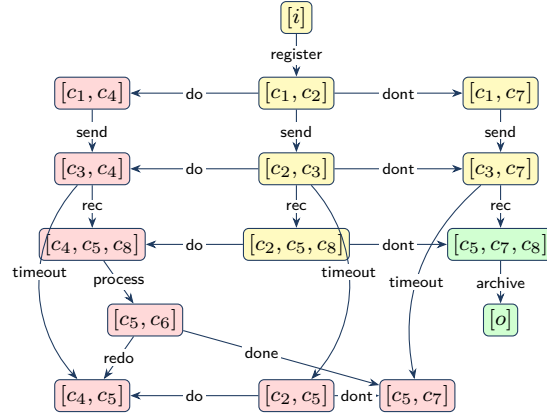
Definition (Non-live sequence). A **non-live sequence** is a firing sequence, as short as possible, after which completion of the case is no longer possible: a witness that the reset transition of N^* is not live. **Fundamental property:** if N^* is bounded and has no dead task, then N^* is live iff N has no non-live sequences.

The property turns the liveness check into an enumeration over the reachability graph (boundedness is required, otherwise the graph is infinite). Colour every node of $RG(N)$ **red** when no path from it reaches a marking with a token in o , **green** when every path from it leads to o , yellow otherwise. Non-live sequences are exactly the minimal traces that step from a non-red node into a red node.

Example (Non-live sequences of the modified registration net). Modify the registration net by one arc: *archive* now consumes c_8 too ($c_5 + c_7 + c_8 \rightarrow o$), so completion needs the returned form to reach the archive. The coloured reachability graph:

⁷⁷<http://www.win.tue.nl/woflan/>

⁷⁸<http://promtools.org/>



The only green run is *register, send, rec, dont, archive* (and its *dont*-first permutations): *rec* must produce c_8 and *process* must never consume it. Every red node lacks a usable c_8 : after *timeout* the form never returns ($[c_4, c_5]$, $[c_2, c_5]$, $[c_5, c_7]$ are dead ends), and after an early *do* the token in c_8 is destined to be eaten by *process*, leaving *archive* disabled ($[c_5, c_6]$, $[c_5, c_7]$). The six non-live sequences (minimal yellow-to-red steps) are

`register,do   register,send,do   register,send,timeout`
`register,send,rec,do   register,send,dont,timeout   register,dont,send,timeout`

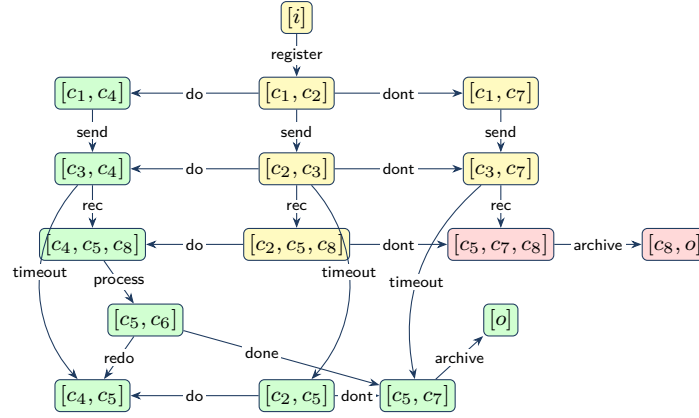
Two failure modes, readable at a glance: firing *do* at all, and the combination *send–timeout*. Woflan reports each as a (transition, marking) pair, the final step and the marking it fires from, under the failed clause “option to complete”.

25.5 Unbounded sequences

Definition (Unbounded sequence). An **unbounded sequence** is a firing sequence of minimal length after which every continuation invalidates proper completion: a witness that N^* is unbounded. **Fundamental property:** N^* is bounded iff N has no unbounded sequences. The **undesired markings** are the two failure shapes of proper completion: markings with an infinite (ω) component, and markings strictly greater than $[o]$.

Boundedness can no longer be assumed, so the colouring runs on the *coverability graph* $CG(N^*)$, finite even for unbounded systems, with reachability read *within the current case*: following ordinary transitions but not the reset $[o] \rightarrow [i]$, which counts as a fresh start: **green** the nodes from which no undesired marking is reachable, **red** those from which no green node is reachable (undesired markings unavoidable), yellow the rest. Unbounded sequences are the minimal yellow-to-red steps. One observation shrinks the graph: infinite-weighted markings only lead to infinite-weighted markings, and they are all red: so the expansion can stop as soon as an ω appears. The result is the **restricted coverability graph** (RCG); on the running example it halves the node count, on real nets the saving is dramatic. For a bounded system, RCG and reachability graph coincide.

Example (Unbounded sequences of the original registration net). Back to the net as drawn above (*archive* consumes $c_5 + c_7$ only). The RCG of N^* has the same skeleton as the previous graph, but *archive* now also fires at $[c_5, c_7]$, and at $[c_5, c_7, c_8]$ it leaves the spare token behind:



The colours flip. The whole left zone is now green: deadlocks like $[c_4, c_5]$ violate option to complete, but no *undesired* marking is reachable from them, and this analysis only asks about proper completion. Red are $[c_5, c_7, c_8]$ (its only move is *archive*, into $[c_8, o] > [o]$) and $[c_8, o]$ itself, an undesired marking whose RCG successors (through the reset) all carry c_8^w and are cut from the graph. The three unbounded sequences are

register, dont, send, rec register, send, dont, rec register, send, rec, dont

one per interleaving: *rec* puts a token in c_8 , and once *dont* has consumed c_2 the only consumer of c_8 (*process*, which needs c_4) is unreachable: the case completes with a leftover token. Woflan’s report flags the failed “proper completion” clause, notes that no transition is dead, lists the two witness pairs (*rec* and *dont* at their firing markings) and points at c_8 via the S-component coverage report: the same place the S-coverability analysis flagged.

25.6 Practice with WoPeD

Three practice nets close the chapter. An *order-processing* net (credit-card charge, packing with backorder subflow, shipping, billing-update merge) passes every check: workflow-net property, boundedness, liveness all green, no wrongly-used operators, no free-choice violations, well-structured: sound. A *questionnaire-handling* net (register, then send/process questionnaire under a timeout, evaluate, optional complaint processing with check-ok/not-ok loop, archive) draws one *free-choice violation*: the cluster around *process questionnaire* and *process complaint* shares input places without sharing all of them; the soundness checks nonetheless pass. A third, denser net (dozens of nodes, several loops) stacks the verdicts: one free-choice violation, fifteen PT-handles and fifteen TP-handles, yet S-coverable with zero uncovered places, and its reachability graph is a fan of hundreds of markings, far too large to inspect by eye. WoPeD’s soundness panel: all green.

Note (Sufficient, not necessary). The structural theorems prune in one direction only. A free-choice or well-structured net that fails S-coverability is certainly unsound; but a net violating free-choice *and* riddled with handles can still be sound, as the third practice net shows. When the structural conditions fail to apply, the verdict comes from the reachability graph and the four soundness checks, and the error-sequence machinery compresses a graph of hundreds of nodes into a handful of minimal witnesses.

The course covered a coherent slice of business process modelling (BPMN and EPC, Petri nets, workflow nets, soundness, free-choice theory, S-coverability and diagnosis, process mining,

conformance, quantitative analysis), one slice of a field that extends into further notations, theory, tools and an entire research literature.

Appendix: BPM Cheat Sheets

A.1: Definitions

Definitions	
Net	$N = (P, T, F)$ where P is the set of places , T is the set of transitions , $F \subseteq (P \times T) \cup (T \times P)$ is the set of arcs (flow relation). A net is a bipartite, oriented graph whose set of nodes is $P \cup T$.
Marking	$M : P \rightarrow \mathbb{N}$ such that $M(p)$ is the number of tokens in the place p . A marking can be seen as a vector $\mathbb{N}^{ P }$ (one entry for each place). A place p is marked at M if $M(p) > 0$, otherwise it is unmarked .
System	(N, M_0) where $N = (P, T, F)$ is a net and $M_0 : P \rightarrow \mathbb{N}$ is the initial marking .
S-net / S-system	Every transition has <i>exactly</i> one input and one output place ($ \bullet t = 1 = t\bullet $).
T-net / T-system	Every place has <i>exactly</i> one input and one output transition ($ \bullet p = 1 = p\bullet $).
Free-choice	Any two transitions must have pre-sets that are either equal or disjoint.
Weakly connected	There exists an undirected path between any two nodes.
Strongly connected	There exists a directed path between any two nodes.
Transition sequence	$\sigma \in T^*$ a possibly empty sequence of transitions.
Parikh vector	$\vec{\sigma} : T \rightarrow \mathbb{N}$ such that $\vec{\sigma}(t)$ counts the number of times t appears in σ . It is seen as a vector $\mathbb{N}^{ T }$ (one entry for each transition).
Incidence matrix	$N \in \mathbb{Z}^{ P \times T }$ has one row for each place and one column for each transition. $N(p, t) = -1$ if there is an arc from p to t but not vice versa; $N(p, t) = 1$ if there is an arc from t to p but not vice versa; $N(p, t) = 0$ otherwise.
S-invariant	$\mathbf{I} \in \mathbb{Q}^{ P }$ is a vector with one entry for each place such that $\mathbf{I} \cdot N = 0$. It is a map $\mathbf{I} : P \rightarrow \mathbb{Q}$ that assigns a weight $\mathbf{I}(p)$ to each place p .
T-invariant	$\mathbf{J} \in \mathbb{Q}^{ T }$ is a vector with one entry for each transition such that $N \cdot \mathbf{J} = 0$. It is a map $\mathbf{J} : T \rightarrow \mathbb{Q}$ that assigns a multiplicity $\mathbf{J}(t)$ to each transition t .
Enabling	$M \xrightarrow{t}$ says that every place in the pre-set of t carries a token. In formulas: $\forall p \in \bullet t. M(p) \geq 1$, written compactly $\bullet t \subseteq M$ (reading the set $\bullet t$ as the marking with one token per input place). $M \rightarrow$ says that some transition is enabled at M .
Firing	$M \xrightarrow{t} M'$ says that t is enabled at M and its firing leads to M' . In formulas: $\bullet t \subseteq M \wedge M' = M - \bullet t + t\bullet$. $M \xrightarrow{\sigma} M'$ says that $\sigma \in T^*$ is enabled at M and its firing leads to M' . In formulas, for $\sigma = t_1 \dots t_n$ a finite sequence of transitions: $\exists M_1, \dots, M_n. M \xrightarrow{t_1} M_1 \xrightarrow{t_2} \dots \xrightarrow{t_n} M_n$.
Reachable markings	$M \rightarrow^* M'$ says that M' is reachable from M , i.e. that $M \xrightarrow{\sigma} M'$ for some $\sigma \in T^*$. $[M]$ is the set of all markings reachable from M .
Infinite firings	$M \xrightarrow{\sigma}$ says that $\sigma \in T^\infty$ is enabled at M . In formulas, for $\sigma = t_1 t_2 \dots t_n \dots$ an infinite sequence of transitions: $\exists M_1, M_2, \dots, M_n, \dots M \xrightarrow{t_1} M_1 \xrightarrow{t_2} \dots \xrightarrow{t_n} M_n \xrightarrow{t_{n+1}} \dots$
Siphon	$R \subseteq P$ such that $\bullet R \subseteq R\bullet$.
Trap	$R \subseteq P$ such that $\bullet R \supseteq R\bullet$.

A.2: Properties

Properties	
Liveness	A transition t is live if at any point in time of the computation it is still possible to enable t in the future. In formulas: $\forall M \in [M_0]. \exists M' \in [M]. M' \xrightarrow{t}$. A system S is live if all its transitions are live. In formulas: $\forall t \in T. \forall M \in [M_0]. \exists M' \in [M]. M' \xrightarrow{t}$.
Place-liveness	A place p is live if at any point in time of the computation it is still possible to mark p in the future. In formulas: $\forall M \in [M_0]. \exists M' \in [M]. M'(p) > 0$. A system S is place-live if all its places are live. In formulas: $\forall p \in P. \forall M \in [M_0]. \exists M' \in [M]. M'(p) > 0$.
Dead transition	A transition t is dead at M if it will never be enabled in the future. In formulas: $\forall M' \in [M]. M' \not\xrightarrow{t}$.
Dead place	A place p is dead at M if it will always remain unmarked. In formulas: $\forall M' \in [M]. M'(p) = 0$.
Non-live node	A place / transition is non-live if it can become dead in the future, ie. if there exists a reachable marking where it is dead.
Deadlock freedom	A system is deadlock free if every reachable marking enables some transition. In formulas: $\forall M \in [M_0]. \exists t \in T. M \xrightarrow{t}$, i.e. more simply, $\forall M \in [M_0]. M \rightarrow$.
Deadlock	A marking M is deadlock if it does not enable any transitions. In formulas: $M \not\rightarrow$.
k-boundedness	A place p is k-bounded if it will never contain more than k tokens. In formulas: $\forall M \in [M_0]. M(p) \leq k$. A system is k-bounded if all its places are k -bounded. In formulas: $\forall p \in P. \forall M \in [M_0]. M(p) \leq k$.
Safeness	1-bounded places and systems are called safe .
Boundedness	A place is bounded if there exists $k \in \mathbb{N}$ such that it is k -bounded.
Positive vector	A vector is positive if all its entries are strictly greater than 0.
Semi-positive vector	A vector is semi-positive if it has no negative entries and at least a positive one.
Uniform vector	A vector is uniform if its entries are either 0 or 1.
Support	$\langle V \rangle$ is the set of elements of the vector V that are assigned positive values.
Stable set	A set of markings $\mathbf{M}$ is stable if every marking reachable from some marking in $\mathbf{M}$ is also in $\mathbf{M}$. In formulas: $\forall M \in \mathbf{M}. \forall M' \in [M]. M' \in \mathbf{M}$.
Workflow net	Given a net (P, T, F) is a workflow net if: 1. There exists a (unique) initial place $i \in P$ such that $\bullet i = \emptyset$. 2. There exists a (unique) final place $o \in P$ such that $o \bullet = \emptyset$. 3. Every node of the net belongs to an oriented path from i to o.
N^*	Given a workflow net N , the net N^* is obtained by adding a reset transition from the final place of N to the initial place of N .
Soundness	Given a workflow net N , N is sound if and only if the following conditions hold:
(Weak soundness = 2 and 3)	1. No dead task: $\forall t \in T. \exists M \in [i]. M \xrightarrow{t}$ (every transition can be executed at least once). 2. Option to complete: $\forall M \in [i]. \exists M' \in [M]. M'(o) \geq 1$ (at any point in time of the computation there is still the possibility to mark o). 3. Proper completion: $\forall M \in [i]. M(o) > 0 \Rightarrow M = [o]$ (any reachable marking that marks the final place must have exactly one token in o and no token elsewhere).

A.3: General Results

General results

Proposition: liveness implies place-liveness. *Consequence:* if a system is not place-live, it cannot be live.

Lemma: liveness implies deadlock freedom.

Proposition: place-liveness implies deadlock freedom. *Consequences:* if a system can deadlock, it is neither live nor place-live.

Enabling proposition. $M \xrightarrow{\sigma}$ if and only if $M \xrightarrow{\sigma'}$ for any (finite) prefix σ' of σ .

Marking equation lemma. If $M \xrightarrow{\sigma} M'$ then $M' = M + N \cdot \vec{\sigma}$.

Monotonicity lemma (1). If $M \xrightarrow{\sigma} M'$ then $M + L \xrightarrow{\sigma} M' + L$ for any $L : P \rightarrow \mathbb{N}$.

Monotonicity lemma (2). If $M \xrightarrow{\sigma}$ then $M + L \xrightarrow{\sigma}$ for any $L : P \rightarrow \mathbb{N}$.

Monotonicity corollary. If $M \xrightarrow{\sigma} M'$ with $M \subseteq M'$ then $M \xrightarrow{\sigma\sigma\cdots}$.

Boundedness lemma. If a system is bounded and $M_0 \xrightarrow{\sigma} M$ with $M_0 \subseteq M$ then $M = M_0$.

Unboundedness lemma. A system is not bounded iff $\exists M \in [M_0]. \exists M' \in [M]. M \subset M'$.

Repetition lemma. If $M \xrightarrow{\sigma} M'$ and $M \xrightarrow{\sigma\sigma\cdots}$, then $M \subseteq M'$.

Strong connectedness theorem (1). If a weakly connected system is live and bounded, then it is strongly connected. *Consequence:* if a system is not strongly connected, then it cannot be live and bounded at the same time.

Main Theorem. A workflow net N is **sound** if and only if N^* is **live and bounded**.

Compositionality Theorem. If N and N' are two sound and safe workflow nets, then $N[N'/t]$ is safe and sound.

Place-liveness = liveness in free-choice systems. If a free-choice system is place-live, then it is live.

Commoner's Theorem. A free-choice system is live if and only if every proper siphon includes a marked trap.

Rank Theorem. A free-choice system (P, T, F, M_0) is live and bounded if and only if

1. It has at least one place and one transition.
2. It is connected.
3. M_0 marks every proper siphon.
4. It has a positive S-invariant.
5. It has a positive T-invariant.
6. The rank of the incidence matrix N equals the number of clusters minus 1.

Fundamental property of siphons. Unmarked siphons remain unmarked.

Liveness and siphons proposition. If a system is live, then any proper siphon is marked. *Corollary:* if a system has an unmarked proper siphon, then it is not live.

Fundamental property of traps. Marked traps remain marked.

Invariants

Proposition. $\mathbf{I}$ is an S-invariant if and only if $\forall t \in T. \sum_{p \in \bullet t} \mathbf{I}(p) = \sum_{q \in t \bullet} \mathbf{I}(q)$.

Vector space. Any linear combination of S-invariants is also an S-invariant.

Fundamental property of S-invariants. $\forall M \in [M_0]. \mathbf{I} \cdot M = \mathbf{I} \cdot M_0$. *Corollary:* if $\mathbf{I} \cdot M \neq \mathbf{I} \cdot M_0$, then M is not reachable from M_0 .

Boundedness theorem for S-invariants (sufficient condition for boundedness). If a net has a positive S-invariant, then it is bounded (for any initial marking). *Corollary:* any place in the support of a semi-positive S-invariant is bounded (for any initial marking).

Liveness theorem for S-invariants (necessary condition for liveness). If a system is live, then for any semi-positive S-invariant $\mathbf{I}$ we have $\mathbf{I} \cdot M_0 > 0$. *Corollary:* if we find a semi-positive S-invariant $\mathbf{I}$ such that $\mathbf{I} \cdot M_0 = 0$ the system is not live.

Proposition. $\mathbf{J}$ is a T-invariant if and only if $\forall p \in P. \sum_{t \in \bullet p} \mathbf{J}(t) = \sum_{u \in p \bullet} \mathbf{J}(u)$.

Fundamental property of T-invariants. Let $M \xrightarrow{\sigma} M'$. Then $\vec{\sigma}$ is a T-invariant if and only if $M' = M$. *Corollary:* any firing sequence whose Parikh vector is a T-invariant leads back to the starting marking.

Reproduction lemma. If a system is bounded and $M_0 \xrightarrow{\sigma}$ with $\sigma \in T^\infty$, then there must exist a semi-positive T-invariant $\mathbf{J}$ such that $\langle \mathbf{J} \rangle \subseteq \{t \in \sigma\}$.

Boundedness + liveness theorem for T-invariants. If a bounded system is live, then it has a positive T-invariant. *Corollary:* every live and bounded system has some $M \in [M_0]$ and $M \xrightarrow{\sigma} M'$ s.t. $\{t \in \sigma\} = T$.

Strong connectedness theorem (2). If a weakly connected system has positive S- and T-invariants, then it is strongly connected. *Consequence:* if a system is not strongly connected, then we cannot find positive S- and T-invariants.

Diagnostic

S-coverability. If a free-choice system is live and bounded, then it is S-coverable. *Corollary:* free-choice + not S-coverable $\Rightarrow$ not live or not bounded.

S-coverability and workflow-nets. If N is sound and free-choice, then N^* is S-coverable. *Corollary:* N free-choice + N^* not S-coverable $\Rightarrow N$ is not sound.

Compositionality of sound free-choice nets. *Lemma:* If a free-choice workflow net is sound, then it is safe. *Proposition:* If N and N' are two sound free-choice workflow nets, then $N[N'/t]$ is sound and free-choice.

Well-structuredness. If N is sound and well-structured, then N^* is S-coverable. *Corollary:* N well structured + N^* not S-coverable $\Rightarrow N$ is not sound.

Fundamental property of non-live error sequences. Let N be such that N^* is bounded and has no dead task. Then, N^* is live if and only if N has no non-live sequences.

Fundamental property of unbounded error sequences. N^* is bounded if and only if N has no unbounded sequences.

A.4: Analysis Summary Table

S-systems	Strongly connected S-systems	T-systems	Strongly connected T-systems	System	Property
✓	✓	$\forall p \in P. \exists \gamma \in \Gamma. p \in \gamma$ (every place belongs to a circuit)	✓	Positive S-invariant	$\Rightarrow$ boundedness
×	×	$\exists p \in P. \forall \gamma \in \Gamma. p \notin \gamma$ (there is a place that is not covered by any circuit)	×	There exist two reachable markings M and M' such that $M \subset M' \wedge M' \in [M]$	$\Rightarrow$ unboundedness
Strongly connected and $M_0(P) > 0$	$M_0(P) > 0$	$\forall \gamma \in \Gamma. M_0(\gamma) > 0$	$\forall \gamma \in \Gamma. M_0(\gamma) > 0$		$\Rightarrow$ liveness
Not strongly connected or $M_0(P) = 0$	$M_0(P) = 0$	$\exists \gamma \in \Gamma. M_0(\gamma) = 0$	$\exists \gamma \in \Gamma. M_0(\gamma) = 0$	There is a semi-positive S-invariant $\mathbf{I}$ such that $\mathbf{I} \cdot M_0 = 0$	$\Rightarrow$ non-liveness
	$M(P) = M_0(P)$				$\Rightarrow M \in [M_0]$ (Reachability)
$M(P) \neq M_0(P)$	$M(P) \neq M_0(P)$	$\exists \gamma \in \Gamma. M(\gamma) \neq M_0(\gamma)$	$\exists \gamma \in \Gamma. M(\gamma) \neq M_0(\gamma)$	$\exists \mathbf{I}. \mathbf{I} \cdot M \neq \mathbf{I} \cdot M_0$	$\Rightarrow M \notin [M_0]$ (Unreachability)

Let N be a workflow net and N^* its corresponding variant with the *reset* transition.

- **Theorem:** N^* is strongly connected.
- **Theorem:** If N is an S-net, then it is sound.
- **Theorem:** If N^* is a T-net whose circuits are all marked, then N is sound.
- **Theorem:** If N^* is a T-net that has at least one unmarked circuit, then N is not sound.
- **Theorem:** If N is free-choice and N^* meets all conditions in the Rank Theorem, then N is sound.
- **Theorem:** If N is free-choice and N^* does not have a positive S-invariant, then N is not sound.
- **Theorem:** If N is free-choice and N^* does not have a positive T-invariant, then N is not sound.
- **Theorem:** If N is free-choice and the rank of N^* is different from the number of its clusters minus 1, then N is not sound.
- **Theorem:** If N is free-choice and the N^* has a proper siphon that does not include the initial place i , then N is not sound.
- **Theorem:** If N is free-choice and N^* is not S-coverable, then N is not sound.
- **Theorem:** If N is well-structured and N^* is not S-coverable, then N is not sound.

Appendix: Process Mining hands-on with Disco

A short tutorial deck by fluxicon walks through a complete process-mining session in their tool *Disco*,⁷⁹ on an event log of purchase orders: 9,119 events for 608 cases, recorded from January to October 2011. Three questions drive the session: *how does the process actually look like? are there deviations from the prescribed process? do we meet the performance targets?* Each step exercises a different Disco view on the same log.

Step 1: Inspect data. Open the CSV (`PurchasingExample.csv`) in pandas or Excel: every row is one event, with case ID, activity, start and complete timestamps, resource, role.

Step 2: Import data. Load the CSV in Disco and assign the columns: case ID → Case ID, the two timestamps → Timestamp, activity → Activity, resource → Resource, role → Other. Start the import.

Step 3: Inspect the process map. Disco discovers a process map: numbers in rectangles are activity frequencies, numbers on arcs are connection frequencies. All 608 cases start with *Create Purchase Requisition*, and a lot of amendments are visible. Two sliders set the level of detail: with *Activities* at 0% only the most frequent variant's activities show; back at 100% even infrequent activities such as *Amend Purchase Requisition* appear. A conservation check on that activity flags an anomaly: 11 cases flow in, only 8 flow out. Raising the *Paths* slider to 100% reveals where the other 3 go: back to *Create Request for Quotation*.

Step 4: Inspect statistics. The *Statistics* tab's overview reports the log-level numbers (9,119 events, 608 cases, January–October 2011). The case-duration histogram sits typically below 15–16 days, with a tail of cases beyond 70–80 days.

Step 5: Inspect cases. The *Cases* tab lists variants. The third most frequent variant (ca. 10.36% of all cases) ends right after *Analyze Purchase Requisition*: the request is cut short. Why are so many requests stopped: do people not know what they can buy? The same pattern shows in the map as a flow leading directly to the end point.

At this stage question 1 is answered: an objective process map has been discovered, and the many amendments and stopped requests call for an update of the purchasing guidelines.

Question 3 is half-answered (not all cases meet the targets, some take longer than 21 days) and raises the next one: is there a bottleneck?

Step 6: Filter on performance. Add a *Performance* filter with 21 days as lower boundary (about 15% of the purchase orders take longer) and apply it to focus on those cases.

Step 7: Spot bottlenecks. The filtered map shows the flow of the 92 slow cases: on average 3 amendments per case (92 cases, 302 amendments). Switching the map from *Frequency* to *Performance* view, *total duration* highlights the high-impact areas and *mean duration* shows that returning from the rework loop to the normal process takes more than 14 days on average (minimum and maximum duration complete the picture).

Step 8: Animate the process. The animation replays the log on the map; dragging the needle along the timeline, the most used paths grow thicker and thicker: the bottleneck visualised in motion. This closes question 3: targets are not met by all cases, and the *Analyze Request for Quotation* activity is a huge bottleneck: a process change is needed.

⁷⁹<https://fluxicon.com/disco/>

Step 9: Compliance check. Exit the animation, remove the performance filter, return to the frequency map and scroll to the end of the process: 10 cases skip the mandatory *Release Supplier's Invoice* activity: a compliance violation. Drilling down (click the path from *Send invoice* to *Authorize Supplier's Invoice payment*, *Filter this path*, then the *Cases* view) lists the ten offending cases. This answers question 2: yes, there are deviations, and the actionable result is to either change the operational system to prevent them or provide targeted training.

Step 10: Organizational view. The last step swaps perspective on the same data: reload the log assigning the *Role* column as Activity (and the original activity column to Other). The map now shows how cases move through the *roles* of the organization rather than through activities: inefficiencies often sit at the borders of organizational units. Here the purchasing agents are causing the biggest delays in the process.

Take-away. The session threads together the recurring process-mining workflow: import → read the map at varying granularities → statistics and variants → filter → spot bottlenecks → animate → compliance check → swap the activity perspective for an organizational one. Each step answers one of the three opening questions, and each answer ends in an actionable recommendation: the value of the exercise is the discipline of the loop, not the tool.

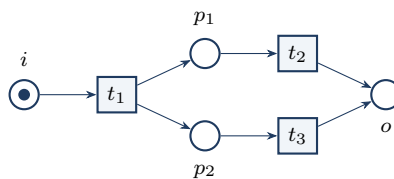
A Worked figures: nets to draw at the exam

This appendix collects small, self-contained nets for the drawing questions that recur at the oral (“draw a net that ...”) and a few diagrams that make a definition concrete. Each figure is minimal on purpose: it carries exactly the structure needed to exhibit one property, so it can be reproduced on paper under exam conditions. Throughout, places are circles, transitions are bars, a filled dot is a token, and for workflow nets i is the unique source and o the unique sink. Every net below was checked by replaying its firing sequences from the stated initial marking.

A.1 Soundness: nets to draw

A.1.1 No dead task, option to complete, but proper completion fails

The single most-requested drawing. The AND-split t_1 opens two branches that *both* flow into the sink o through t_2 and t_3 , with no join synchronising them. Firing t_1 then t_2 reaches $\{o, p_2\}$: the sink carries a token while a sibling token still waits on p_2 . That marking violates proper completion (a token on o must mean the *only* token). No transition is dead: t_1 fires from $\{i\}$, and both t_2 and t_3 fire after it.

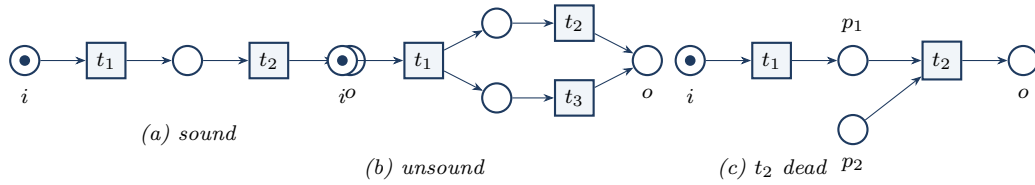


Note (The subtle point). Under the *strict* soundness definition (the good final marking is exactly $\{o\}$), this net actually fails *both* option to complete and proper completion: every maximal run ends at $\{o, o\}$, never at the clean $\{o\}$. The witness above (reach $\{o, p_2\}$) is the direct, unambiguous demonstration that

proper completion is broken, which is what the question asks. Merging t_2 and t_3 into a single AND-join that consumes p_1 and p_2 together restores soundness.

A.1.2 Sound, unsound, and dead

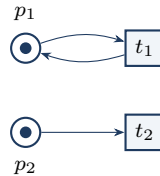
Three minimal nets, one per verdict. (a) A sequence $i \rightarrow t_1 \rightarrow p_1 \rightarrow t_2 \rightarrow o$: live, safe, properly completing: **sound**. (b) An AND-split t_1 feeding an XOR-join at o : firing t_1 then t_2 leaves a stranded token on p_2 while o is marked, and t_3 then yields $\{o, o\}$: **unsound** (proper completion and safeness both broken). (c) A transition t_2 that needs both p_1 and p_2 , but p_2 has no producer: t_2 can never fire: a **dead** transition. Net (c) is deliberately *not* a WF-net: p_2 is disconnected from the source, which is exactly why t_2 starves.



A.2 Liveness and boundedness: nets to draw

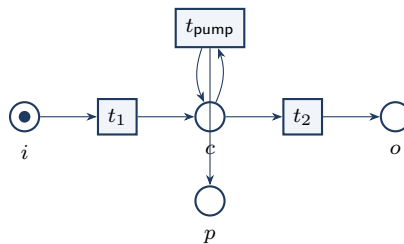
A.2.1 Deadlock-free but not live

A net is *deadlock-free* when every reachable marking still enables some transition, and *live* when every transition can always fire again. The two differ: t_1 's self-loop keeps a token cycling on p_1 , so *some* transition is forever enabled (no deadlock), yet t_2 fires at most once (it drains p_2 , which nothing refills) so t_2 dies. Deadlock-free, not live.



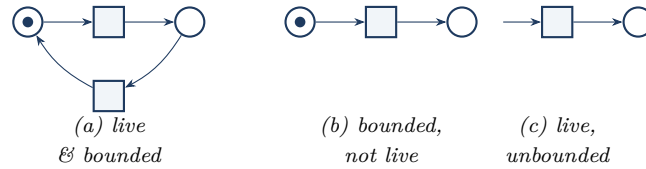
A.2.2 Unbounded

The self-loop transition t_{pump} returns the control token to c while depositing one extra token on p at every firing. Nothing ever consumes from p , so its count grows without limit: from $M_0 = \{i\}$ the sequence $t_1 t_{\text{pump}}^n$ reaches $\{c, p^n\}$ for every n . The covering pair $M'_0 = (c:1) < (c:1, p:1) = M''$ with M'' reachable from M'_0 is the unboundedness witness.



A.2.3 The three liveness/boundedness combinations

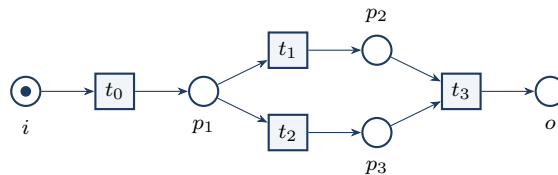
One tiny net per case. (a) A one-token cycle $a_i \rightarrow a_{t1} \rightarrow a_o \rightarrow a_{t2} \rightarrow a_i$: both transitions fire forever, one token throughout: **live and bounded**. (b) A fire-once transition b_{t1} : after it fires the net deadlocks, bounded but **not live**. (c) A source transition c_{t1} (no input place) endlessly feeding c_p : always fireable, but c_p grows without bound: **live but unbounded**.



A.3 Handles: nets to draw

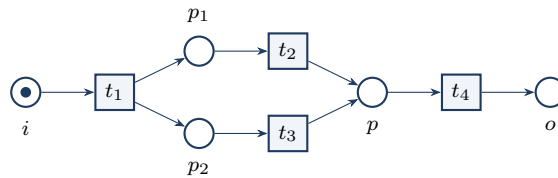
A.3.1 PT-handle: deadlock

A **PT-handle** joins a place p_1 to a transition t_3 by two node-disjoint paths. Here p_1 is an XOR-split (its token enables t_1 or t_2 , not both), while t_3 is an AND-join (it needs p_2 and p_3). The mismatch deadlocks: fire t_0 then t_1 , and the token sits on p_2 while p_3 stays empty, so t_3 never fires and o is never reached. XOR-out synchronised by AND-in $\Rightarrow$ deadlock.



A.3.2 TP-handle: unsafeness

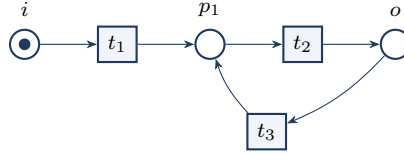
A **TP-handle** joins a transition t_1 to a place p by two node-disjoint paths. Here t_1 is an AND-split (it marks p_1 and p_2), while p is an XOR-join (its consumer t_4 needs only one token). Firing t_1, t_2, t_3 drops *two* tokens on p : the net is no longer 1-safe. AND-out merged by XOR-in $\Rightarrow$ unsafeness.



A.4 S-nets, T-nets, invariants

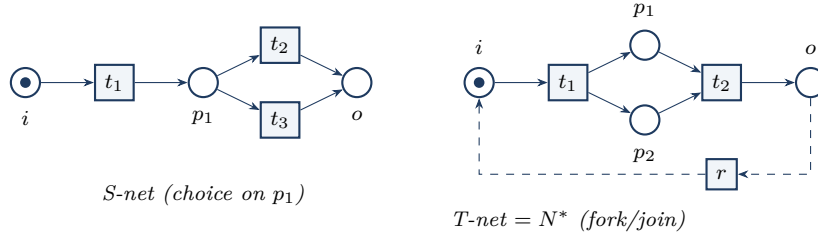
A.4.1 S-net with a non-live transition

In an **S-net** every transition has exactly one input and one output place ($|\bullet t| = 1 = |t\bullet|$). This one carries a single token. t_1 fires once, moving the token off the source i ; since nothing ever re-marks i , t_1 is *non-live* (dead after its first firing) while t_2 and t_3 cycle forever around $p_1 \leftrightarrow o$. Being non-strongly-connected is what lets an S-net starve a transition.



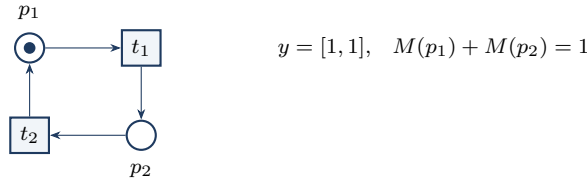
A.4.2 S-net versus T-net

The structural duality. In an **S-net** every transition is one-in/one-out, so branching lives on *places* (choice): here p_1 chooses t_2 or t_3 . In a **T-net** every *place* is one-in/one-out, so branching lives on *transitions* (fork/join): t_1 forks, t_2 joins. The right panel doubles as a short-circuit N^* , its reset r closing o back to i .



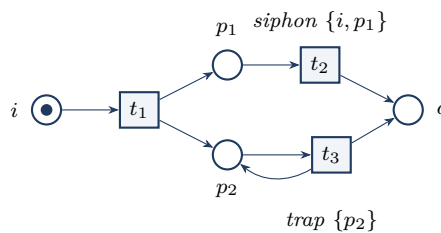
A.4.3 A positive S-invariant certifies boundedness

The two-place loop conserves its single token: $M(p_1) + M(p_2) = 1$ at every reachable marking. That conservation *is* the positive S-invariant $y = [1, 1]$ (it satisfies $y \cdot \mathbf{N} = 0$), and it bounds the net without exploring the state space: each place holds at most 1 token because the weighted sum is fixed at 1.



A.5 Siphons and traps

One net holds both. The set $R = \{i, p_1\}$ is a **siphon**: its only producer is t_1 , which draws solely from within R (from i), so $\bullet R \subseteq R^\bullet$: once R is emptied it stays empty. The place p_2 is a **trap**: its only consumer t_3 loops a token straight back to it, so $R'^\bullet \subseteq \bullet R'$ for $R' = \{p_2\}$: once marked it stays marked.



A.6 The short-circuit N^*

A sound WF-net N (source i , AND-split t_1 , AND-join t_2 , sink o) beside its short-circuit N^* : the dashed reset t_* returns the token from o to i . The loop $t_1 t_2 t_*$ brings the marking back to $\{i\}$, and N^* is live and bounded *exactly* because N is sound: the content of the main theorem.

